# Embedded Linux

BeagleBone Black ile Adım Adım

# İçindekiler


- Önsöz

**Kısım I — Başlarken**

- Bölüm 1 — BeagleBone Black Donanımı
- Bölüm 2 — Seri Konsol ve Minicom

**Kısım II — Boot Süreci**

- Bölüm 3 — Linux Boot Süreci: RBL → SPL → U-Boot
- Bölüm 4 — SD Karttan Boot
- Bölüm 5 — eMMC'den Boot

**Kısım III — U-Boot ile Çalışmak**

- Bölüm 6 — U-Boot Komut Satırı ve uEnv.txt
- Bölüm 7 — Seri Porttan (UART) Boot
- Bölüm 8 — TFTP ile Ağdan Boot
- Bölüm 9 — Cross Toolchain, SPL ve U-Boot Derleme

**Kısım IV — Linux Kernel**

- Bölüm 10 — Linux Kernel: Kaynak Ağacı ve Derleme
- Bölüm 11 — Linux Device Tree

**Kısım V — Root Filesystem**

- Bölüm 12 — BusyBox ile Minimal Root Filesystem

# Önsöz

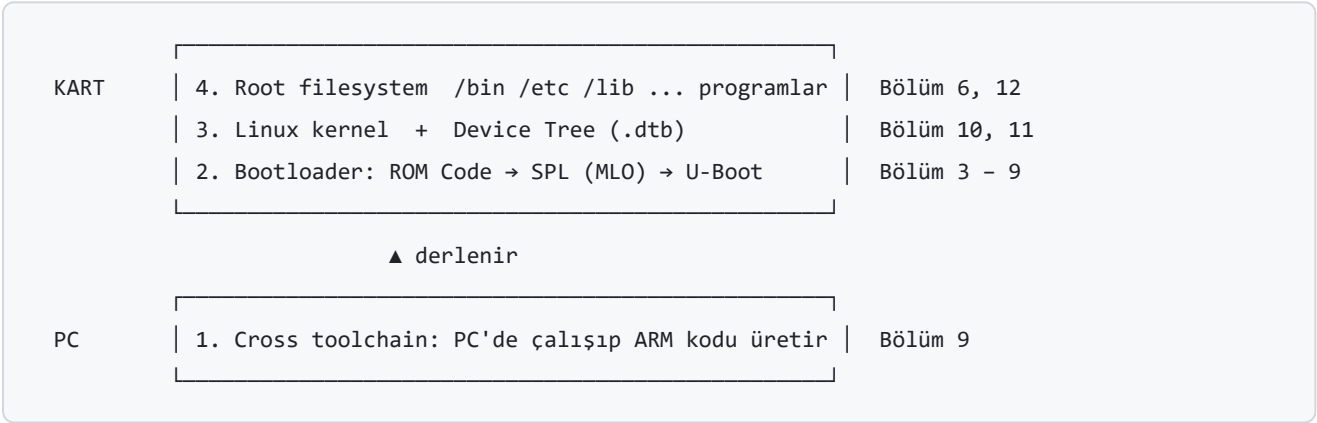
Bu kitap, gömülü Linux'a (Embedded Linux) yeni başlayan biri için yazıldı. Bir **BeagleBone Black** kartını sıfırdan alıp önce hazır dosyalarla, sonra kendi derlediğin bootloader ile açacaksın. Her adımda **neyi neden yaptığını** görmen hedefleniyor.

Kitap, bir Embedded Linux kursunu (https://github.com/niekiran/EmbeddedLinuxBBB) takip ederken alınan notlardan oluştu. Kursla birlikte de tek başına da okunabilir.

---

## Bir Embedded Linux Sistemi Nelerden Oluşur?

Masaüstü bilgisayarda bu parçaları hiç görmeyiz; hepsi hazır gelir. Gömülü sistemde ise her birini tek tek hazırlamak, karta koymak ve birbirine bağlamak bizim işimizdir:




| Parça | Görevi | Kitaptaki yeri |
|---|---|---|
| **Cross toolchain** | PC'de (x86) çalışıp karta (ARM) uygun program üreten derleyici ve araçlar | Bölüm 9 |
| **Bootloader** | Kart açılınca belleği hazırlar, kernel'i ve device tree'yi RAM'e yükleyip başlatır | Bölüm 3 – Bölüm 9 |
| **Linux kernel + Device Tree** | İşletim sisteminin çekirdeği ve kartın donanımını kernel'e tarif eden dosya | Bölüm 10, Bölüm 11 |
| **Root filesystem** | Kernel açıldıktan sonra çalışan her şey: kabuk, komutlar, ayar dosyaları | Bölüm 6, Bölüm 12 |

---

## Kitabın Haritası

| Kısım | Bölüm | Ne yapacaksın? |
|---|---|---|
| **I — Başlarken** | 1. BeagleBone Black Donanımı | Kartı, butonları, LED'leri ve debug header'ını tanı |
| | 2. Seri Konsol ve Minicom | Kartın konsolunu bilgisayarda gör |

| Kısım | Bölüm | Ne yapacaksın? |
|---|---|---|
| **II — Boot Süreci** | 3. Linux Boot Süreci | Kart açılırken hangi programın hangi sırayla çalıştığını öğren |
| | 4. SD Karttan Boot | SD kartı hazırla, kartı adım adım SD karttan aç |
| | 5. eMMC'den Boot | Kartın dahili belleğine güncel Debian yükle |
| **III — U-Boot ile Çalışmak** | 6. U-Boot Komut Satırı ve uEnv.txt | U-Boot'u durdur, ayarlarını değiştir, rootfs'i tanı |
| | 7. Seri Porttan (UART) Boot | Kartı SD kart ve eMMC olmadan, sadece seri kabloyla aç |
| | 8. TFTP ile Ağdan Boot | Kernel'i her seferinde ağdan çekerek geliştirmeyi hızlandır |
| | 9. Cross Toolchain, SPL ve U-Boot Derleme | Bootloader'ı kaynak koddan kendin derle |
| **IV — Linux Kernel** | 10. Linux Kernel: Kaynak Ağacı ve Derleme | Kernel kaynağında gezin, kernel'i ve modülleri derle |
| | 11. Linux Device Tree | Kernel'in donanımı nasıl tanıdığını öğren |
| **V — Root Filesystem** | 12. BusyBox ile Minimal Root Filesystem | Kendi rootfs'ini sıfırdan kur, NFS ile bilgisayardan aç |

**Hazırlanan bölümler:** Kursun kalan konuları kitaba eklendikçe bu tabloya girecek: Buildroot, Eclipse ile uygulama geliştirme, AM335x GPIO'ları ve sysfs, harici LED, 7-segment gösterge, MPU6050 ivmeölçer/jiroskop, 16x2 LCD.

**Okuma sırası:** Bölümler birbirinin üzerine kurulu. İlk kez okuyorsan sırayla git. Her bölümün başındaki **"Ön koşul"** satırı, o bölümden önce hangilerini bilmen gerektiğini söyler.

Her bölüm aynı düzende yazıldı:

1. **Bu bölümde:** Bölümün sonunda neleri yapabileceğin.
2. **Konu anlatımı ve adım adım uygulama:** Komutlar hem terminal görüntüsü olarak hem de kopyalanabilir kod bloğu olarak verilir.
3. **Özet:** Bölümün tek bakışta tekrarı.
4. **Karşılaşılabilecek problemler:** Sık görülen hatalar, sebepleri ve çözümleri.

---

## Gerekenler

| Donanım | Neden? |
|---|---|
| BeagleBone Black (Rev C) | Kitabın kartı |
| microSD kart (en az 4 GB) + kart okuyucu | SD karttan boot için |
| USB-TTL seri dönüştürücü (**3.3V**) | Kartın konsolunu görmek için. En önemli debug aracın. |

| Donanım | Neden? |
|---|---|
| 5V / 2A adaptör (barrel jack) | Seri boot ve TFTP bölümlerinde karta güvenilir güç için |
| Ethernet kablosu | TFTP ile ağdan boot için |
| Mini USB kablo | Karta güç vermek ve USB üzerinden SSH için |

| Yazılım | Not |
|---|---|
| **Ubuntu** (host PC) | Gerçek kurulum ya da sanal makine. Sanal makinede ağ ayarı **Bridged** olmalı (Bölüm 8). |
| `minicom`, `lrzsz` | Seri konsol ve Xmodem/Ymodem ile dosya gönderme (Bölüm 2) |
| `gparted` | SD kartı bölümlemek için (Bölüm 4) |
| `tftpd-hpa` | TFTP sunucusu (Bölüm 8) |
| `nfs-kernel-server` | Rootfs'i ağdan bağlamak için NFS sunucusu (Bölüm 12) |
| Cross toolchain, U-Boot, kernel, BusyBox kaynakları | Derleme bölümlerinde kurulur (Bölüm 9, Bölüm 10, Bölüm 12) |

---

## Bu Kitaptaki Sürümler

Komutlarda ve ekran görüntülerinde geçen sürümler şunlardır. Kendi kurulumunda farklı bir sürüm kullanıyorsan komutlardaki sürüm numaralarını ona göre değiştir.

| Ne? | Değer | Bölüm |
|---|---|---|
| Kursun ilk hazır imajları (Ångström): U-Boot / kernel | 2013.04 / 3.8.13 | 4, 6, 7, 8 |
| Kursun seri boot imajları: SPL / U-Boot | 2017.03 / 2016.11-rc1 | 7 |
| Kursun güncel SD kart imajları: U-Boot / kernel | 2022.04 / 5.10.168 | 4, 8 |
| eMMC'ye yüklenen Debian imajı | `bone-debian-10.3-iot-armhf-2020-04-06-4gb.img.xz` | 5 |
| Kendi derlediğimiz U-Boot | 2022.04 ( `am335x_evm_defconfig` ) | 9 |
| Cross toolchain | Linaro GCC 7.5.0 (7.5-2019.12) | 9, 10, 12 |
| Kendi derlediğimiz kernel | 5.10.162 (BeagleBoard, dal `5.10` , `bb.org_defconfig` ) | 10 |
| BusyBox / Dropbear | 1.36.1 / 2022.83 | 12 |
| Host PC IP / kart IP | 192.168.27.1 / 192.168.27.2 | 8 |

---

## Yazım Kuralları

**Komut satırları:** Bir komutun nerede çalıştırılacağını başındaki işaretten anlarsın.

| Görünüm | Nerede? |
|---|---|
| `mehmet@ubuntu:~$ komut` veya kod bloğunda `bash` | Host PC'deki Ubuntu terminali |
| `U-Boot# komut` veya `=> komut` | Kartın U-Boot komut satırı (seri konsolda). Eski U-Boot `U-Boot#`, yenisi `=>` gösterir. |
| `root@beaglebone:~# komut` veya `/ # komut` | Kartta açılmış Linux (seri konsol veya SSH) |

**Ekran görüntüleri:** Koyu renkli terminal görüntüleri adımları göstermek için hazırlanmış **temsili** çıktılardır. Sürüm, tarih, boyut ve hız değerleri senin kartında farklı görünebilir. Kursun slaytlarından ve programların kendisinden alınan görüntüler ise olduğu gibi kullanıldı.

**Kutular:**

> **Not**
>
> Ek bilgi, arka plan.

> **İpucu**
>
> İşini kolaylaştıracak bir yol.

> **Önemli**
>
> Atlarsan adımın çalışmayacağı bilgi.

> **Uyarı**
>
> Dikkat etmezsen zaman kaybettirecek veya bir şeyi bozacak durum.

> **Dikkat**
>
> Veri kaybına veya donanım hasarına yol açabilecek durum.

---

## Kaynaklar

- Kursun deposu: https://github.com/niekiran/EmbeddedLinuxBBB
- BeagleBone Black resmi dokümanı
- AM335x Technical Reference Manual (TRM): repodaki `am335x_referance_manuel.pdf`
- U-Boot dokümantasyonu
- Bootlin eğitim materyalleri (CC BY-SA 3.0)

# Kısım I
# Başlarken

# Bölüm 1 — BeagleBone Black Donanımı

> **Bu bölümde:**
>
> - BeagleBone Black'in işlemcisini, belleklerini ve bağlantı noktalarını tanıyacaksın.
> - Kartın üç butonunun (Power, Reset, Boot) ne işe yaradığını öğreneceksin.
> - Seri konsol için kullanacağın J1 debug header'ının pinlerini göreceksin.
>
> **Ön koşul:** Yok. Kitabın ilk bölümü.

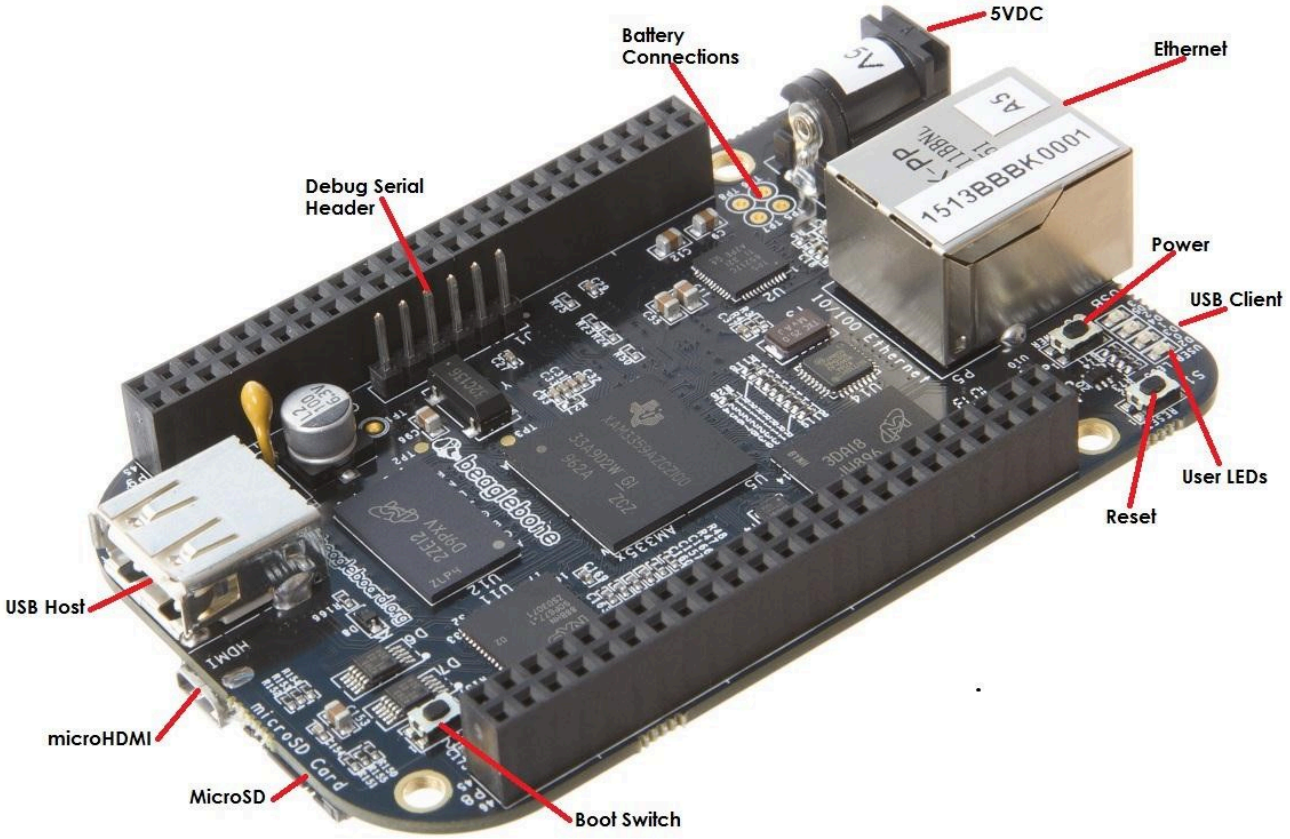
> Kaynak: BeagleBone Black resmi dokümanı (docs.beagleboard.org). Bu bölümdeki kart görselleri bu dokümandan alınmıştır (CC BY-SA).

---

## 1.1 Genel Özellikler

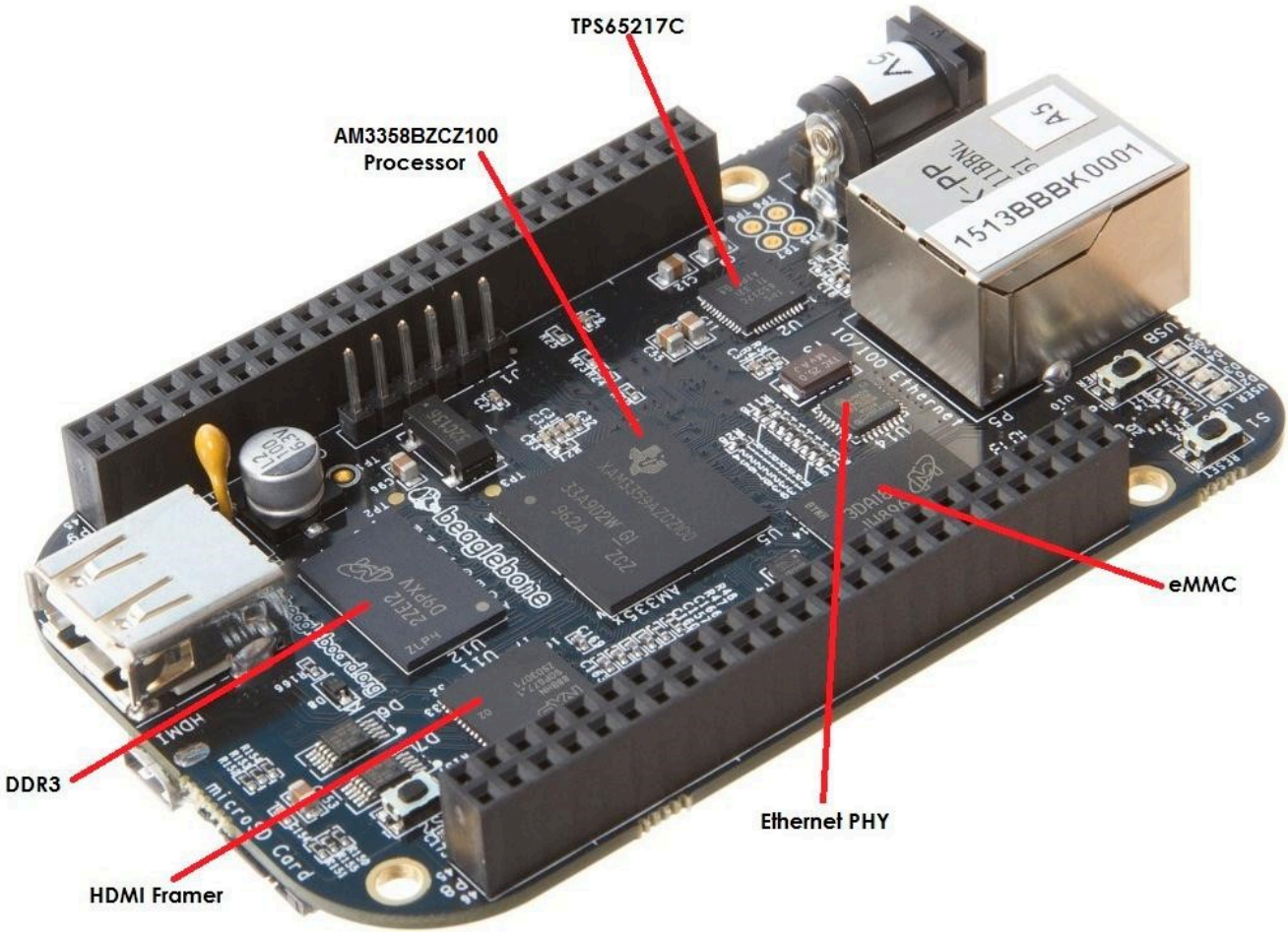
| Özellik | Değer |
|---|---|
| **İşlemci (SoC)** | TI Sitara **AM3358BZCZ100**, ARM **Cortex-A8**, **1 GHz**'e kadar |
| **Cache** | 64 KB L1, 256 KB L2 |
| **RAM** | **512 MB DDR3L** (400 MHz clock, 800 MHz efektif) |
| **Dahili depolama** | **4 GB eMMC** (Rev C, eski revizyonlarda 2 GB), MMC1 üzerinden 8-bit |
| **Harici depolama** | **microSD** yuvası (MMC0) |
| **EEPROM** | 4 KB, I2C0 üzerinde. Kart adı, seri numarası ve revizyon bilgisini tutar. |
| **PMIC** | **TPS65217C** (güç yönetimi) + 3.3V için harici LDO (TLV70233) |
| **Ethernet** | 10/100 Mbit (RJ45) |
| **USB** | USB0: mini-USB **client** (PC bağlantısı), USB1: Type-A **host** (500 mA) |
| **Görüntü** | micro-HDMI (NXP TDA19988 HDMI framer) |
| **Genişleme** | **P8** ve **P9** (2 x 46 pin) header'ları, en fazla 4 cape takılabilir |
| **PRU-ICSS** | 2 adet 200 MHz gerçek zamanlı yardımcı çekirdek (PRU) |
| **Debug** | **J1**: 6 pinli seri debug header (UART0), opsiyonel 20 pinli JTAG (lehimli gelmez) |

## 1.2 Kart Üzerindeki Bileşenler



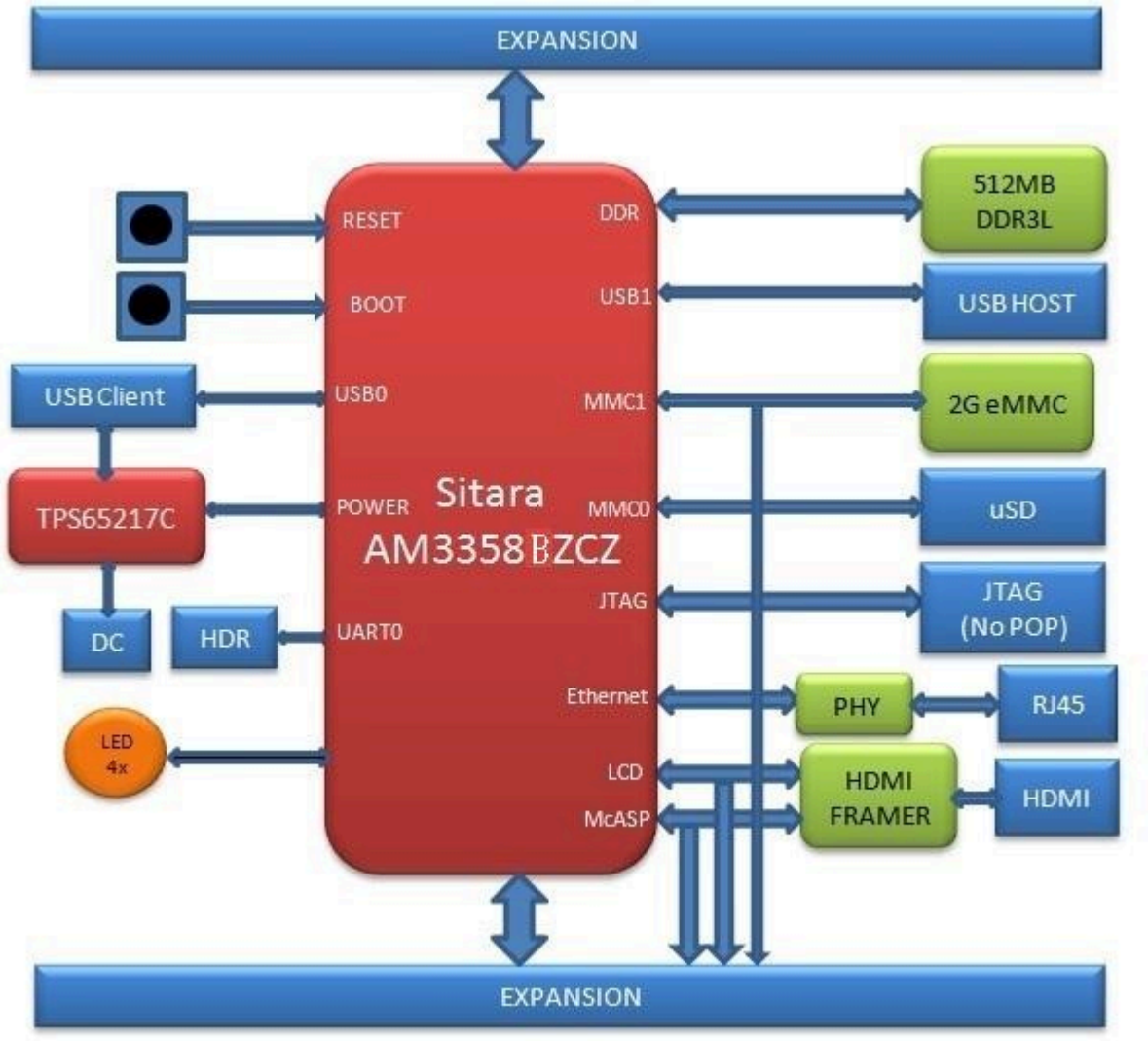

- **5VDC**: Barrel jack güç girişi (5V, en az 1A).
- **USB Client (mini-USB)**: Kartı bilgisayara bağlarsın. Hem güç verir hem de USB üzerinden ağ ve depolama sağlar. Kartın USB ağ adresi Windows'ta `192.168.7.2`, Linux ve macOS'ta `192.168.6.2`'dir (5.4).
- **Debug Serial Header (J1)**: Seri konsol bağlantısı. Kitap boyunca en çok kullanacağın yer burası.
- **Power / Reset / Boot Switch**: Kartın üç butonu (1.5).
- **User LEDs**: 4 adet mavi LED (USR0-USR3).

## Kart üzerindeki çipler




| Çip | Görevi |
|---|---|
| **AM3358BZCZ100** | Ana işlemci (SoC) |
| **DDR3** | 512 MB RAM |
| **eMMC** | Kart üstündeki flash bellek, varsayılan boot kaynağı |
| **TPS65217C** | PMIC. Tüm voltaj raylarını üretir ve power butonunu yönetir. |
| **Ethernet PHY** | Ethernet fiziksel katman çipi |
| **HDMI Framer** | LCD sinyallerini HDMI'a çevirir |

## 1.3 Blok Diyagram




> **Not**
>
> Diyagramda **2G eMMC** yazıyor çünkü eski revizyona ait. Rev C kartlarda eMMC **4 GB**'dır.

Diyagramdan çıkan önemli noktalar:

- **MMC1** → eMMC, **MMC0** → microSD. Kernel ve U-Boot tarafında bu isimleri sık göreceksin.
- **UART0** → Debug header (J1). Linux'ta `/dev/ttyS0` (eski kernel'lerde `/dev/ttyO0`) olarak görünür.
- **USB0** → Client portu, **USB1** → Host portu.

## 1.4 Güç

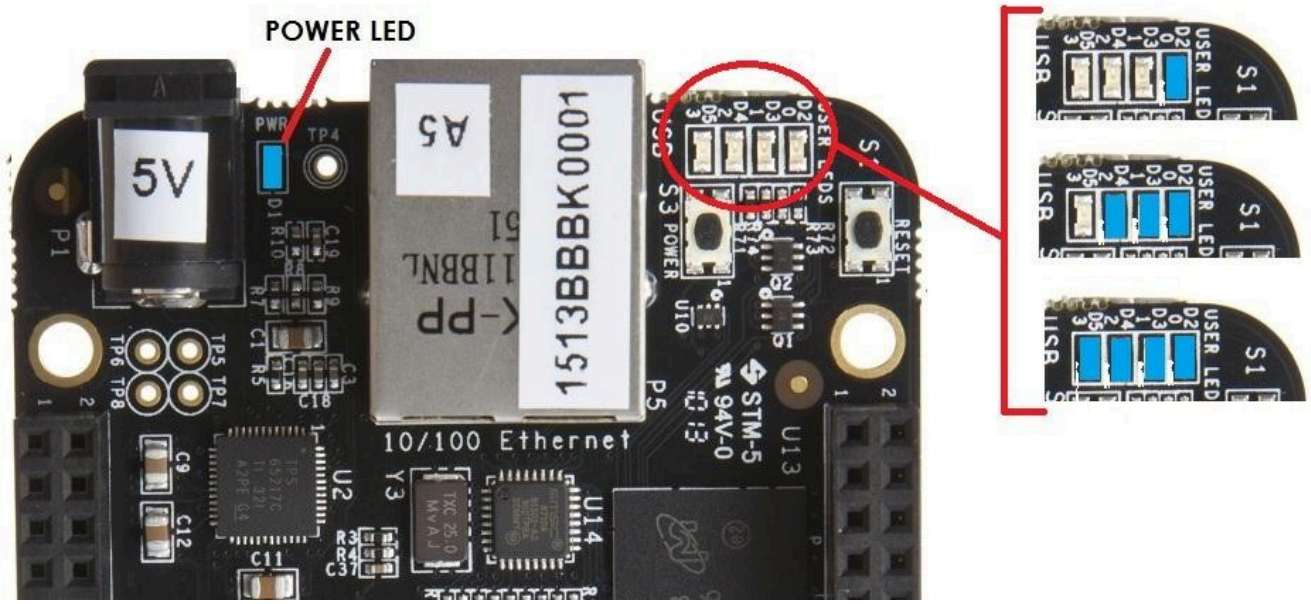
- **USB'den güç**: En fazla ~500 mA çeker. Bu modda P9'daki **VDD_5V** pinlerine güç gelmez.
- **DC jack'ten güç**: 5V ±0.25V, 1A önerilir. Cape takıyorsan daha yüksek akımlı bir adaptör gerekebilir.
- **Tipik tüketim (5V'ta)**: U-Boot'ta boşta ~210 mA, kernel boot sırasında ~460 mA (tepe), kernel boşta ~350 mA.

> **Uyarı**
>
> Kartı kapatmak için kabloyu çekme. `sudo poweroff` komutunu kullan veya **Power** butonuna kısa bas. Ani güç kesilmesi eMMC/SD karttaki dosya sistemini bozabilir.

## 1.5 Butonlar

| Buton | Kart üstündeki adı | Görevi |
|---|---|---|
| **Power** | S3 | Kısa basış işletim sistemine düzgün kapanma sinyali gönderir. **8 saniyeden uzun** basılı tutarsan kart zorla kapanır. |
| **Reset** | S1 | Kartı yeniden başlatır (warm reset). Boot modunu **değiştirmez**. |
| **Boot** | S2 | Güç verilirken basılı tutulursa boot sırasını değiştirir (1.7). |

## 1.6 LED'ler




- **Power LED** (mavi): Güç varken yanar. Açılışta yanıp sönüyorsa PMIC aşırı akım nedeniyle kartı kapatmış demektir.
- **User LED'ler (USR0-USR3)**: Yazılımdan GPIO ile kontrol edilir. Hazır Debian imajındaki varsayılan görevleri:

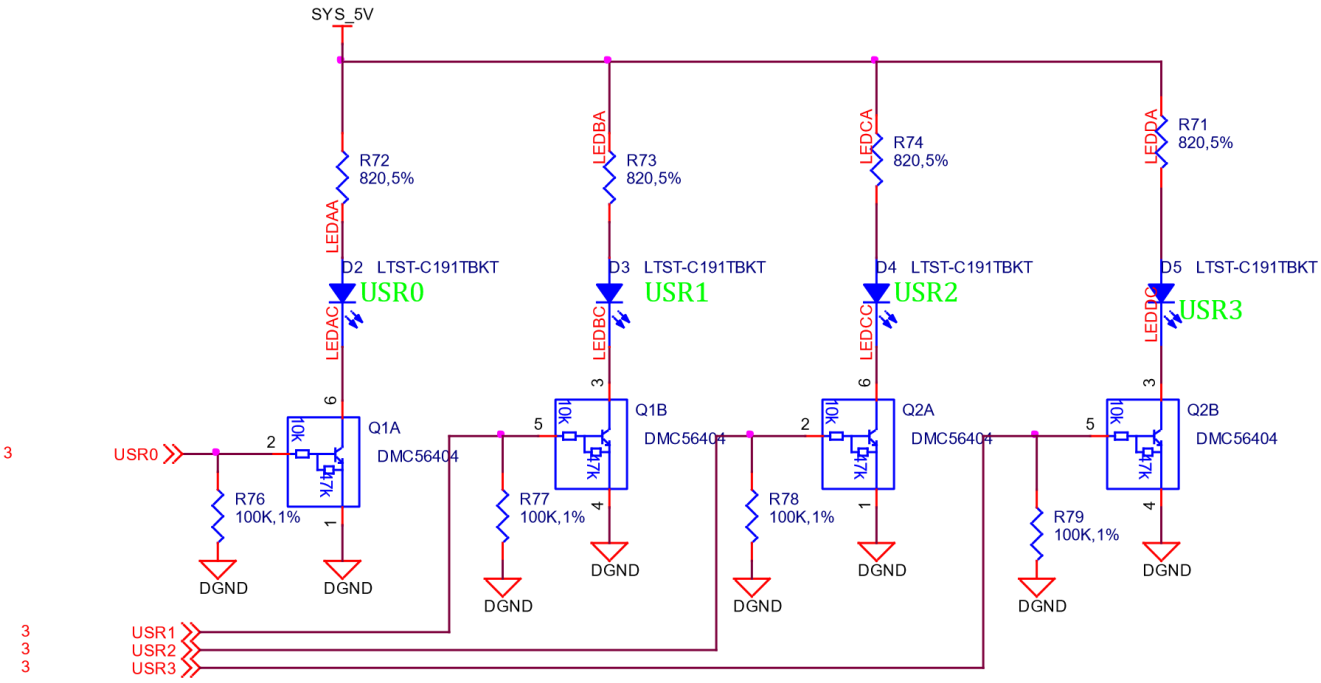
| LED | GPIO | Varsayılan görev (Debian imajı) |
|---|---|---|
| USR0 | GPIO1_21 | **heartbeat**: Kalp atışı gibi yanıp söner, "kernel çalışıyor" demektir |
| USR1 | GPIO1_22 | microSD erişimi |
| USR2 | GPIO1_23 | CPU aktivitesi |

| LED | GPIO | Varsayılan görev (Debian imajı) |
|---|---|---|
| USR3 | GPIO1_24 | eMMC erişimi |

Linux'ta LED'leri şu dizinden kontrol edebilirsin:

```
ls /sys/class/leds/
echo none > /sys/class/leds/beaglebone:green:usr0/trigger
echo 1 > /sys/class/leds/beaglebone:green:usr0/brightness
```

User LED'lerin devre şeması (her LED bir transistör üzerinden GPIO ile sürülür):




> **İpucu**
>
> USR0'ın kalp atışı gibi yanıp sönmesi kernel kodunda değil, **Device Tree**'de tanımlıdır (11.5).

## 1.7 Boot Sırası ve S2 Butonu

Kitap açısından **en önemli başlık budur.**

AM3358 açılırken **SYSBOOT** pinlerinin seviyesine bakar ve hangi kaynaktan boot edeceğine bu pinlere göre karar verir. **Boot (S2)** butonu bu bitlerden birini değiştirir.

| SYSBOOT[15:14] | SYSBOOT[13:12] | SYSBOOT[11:10] | SYSBOOT[9] | SYSBOOT[8] | SYSBOOT[7:6] | SYSBOOT[5] | SYSBOOT[4:0] | Boot Sequence | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz<br>11b = 26MHz | 00b<br>(all other values reserved) | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 enabled | 11100b | MMC1 | MMC0 | UART0 | USB0[5] |
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz<br>11b = 26MHz | 00b<br>(all other values reserved) | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 enabled | 11000b | SPI0 | MMC0 | USB0[5] | UART0 |

| Durum | Boot sırası |
|---|---|
| **S2 basılı değil** (varsayılan) | **MMC1 (eMMC)** → MMC0 (microSD) → UART0 → USB0 |
| **S2 basılı** (güç verilirken) | SPI0 → **MMC0 (microSD)** → USB0 → UART0 |

> **Önemli**
>
> Kendi derlediğin U-Boot/kernel'i **SD karttan** açmak için: Kartın gücünü kes, **S2**'yi basılı tut, gücü ver, birkaç saniye sonra bırak. Boot modu **sadece güç verilirken** okunur. Reset butonu boot modunu değiştirmez.

Boot zincirinin (ROM Code → SPL → U-Boot → Kernel) ayrıntıları ve SYSBOOT tablosunun nasıl okunduğu Bölüm 3'te anlatılıyor.

## 1.8 Seri Debug Header (J1)

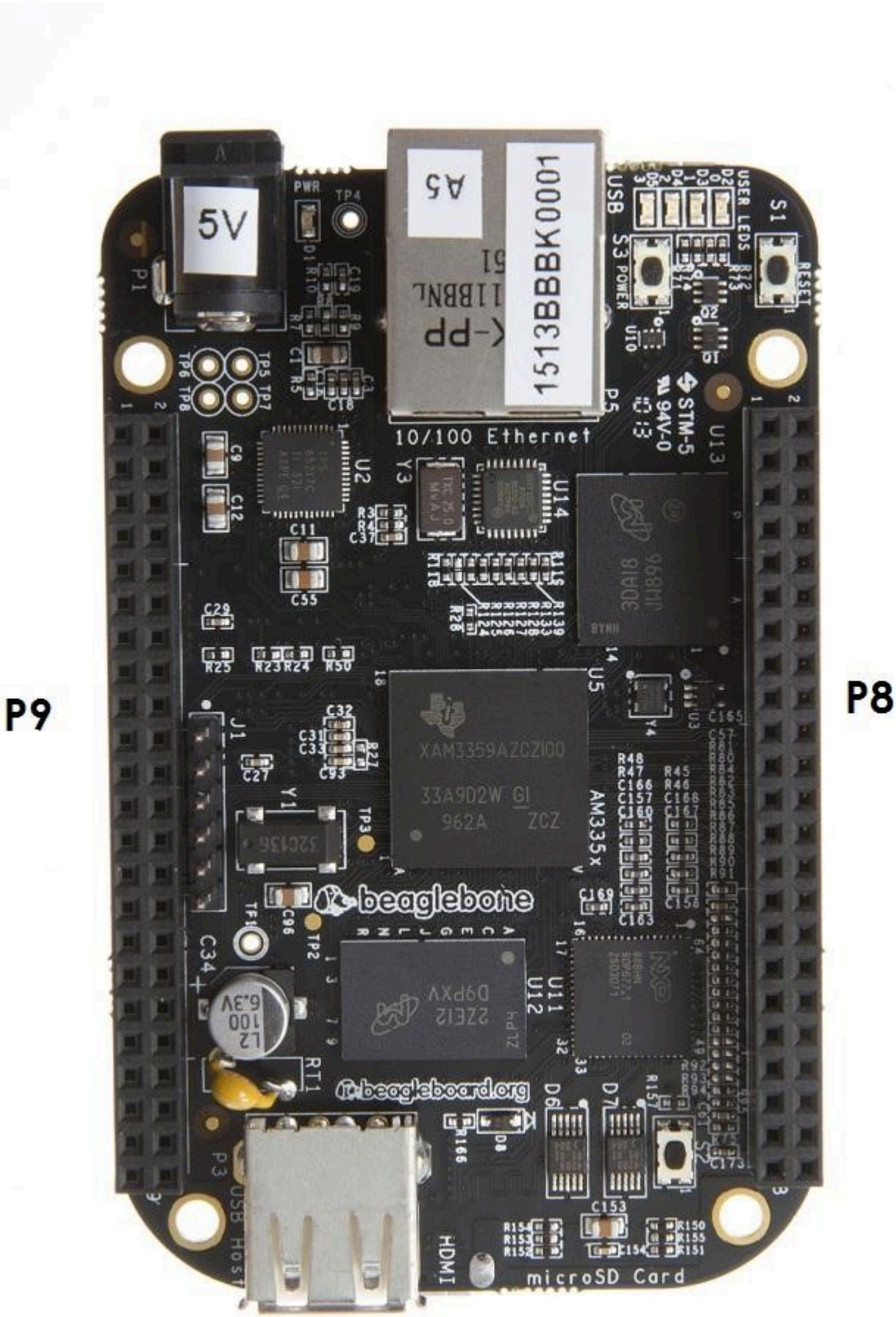
| J1 Pini | Sinyal | USB-TTL dönüştürücüde bağlanacağı yer |
|---|---|---|
| **1** | GND | GND |

| J1 Pini | Sinyal | USB-TTL dönüştürücüde bağlanacağı yer |
|---|---|---|
| 2 | — | — |
| 3 | — | — |
| **4** | RX (kartın alıcısı) | Dönüştürücünün **TX**'i |
| **5** | TX (kartın vericisi) | Dönüştürücünün **RX**'i |
| 6 | — | — |

- 1 numaralı pin kart üzerinde **beyaz nokta** ile işaretlidir. FTDI kablosu kullanıyorsan siyah kablo (GND) bu pine gelir.
- Ayarlar: **115200 8N1**, flow control **kapalı** (Bölüm 2).

> **Uyarı**
>
> Sinyaller **3.3V**'tur. USB-TTL dönüştürücünün **3.3V** modda olduğundan emin ol, 5V'luk dönüştürücü kartı bozabilir. Dönüştürücünün **VCC** pinini karta **bağlama**, sadece GND, TX ve RX yeterli.

## 1.9 Genişleme Header'ları (P8 ve P9)



- Her biri 2x23 = **46 pin**. Toplam 92 pin.
- Pinler **multiplexed**: Aynı pin GPIO, UART, I2C, SPI, PWM gibi farklı görevlerde kullanılabilir. Hangi görevin seçileceği **pinmux** (device tree / overlay) ile belirlenir.

| Pin(ler) | Sinyal |
|---|---|
| P8_1, P8_2, P9_1, P9_2, P9_43-46 | **GND** |
| P9_3, P9_4 | **3.3V** |
| P9_5, P9_6 | **VDD_5V** (sadece DC jack'ten güç verilirken var) |
| P9_7, P9_8 | **SYS_5V** |
| P9_9 | PWR_BUT |
| P9_10 | SYS_RESETn |
| P9_32 | VADC (ADC referansı) |
| P9_33-40 | **AIN0-AIN6** (analog girişler) |
| P9_34 | AGND |

> **Dikkat**
>
> - GPIO pinleri **3.3V**'tur ve **5V toleranslı değildir**. 5V sinyal bağlarsan işlemci bozulur.
> - Analog girişler (AIN) en fazla **1.8V** kabul eder.
> - Kart tamamen açılmadan (SYS_RESETn HIGH olmadan) pinlere dışarıdan sinyal verme.

## 1.10 Özet

1. Seri konsol (J1, 115200 8N1) en önemli debug aracın.
2. Varsayılan boot **eMMC**'den. Kendi imajını **SD karttan** açmak için **S2**'yi basılı tutarak güç ver.
3. Boot zinciri: **ROM → MLO (SPL) → U-Boot → Kernel + DTB → RootFS**.
4. eMMC = **MMC1**, microSD = **MMC0**.
5. Tüm I/O **3.3V**, ADC **1.8V**.

# Bölüm 2 — Seri Konsol ve Minicom

> **Bu bölümde:**
>
> - USB-TTL dönüştürücüyü bilgisayara tanıtıp port adını ( `/dev/ttyUSB0` ) bulacaksın.
> - Minicom'u kurup BeagleBone Black için doğru ayarlarla (115200 8N1, flow control kapalı) kaydedeceksin.
> - Minicom'un sık kullanılan kısayollarını ve Windows'taki karşılığı Tera Term'i öğreneceksin.
>
> **Ön koşul:** Bölüm 1 — J1 debug header'ının pinleri.

---

## 2.1 Seri Konsol Neden Önemli?

Kart açılırken ekrana bir şey basan ilk programlar (ROM Code, SPL, U-Boot) HDMI veya ağ kullanmaz. Bütün mesajlarını **UART0** üzerinden, yani J1 header'ından gönderirler. Bu mesajları görmenin ve U-Boot'a komut yazmanın tek yolu seri konsoldur:

```
[Klavye] ⇄ [minicom / Tera Term] ⇄ USB-TTL kablo ⇄ J1 (UART0) ⇄ [BeagleBone: ROM / SPL / U-Boot / Linux]
```

Bilgisayardaki program sadece bir penceredir. Komutları bilgisayar değil, **kartın üzerindeki** program (U-Boot veya Linux) çalıştırır.

## 2.2 Minicom'u Kur

**Minicom**, Linux'ta kullanılan bir seri port terminal programıdır. `lrzsz` paketi de Minicom'un Xmodem/Ymodem ile dosya göndermesi için gerekir (Bölüm 6 ve Bölüm 7'de kullanacağız):

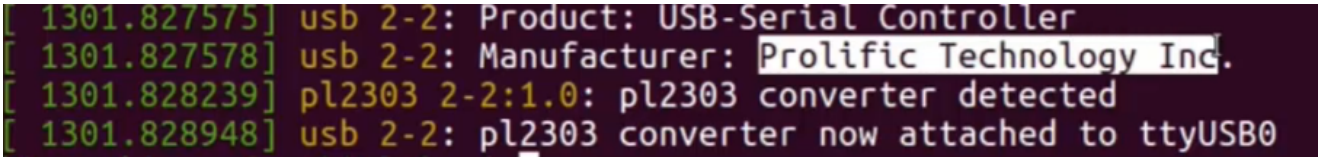
```
sudo apt update
sudo apt install minicom lrzsz
```

## 2.3 USB-TTL Dönüştürücünün Port Adını Bul

USB-TTL dönüştürücüyü bilgisayara tak ve terminalde şunu çalıştır:

```
sudo dmesg | grep tty
```

Çıktı aşağıdakine benzer olmalı:




- Kernel, dönüştürücünün çipini tanıyıp sürücüsünü yükler. Bu örnekte çip **pl2303**. Senin dönüştürücünde **cp210x**, **ftdi_sio** veya **ch341** de yazabilir.
- Satırın sonundaki `ttyUSB0`, Minicom'da kullanacağımız portun adıdır: `/dev/ttyUSB0`.

Portu her seferinde `sudo` ile açmamak için kullanıcını `dialout` grubuna ekle (bir kez yapılır, sonra oturumu kapatıp açman gerekir):

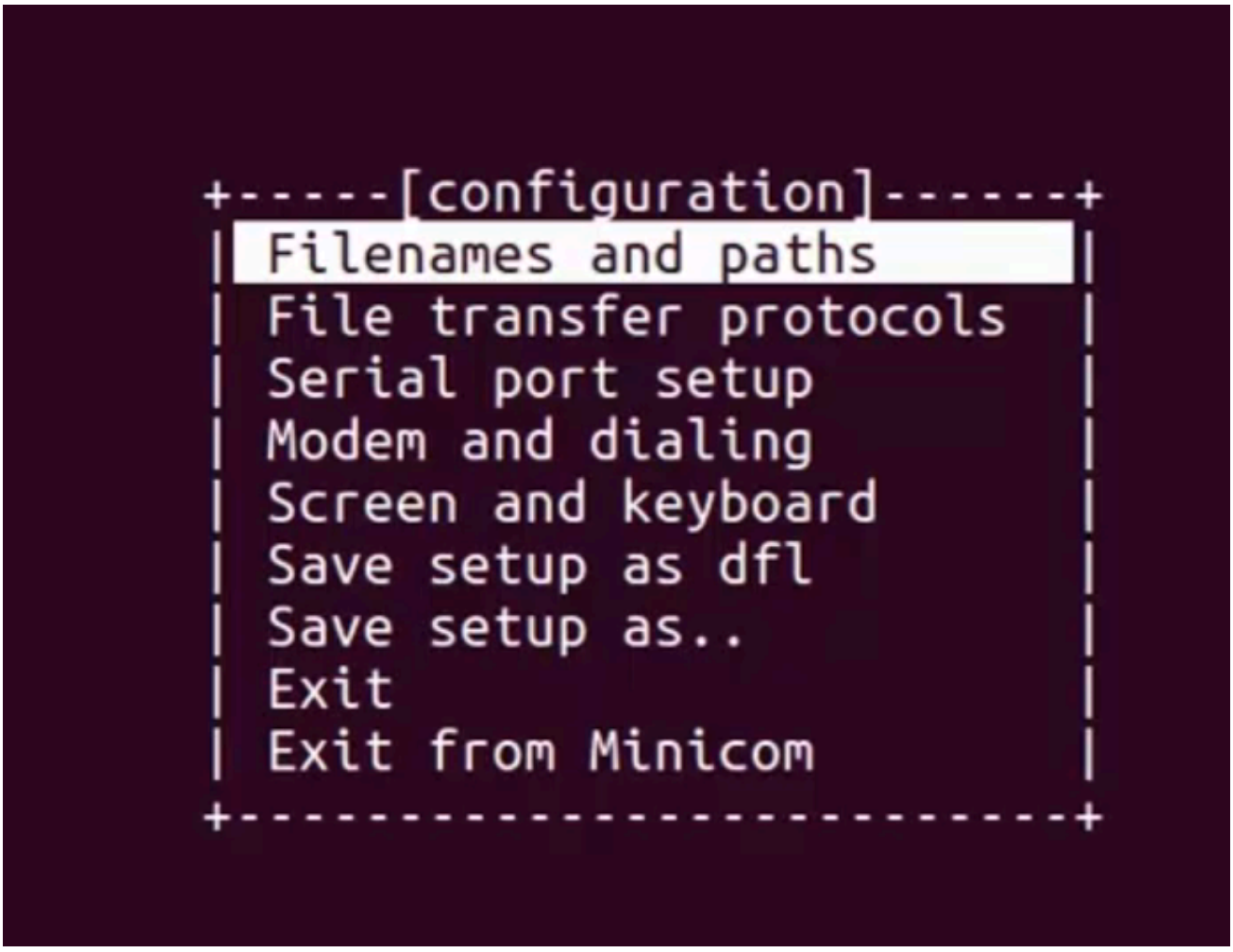
```
sudo usermod -aG dialout $USER
```

## 2.4 Minicom Ayarlarını Yap

Ayar ekranını aç:

```
sudo minicom -s
```

Yön tuşlarıyla menüde gezinebilirsin. **Serial port setup**'ı seç:

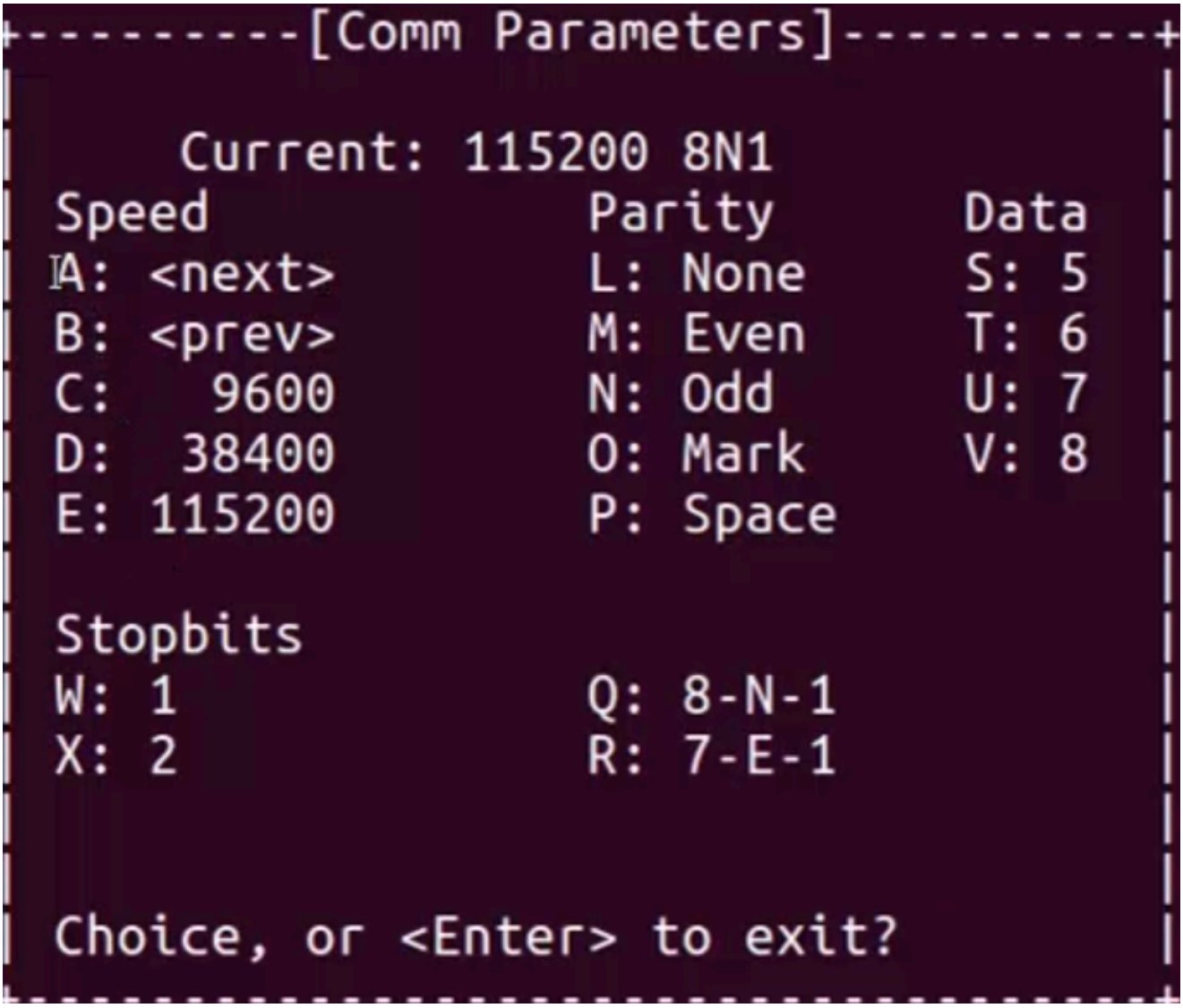


Seri port ayarları ekranı açılır:

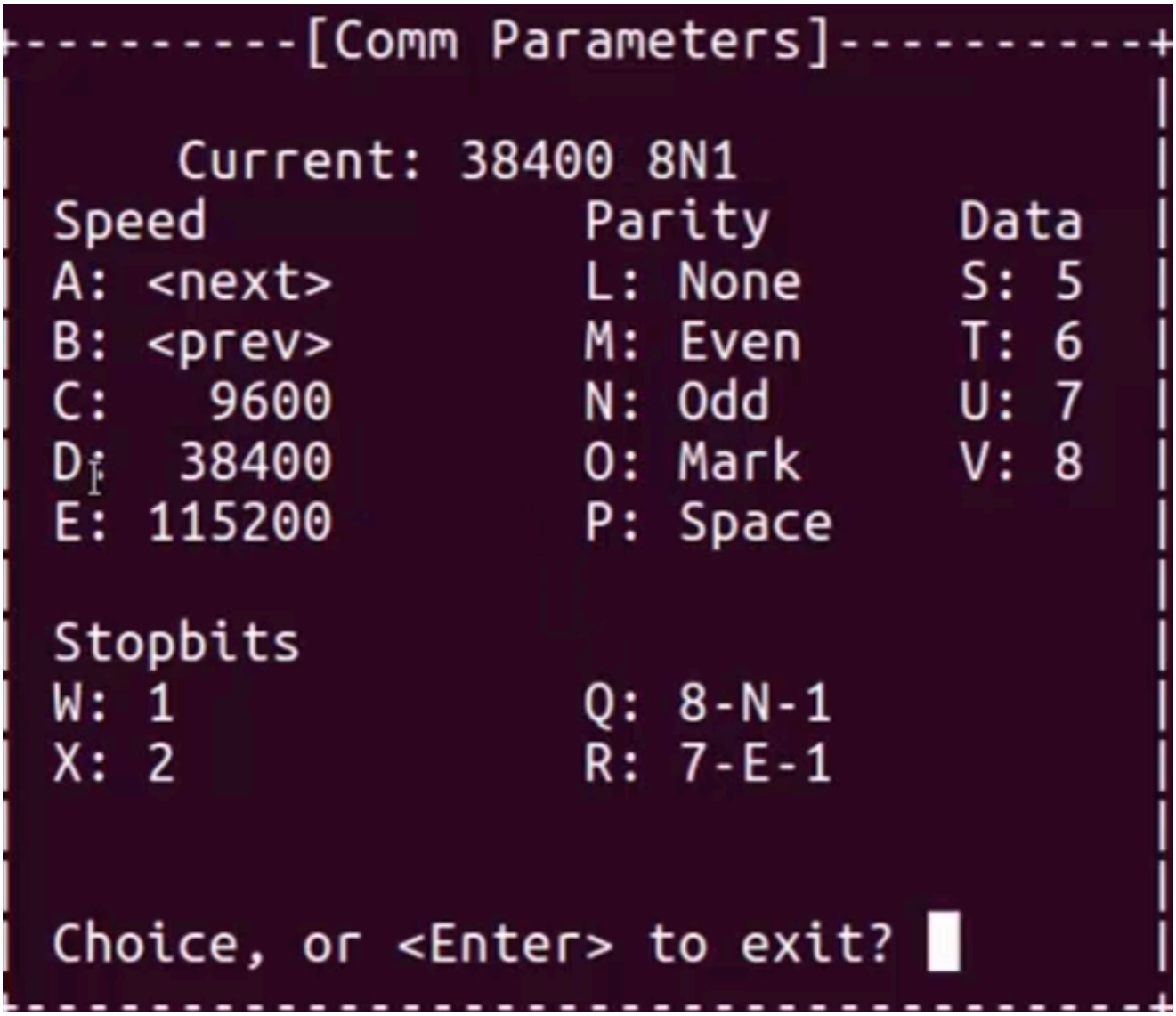
Bu ekranda her ayarın başında bir harf vardır. Değiştirmek istediğin ayarın harfine basarsın. Örneğin **Serial Device**'ı değiştirmek için **A**'ya bas. İmleç o satıra geçer:

Eski değeri silip `/dev/ttyUSB0` yaz ve **Enter**'a bas.

Baud rate'i değiştirmek için **E**'ye bas. Hız ve veri formatı ekranı açılır:




Bu ekranda da her seçeneğin bir harfi var. Örneğin **D**'ye basınca hız 38400 olur:




Biz BeagleBone Black için **115200** seçeceğiz. Bu ekrandaki harflerle hızı 115200, formatı **8N1** (8 bit, parity yok, 1 stop bit) yap ve Enter ile çık.

Ayarların son hali:

| Ayar | Değer |
|---|---|
| Serial Device | `/dev/ttyUSB0` |
| Bps/Par/Bits | `115200 8N1` |
| Hardware Flow Control | **No** |
| Software Flow Control | **No** |

> **Önemli**
>
> BeagleBone Black'in seri hattında **CTS** ve **RTS** pinleri yoktur. Bu yüzden **Hardware Flow Control** kullanılamaz ve **No** olmalıdır. Yes kalırsa ekranda kartın mesajlarını görürsün ama klavyeden yazdıkların karta gitmez. **Software Flow Control** da **No** kalsın.

Ayarları bitirince ana menüye dön ve **Save setup as dfl**'yi seç. Böylece ayarlar varsayılan olarak kaydedilir ve her seferinde yeniden yapman gerekmez. Sonra **Exit** ile ayar ekranından çık:

## 2.5 Minicom'u Kullanmak

Ayarları kaydettikten sonra Minicom'u doğrudan açabilirsin. Ayarları komut satırında açıkça vermek de mümkün:

```
minicom                                 # kayıtlı ayarlarla aç
minicom -D /dev/ttyUSB0 -b 115200       # portu ve hızı açıkça ver
```

Minicom'da bütün kısayollar önce `Ctrl-A`, ardından bir harf ile kullanılır:

| Kısayol | Görevi |
|---|---|
| `Ctrl-A` → `Z` | Yardım: bütün kısayolların listesi |
| `Ctrl-A` → `S` | Dosya gönder (xmodem / ymodem). Bölüm 6 ve Bölüm 7'de kullanılır. |
| `Ctrl-A` → `P` | Baud rate'i değiştir |
| `Ctrl-A` → `O` | Ayar menüsü ( `minicom -s` ile aynı) |
| `Ctrl-A` → `C` | Ekranı temizle |
| `Ctrl-A` → `X` | Minicom'dan çık |

> **İpucu**
>
> Minicom'u kartı açmadan **önce** başlat. Böylece açılıştaki ilk mesajları (SPL, U-Boot) kaçırmazsın.

## 2.6 Windows'ta: Tera Term

Kursun bazı derslerinde Windows'ta **Tera Term** kullanılıyor. Bu bir zorunluluk değil, Tera Term de Minicom gibi bir seri terminal programıdır. Hangisini kullanırsan kullan, komutlar ve çıktılar aynıdır.

| | **Linux** | **Windows** |
|---|---|---|
| Program | `minicom` | Tera Term |
| Port adı | `/dev/ttyUSB0` | `COM3` gibi (Aygıt Yöneticisi → Bağlantı Noktaları'na bak) |
| Ayarlar | 115200, 8N1, flow control yok | Setup → Serial port: 115200, 8 bit, none, 1 stop, Flow control: none |

## 2.7 Özet

| **Ne?** | **Komut / Değer** |
|---|---|
| Kurulum | `sudo apt install minicom lrzsz` |
| Port adını bulmak | `sudo dmesg \| grep tty` → `/dev/ttyUSB0` |
| Ayarlar | `sudo minicom -s` → 115200 8N1, Hardware/Software Flow Control: **No** → Save setup as dfl |
| Açmak | `minicom -D /dev/ttyUSB0 -b 115200` |
| Çıkmak | `Ctrl-A` → `X` |

## 2.8 Karşılaşılabilecek Problemler

| **Belirti** | **Sebep** | **Çözüm** |
|---|---|---|
| `cannot open /dev/ttyUSB0: Permission denied` | Kullanıcın `dialout` grubunda değil | `sudo usermod -aG dialout $USER`, sonra oturumu kapatıp aç |
| `dmesg`'de `ttyUSB` hiç görünmüyor | Kablo/sürücü sorunu ya da sanal makine USB'yi görmüyor | Başka USB portu dene. Sanal makinedeysen USB aygıtını VM'e bağla (VirtualBox: Devices → USB). |
| Ekranda hiçbir şey yok | TX/RX düz bağlanmış ya da GND bağlı değil | Dönüştürücünün TX'i → kartın RX'i (J1 pin 4), RX'i → kartın TX'i (J1 pin 5), GND → pin 1 (1.8) |
| Anlamsız / bozuk karakterler | Baud rate yanlış | 115200 8N1 olmalı |

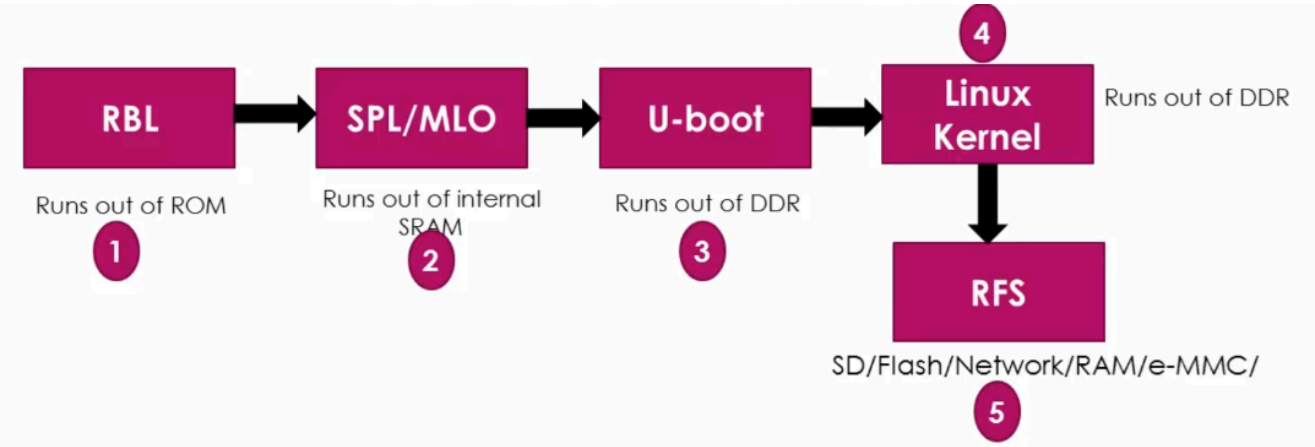
| Belirti | Sebep | Çözüm |
| --- | --- | --- |
| Mesajlar geliyor ama yazdıklarım gitmiyor | Hardware Flow Control açık | `minicom -s` → Serial port setup → Hardware Flow Control: **No** |
| `sx: not found` (dosya gönderirken) | `lrzsz` kurulu değil | `sudo apt install lrzsz` |

# Kısım II
# Boot Süreci

# Bölüm 3 — Linux Boot Süreci: RBL → SPL → U-Boot

> **Bu bölümde:**
>
> - Kart açılırken çalışan programları (RBL, SPL/MLO, U-Boot, kernel) ve her birinin görevini öğreneceksin.
> - Neden tek bir bootloader yerine bir **zincir** olduğunu anlayacaksın.
> - SYSBOOT pinlerinin boot sırasını nasıl belirlediğini ve S2 butonunun bunu nasıl değiştirdiğini göreceksin.
>
> **Ön koşul:** Bölüm 1 — kartın bellekleri (SRAM, DDR, eMMC, SD) ve S2 butonu.

---




BeagleBone Black'e güç verildiğinde Linux tek adımda açılmaz. Açılış, birbirini yükleyen bir **zincir** şeklinde ilerler. Her aşama bir sonrakini belleğe yükler ve kontrolü ona devreder:

```
Güç → RBL (ROM) → SPL / MLO (SRAM) → U-Boot (DDR) → Linux kernel (DDR) → Root filesystem → init
```

Şemadaki **"Runs out of ..."** ifadeleri her aşamanın **hangi bellekte çalıştığını** gösterir. Zincirin neden bu kadar uzun olduğu da buradan anlaşılır.

> **Not**
>
> **Neden tek bir bootloader yok?** Açılışın ilk anında 512 MB'lık **DDR RAM henüz kullanılamaz**. DDR'ın çalışması için önce saat (clock), zamanlama ve voltaj ayarlarının yapılması gerekir ve bu ayarlar karttan karta değişir. İşlemcinin hemen kullanabildiği tek RAM, SoC'nin içindeki **çok küçük dahili SRAM**'dir. Bu yüzden önce SRAM'e sığacak kadar küçük bir program (SPL) çalışır, DDR'ı hazırlar, ardından büyük programlar (U-Boot, Kernel) DDR'a yüklenebilir.

---

## 3.1 RBL (ROM Boot Loader)

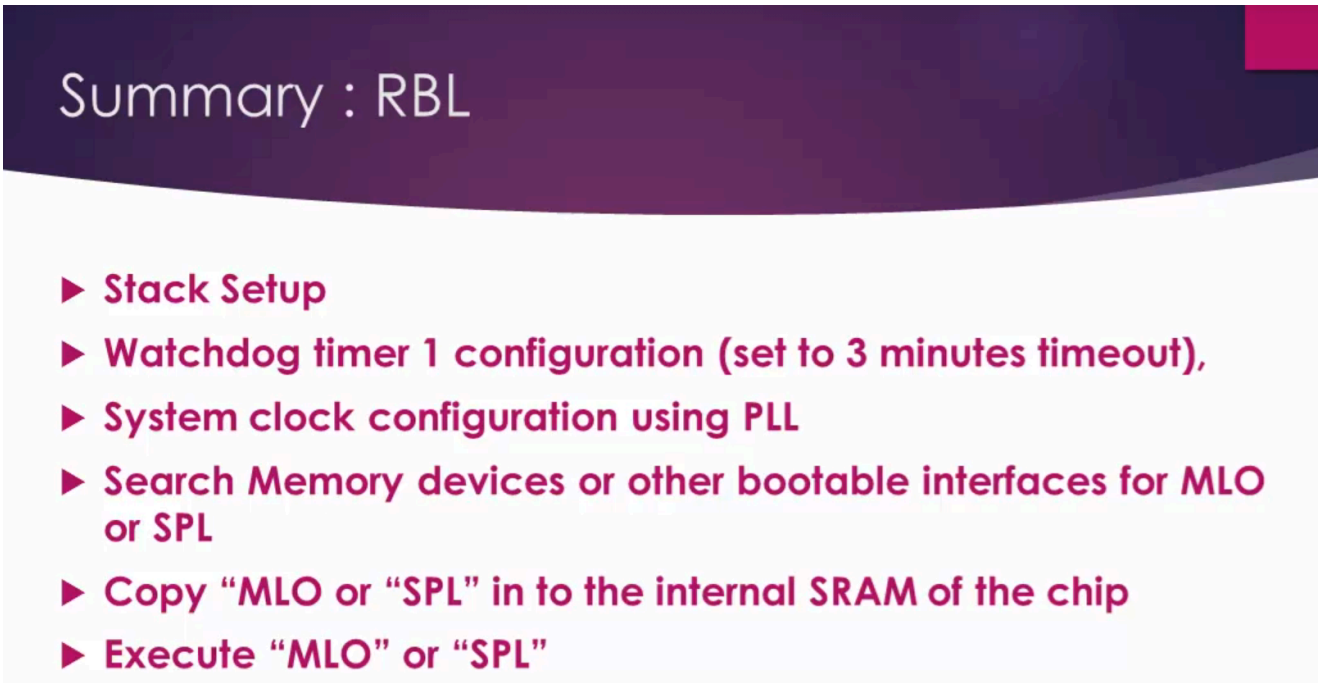
- Karta enerji verildiğinde SoC (System on Chip) üzerinde çalışan ***ilk kod parçasıdır***.
- Üretici firma (Texas Instruments) tarafından yazılır ve çip üretilirken SoC'nin içindeki **ROM** belleğe kaydedilir. ROM'da olduğu için **değiştirilemez**.
- **Nerede çalışır:** Doğrudan **ROM** üzerinden (Runs out of ROM).

**Görevleri:**

1. İşlemcinin temel donanımını (saat, watchdog vb.) minimum seviyede başlatır.
2. **SYSBOOT** pinlerinin seviyesini okuyarak **boot sırasını** belirler. Beaglebone'da bu sırayı **S2 (Boot)** butonu değiştirir: - S2 basılı değilse: **eMMC** → microSD → UART → USB - S2 basılıysa: SPI → **microSD** → USB → UART
3. Bu sırayla cihazları tek tek dener ve ilk bulduğu geçerli **ikinci aşama bootloader**'ı (**MLO**) dahili SRAM'e kopyalar.
4. Kontrolü MLO'ya devreder.

> **Not**
>
> SD karttan boot ederken RBL, kartın **FAT formatlı ilk bölümünde** (boot partition) `MLO` adlı dosyayı arar. Bu yüzden kendi boot SD kartımızı hazırlarken MLO dosyasını bu bölüme koyacağız (Bölüm 4).

### RBL'in yaptığı işler

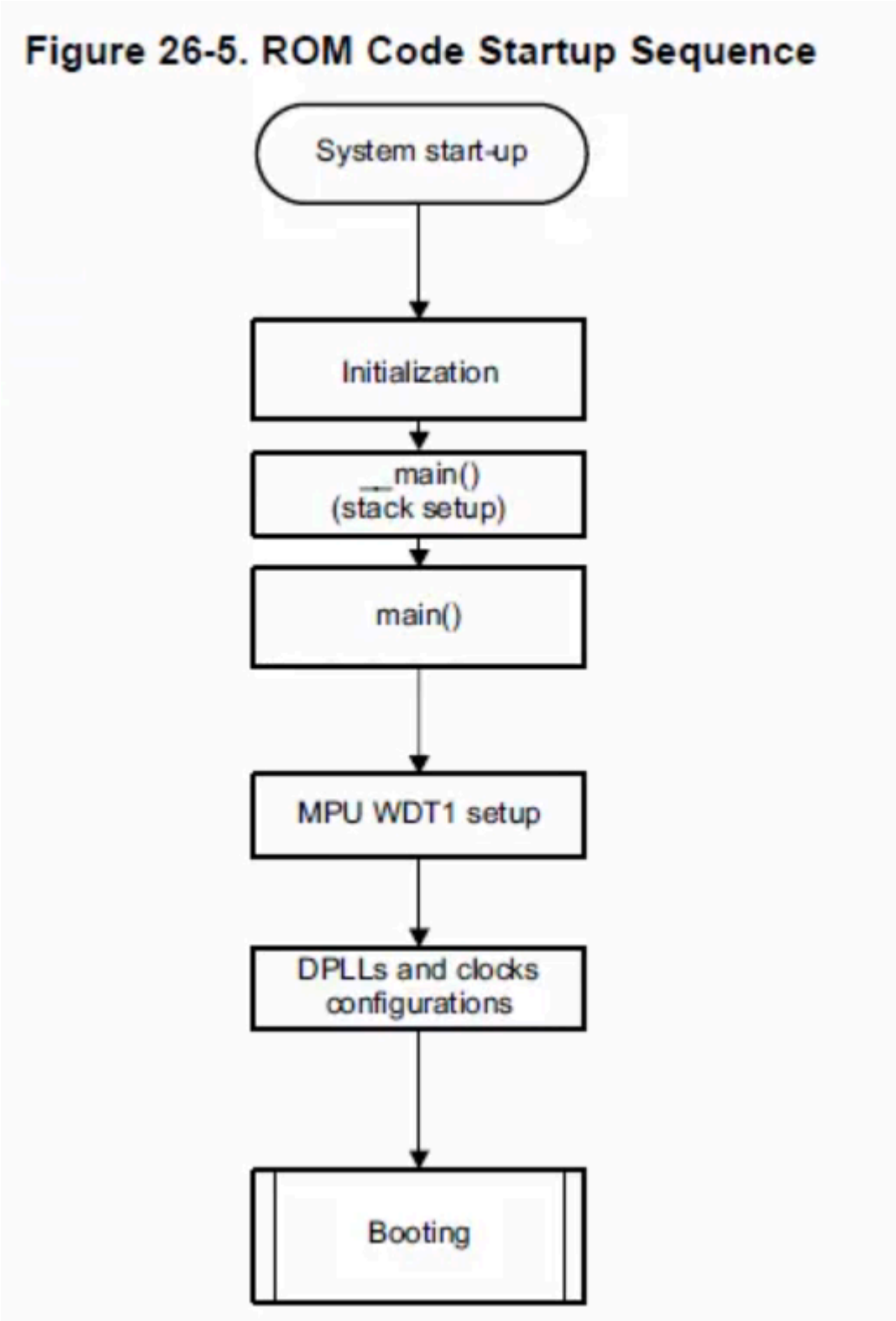



- **Stack Setup:** RBL'in C ile yazılmış fonksiyonları çalıştırabilmesi için önce bir **stack** (yığın) alanı gerekir. Fonksiyon çağrıları ve yerel değişkenler burada tutulur. RBL stack'i dahili SRAM'de kurar.
- **Watchdog Timer 1 (3 dakika):** Watchdog, belirli bir süre içinde "beslenmezse" (sıfırlanmazsa) sistemi **resetleyen** bir sayaçtır. Amacı, yazılım bir yerde takılırsa kartı kendiliğinden yeniden başlatmaktır. RBL bu sayacı **3 dakikaya** kurar ve açık bırakır. Bu yüzden sonraki aşamalar (SPL, U-

Boot, Linux) bu watchdog'u ya kapatmalı ya da düzenli olarak beslemelidir. Yoksa kart 3 dakika sonra resetlenir.

- **System clock configuration using PLL:** İşlemcinin ve çevre birimlerinin saatlerini PLL ile kurar (aşağıda açıklanıyor).
- **MLO/SPL'i arar, SRAM'e kopyalar ve çalıştırır:** Yukarıdaki görev listesinin 2-4. maddeleri.

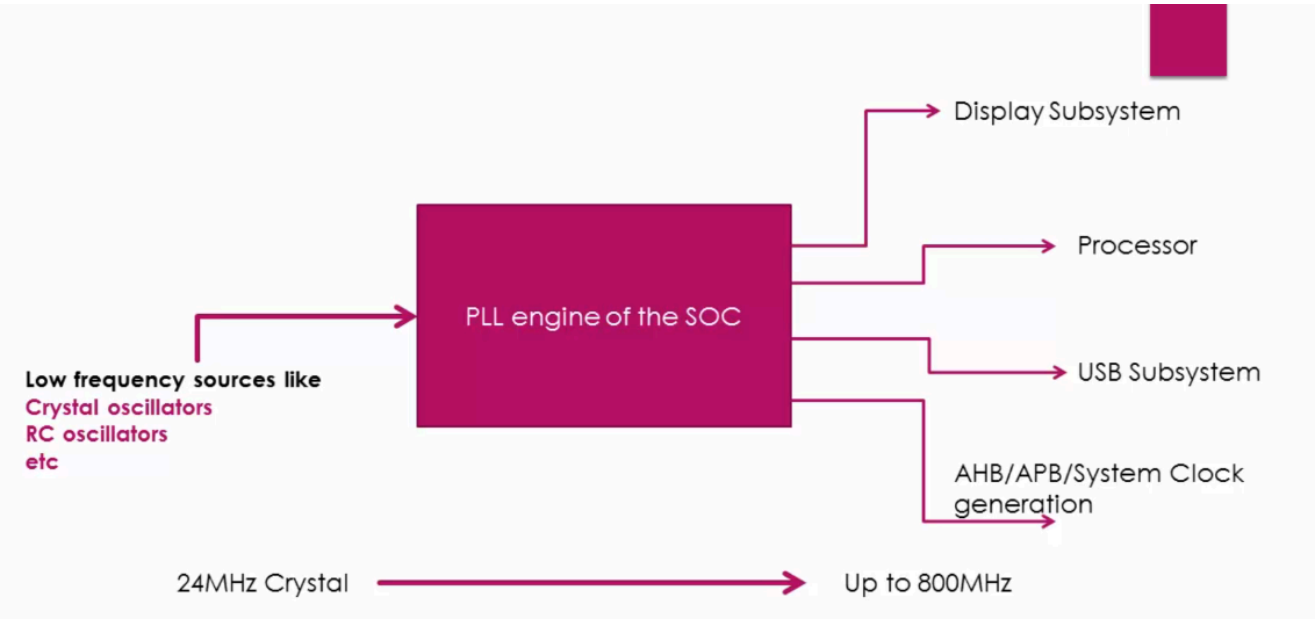
### ROM Code Startup Sequence




Bu akış şeması AM335x TRM'den (Figure 26-5) alınmıştır ve RBL'in açılışta yaptığı işleri sırasıyla gösterir:

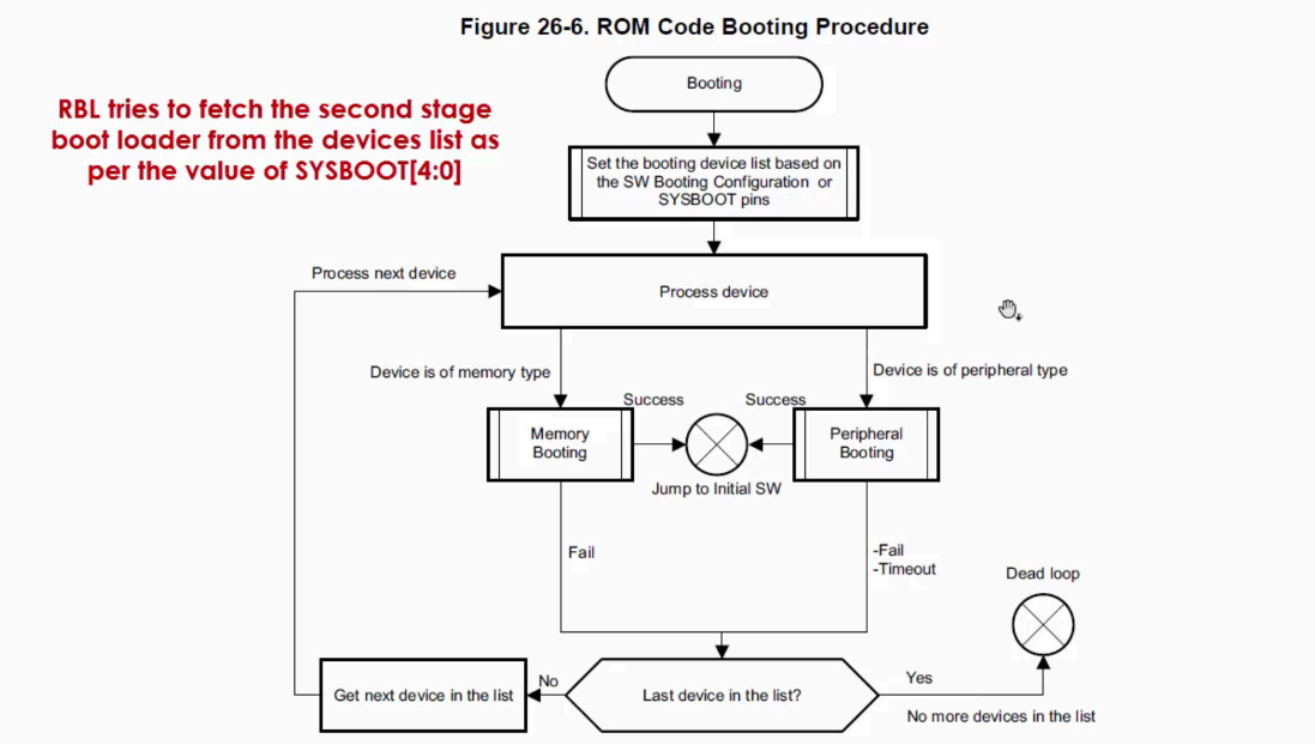
1. **System start-up:** Karta güç verilir veya reset gelir.
2. **Initialization:** İşlemcinin en temel ayarları yapılır.
3. **__main() (stack setup):** Stack kurulur. Bundan sonra C kodu çalışabilir.
4. **main():** RBL'in ana fonksiyonu başlar.
5. **MPU WDT1 setup:** Watchdog Timer 1 kurulur (3 dakika).
6. **DPLLs and clocks configuration:** PLL'ler ve saatler ayarlanır.
7. **Booting:** Boot cihazları aranır ve MLO yüklenir. Bu adımın detayı aşağıdaki **ROM Code Booting Procedure** şemasındadır.

## PLL (Phase Locked Loop) nedir?




- Kart üzerindeki kristal osilatör **düşük frekanslıdır** (BBB'de **24 MHz**). Ama işlemci yüzlerce MHz ile çalışmak ister.
- **PLL**, SoC içindeki bir devredir. Kristalden gelen düşük frekansı **çarparak** (ve gerektiğinde bölerek) yüksek frekanslar üretir. Şemadaki örnekte 24 MHz'lik kristalden **800 MHz**'e kadar frekans elde edilir.
- Her birim farklı bir frekansa ihtiyaç duyar. Bu yüzden SoC'de birden fazla PLL vardır ve her biri farklı bir birimi besler: **işlemci**, **USB**, **ekran (display)**, **bus saatleri** (AHB/APB) gibi.
- Doğrudan yüksek frekanslı bir kristal kullanılmamasının sebebi: Düşük frekanslı kristaller ucuz ve kararlıdır. Tek bir kristalden PLL'ler ile ihtiyaç duyulan tüm frekansları üretmek çok daha pratiktir.

## ROM Code Booting Procedure




Bu şema (TRM Figure 26-6), RBL'in ikinci aşama bootloader'ı (MLO) nasıl aradığını gösterir:

1. **Boot cihaz listesi oluşturulur:** RBL, **SYSBOOT[4:0]** değerine bakarak hangi cihazları hangi sırayla deneyeceğini belirler (3.5).
2. **Process device:** Listedeki sıradaki cihaz denenir. Cihazlar iki türe ayrılır: - **Memory type (bellek türü):** eMMC, SD kart, NAND, SPI flash gibi. MLO bu belleğin içinde **hazır durur**, RBL sadece okur. - **Peripheral type (çevre birimi türü):** UART, USB, Ethernet gibi. MLO kartta yoktur, **dışarıdan** (bir bilgisayardan) gönderilmesi beklenir. Bu yüzden burada **timeout** (zaman aşımı) vardır: Belirli süre içinde imaj gelmezse başarısız sayılır.
3. **Success:** MLO bulunduysa **"Jump to Initial SW"**, yani MLO'ya atlanır ve kontrol ona geçer.
4. **Fail / Timeout:** Listede başka cihaz varsa **bir sonrakine** geçilir.
5. **Dead loop:** Listedeki hiçbir cihazda MLO bulunamazsa RBL sonsuz bir döngüye girer ve kart hiçbir şey yapmaz.

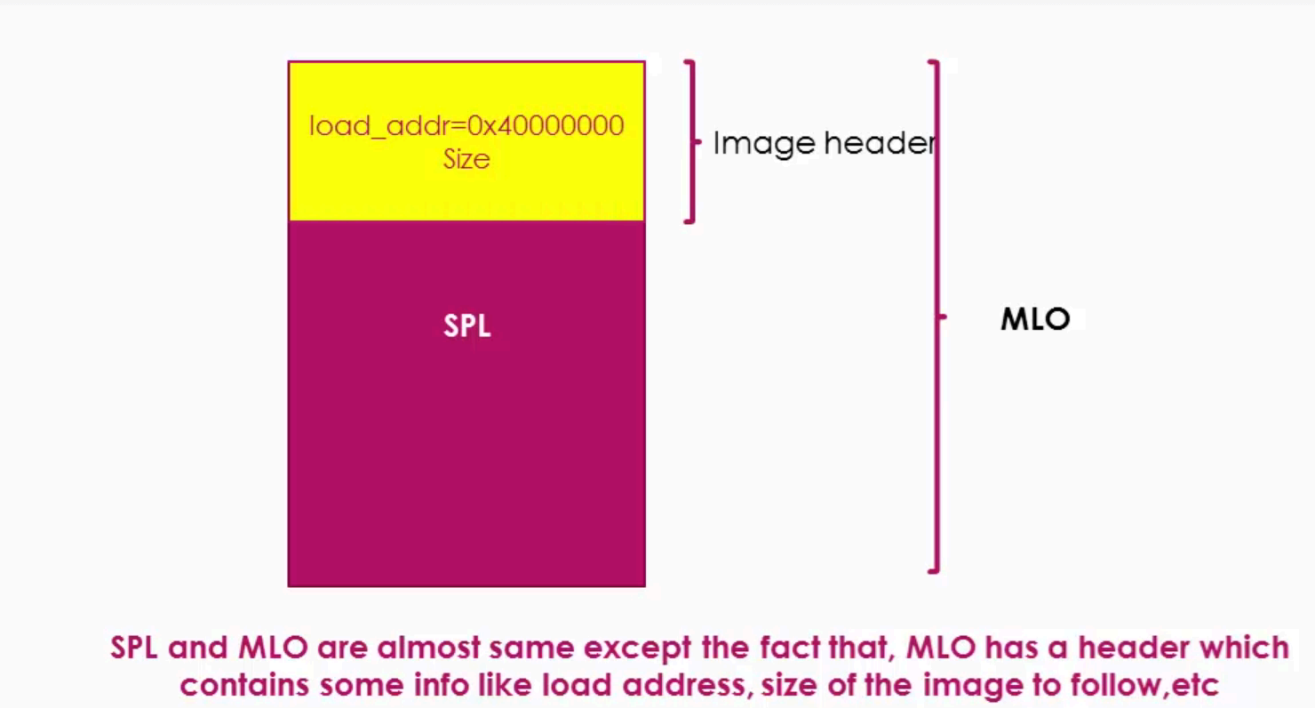
> **İpucu**
>
> BBB'nin varsayılan boot listesinde **UART0** da vardır. eMMC'de ve SD kartta geçerli bir MLO yoksa RBL sırayı UART0'a getirir ve seri konsoldan X-Modem ile imaj bekler. Bu sırada seri konsolda sürekli `CCCCCCCC...` yazdığını görürsün. Bu çıktı, **"RBL MLO bulamadı"** anlamına gelir. Bu davranışı Bölüm 7'de kartı seri porttan açmak için kullanacağız.

---

## 3.2 SPL / MLO (Secondary Program Loader)

- **SPL** = Secondary Program Loader (İkinci Aşama Önyükleyici). TI işlemcilerinde bu dosyanın adı **MLO**'dur (MLO = MMC Loader). SPL ile MLO **neredeyse** aynı şeydir (farkı aşağıda).
- SPL ayrı bir program değildir. **U-Boot'un kaynak kodundan derlenir**, ama sadece gerekli kısımları içeren çok küçük bir versiyonudur. U-Boot'u derlediğimizde hem `MLO` hem de `u-boot.img` dosyaları birlikte oluşur (Bölüm 9).
- **Nerede çalışır:** SoC içindeki **dahili SRAM** üzerinde (Runs out of internal SRAM). SRAM çok küçük olduğu için SPL de küçük olmak zorundadır.

### SPL ile MLO arasındaki fark




- **MLO = Image header + SPL**. Yani MLO, başına küçük bir **başlık (header)** eklenmiş SPL'dir.
- Header'da RBL'in ihtiyaç duyduğu bilgiler bulunur:
- **load_addr:** SPL'in belleğin **hangi adresine** kopyalanacağı
- **Size:** Header'dan sonra gelen SPL kodunun **boyutu**
- RBL önce header'ı okur. Böylece SPL'i nereye kopyalayacağını ve ne kadar veri okuyacağını öğrenir, kopyalamayı yapar ve o adrese atlar.
- U-Boot derlendiğinde önce header'sız SPL ( `u-boot-spl.bin` ) oluşur. Başına header eklenince `MLO` dosyası elde edilir.

> **Not**
>
> Slayttaki `0x40000000` adresi bir örnektir. AM335x'te MLO gerçekte dahili SRAM'deki `0x402F0400` adresine yüklenir.

**Görevleri:**

1. **DDR RAM'i başlatır.** En kritik görevi budur. Saat (PLL) ve DDR zamanlama ayarlarını yapar, böylece 512 MB'lık RAM kullanılabilir hale gelir.
2. Seri konsol (UART0) gibi gerekli birkaç çevre birimini hazırlar. Seri konsolda gördüğün ilk `U-Boot SPL ...` satırı buradan gelir.
3. Üçüncü aşama bootloader'ı, yani **U-Boot**'u (`u-boot.img`) eMMC/SD karttan okuyup **DDR**'a yükler.
4. Kontrolü U-Boot'a devreder.

### Soru: AM335x'in RBL'i neden U-Boot'u doğrudan DDR'a yükleyemez?

**Cevap:** Çünkü RBL, karta **hangi DDR belleğin takılı olduğunu bilmez**. Bilmediği bir belleği de başlatamaz.

- DDR RAM'den veri okuyup yazabilmek için önce SoC içindeki **DDR kontrolcüsünün** ve RAM'in **doğru parametrelerle** ayarlanması gerekir: bellek tipi (DDR2, DDR3, DDR3L...), boyut, hız, zamanlama (timing) değerleri vb.
- Bu parametreler **tamamen karta (ürüne) özeldir**. Aynı AM335x işlemcisini kullanan farklı firmalar farklı bellekler seçebilir:
- X firması: Micron marka **DDR3**
- Y firması: Başka bir markanın **DDR2**'si
- Z firması: Hiç DDR kullanmayabilir
- RBL çip üretilirken ROM'a yazılır. O an çipin hangi kartta, hangi bellekle kullanılacağı belli değildir. Bu yüzden RBL **DDR'ı hiç başlatmaz**. Sadece her işlemcide garanti olarak bulunan **dahili SRAM**'i kullanır.
- U-Boot ise dahili SRAM'e **sığmayacak kadar büyüktür**. Yani RBL'in U-Boot'u yükleyebileceği bir yer yoktur.

**Çözüm SPL'dir:** SPL, dahili SRAM'e sığacak kadar küçüktür ve kartın DDR ayarlarını **bilir**, çünkü o kart için derlenmiştir. RBL, SPL'i SRAM'e yükler. SPL de DDR'ı başlatıp U-Boot'u DDR'a yükler.

> **Not**
>
> Kendi kartını tasarlayıp BBB'den farklı bir DDR kullanırsan, SPL kaynak kodundaki **DDR parametrelerini** kendi belleğine göre değiştirip SPL'i yeniden derlemen gerekir.

> **Not**
>
> **Saat (clock) ayarları:** RBL'in kodu ROM'da olduğu için saat ayarları **değiştirilemez**. SYSBOOT[15:14] ile sadece karttaki kristalin frekansı bildirilir. İşlemci (MPU), DDR ve çevre birimi frekanslarını asıl kuran **SPL**'dir. SPL'i kendimiz derlediğimiz için bu ayarları kaynak kodda değiştirip yeniden derleyebiliriz. Açılıştan sonra Linux da **cpufreq** ile işlemci frekansını çalışma sırasında değiştirebilir.

---

## 3.3 U-Boot (Universal Bootloader)

- Açık kaynak kodlu ve gömülü sistemlerde en yaygın kullanılan bootloader'dır. Üçüncü (ve son) aşama önyükleyicidir.
- **Nerede çalışır: DDR RAM** üzerinde (Runs out of DDR). Artık bellek kısıtı olmadığı için çok daha fazla özellik içerebilir.

**Görevleri:**

1. Linux Kernel'i çalıştırmak için gereken donanımları (eMMC, SD kart, Ethernet, USB vb.) başlatır.
2. **Linux Kernel** imajını ( `zImage` / `uImage` ) bir kaynaktan (SD kart, eMMC, seri port, ağ üzerinden TFTP vb.) okuyup **DDR**'a yükler.
3. Kartın donanımını tarif eden **Device Tree** dosyasını (BeagleBone Black için `am335x-boneblack.dtb` ) DDR'a yükler.
4. Kernel'e **bootargs** (komut satırı parametreleri) iletir. Örneğin root filesystem'in nerede olduğu ( `root=` ) veya hangi konsolun kullanılacağı ( `console=` ) bu parametrelerle bildirilir.
5. Kontrolü Linux Kernel'e devreder. Bu noktadan sonra U-Boot'un işi biter.

**Neden önemli:**

- U-Boot'un kendi **komut satırı** vardır. Açılışta seri konsolda geri sayım ( `Hit any key to stop autoboot` ) sırasında bir tuşa basarsan U-Boot komut satırına ( `U-Boot#` veya `=>` ) düşersin. Buradan bellek okuma/yazma, dosya yükleme, boot ayarlarını değiştirme gibi işlemler yapılabilir (Bölüm 6).
- Boot ayarları **environment değişkenlerinde** ( `bootcmd` , `bootargs` vb.) tutulur ve `printenv` / `setenv` komutlarıyla görüntülenip değiştirilebilir.

---

## 3.4 Linux Kernel ve Root Filesystem

U-Boot `bootm` (veya `bootz` ) komutuyla kernel'e atladıktan sonra zincirin son iki halkası çalışır:

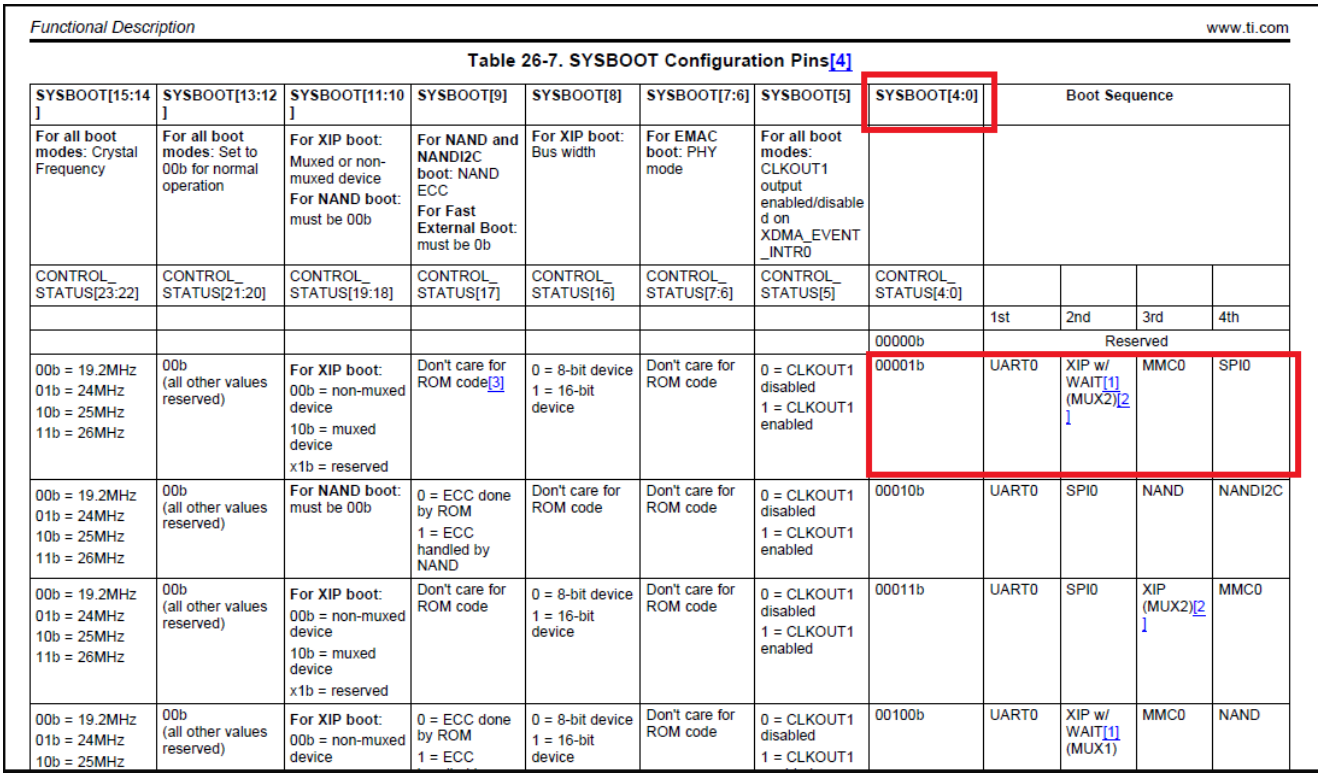
1. **Kernel** kendini RAM'de açar ( `zImage` sıkıştırılmıştır), U-Boot'un verdiği **Device Tree**'yi okuyup kartta hangi donanımın olduğunu öğrenir ve sürücüleri başlatır (Bölüm 11).
2. Kernel, `bootargs` 'taki **`root=`** parametresine bakarak **root filesystem**'i ( `/` ) bağlar. Bu bir SD kart bölümü ( `/dev/mmcblk0p2` ), eMMC bölümü veya RAM'deki bir initramfs olabilir.
3. Rootfs'teki ilk program olan **`init`** 'i (systemd veya BusyBox init) çalıştırır. Bundan sonra servisler başlar ve `login:` ekranı gelir.

> **Not**
>
> `console=` parametresindeki konsol adı kernel sürümüne göre değişir. Kursun hazır kernel'inde (3.8.13) UART0'ın adı `ttyO0` , yeni kernel'lerde `ttyS0` 'dır. Yanlış ad yazılırsa kernel açılır ama mesajlarını göremezsin.

---

## 3.5 Boot Kaynakları ve Boot Sırası (SYSBOOT Pinleri)

AM335x işlemcisi tek bir yerden boot etmek zorunda değildir. **Birçok farklı çevre biriminden** (eMMC, SD kart, UART, USB, SPI flash, NAND, Ethernet...) boot edebilir. Peki RBL hangisinden boot edeceğine nasıl karar verir? Bunu işlemcinin **SYSBOOT** pinleri belirler.



*Functional Description* www.ti.com

**Table 26-7. SYSBOOT Configuration Pins[4]**

| SYSBOOT[15:14] | SYSBOOT[13:12] | SYSBOOT[11:10] | SYSBOOT[9] | SYSBOOT[8] | SYSBOOT[7:6] | SYSBOOT[5] | SYSBOOT[4:0] | Boot Sequence | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| For all boot modes: Crystal Frequency | For all boot modes: Set to 00b for normal operation | For XIP boot: Muxed or non-muxed device<br>For NAND boot: must be 00b | For NAND and NANDI2C boot: NAND ECC<br>For Fast External Boot: must be 0b | For XIP boot: Bus width | For EMAC boot: PHY mode | For all boot modes: CLKOUT1 output enabled/disabled on XDMA_EVENT_INTR0 | | | | | |
| CONTROL_STATUS[23:22] | CONTROL_STATUS[21:20] | CONTROL_STATUS[19:18] | CONTROL_STATUS[17] | CONTROL_STATUS[16] | CONTROL_STATUS[7:6] | CONTROL_STATUS[5] | CONTROL_STATUS[4:0] | | | | |
| | | | | | | | | 1st | 2nd | 3rd | 4th |
| | | | | | | | 00000b | Reserved | | | |
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz<br>11b = 26MHz | 00b (all other values reserved) | For XIP boot:<br>00b = non-muxed device<br>10b = muxed device<br>x1b = reserved | Don't care for ROM code[3] | 0 = 8-bit device<br>1 = 16-bit device | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 enabled | 00001b | UART0 | XIP w/ WAIT[1] (MUX2)[2] | MMC0 | SPI0 |
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz<br>11b = 26MHz | 00b (all other values reserved) | For NAND boot: must be 00b | 0 = ECC done by ROM<br>1 = ECC handled by NAND | Don't care for ROM code | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 enabled | 00010b | UART0 | SPI0 | NAND | NANDI2C |
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz<br>11b = 26MHz | 00b (all other values reserved) | For XIP boot:<br>00b = non-muxed device<br>10b = muxed device<br>x1b = reserved | Don't care for ROM code | 0 = 8-bit device<br>1 = 16-bit device | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 enabled | 00011b | UART0 | SPI0 | XIP (MUX2)[2] | MMC0 |
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz | 00b (all other values reserved) | For XIP boot:<br>00b = non-muxed device | 0 = ECC done by ROM<br>1 = ECC | 0 = 8-bit device<br>1 = 16-bit device | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 | 00100b | UART0 | XIP w/ WAIT[1] (MUX1) | MMC0 | NAND |

Bu tablo, AM335x'in **Technical Reference Manual (TRM)** dokümanındaki **Table 26-7: SYSBOOT Configuration Pins** tablosudur.

### SYSBOOT pinleri nedir?

- **SYSBOOT[15:0]**, işlemcinin 16 adet pinidir. Karta **güç verildiği anda** RBL bu pinlerin seviyesini (1 veya 0) okur.
- Okunan değer, **CONTROL_STATUS** adlı bir register'a kaydedilir. Tablonun 3. satırındaki `CONTROL_STATUS[...]` ifadeleri, her SYSBOOT bitinin bu register'da hangi bite karşılık geldiğini gösterir.
- Bu pinlerin seviyesi kart üzerindeki **pull-up / pull-down dirençleriyle** sabitlenir. Yani boot ayarı, kartı tasarlayan kişi tarafından donanımsal olarak belirlenir.

### Tablonun sütunları

| Sütun | Ne işe yarar? |
|---|---|
| **SYSBOOT[15:14]** | Karttaki kristalin (osilatörün) frekansını bildirir: 19.2 / 24 / 25 / 26 MHz. Beaglebone Black **24 MHz** kristal kullanır (**01b**). |
| **SYSBOOT[13:12]** | Normal çalışma için **00b** olmalıdır. |

| Sütun | Ne işe yarar? |
|---|---|
| **SYSBOOT[11:10], [9], [8]** | Sadece belirli boot türlerinde (XIP, NAND) kullanılan ek ayarlar. Diğer boot türlerinde önemsizdir ("Don't care"). |
| **SYSBOOT[7:6]** | Ethernet (EMAC) üzerinden boot için PHY modu. |
| **SYSBOOT[5]** | CLKOUT1 saat çıkışının açık/kapalı olması. |
| **SYSBOOT[4:0]** | **En önemli kısım.** Boot modunu, yani **hangi cihazların hangi sırayla** deneneceğini belirler. |
| **Boot Sequence** | SYSBOOT[4:0] değerine göre RBL'in deneyeceği cihazlar: **1st → 2nd → 3rd → 4th**. |

## Boot Sequence (Boot sırası) nasıl çalışır?

Resimde kırmızıyla işaretlenen satırı örnek alalım. **SYSBOOT[4:0] = 00001b** ise boot sırası şöyledir:

```
1st: UART0  →  2nd: XIP (NOR flash)  →  3rd: MMC0 (SD kart)  →  4th: SPI0
```

1. RBL önce **1. cihazı** (UART0) dener.
2. O cihazda geçerli bir boot imajı (MLO) bulamazsa veya cihaz yanıt vermezse **bir sonraki cihaza** geçer.
3. Geçerli bir imaj bulduğu **ilk cihazdan** boot eder ve diğerlerine bakmaz.
4. Dört cihazın hiçbirinde imaj bulamazsa boot başarısız olur.

SYSBOOT[4:0] 5 bit olduğu için 32 farklı kombinasyon vardır. Her biri farklı bir boot sırası tanımlar. Resimde bunların sadece ilk birkaçı görünüyor, tablonun tamamı TRM'de.

## Tablodaki boot kaynakları

| İsim | Anlamı |
|---|---|
| **MMC0** | Beaglebone'da **microSD kart** yuvası |
| **MMC1** | Beaglebone'da kart üstündeki **eMMC** |
| **UART0** | Seri port üzerinden boot. İmaj bilgisayardan seri hat ile (X-Modem protokolü) gönderilir (Bölüm 7). |
| **USB0** | USB üzerinden boot. Kart, bilgisayara bir ağ cihazı gibi görünür ve imajı oradan alır. |
| **SPI0** | SPI ile bağlı harici flash bellekten boot |
| **XIP** | e**X**ecute **I**n **P**lace: Kod, NOR flash'a kopyalanmadan doğrudan orada çalıştırılır. |
| **NAND / NANDI2C** | Harici NAND flash bellekten boot |
| **EMAC** | Ethernet üzerinden ağdan boot |

> **Not**
>
> UART0 ve USB0 boot modları, kartta hiç yazılım olmadığında (örneğin eMMC boşsa veya bozulduysa) kartı **kurtarmak** için kullanılır.

### BeagleBone Black'te durum

BeagleBone Black'in boot ayarları, resmi dokümandaki şu tabloda görünür:

| SYSBOOT[15:14] | SYSBOOT[13:12] | SYSBOOT[11:10] | SYSBOOT[9] | SYSBOOT[8] | SYSBOOT[7:6] | SYSBOOT[5] | SYSBOOT[4:0] | Boot Sequence | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz<br>11b = 26MHz | 00b<br>(all other values reserved) | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 enabled | 11100b | MMC1 | MMC0 | UART0 | USB0[5] |
| 00b = 19.2MHz<br>01b = 24MHz<br>10b = 25MHz<br>11b = 26MHz | 00b<br>(all other values reserved) | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | Don't care for ROM code | 0 = CLKOUT1 disabled<br>1 = CLKOUT1 enabled | 11000b | SPI0 | MMC0 | USB0[5] | UART0 |

| Durum | SYSBOOT[4:0] | Boot Sequence |
|---|---|---|
| **S2 basılı değil** (varsayılan) | **11100b** | **MMC1 (eMMC)** → MMC0 (SD) → UART0 → USB0 |
| **S2 basılı** (güç verilirken) | **11000b** | SPI0 → **MMC0 (SD)** → USB0 → UART0 |

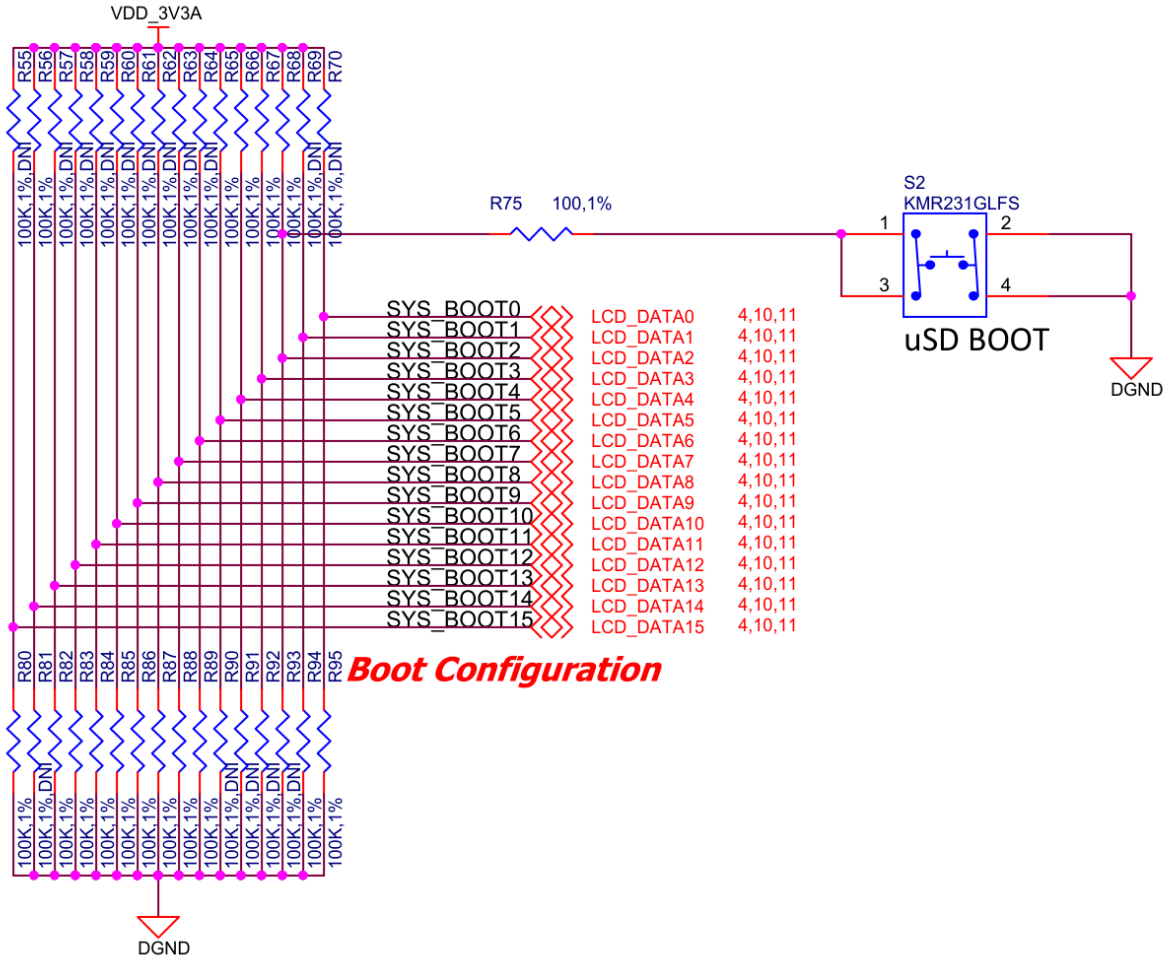
- İki değer arasında sadece **SYSBOOT[2]** biti farklıdır: 11**1**00 → 11**0**00. Boot (**S2**) butonu tam olarak bu biti **0**'a çeker.
- Varsayılan durumda eMMC ilk sırada olduğu için kart **eMMC'deki** hazır Debian'dan açılır.
- S2'ye basılıyken eMMC listede **hiç yoktur**. SPI0'da flash olmadığı için o adım atlanır ve kart **SD karttan** açılır. Kendi derlediğimiz MLO/U-Boot/Kernel'i bu yüzden SD karta koyup S2 ile boot edeceğiz.
- Beaglebone'da SYSBOOT pinleri **LCD veri pinleriyle** (LCD_DATA0–15) ortaktır ve P8 header'ına çıkar. Bu pinlere boot sırasında dışarıdan sinyal verilirse boot sırası bozulabilir. Bu yüzden kart açılmadan P8'e bir şey bağlarken dikkatli olmak gerekir.
- Listede **EMAC (Ethernet) yoktur**. ROM Code BBB'de ağdan boot edemez. Ağdan boot için önce SD karttan U-Boot'u açmak gerekir (Bölüm 8).

> **Önemli**
>
> SYSBOOT pinleri **sadece güç verildiği anda** okunur. Kart açıldıktan sonra S2'ye basmak veya Reset butonuna basmak boot sırasını değiştirmez. S2 ile boot etmek için gücü tamamen kesip, S2'yi basılı tutarak tekrar güç vermelisin.

---

## 3.6 Schematic'te Boot Devresi

Yukarıda anlatılan boot ayarının kart üzerindeki karşılığı, BBB schematic'inin **6. sayfasındaki** devredir.




> Kaynak: BeagleBone Black Rev C schematic, sayfa 6 (niekiran/EmbeddedLinuxBBB reposundaki `BBB_SCH.pdf` )

Schematic'te dikkat edilecekler:

- Her **SYS_BOOTx** hattına üstte bir pull-up (R55–R70), altta bir pull-down (R80–R95) direnci için yer var. Bunlardan sadece biri karta takılıdır. **DNI** (Do Not Install) yazan direnç kartta **yoktur**.
- Takılı dirençlere bakınca SYS_BOOT[4:0] = **11100b** çıkar. Bu değer, yukarıdaki tablodaki varsayılan değerle aynıdır.
- **S2** butonu (üzerinde **"uSD BOOT"** yazar), **R75** üzerinden **SYS_BOOT2** hattını **GND**'ye bağlar.

---

## 3.7 Özet

| Aşama | Kim yazar? | Nerede saklanır? | Nerede çalışır? | Ana görevi |
|---|---|---|---|---|
| **RBL** | TI (çip üreticisi) | SoC içindeki ROM | ROM | Boot cihazını seçip MLO'yu SRAM'e yüklemek |

| Aşama | Kim yazar? | Nerede saklanır? | Nerede çalışır? | Ana görevi |
|---|---|---|---|---|
| **SPL/MLO** | U-Boot projesi (biz derleriz) | eMMC / SD kart | Dahili SRAM | DDR'ı başlatıp U-Boot'u DDR'a yüklemek |
| **U-Boot** | U-Boot projesi (biz derleriz) | eMMC / SD kart | DDR | Kernel + DTB'yi yükleyip Kernel'i başlatmak |
| **Linux kernel** | Linux projesi (biz derleriz) | eMMC / SD kart / ağ | DDR | Donanımı başlatmak, rootfs'i bağlamak, `init`'i çalıştırmak |

- Boot sırasını **SYSBOOT[4:0]** pinleri belirler. BBB'de varsayılan **eMMC → SD → UART0 → USB0**, S2 basılıyken **SPI0 → SD → USB0 → UART0**.
- S2 sadece **güç verilirken** etkilidir.

# Bölüm 4 — SD Karttan Boot

> **Bu bölümde:**
>
> - Bir microSD kartı GParted ile BOOT (FAT) ve ROOTFS (ext3) bölümlerine ayıracaksın.
> - Kursun hazır dosyalarını karta **tek tek** koyup her seferinde kartı açarak boot zincirinin her halkasının çalıştığını göreceksin.
> - `uEnv.txt` 'nin bu açılışta ne yaptığını ve `uImage` dosyasının yapısını öğreneceksin.
> - U-Boot'u durdurup kernel'in başlığını (header) bellekten byte byte okuyacaksın.
>
> **Ön koşul:** Bölüm 2 — seri konsol, Bölüm 3 — boot zinciri.

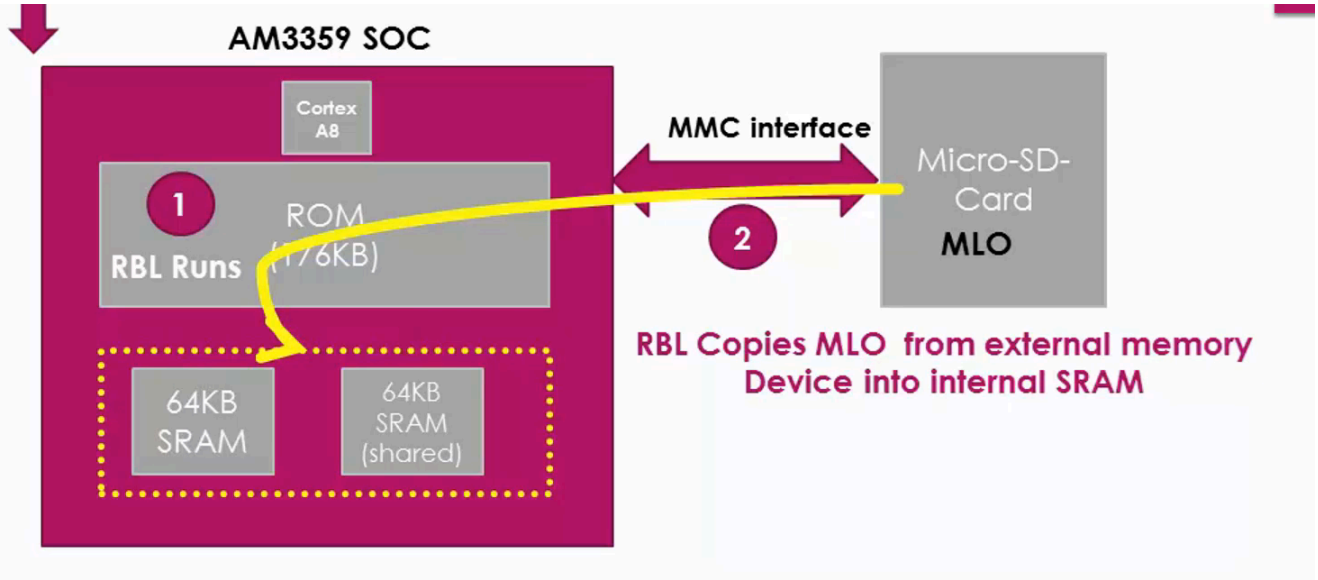
---

## 4.1 Büyük Resim

Kart açılırken programlar sırayla birbirini çalıştırır. Buna **boot zinciri** denir (Bölüm 3):

```
Güç → RBL → MLO → U-Boot → Linux Kernel → Root Filesystem → login ekranı
```

| Aşama | Ne yapar? | Nerede durur? |
|---|---|---|
| **RBL** (ROM Boot Loader) | Kart açılınca ilk çalışan kod. MLO'yu bulup çalıştırır. | İşlemcinin içinde, değiştirilemez |
| **MLO** | RAM'i (DDR) hazırlar, U-Boot'u yükler. | SD kart → BOOT bölümü |
| **U-Boot** | Kernel'i ve device tree'yi RAM'e yükleyip Linux'u başlatır. | SD kart → BOOT bölümü |
| **Linux Kernel** | İşletim sisteminin çekirdeği. | SD kart → ROOTFS bölümü ( `/boot` ) |
| **Root Filesystem** | Linux'un tüm dosyaları ( `/bin` , `/etc` , ...). | SD kart → ROOTFS bölümü |




**SD kartın son hali** şöyle olacak:

```
SD Kart
├── BOOT   (FAT16, ~1 GB)  →  MLO, u-boot.img, uEnv.txt
└── ROOTFS (ext3, kalan)   →  Linux dosyaları (kernel ve .dtb dahil)
```

### Yol haritası

Dosyalar karta **tek tek** eklenir ve her seferinde kart açılıp test edilir. Böylece zincirin her halkasının çalıştığı görülür:

| Adım | Ne yapılıyor? | Sonuç |
|---|---|---|
| 4.2 – 4.4 | Dosyaları indir, SD kartı bölümle | SD kart hazır |
| 4.6 | MLO'yu kopyala, test et | RBL ✓ → MLO ✓ → U-Boot ✗ |
| 4.7 | U-Boot'u kopyala, test et | … → U-Boot ✓ |
| 4.8 | Rootfs ve uEnv.txt'yi kopyala, test et | Linux açılır ✓ |
| 4.10 | U-Boot'u durdur, uImage header'ını elle oku | Header çözüldü ✓ |

Her testte kart aynı şekilde açılır. Bu işlem 4.5'te bir kez anlatılıyor.

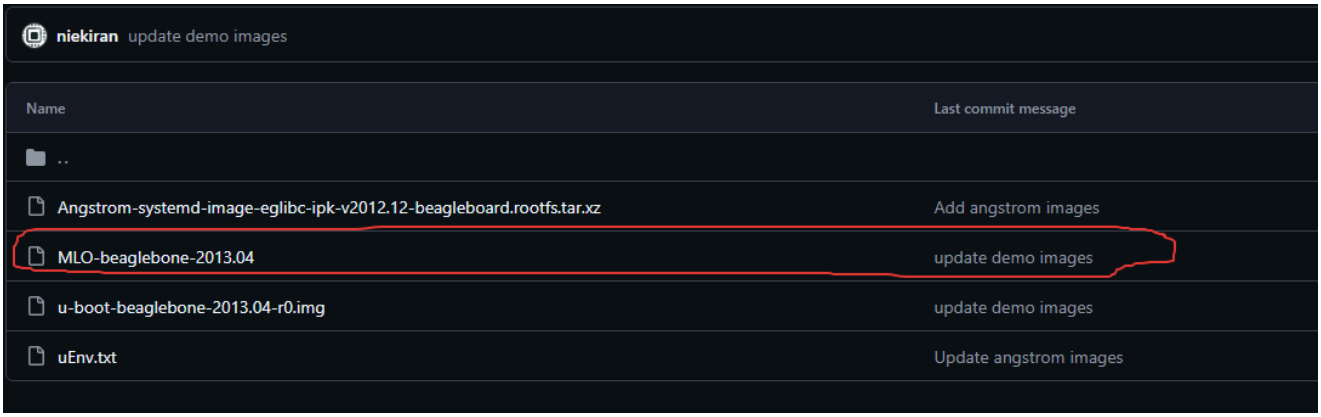
---

## 4.2 Hazırlık

**Gerekenler:**

- microSD kart (en az 4 GB) ve kart okuyucu
- USB-TTL seri dönüştürücü (3.3V). Kartın açılış mesajlarını görmek için gerekli.
- Mini USB kablo (karta güç vermek için)

**Dosyaları indir.** Kursun GitHub reposundaki **4 dosyanın hepsini** indir:
https://github.com/niekiran/EmbeddedLinuxBBB/tree/master/pre-built-images/Angstrom_Demo



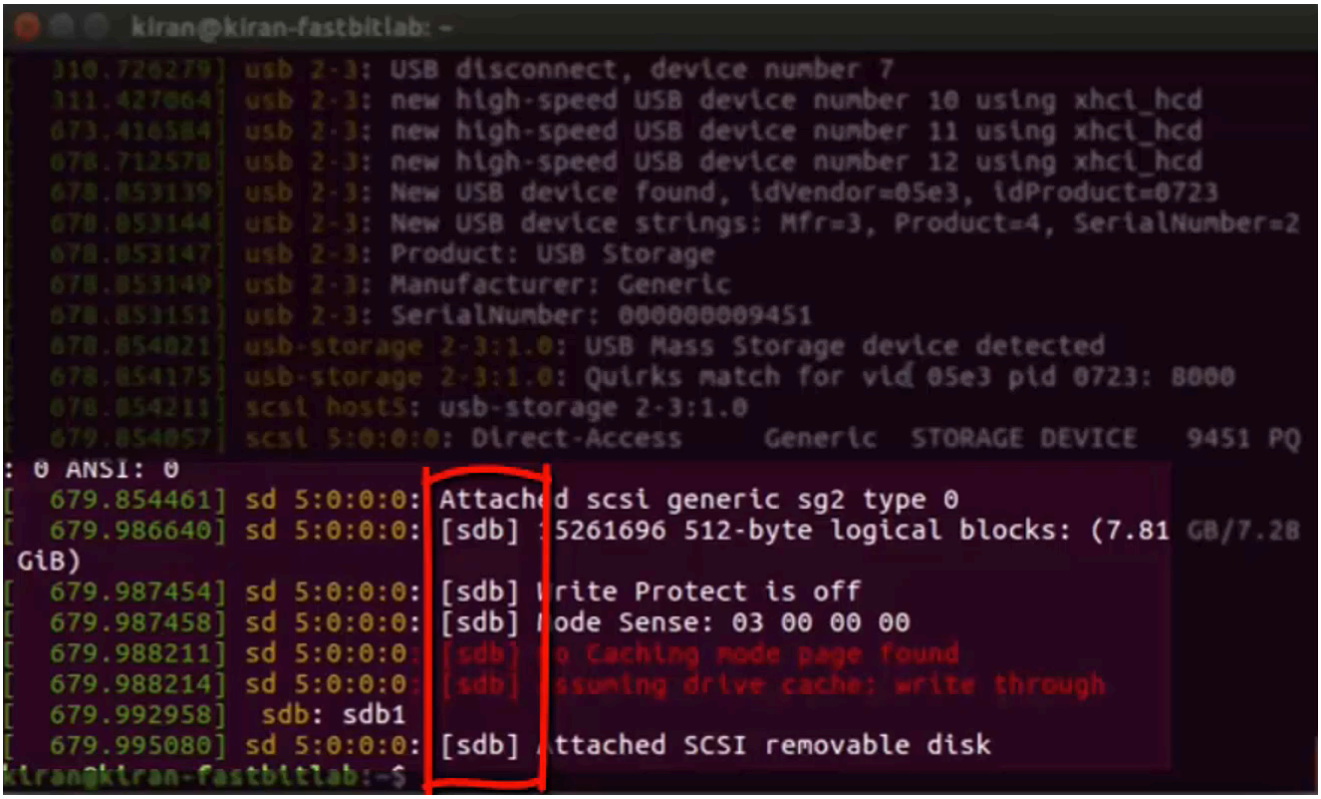

| İndirilen dosya | Ne? | Nereye kopyalanacak? |
|---|---|---|
| `MLO-beaglebone-2013.04` | MLO | BOOT → adı `MLO` olacak |
| `u-boot-beaglebone-2013.04-r0.img` | U-Boot | BOOT → adı `u-boot.img` olacak |
| `uEnv.txt` | U-Boot ayar dosyası | BOOT → adı aynı kalacak |
| `Angstrom-systemd-image-eglibc-ipk-v2012.12-beagleboard.rootfs.tar.xz` | Root filesystem (içinde kernel `/boot/uImage` ve device tree `/boot/am335x-boneblack.dtb` da var) | ROOTFS → arşivden çıkarılacak |

---

## 4.3 SD Kartı Bilgisayarda Bulma

Kartı takıp terminalde çalıştır:

```
sudo dmesg
```




- `sdb` → SD kartın adı ( `/dev/sdb` ). Yanında boyutu yazar (7.81 GB). Doğru cihaz olduğunu boyuttan anlarsın.
- `sdb1` → kartta zaten var olan bölüm (partition).

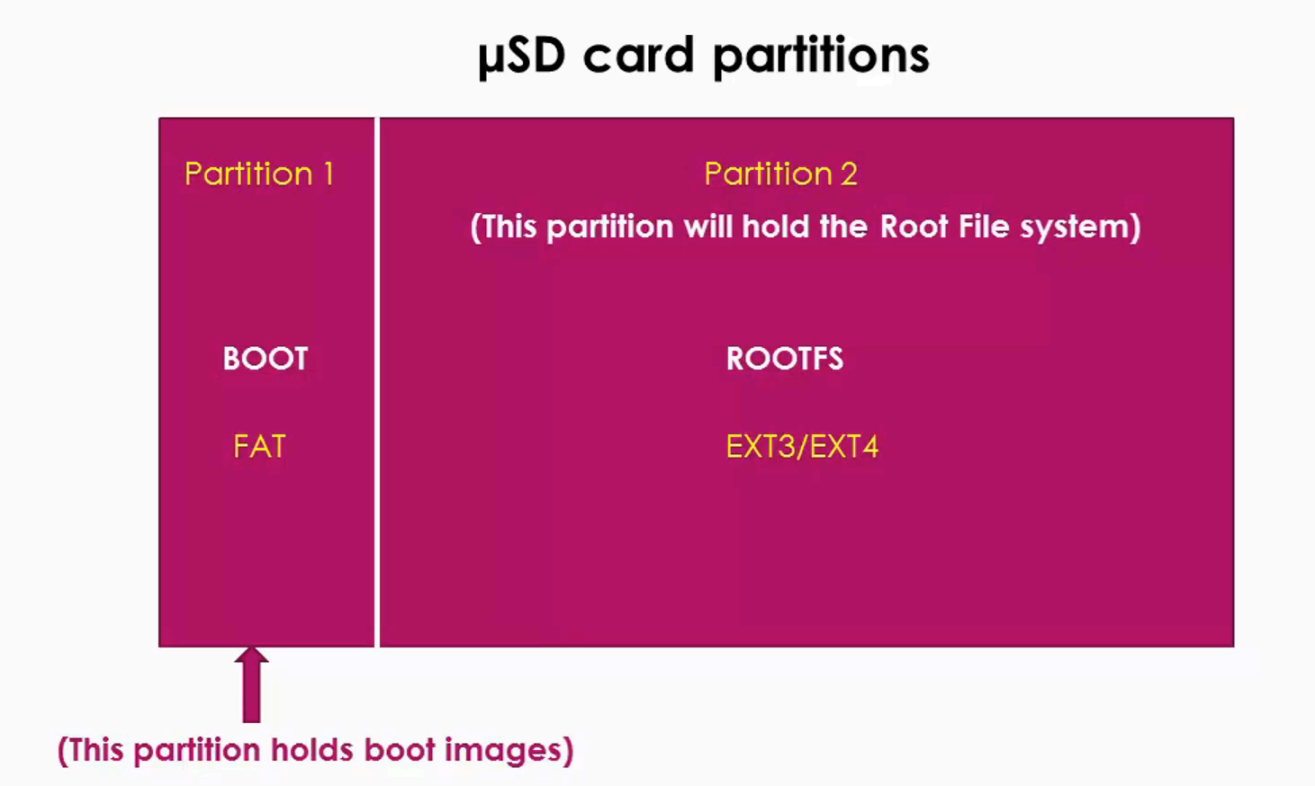
> **İpucu**
>
> `lsblk` komutu tüm diskleri boyutlarıyla birlikte daha okunaklı gösterir.

> **Not**
>
> GParted'da aynı kart **7.28 GiB** görünür. Kart eksik değil, birimler farklı: 1 GB = 1.000.000.000 byte, 1 GiB = 1.073.741.824 byte.

---

## 4.4 SD Kartı Bölümlere Ayırma (GParted)

Kart 2 bölüme ayrılacak. **Karttaki tüm veriler silinir.**

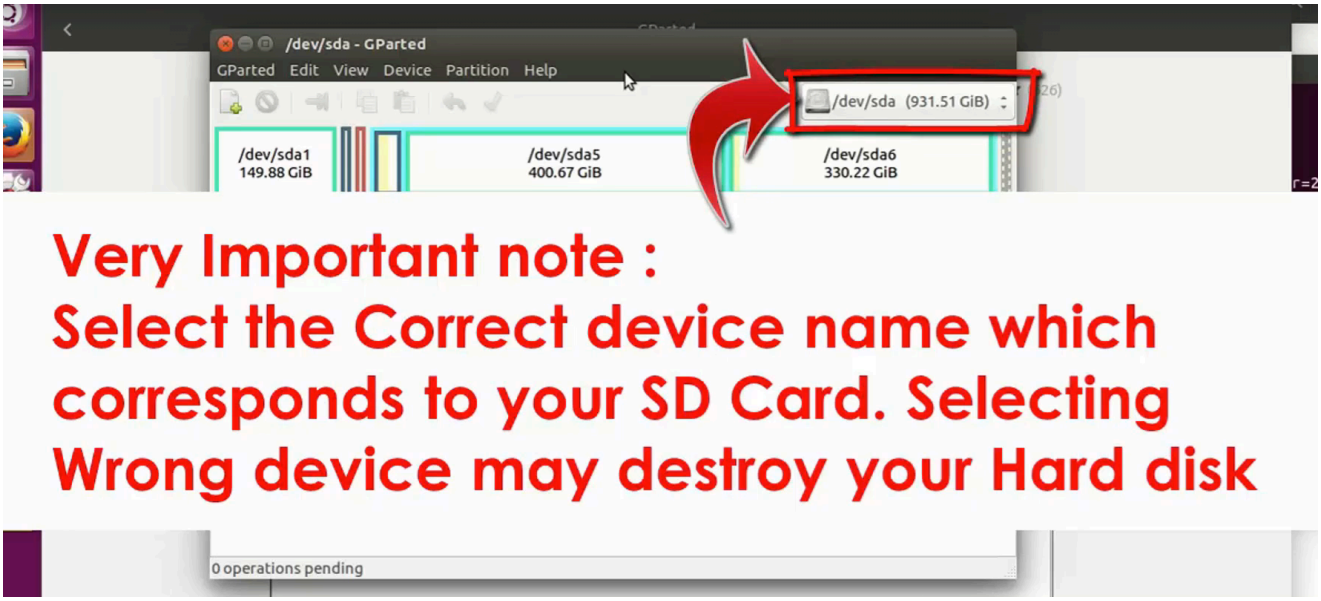



| Bölüm | Label | Dosya sistemi | Boyut | Neden? |
|---|---|---|---|---|
| 1 | `BOOT` | `fat16` | ~1 GB | RBL, MLO'yu sadece FAT bölümde arar |
| 2 | `ROOTFS` | `ext3` | Kalan alan | Linux dosyaları Linux dosya sistemi ister |

GParted'ı kur ve aç:

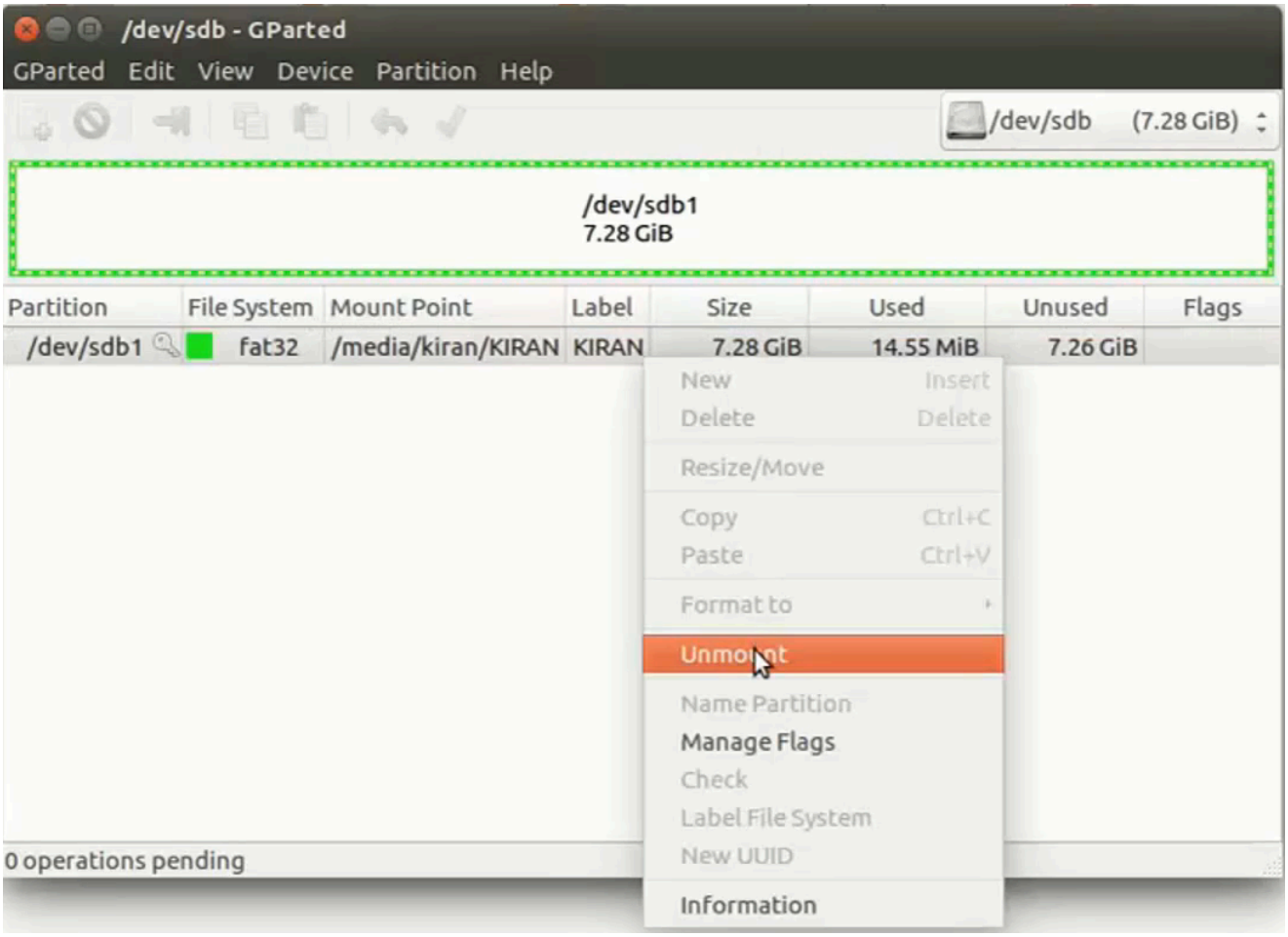
```
sudo apt install gparted
sudo gparted
```

> **Dikkat**
>
> Sağ üstteki listeden **SD kartı seç** ( `/dev/sdb` , 7.28 GiB). Yanlışlıkla bilgisayarının diskini ( `/dev/sda` ) seçersen **bilgisayarındaki veriler silinir**.



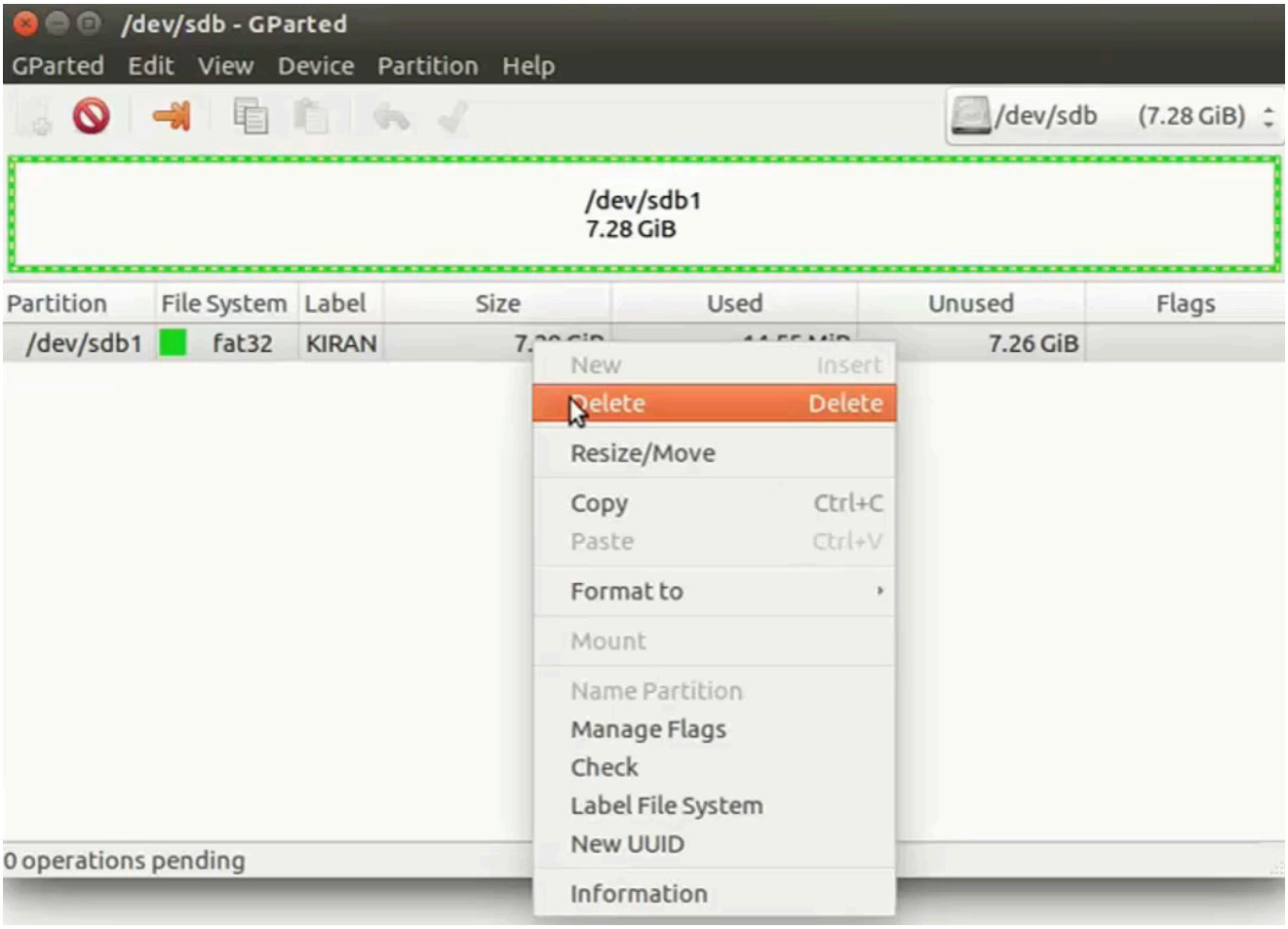

### 4.4.1 Unmount et

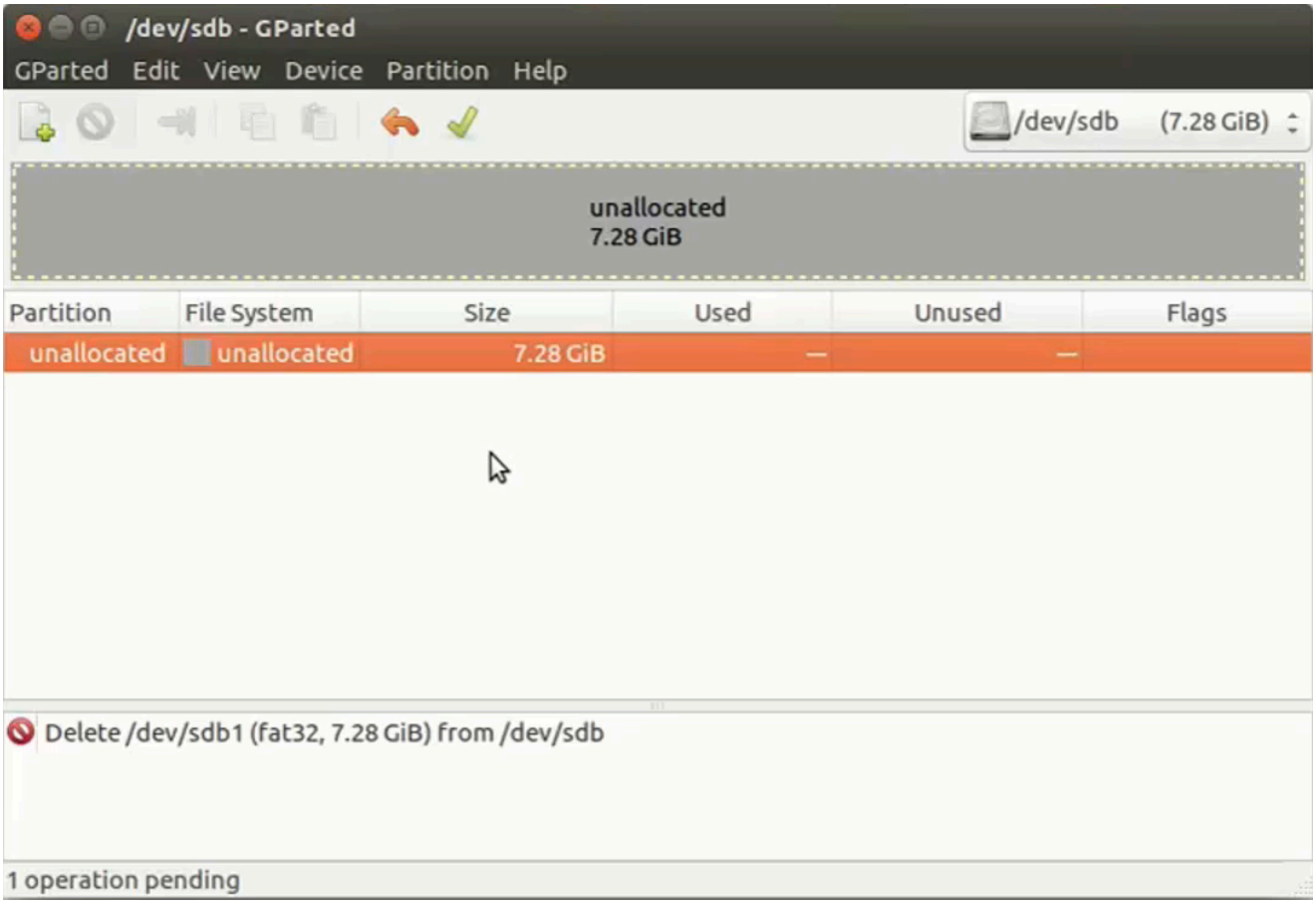
Bölüm şu an sistem tarafından kullanılıyor (yanındaki 🔑 simgesi). Kullanımdayken silinemez. **Sağ tık → Unmount.**




### 4.4.2 Eski bölümü sil

**Sağ tık → Delete.** Kartın tamamı `unallocated` (boş alan) olur.

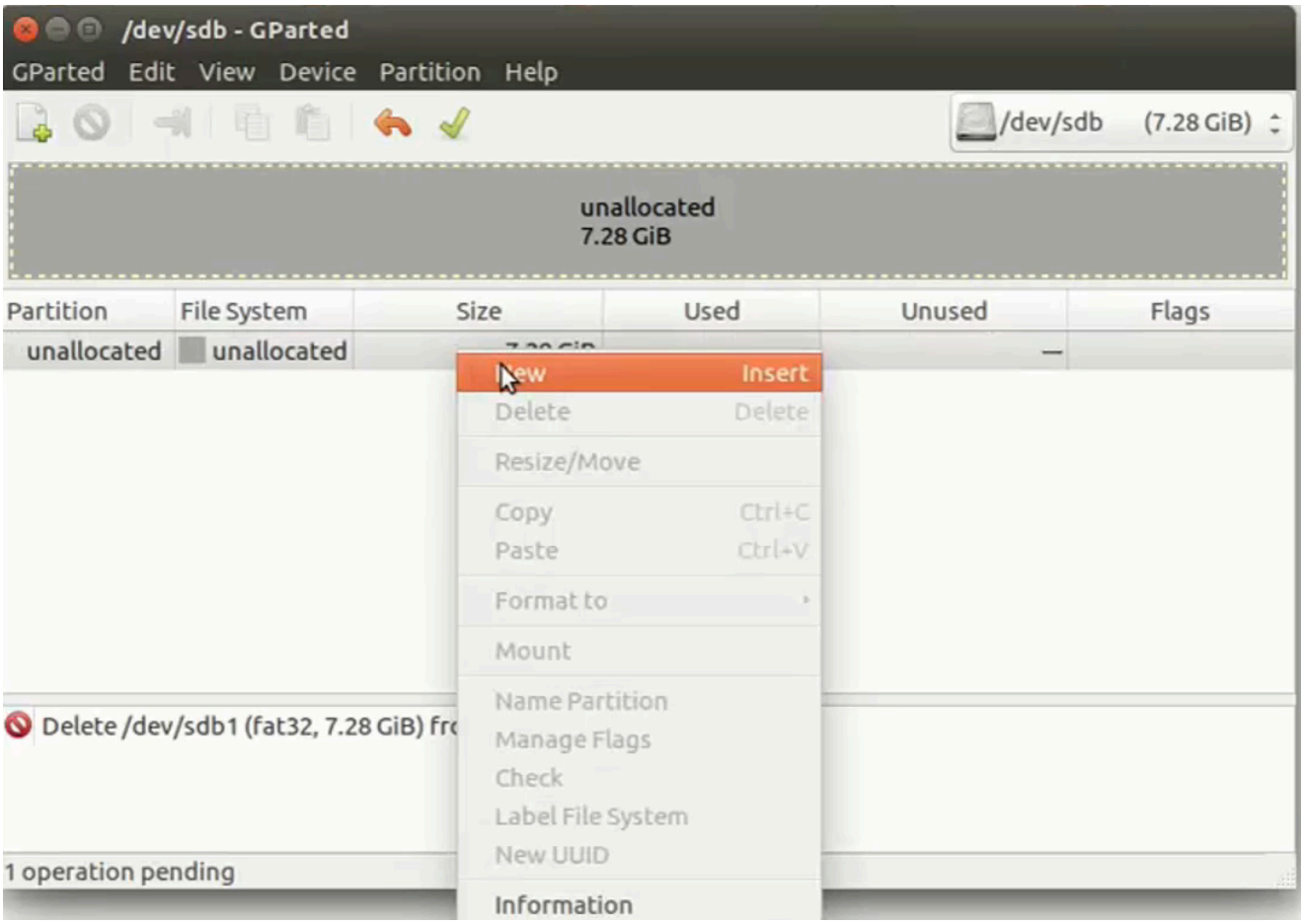

/dev/sdb - GParted
GParted Edit View Device Partition Help
/dev/sdb (7.28 GiB)
/dev/sdb1
7.28 GiB
Partition File System Label Size Used Unused Flags
/dev/sdb1 fat32 KIRAN 7.26 GiB
New Insert
Delete Delete
Resize/Move
Copy Ctrl+C
Paste Ctrl+V
Format to
Mount
Name Partition
Manage Flags
Check
Label File System
New UUID
Information
0 operations pending


/dev/sdb - GParted
GParted Edit View Device Partition Help
/dev/sdb (7.28 GiB)
unallocated
7.28 GiB
Partition File System Size Used Unused Flags
unallocated unallocated 7.28 GiB — —
Delete /dev/sdb1 (fat32, 7.28 GiB) from /dev/sdb
1 operation pending

> **Not**
>
> GParted işlemleri hemen uygulamaz, sıraya koyar ("1 operation pending"). Hepsi en sonda 4.4.5'te uygulanır. O ana kadar her şey geri alınabilir.

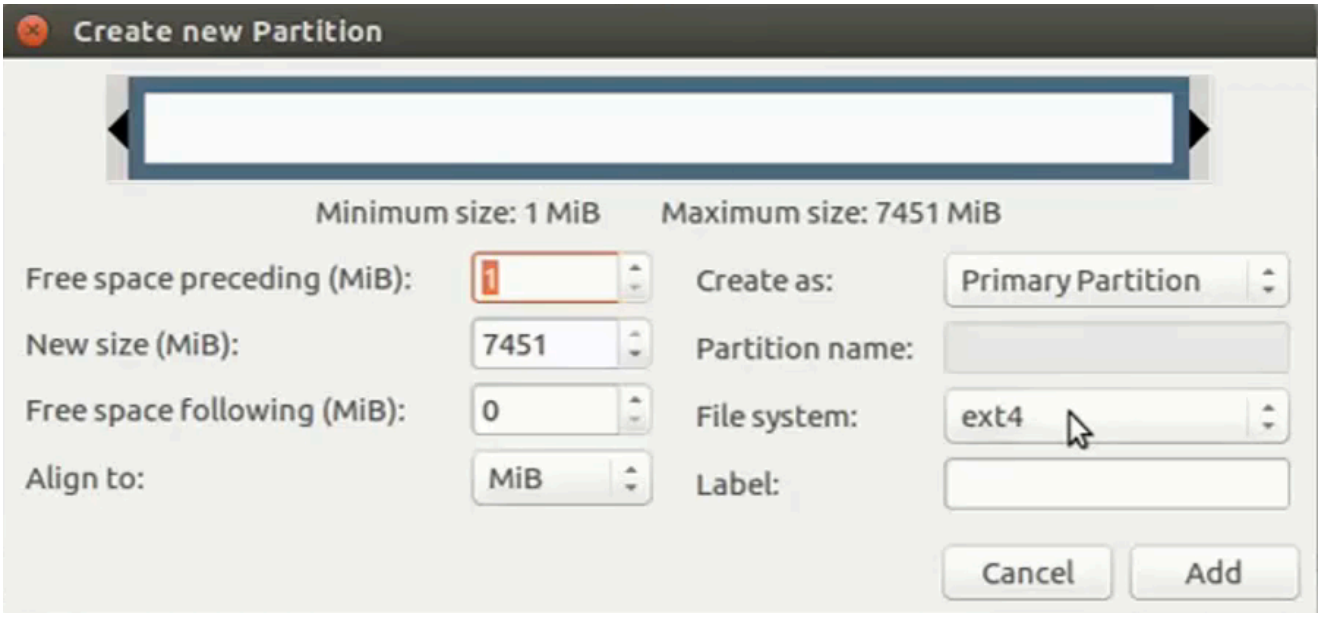
### 4.4.3 BOOT bölümünü oluştur

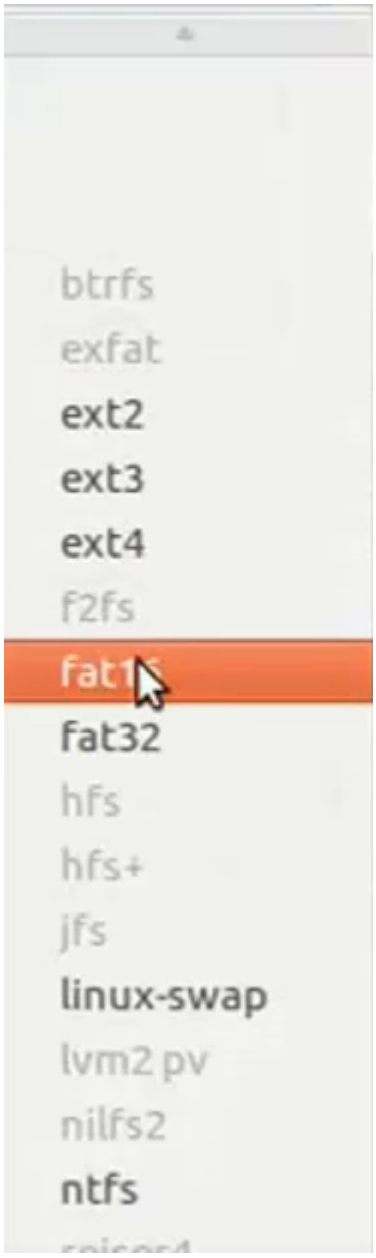
`unallocated` alana **sağ tık → New.**

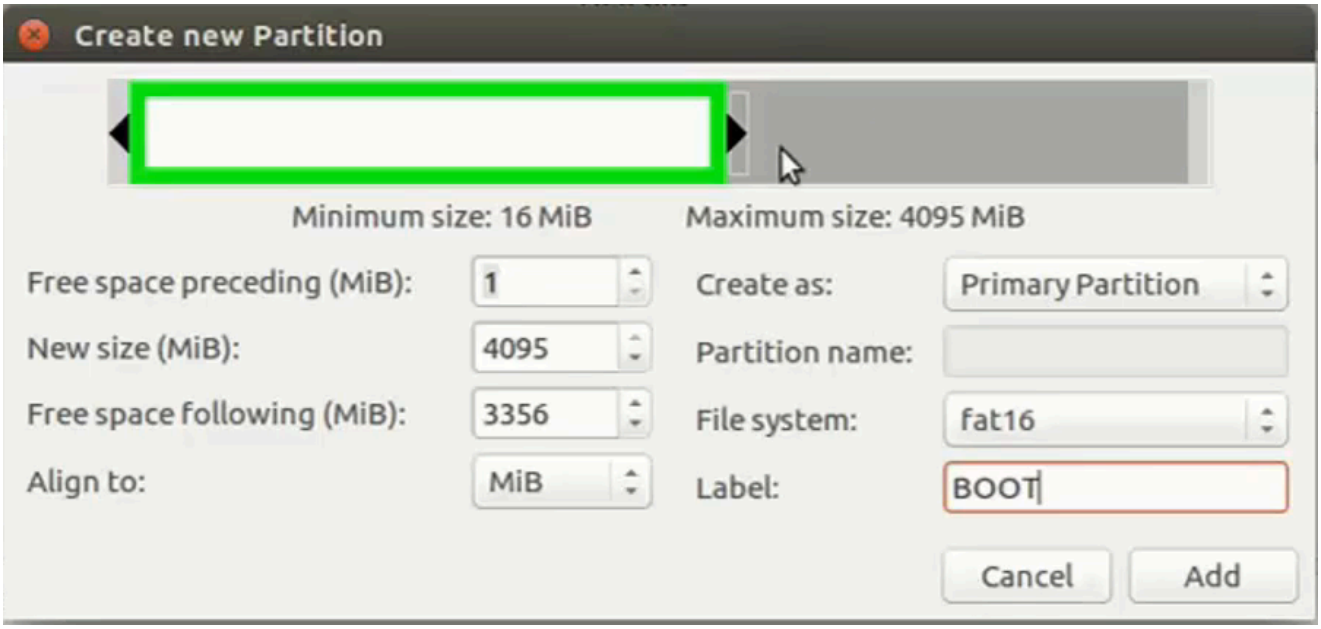



Açılan pencerede 3 ayar yap, sonra **Add**'e bas:

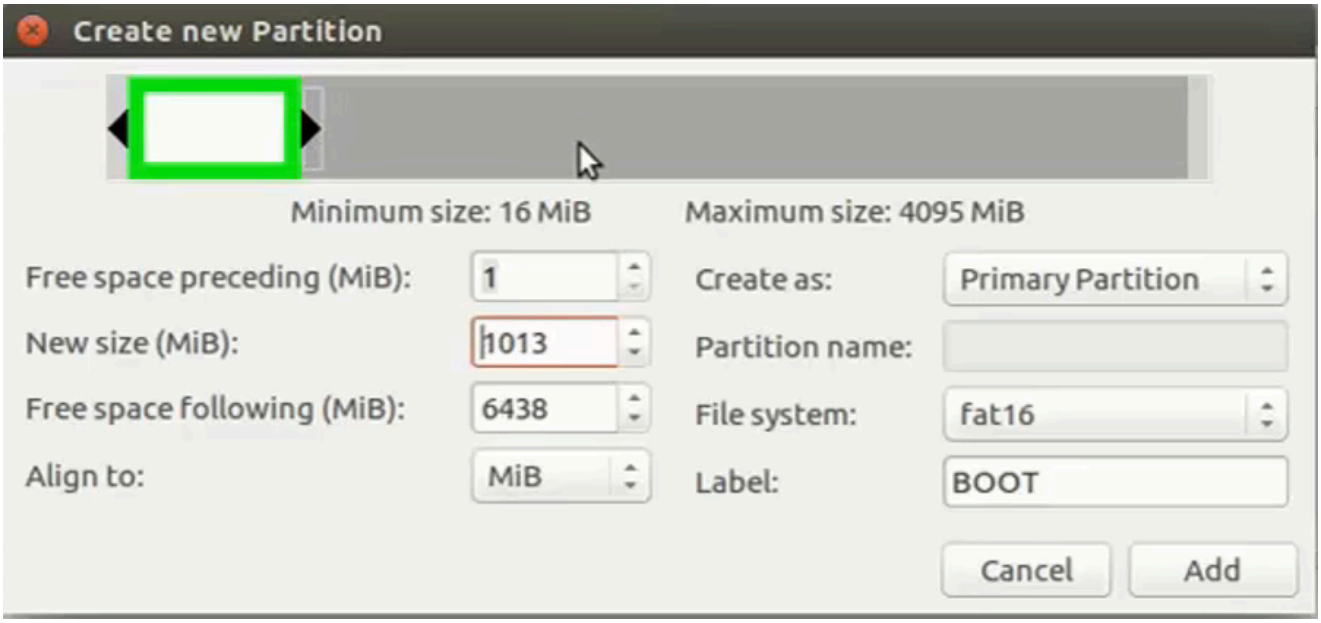
| Alan | Değer |
|---|---|
| File system | `fat16` |
| Label | `BOOT` |
| New size (MiB) | `1013` (yaklaşık 1 GB) |

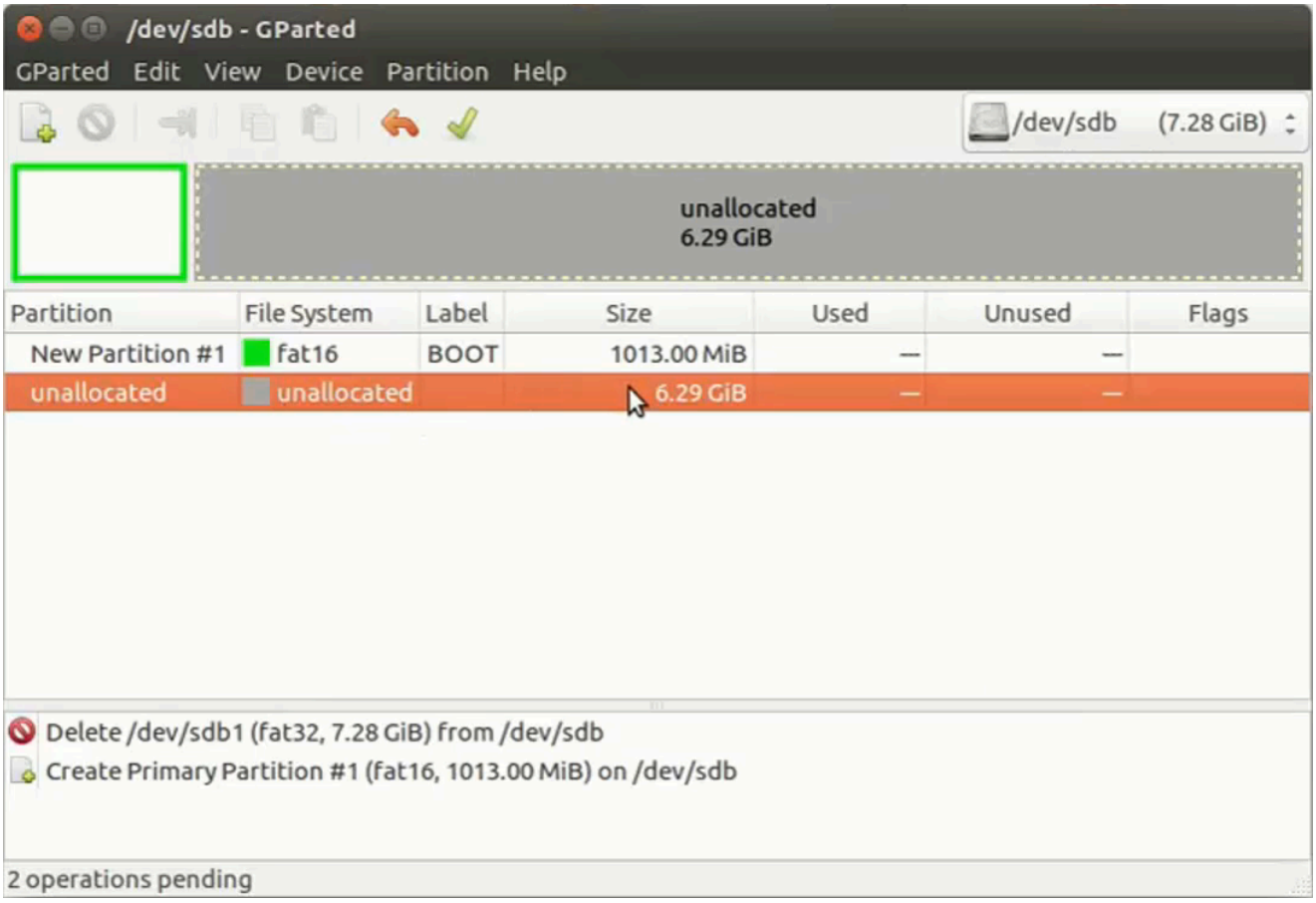

Create new Partition
Minimum size: 1 MiB
Maximum size: 7451 MiB
Free space preceding (MiB):
1
New size (MiB):
7451
Free space following (MiB):
0
Align to:
MiB
Create as:
Primary Partition
Partition name:
File system:
ext4
Label:
Cancel
Add


btrfs
exfat
ext2
ext3
ext4
f2fs
fat16
fat32
hfs
hfs+
jfs
linux-swap
lvm2 pv
nilfs2
ntfs




> **Not**
>
> fat16 seçilince boyut kendiliğinden **4095 MiB** olur, çünkü FAT16 en fazla ~4 GiB olabilir. Bunu 1013'e düşürüyoruz. Boot dosyaları zaten birkaç MB.

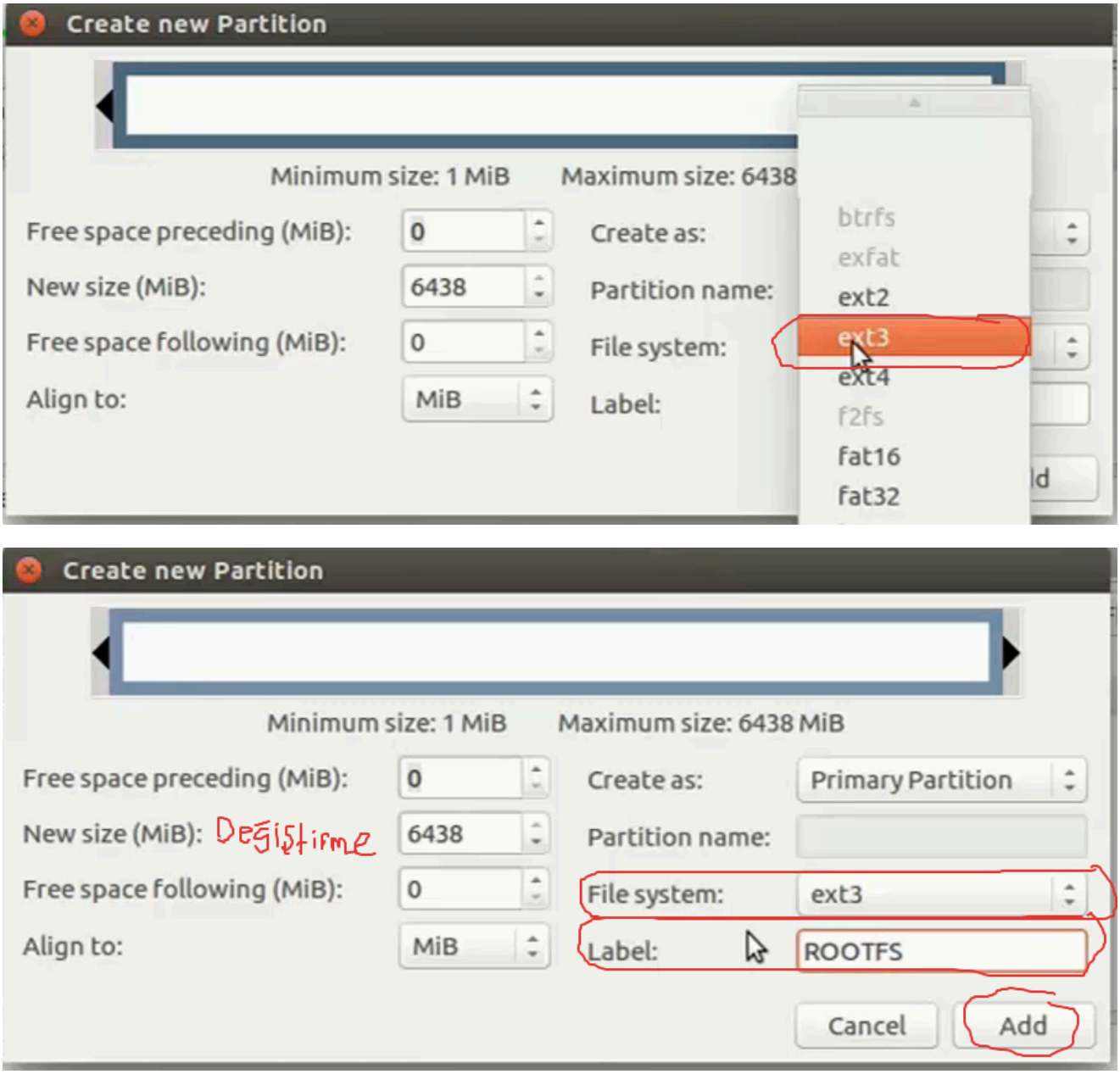





### 4.4.4 ROOTFS bölümünü oluştur

Kalan `unallocated` alana yine **sağ tık** → **New**, sonra **Add**:

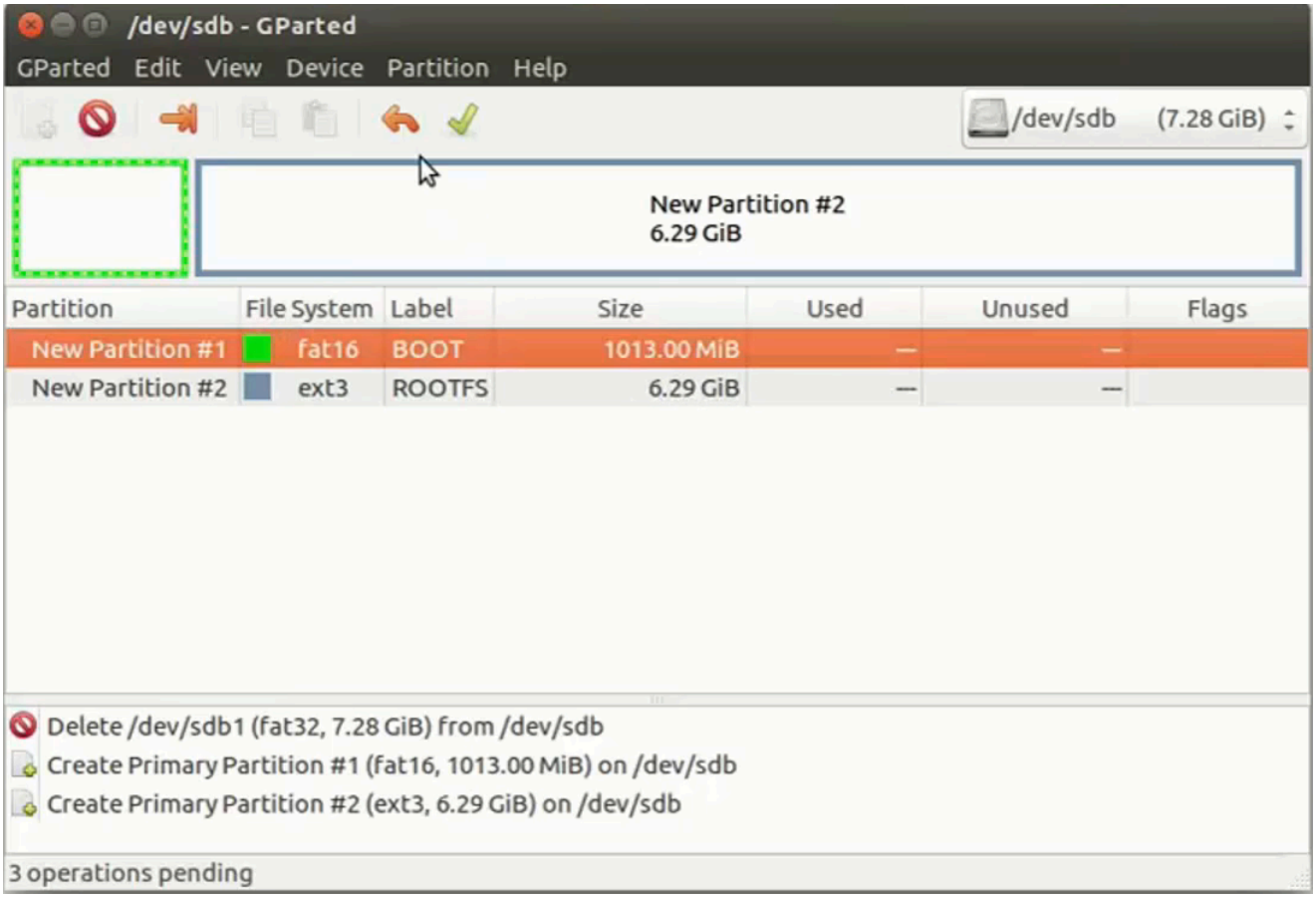
| Alan | Değer |
|---|---|
| File system | `ext3` |
| Label | `ROOTFS` |
| New size (MiB) | Değiştirme (kalan alanın tamamı) |







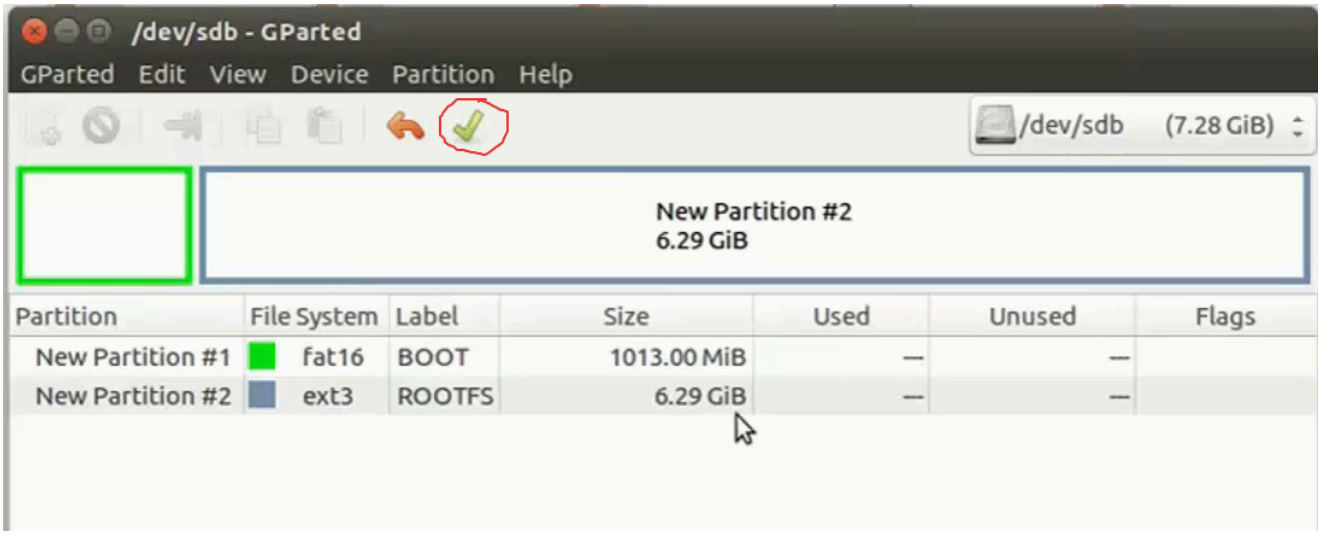
Planlanan durum: 1 GB BOOT + 6.29 GiB ROOTFS.

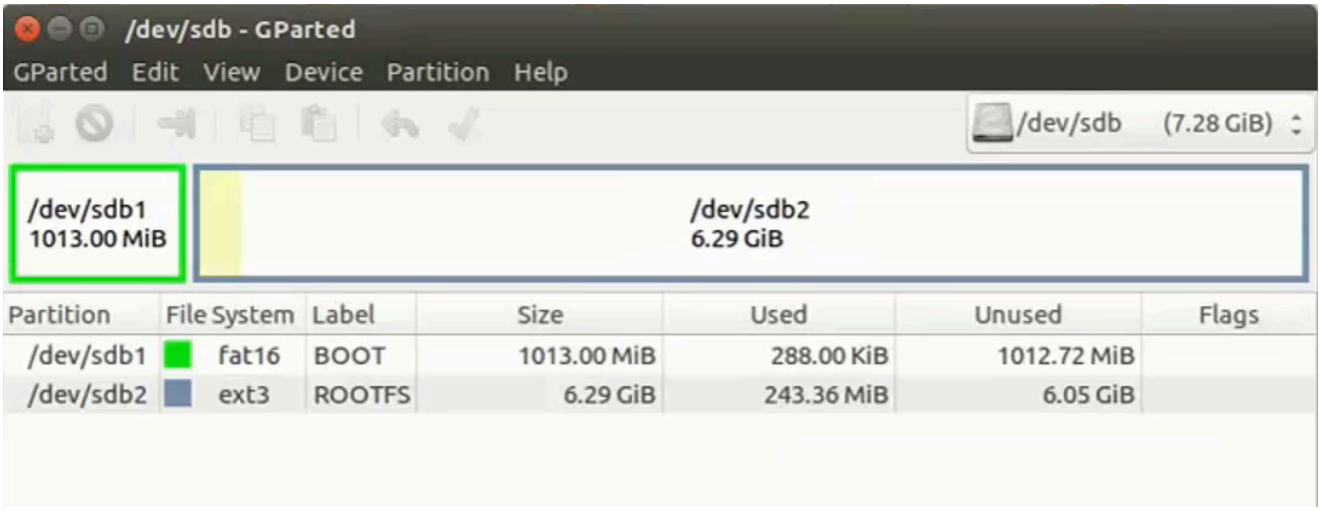



### 4.4.5 Değişiklikleri uygula

**Yeşil tik** (Apply All Operations) → onay penceresinde **Apply**.

> **Uyarı**
>
> Bu adımdan sonra geri dönüş yok, karttaki eski veriler silinir.

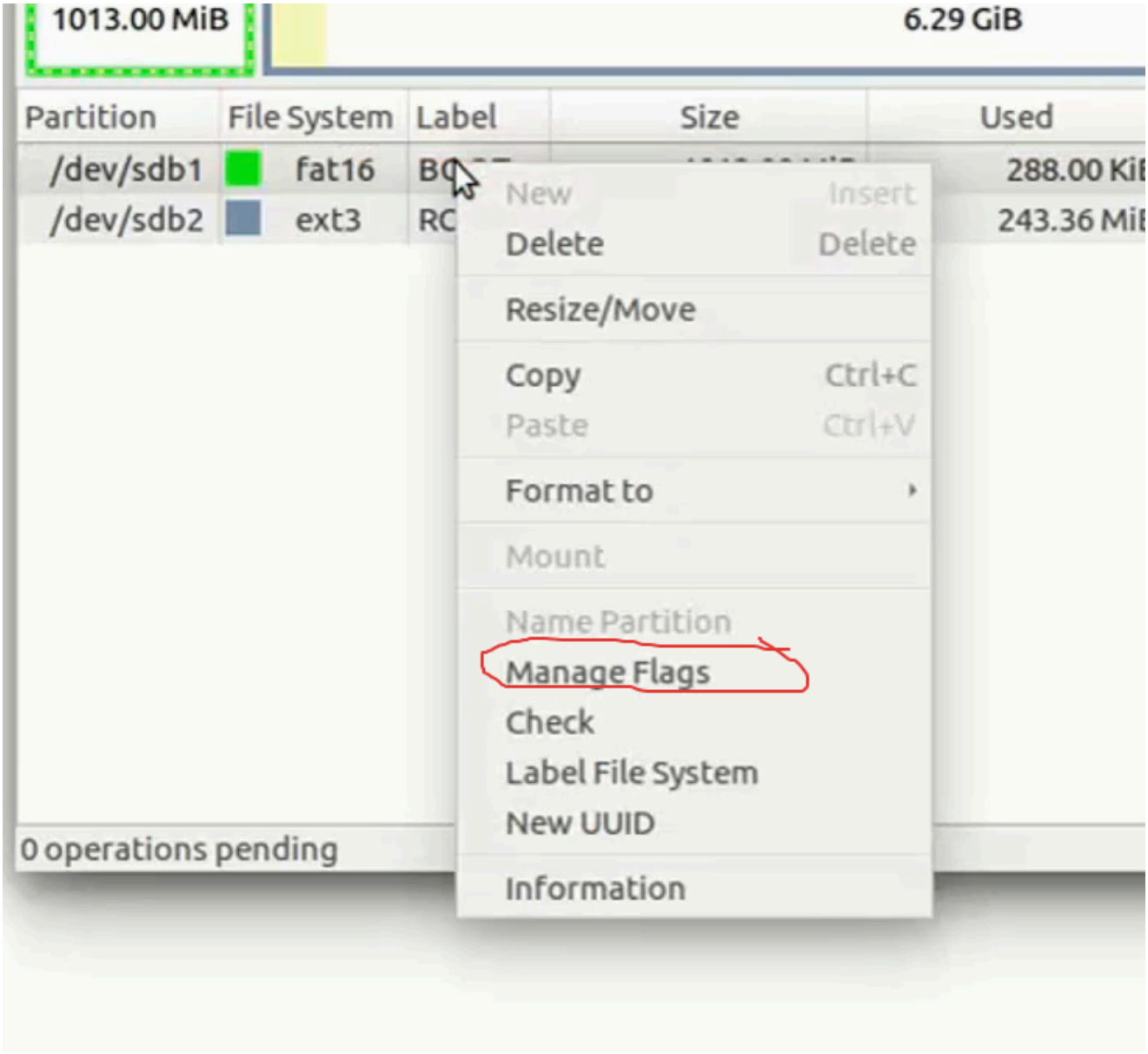


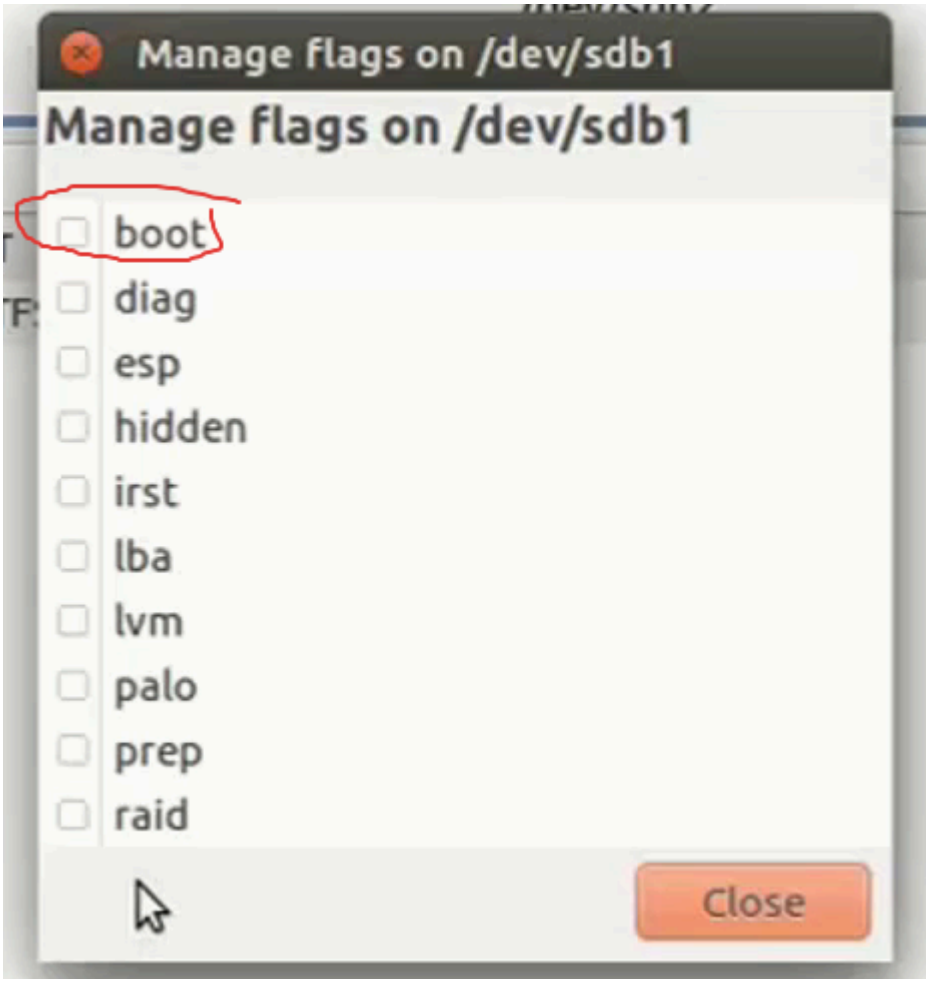



### 4.4.6 BOOT bölümüne `boot` bayrağı ekle

Bu bayrak, bölümü "açılış bölümü" olarak işaretler. RBL, MLO'yu bu bölümde arar.

**BOOT'a sağ tık → Manage Flags → `boot` kutucuğunu işaretle → Close.**






Flags sütununda `boot` yazıyorsa tamam. GParted'ı kapatabilirsin.

| Partition | File System | Label | Size | Used | Unused | Flags |
|---|---|---|---|---|---|---|
| /dev/sdb1 | fat16 | BOOT | 1013.00 MiB | 288.00 KiB | 1012.72 MiB | boot |
| /dev/sdb2 | ext3 | ROOTFS | 6.29 GiB | 243.36 MiB | 6.05 GiB | |

---

## 4.5 Boot Prosedürü (her testte tekrarlanır)

Aşağıdaki aşamaların her birinde dosya kopyaladıktan sonra bu adımları uygula.

### 4.5.1 Kartı bilgisayardan güvenli çıkar

```
sync
sudo umount /media/$USER/BOOT
sudo umount /media/$USER/ROOTFS
```

- `sync` → Bekleyen yazma işlemlerini karta yazar. Atlanırsa kopyalanan dosya eksik kalabilir.
- `umount` → Bölümlerin bağlantısını keser. Artık kart çıkarılabilir.

### 4.5.2 Seri konsolu hazırla (sadece ilk seferde)

1. USB-TTL'yi kartın **J1** header'ına bağla (1.8).
2. Minicom'u aç: **115200 baud, 8N1, Hardware Flow Control kapalı** (Bölüm 2).

### 4.5.3 Kartı SD karttan aç (S2 butonu)

BeagleBone Black normalde **kendi eMMC belleğinden** açılır. SD karttan açılması için **kart açılırken S2 (BOOT) butonu basılı olmalı**. RBL bu butona sadece açılış anında bakar (3.5).

SD kartı BeagleBone'a tak, sonra iki yöntemden birini uygula:

| Yöntem | Ne zaman? | Adımlar |
|---|---|---|
| **A: Güç vererek** | Kartta güç yokken | S2'yi basılı tut → USB kablosunu tak → 2–3 sn sonra S2'yi bırak |
| **B: Power butonuyla** | Kart zaten güç alıyorsa | Power'a LED'ler sönene kadar uzun bas (kapanır) → S2'yi basılı tut → Power'a kısa bas (açılır) → S2'yi bırak |

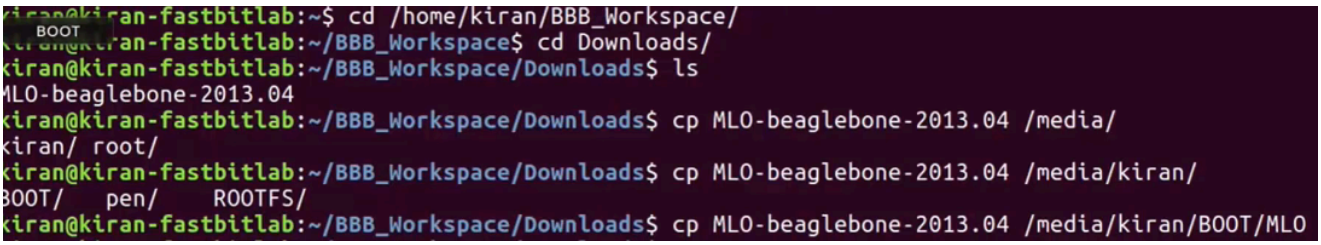
---

## 4.6 Aşama 1: MLO

### Kopyala

İndirme klasörüne git ve MLO'yu BOOT bölümüne `MLO` **adıyla** kopyala:

```
cd ~/BBB_Workspace/Downloads/
cp MLO-beaglebone-2013.04 /media/$USER/BOOT/MLO
```

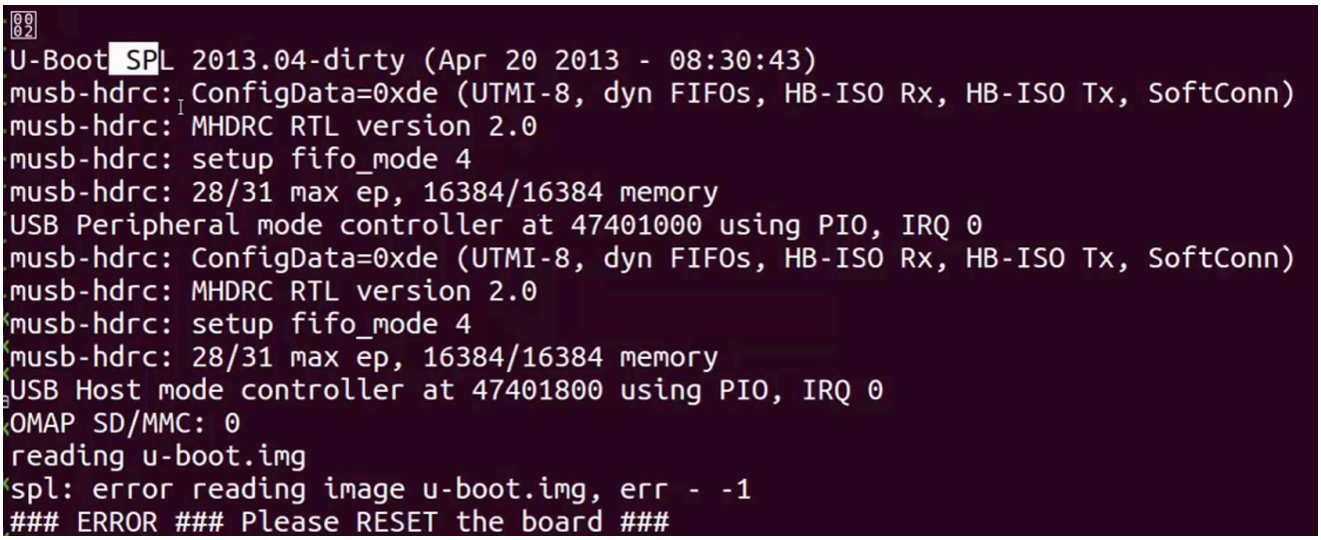



> **Önemli**
>
> **Neden sonda `MLO` yazıyoruz?** `cp` komutunda hedefin sonuna bir ad yazılırsa dosya o adla kopyalanır:
>
> ```
> cp  MLO-beaglebone-2013.04   /media/$USER/BOOT/MLO
>     └──────── kaynak ────────┘     └─ hedef klasör + YENİ AD ─┘
> ```
>
> RBL, BOOT bölümünde **tam olarak `MLO` adlı** dosyayı arar. Dosya uzun adıyla kalırsa bulunamaz.

> **İpucu**
>
> `$USER` senin kullanıcı adındır (kursta `kiran`). Dosya adlarını elle yazmak yerine ilk birkaç harfi yazıp **Tab**'a bas, terminal tamamlar.

### Test et

4.5'teki adımlarla kartı aç. Minicom'da şunu görmelisin:




| Satır | Anlamı |
|---|---|
| `U-Boot SPL 2013.04-dirty ...` | **MLO çalıştı.** RBL dosyayı buldu. |
| `musb-hdrc ...`, `USB ... controller` | MLO USB donanımını başlatıyor, önemli değil. |
| `reading u-boot.img` | MLO bir sonraki aşamayı (U-Boot) arıyor. |
| `spl: error reading image u-boot.img` | U-Boot bulunamadı, çünkü henüz kopyalamadık. **Beklenen hata.** |

Sonuç: **RBL ✓ → MLO ✓ → U-Boot ✗**

> **Not**
>
> "dirty" bir hata değildir. MLO derlenirken kaynak kodda kaydedilmemiş (commit edilmemiş) değişiklik olduğunu belirtir.

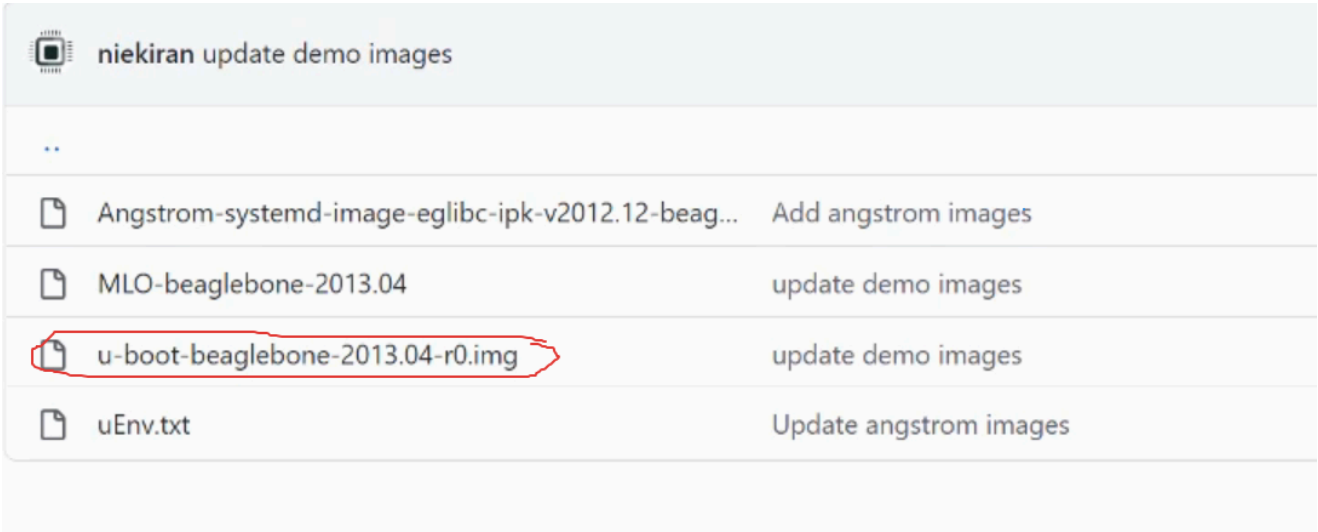
---

## 4.7 Aşama 2: U-Boot

### Kopyala

Kartı tekrar bilgisayara tak ve U-Boot'u `u-boot.img` **adıyla** kopyala. MLO, U-Boot'u bu adla arar:

```
cp u-boot-beaglebone-2013.04-r0.img /media/$USER/BOOT/u-boot.img
```




### Test et

4.5'teki adımlarla kartı aç.

- `U-Boot SPL` satırlarından sonra **U-Boot'un kendi mesajları** görünüyorsa bu aşama başarılı.
- Kernel ve `uEnv.txt` henüz olmadığı için Linux SD karttan açılmaz.

Sonuç: **RBL** ✓ → **MLO** ✓ → **U-Boot** ✓ → **Kernel** ✗

---

## 4.8 Aşama 3: Root Filesystem ve uEnv.txt

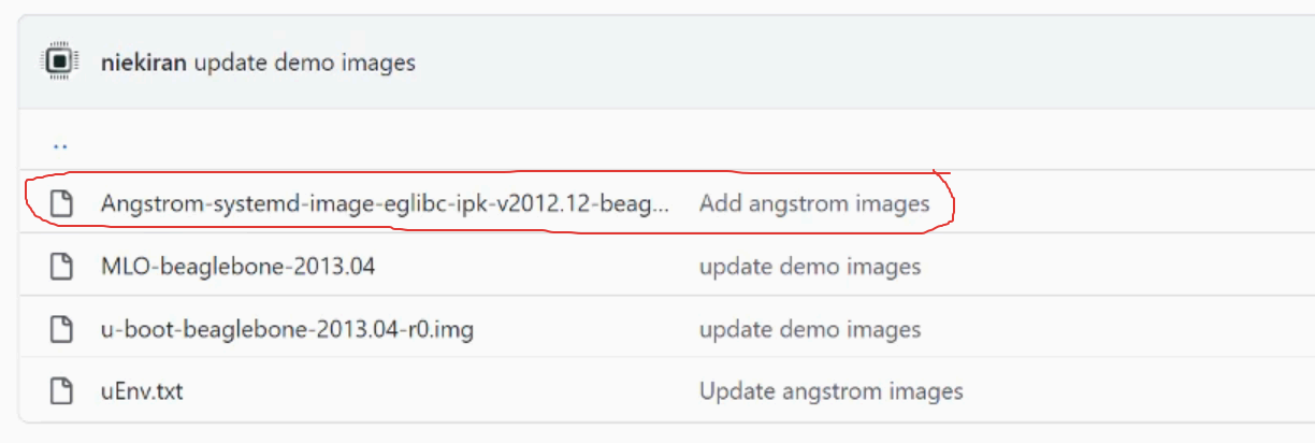
### Kopyala

Kartı tekrar bilgisayara tak.

**1) Root filesystem'i ROOTFS bölümüne çıkar:**

```
sudo tar -xvf Angstrom-systemd-image-eglibc-ipk-v2012.12-beagleboard.rootfs.tar.xz -C
/media/$USER/ROOTFS
```

| Parça | Anlamı |
|---|---|
| `sudo` | Linux dosyalarının sahibi `root`. Dosya izinlerinin korunması için gerekli. |
| `-x` `-v` `-f` | Arşivi aç (e**x**tract), açılan dosyaları göster (**v**erbose), arşiv dosyası (**f**ile) |
| `-C /media/$USER/ROOTFS` | Dosyaları bu klasöre çıkar |




**2) uEnv.txt'yi BOOT bölümüne kopyala** (adı değişmiyor, U-Boot bu adla arar):

```
cp uEnv.txt /media/$USER/BOOT/
```

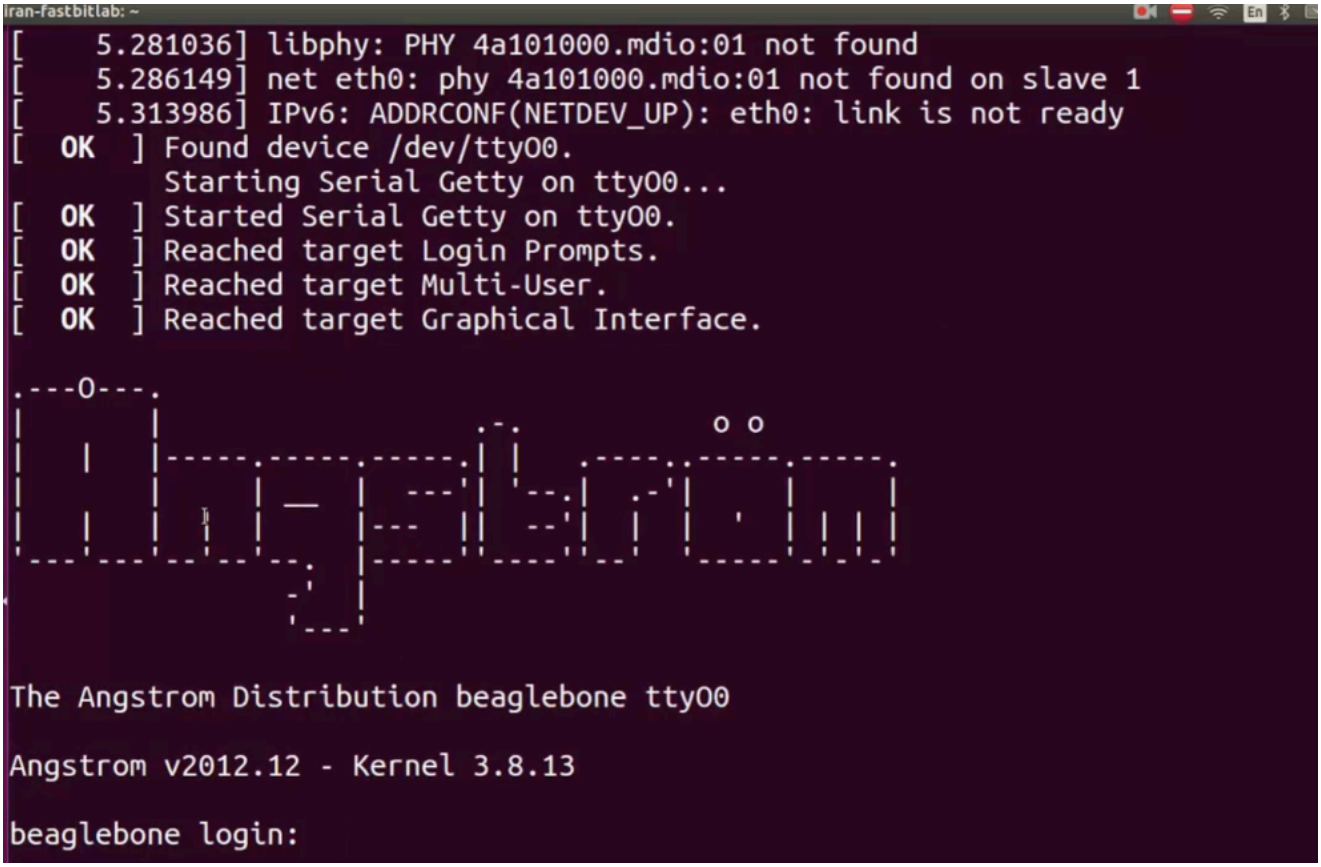
**Kontrol et:**

```
ls /media/$USER/BOOT/           # MLO  u-boot.img  uEnv.txt
ls /media/$USER/ROOTFS/boot/    # uImage ve am335x-boneblack.dtb burada olmalı
```

## Test et

4.5'teki adımlarla kartı aç. Rootfs büyük olduğu için `sync` biraz uzun sürebilir, bitmesini bekle.

Kernel mesajları akar ve sonunda **Ångström** logosu ile `beaglebone login:` satırı çıkar:




Sonuç: **RBL ✓ → MLO ✓ → U-Boot ✓ → Kernel ✓ → Rootfs ✓** 🎉

> **İpucu**
>
> Giriş için kullanıcı adı `root` yaz. Bu demo imajında genelde şifre yoktur.

## Bu açılışta uEnv.txt ne yaptı?

`uEnv.txt`, U-Boot'un davranışını değiştiren ayar dosyasıdır. BBB'nin U-Boot'u açılırken BOOT bölümündeki bu dosyayı okur ve içindeki `uenvcmd` komutunu çalıştırır.

```
console=ttyO0,115200n8
ipaddr=192.168.7.2
serverip=192.168.7.1
loadaddr=0x82000000 ──→ DDR Temel Adresi
fdtaddr=0x88000000 ──→ DTB File Adresi
loadfromsd=load mmc 0:2 ${loadaddr} /boot/uImage;load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb
linuxbootargs=setenv bootargs console=${console} root=/dev/mmcblk0p2 rw
uenvcmd=setenv autoload no; run loadfromsd; run linuxbootargs; bootm ${loadaddr} - ${fdtaddr}
```

| Satır | Anlamı |
|---|---|
| `console=ttyO0,115200n8` | Linux mesajları UART0'a (J1 header) 115200 baud ile yazılsın. |
| `ipaddr`, `serverip` | Ağdan boot için IP adresleri. SD karttan boot'ta kullanılmıyor. |
| `loadaddr=0x82000000` | Kernel'in (uImage) **RAM'de (DDR) yükleneceği adres**. |
| `fdtaddr=0x88000000` | Device tree (.dtb) dosyasının RAM'de yükleneceği adres. |

| Satır | Anlamı |
|---|---|
| `loadfromsd=load mmc 0:2 ...` | `mmc 0:2` = SD kart, 2. bölüm (ROOTFS). Oradaki `/boot/uImage` → `loadaddr`'e, `/boot/am335x-boneblack.dtb` → `fdtaddr`'e yüklenir. |
| `linuxbootargs=... root=/dev/mmcblk0p2 rw` | Kernel'e "root filesystem SD kartın 2. bölümünde, okuma-yazma modunda" bilgisini verir. |
| `uenvcmd=...` | U-Boot'un çalıştırdığı komut: önce `loadfromsd`, sonra `linuxbootargs`, en son `bootm` ile kernel'i başlat. `bootm`'deki `-` "initramfs yok" demek. |

> **Not**
>
> AM335x'te DDR'in başlangıç adresi `0x80000000`'dır. `0x82000000` ise DDR'in içinde, kernel için seçilmiş bir adres.

Aynı dosya farklı yazılarak U-Boot'a kernel'i **ağdan (TFTP)**, rootfs'u da **bilgisayardaki bir klasörden (NFS)** aldırmak mümkün (Bölüm 8):

```
console=ttyO0,115200n8
ipaddr=192.168.7.2
serverip=192.168.7.1
absolutepath=/var/lib/tftp/
rootpath=/srv/nfs/bbb,nolock,wsize=1024,rsize=1024 rootwait rootdelay=5
loadtftp=echo Booting from network ...;tftpboot ${loadaddr} ${absolutepath}uImage; tftpb
netargs=setenv bootargs console=${console} root=/dev/nfs rw nfsroot=${serverip}:${rootpa
uenvcmd=setenv autoload no; run loadtftp; run netargs; bootm ${loadaddr} - ${fdtaddr}
```

`uEnv.txt`'nin her satırını ve U-Boot komut satırını Bölüm 6'da ayrıntılı göreceğiz.

---

## 4.9 uImage Nedir?

`uImage`, U-Boot'un `bootm` komutuyla başlatabildiği kernel formatıdır. Yeni bir derleme yapılmaz. `zImage` dosyasının başına **64 byte'lık bir U-Boot header'ı** eklenir. Bu işi `mkimage` aracı yapar.

```
+-------------------+--------------------------------------+
| U-Boot Header     |               zImage                 |
|   (64 byte)       | (sıkıştırılmış, kendini açan kernel) |
+-------------------+--------------------------------------+
|<----------------------- uImage ------------------------->|
```

**Kernel imajları zinciri:**

| Dosya | Açıklama |
|---|---|
| `vmlinux` | Derlenmiş kernel. ELF formatında, sıkıştırılmamış. |
| `Image` | `vmlinux`'tan `objcopy` ile çıkarılmış ham binary |
| `zImage` | Sıkıştırılmış `Image` ve başında onu RAM'de açan küçük bir program |
| `uImage` | 64 byte U-Boot header + `zImage` |

**Header'ın içeriği** ( `struct image_header`, tüm alanlar big-endian):

| Offset | Boyut | Alan | Anlamı |
|---|---|---|---|
| 0x00 | 4 | `ih_magic` | Sihirli sayı: `0x27051956` |
| 0x04 | 4 | `ih_hcrc` | Header'ın CRC32 değeri |
| 0x08 | 4 | `ih_time` | Oluşturulma zamanı |
| 0x0C | 4 | `ih_size` | zImage'in boyutu |
| 0x10 | 4 | `ih_load` | RAM'de kopyalanacağı adres |
| 0x14 | 4 | `ih_ep` | Çalıştırmanın başlayacağı adres (entry point) |
| 0x18 | 4 | `ih_dcrc` | zImage'in CRC32 değeri |
| 0x1C | 1 | `ih_os` | İşletim sistemi (Linux) |
| 0x1D | 1 | `ih_arch` | Mimari (ARM) |
| 0x1E | 1 | `ih_type` | İmaj tipi (kernel, ramdisk, script...) |
| 0x1F | 1 | `ih_comp` | Sıkıştırma tipi |
| 0x20 | 32 | `ih_name` | İmaj adı |

U-Boot bu header sayesinde dosyanın geçerli bir imaj olup olmadığını (magic number), bozulup bozulmadığını (CRC) ve RAM'de nereye yüklenip nereden çalıştırılacağını anlar.

---

## 4.10 Uygulama: uImage Header'ını U-Boot'ta Elle Okuma

**Amaç:** uImage'ın başındaki 64 byte'lık U-Boot header'ını (4.9) kendi gözümüzle görmek.

**Neredeyiz?** 4.8'de U-Boot, `uEnv.txt`'deki komutları **otomatik** çalıştırıp Linux'u açtı. Bu sefer U-Boot'u durdurup aynı işin ilk yarısını **elle** yapacağız:

```
4.8:   RBL → MLO → U-Boot → Kernel → Rootfs → login     (otomatik)
4.10:  RBL → MLO → U-Boot ■ DUR → komutları elle yaz
```

| Adım | Komut | Ne yapar? |
|---|---|---|
| 1 | Açılışta bir tuşa bas | U-Boot'u durdurur, `U-Boot#` komut satırı gelir |
| 2 | `load mmc 0:2 0x82000000 /boot/uImage` | uImage'ı SD karttan DDR'e kopyalar |
| 3 | `md.b 0x82000000 40` | DDR'deki ilk 64 byte'ı (header) ekrana döker |
| 4 | — | Byte'ları header tablosuyla eşleştirip çözeriz |

> **Not**
>
> **Neden önce DDR'e yüklüyoruz?** `md` (memory display) komutu sadece **belleği** okuyabilir. SD kart bir bellek adresi değil, depolama cihazıdır. Bu yüzden dosyayı önce RAM'e kopyalıyoruz.

Kursun bu dersinde Windows'ta Tera Term kullanılıyor. Minicom ile de komutlar ve çıktılar aynıdır (2.6).

### 4.10.1 U-Boot'u durdur

1. Seri terminali aç ve kartı 4.5.3'teki gibi SD karttan başlat.
2. Ekranda `Hit any key to stop autoboot` yazısı görünür görünmez bir tuşa (ör. **Space**) bas.
3. `U-Boot#` komut satırı geldiyse hazırsın.

> **İpucu**
>
> Geri sayım çok kısa (1–3 saniye). Kaçırırsan Linux açılır. Kartı yeniden başlatıp daha erken bas. Kart açılırken Space'e art arda basmak işe yarar.

### 4.10.2 uImage'ı DDR'e yükle

Önce dosyanın orada olduğunu gör:

```
U-Boot# ls mmc 0:2 /boot
```

Sonra DDR'e yükle:

```
U-Boot# load mmc 0:2 0x82000000 /boot/uImage
```

| Parça | Anlamı |
|---|---|
| `load` | Bir dosya sisteminden dosyayı belleğe kopyala |
| `mmc 0:2` | MMC cihazı 0 (SD kart), 2. bölüm (ROOTFS) |
| `0x82000000` | DDR'de dosyanın yazılacağı adres (`uEnv.txt`'deki `loadaddr` ile aynı) |
| `/boot/uImage` | Bölüm içindeki dosyanın yolu |

### `mmc X:Y` ne demek?

```
load  mmc 0:2      0x82000000   /boot/uImage
      └─ NEREDEN ─┘ └─ NEREYE ─┘ └─ HANGİ DOSYA ┘
      SD kart,      DDR'deki      partition içindeki
      2. partition  bu adrese     dosya yolu
```

- `X` = **hangi MMC cihazı.** BeagleBone Black'te:
- `mmc 0` → **microSD kart**
- `mmc 1` → **eMMC** (karta lehimli dahili bellek)
- `Y` = **o cihazın kaçıncı partition'ı.** Numaralandırma **1'den** başlar.

> **Önemli**
>
> `load` her zaman **DDR'e** yükler. `mmc 0:2` dosyanın **nereye** yazılacağını değil, **nereden okunacağını** söyler. Yani "2. partition'a yükle" değil, "**2. partition'dan oku**, DDR'e yükle".

Bizim SD kartımız için:

| Yazım | Anlamı | İçinde ne var? |
|---|---|---|
| `mmc 0:1` | SD kart, 1. partition | BOOT (FAT16): `MLO`, `u-boot.img`, `uEnv.txt` |
| `mmc 0:2` | SD kart, 2. partition | ROOTFS (ext3): `/boot/uImage`, `/boot/am335x-boneblack.dtb`, Linux dosyaları |
| `mmc 1:...` | eMMC'nin partition'ları | Kartla fabrikadan gelen sistem (ör. Ångström/Debian) |

Örnekler:

```
U-Boot# ls mmc 0:1                              # BOOT bölümünü listele
U-Boot# ls mmc 0:2 /boot                        # ROOTFS'taki /boot klasörünü listele
U-Boot# load mmc 0:1 0x82000000 uEnv.txt        # BOOT'taki uEnv.txt'yi DDR'e yükle
U-Boot# mmc list                                # U-Boot'un gördüğü MMC cihazlarını listele
U-Boot# mmc dev 0                               # SD kartı seç
U-Boot# mmc part                                # Seçili cihazın partition tablosunu göster
```

> **Not**
>
> Linux'ta da benzer bir numaralandırma vardır: `/dev/mmcblk0p2` = MMC cihazı 0, partition 2. `uEnv.txt`'deki `root=/dev/mmcblk0p2` ifadesi, U-Boot'taki `mmc 0:2` ile aynı yeri gösterir. Ancak Linux'un numaraları kernel sürümüne göre U-Boot'tan farklı olabilir, bu yüzden ikisinin her zaman aynı olacağını varsayma.

Çıktıda kaç byte okunduğu yazar ( `xxxxxxx bytes read` ). Bu sayı uImage'ın boyutudur, yani 64 byte header + zImage.

> **Not**
>
> Bu komut `uEnv.txt` 'deki `loadfromsd` satırının yaptığı işin aynısı. Fark sadece bu sefer elle yazmamız.

> **İpucu**
>
> `load` komutu tanınmazsa eski adı olan `ext2load mmc 0:2 0x82000000 /boot/uImage` komutunu kullan (ext3 de bu komutla okunur). `ls` yerine de `ext2ls mmc 0:2 /boot` kullanılır.

### 4.10.3 Header'ı ekrana dök

```
U-Boot# md.b 0x82000000 40
```

| Parça | Anlamı |
|---|---|
| `md` | memory display: belleği ekrana dök |
| `.b` | byte byte göster |
| `0x82000000` | Başlangıç adresi (uImage'ı yüklediğimiz yer) |
| `40` | Kaç byte gösterilecek. **U-Boot sayıları hex kabul eder:** `0x40` = **64 byte** = header'ın tamamı |

Çıktı 4 satırdır. Her satırda adres, 16 byte ve sağda bu byte'ların ASCII karşılığı bulunur.

> **Uyarı**
>
> `md` komutu nokta olmadan ( `md 0x82000000 10` ) yazılırsa veriyi **4 byte'lık word'ler** halinde gösterir. ARM işlemci **little-endian**, header ise **big-endian** olduğu için byte'lar ters görünür. Örneğin magic number `27051956` yerine **`56190527`** olarak görünür. Header'ı okurken `md.b` kullanmak daha kolaydır.

### 4.10.4 Byte'ları çöz

Çıktının her satırı header'ın şu alanlarına denk gelir:

```
82000000: [27 05 19 56] [.. .. .. ..] [.. .. .. ..] [.. .. .. ..]
           ih_magic      ih_hcrc       ih_time       ih_size
82000010: [.. .. .. ..] [.. .. .. ..] [.. .. .. ..] [..][..][..][..]
           ih_load       ih_ep         ih_dcrc      os arch type comp
82000020: [ ih_name: 32 byte imaj adı, sağdaki ASCII sütununda okunur ...
82000030:   ...                                                      ]
```

| Alan | Nasıl kontrol edilir? |
|---|---|
| `ih_magic` | `27 05 19 56` olmalı. Değilse bu adreste geçerli bir uImage yok. |
| `ih_time` | Unix zaman damgası. Linux'ta tarihe çevir: `date -d @$((0xXXXXXXXX))` |
| `ih_size` | zImage boyutu. Ondalığa çevir: `echo $((0xXXXXXXXX))`. Bu değer + 64 = `load` komutunun okuduğu byte sayısı olmalı. |
| `ih_load`, `ih_ep` | Kernel'in DDR'de çalışacağı adres ve başlangıç noktası |
| `os` | `05` = Linux |
| `arch` | `02` = ARM |
| `type` | `02` = Kernel |
| `comp` | `00` = sıkıştırma yok. zImage zaten kendini açtığı için U-Boot ayrıca açmaz. |
| `ih_name` | Sağdaki ASCII sütununda imajın adı okunur |

> **İpucu**
>
> Hex'i ondalığa çevirmek için Windows Hesap Makinesi'nin **Programcı** modunu da kullanabilirsin.

### 4.10.5 Sonucu U-Boot'a doğrulat

Elle çözdüğün değerleri U-Boot'un kendi çözümüyle karşılaştır:

```
U-Boot# iminfo 0x82000000
```

`iminfo` komutu header'ı okuyup Image Name, Created, Image Type, Data Size, Load Address ve Entry Point bilgilerini yazar. Ayrıca CRC'yi kontrol eder (`Verifying Checksum ... OK`). Değerler senin bulduklarınla aynı olmalı.

### 4.10.6 Linux'u başlat

İşin bitince Linux'u normal şekilde açmak için:

```
U-Boot# boot
```

Ya da kartı yeniden başlat (`reset` veya Power butonu).

### Komut özeti

| Komut | Ne yapar? |
|---|---|
| `ls mmc 0:2 /boot` | SD kartın 2. bölümündeki `/boot` klasörünü listeler |

| Komut | Ne yapar? |
|---|---|
| `load mmc 0:2 0x82000000 /boot/uImage` | uImage'ı DDR'de `0x82000000` adresine yükler |
| `md.b 0x82000000 40` | O adresten başlayarak 64 byte'ı byte byte gösterir |
| `iminfo 0x82000000` | O adresteki uImage header'ını çözüp yazar |
| `boot` | Normal açılışa devam eder (Linux'u başlatır) |
| `reset` | Kartı yeniden başlatır |
| `help` | Tüm U-Boot komutlarını listeler ( `help md` gibi tek komut için de olur) |

---

## 4.11 Kursun Güncel SD Kart İmajları (U-Boot 2022.04 + Kernel 5.10.168)

Bu bölümde kullandığımız Ångström dosyaları kursun ilk sürümünden kalma (2013). Kursun reposundaki `pre-built-images/SD-boot` klasöründe güncel bir set de var:

| Dosya | İçerik | SD kartta nereye? |
|---|---|---|
| `MLO` , `u-boot.img` | U-Boot 2022.04 | BOOT |
| `uImage` | Linux 5.10.168 | BOOT |
| `am335x-boneblack.dtb` | Aynı kernel'in device tree'si | BOOT |
| `uEnv.txt` | Aşağıda | BOOT |

Farklar:

- Kernel ve DTB artık ROOTFS'teki `/boot` klasöründe değil, **BOOT (FAT) bölümünde** duruyor. `load mmc 0:1 ...` ile okunuyor.
- Bu set bir rootfs içermiyor. ROOTFS bölümüne Bölüm 12'de BusyBox ile hazırlayacağımız dosya sistemini koyacağız. Kernel 5.10 ile 2012 tarihli Ångström rootfs'i birlikte çalışmaz.
- Bu U-Boot'ta `bootdelay=0` : geri sayım yok. U-Boot'u durdurmak için kart açılırken **Space'e basılı tut**. Bekleme süresini uzatmak 9.4'ün konusu.

Kursun `uEnv.txt` 'si:

```
console=ttyS0,115200n8
netargs=setenv bootargs console=ttyO0,115200n8 root=/dev/mmcblk0p2 rw rootfstype=ext4 rootwait
debug earlyprintk mem=512M
netboot=echo Booting from microSD ...; setenv autoload no ; load mmc 0:1 ${loadaddr} uImage ;
load mmc 0:1 ${fdtaddr} am335x-boneblack.dtb ; run netargs ; bootm ${loadaddr} - ${fdtaddr}
uenvcmd=run netboot
```

| Parça | Anlamı |
|---|---|
| `rootfstype=ext4` | Rootfs'in dosya sistemi türü. ext4 sürücüsü ext3 bölümleri de bağlayabilir. |
| `rootwait` | Kernel, rootfs'in bulunduğu aygıt (SD kart) hazır olana kadar bekler. MMC aygıtları geç algılandığı için önemli. |
| `debug earlyprintk` | Açılışta daha fazla mesaj basar. Sorun ayıklarken işe yarar. |
| `mem=512M` | Kernel'e kullanacağı RAM miktarını söyler. |
| `${loadaddr}`, `${fdtaddr}` | U-Boot'un kendi varsayılan değişkenleri ( `0x82000000` , `0x88000000` ). Dosyada tanımlanmadıkları halde çalışır. |

> **Uyarı**
>
> Dosyanın ilk satırında `console=ttyS0` tanımlanmış ama `netargs` bu değişkeni kullanmıyor, `console=tty00` yazıyor. Kernel 5.10'da UART0'ın adı `ttyS0` 'dır (3.4). Açılışta `Starting kernel ...` 'dan sonra mesaj göremezsen `netargs` içinde `console=${console}` kullan.

## 4.12 Özet

- SD kart iki bölümden oluşur: **BOOT** (FAT16, `boot` bayraklı: `MLO` , `u-boot.img` , `uEnv.txt` ) ve **ROOTFS** (ext3: kernel, DTB ve Linux dosyaları).
- Dosya adları önemli: RBL tam olarak **`MLO`** 'yu, MLO tam olarak **`u-boot.img`** 'yi arar.
- Kartı SD karttan açmak için güç verirken **S2**'yi basılı tut.
- U-Boot `uEnv.txt` 'deki `uenvcmd` 'yi çalıştırır: kernel ve DTB'yi `mmc 0:2` 'den RAM'e yükler, `bootargs` 'ı ayarlar, `bootm` ile başlatır.
- `uImage` = 64 byte U-Boot header + `zImage` . `md.b` ile byte byte, `iminfo` ile çözülmüş halde okunabilir.

## 4.13 Karşılaşılabilecek Problemler

| Belirti | Olası sebep |
|---|---|
| Minicom'da hiçbir şey yok | TX/RX çapraz bağlı mı? Baud 115200 mü? Hardware Flow Control kapalı mı? (2.8) |
| Sürekli `CCCCCCCC...` | RBL, MLO'yu bulamadı. Dosya adı tam `MLO` mu? BOOT bölümü FAT mı? `boot` bayrağı var mı? |
| `error reading image u-boot.img` | BOOT'ta `u-boot.img` yok veya adı yanlış. |
| Eski Linux açılıyor (Debian vb.) | Kart eMMC'den açıldı. S2 açılış anında basılı değildi. |
| U-Boot `/boot/uImage` bulamıyor | Rootfs ROOTFS bölümüne çıkarılmamış veya yanlış klasöre çıkarılmış. |
| `Kernel panic ... Unable to mount root fs` | ROOTFS bölümü boş veya `sync` yapılmadan kart çıkarılmış. |

# Bölüm 5 — eMMC'den Boot

> **Bu bölümde:**
>
> - eMMC'nin ne olduğunu ve DDR RAM'den farkını öğreneceksin.
> - Güncel bir Debian imajını önce SD karta yazıp sonra **flasher script** ile eMMC'ye aktaracaksın.
> - Karta USB üzerinden SSH ile bağlanmayı göreceksin.
>
> **Ön koşul:** Bölüm 3 — boot sırası ve S2, Bölüm 4 — SD karttan boot.

---

## 5.1 Hatırlatma: eMMC Nedir?

**eMMC (embedded MultiMediaCard)**, BeagleBone Black'in üzerine lehimli olarak gelen dahili depolama birimidir. Aslında içinde bir SD kart vardır ve doğrudan karta lehimlenmiştir. Kapasitesi Rev C'de **4 GB**, eski Rev B'de **2 GB**'tır.

- Kartın **kalıcı depolama alanıdır**. Bilgisayardaki HDD/SSD gibi düşünebilirsin.
- Kart fabrikadan eMMC'de yüklü bir Debian ile gelir. Bu yüzden SD kart takmadan güç verildiğinde Linux açılır.
- SD karta göre **daha hızlı ve güvenilirdir**: 8-bit veri yolu ve dahili denetleyicisi vardır. Kartın üzerinde olduğu için çıkmaz, kaybolmaz, temas sorunu yaşamaz. Bu yüzden ürünlerde (sahadaki cihazlarda) tercih edilir.

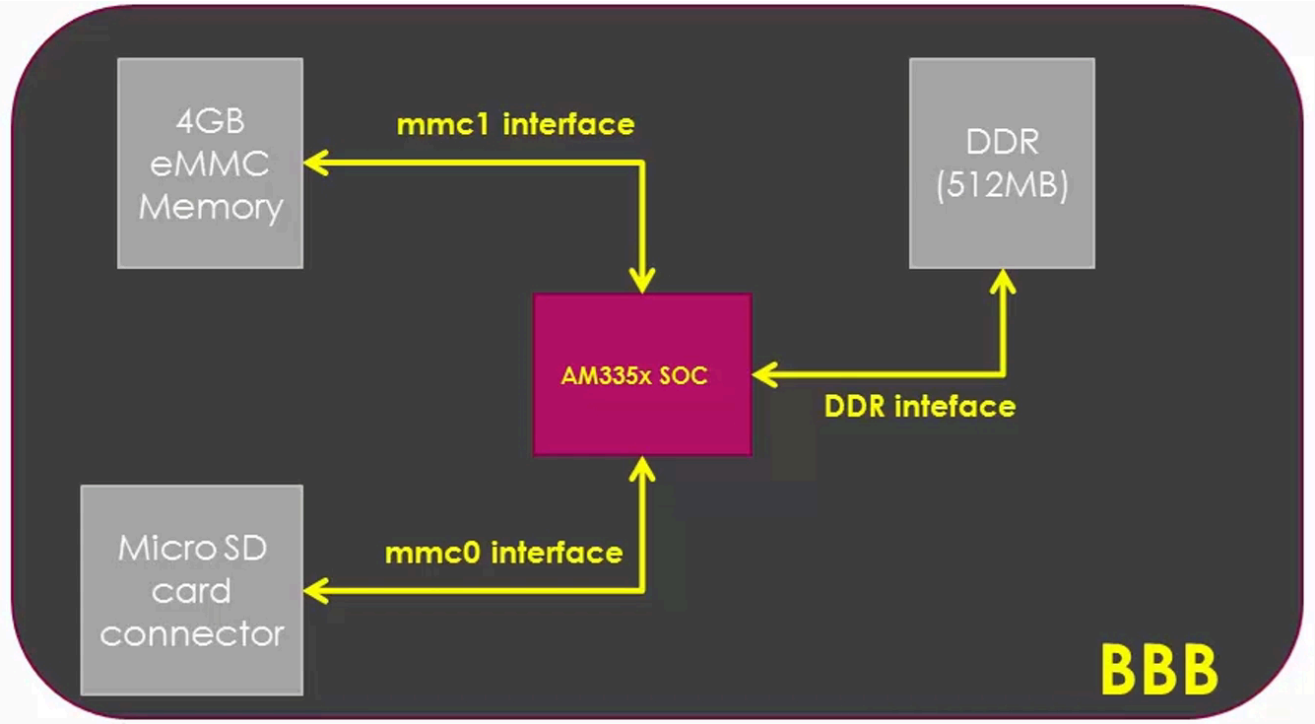
### DDR RAM ile farkı

| | **DDR RAM** | **eMMC** |
|---|---|---|
| **PC karşılığı** | RAM | HDD / SSD |
| **Kalıcılık** | Geçici, güç gidince silinir | Kalıcı |
| **Görevi** | Çalışma alanı | Depolama |
| **BBB'de boyut** | 512 MB DDR3L | 4 GB (Rev C) |

> **Not**
>
> İşlemci kodu doğrudan eMMC'den çalıştırmaz. Açılışta MLO, U-Boot, kernel ve device tree eMMC'den okunur, adım adım RAM'e yüklenir ve orada çalıştırılır.

---

## 5.2 Donanım Bağlantısı




| Bileşen | AM335x'e bağlantısı | Linux'ta (genelde) |
|---|---|---|
| **4 GB eMMC** | **mmc1** arayüzü (8-bit) | `mmcblk1` |
| **microSD kart** | **mmc0** arayüzü | `mmcblk0` |
| **DDR (512 MB)** | DDR arayüzü | - |

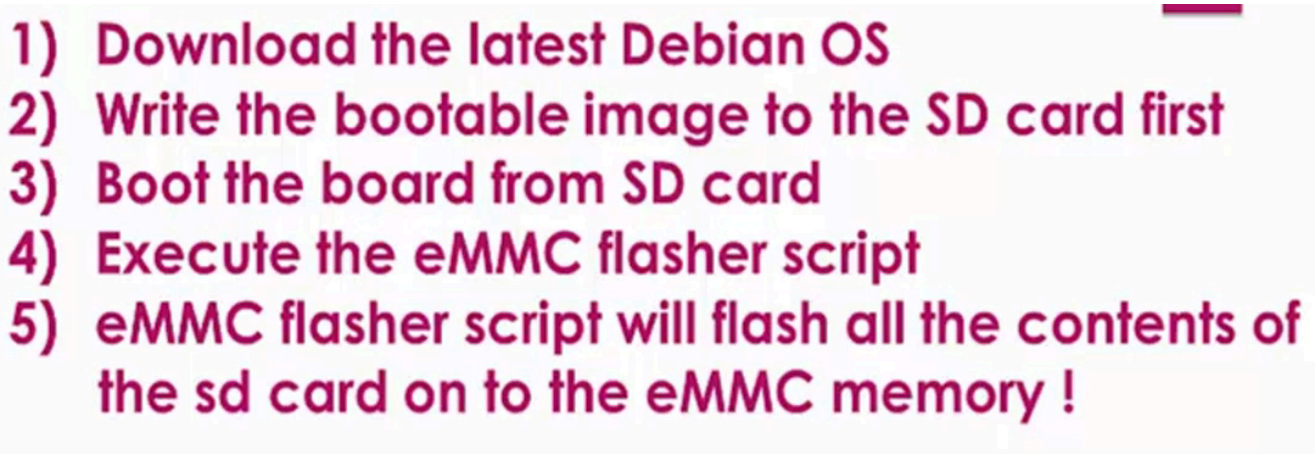
> **Dikkat**
>
> Numaralandırma kernel sürümüne ve device tree'ye göre değişebilir. Bir diske bir şey yazmadan önce `lsblk` **ile mutlaka kontrol et**. Yanlış diske yazmak oradaki sistemi siler.

Varsayılan boot sırasında eMMC ilk sıradadır. eMMC'de geçerli bir MLO varsa kart oradan açılır, SD kart takılı olsa bile ona bakmaz. SD karttan açmak için **güç verirken S2'ye basılı tutulmalıdır** (3.5).

---

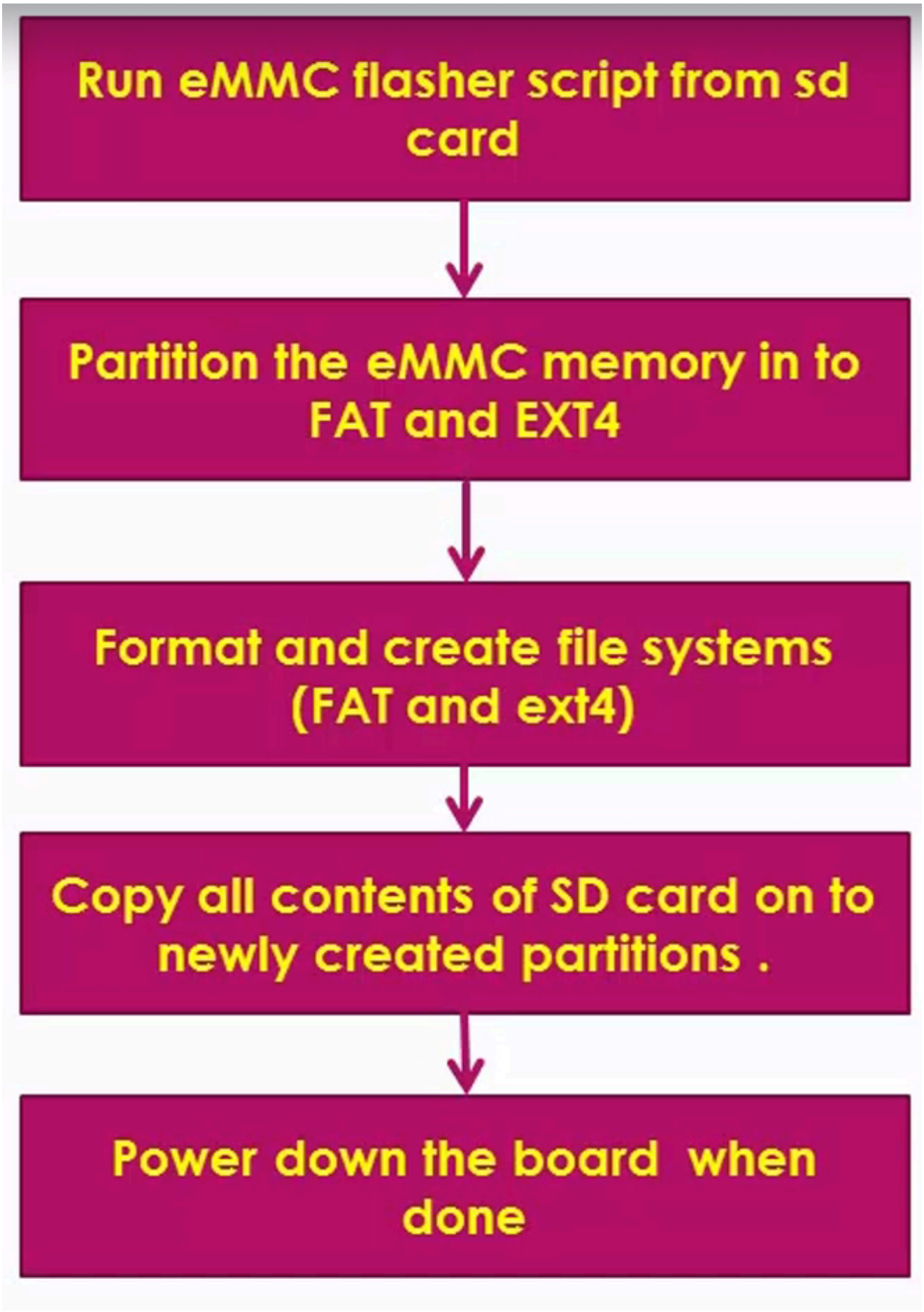
## 5.3 eMMC'ye Debian Yükleme (Teori)

eMMC'ye doğrudan bilgisayardan yazılamaz, çünkü kartın üzerine lehimlidir. Bu yüzden önce **SD karttan** boot edilir, sonra SD karttaki her şey eMMC'ye kopyalanır.




1. En güncel **Debian OS** image'ını indir.
2. Bootable image'ı önce **SD karta** yaz.
3. Kartı **SD karttan boot et** (güç verirken S2'ye basılı tut).
4. **eMMC flasher script**'ini çalıştır.
5. Flasher script, **SD karttaki her şeyi eMMC'ye kopyalar**.

**eMMC flasher script ne yapar?**



1. Script **SD karttan** çalıştırılır.
2. eMMC iki bölüme (partition) ayrılır: **FAT** ve **EXT4**.
3. Bu bölümler formatlanır ve dosya sistemleri oluşturulur: - **FAT**: Boot dosyaları (MLO, u-boot.img vb.) - **EXT4**: Root dosya sistemi (rootfs)
4. SD karttaki tüm içerik yeni oluşturulan bölümlere kopyalanır.
5. İş bitince kart **kapatılır**.

> **Not**
>
> Bu, Bölüm 4'te SD kart için GParted ile elle yaptığımız bölümlendirme ve kopyalama işleminin eMMC için otomatik yapılan halidir.

---

## 5.4 Uygulama: eMMC'yi Güncelleme

### 5.4.1 Kartın IP adresini öğren

Kartı bilgisayara bağladığın yönteme göre IP adresi değişir. Güncel liste: https://beagleboard.org/getting-started

| IP Address | Connection Type | Operating System(s) | Status |
|---|---|---|---|
| 192.168.7.2 | USB | Windows | |
| 192.168.6.2 | USB | Mac OS X, Linux | |
| 192.168.8.1 | WiFi | all | |
| beaglebone.local | all | mDNS enabled | |
| beaglebone-2.local | all | mDNS enabled | |

| Bağlantı | Bilgisayarın işletim sistemi | IP adresi |
|---|---|---|
| USB | Windows | `192.168.7.2` |
| USB | Mac OS X, **Linux** | `192.168.6.2` |
| WiFi | Hepsi | `192.168.8.1` |
| Hepsi (mDNS) | Hepsi | `beaglebone.local` |

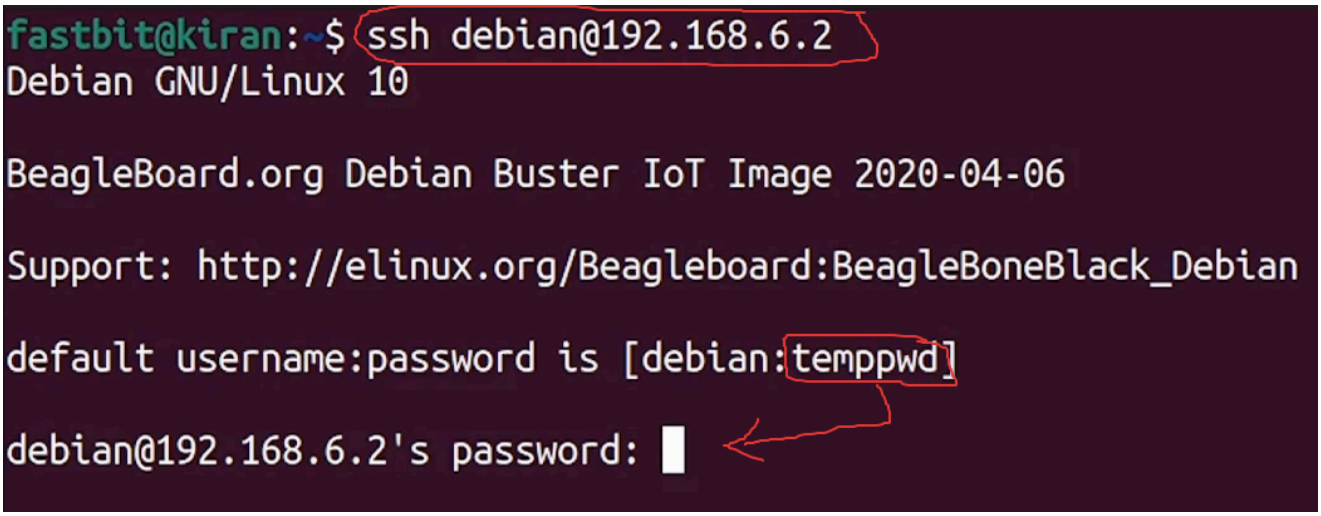
Bağlantıyı test etmek için:

```
ping 192.168.6.2
```

### 5.4.2 SSH ile karta bağlan

> **İpucu**
>
> SSH'ın bilgisayarında yüklü olup olmadığını `ssh -V` komutuyla kontrol edebilirsin. Yüklü değilse Ubuntu'da `sudo apt install openssh-client`, Fedora'da `sudo dnf install openssh-clients` ile kurabilirsin.

```
ssh debian@192.168.6.2
```




Bağlanınca kart karşılama mesajında varsayılan kullanıcı adını ve şifreyi gösterir. `password:` satırına şifreyi yaz:

| Kullanıcı adı | Şifre |
|---|---|
| `debian` | `temppwd` |

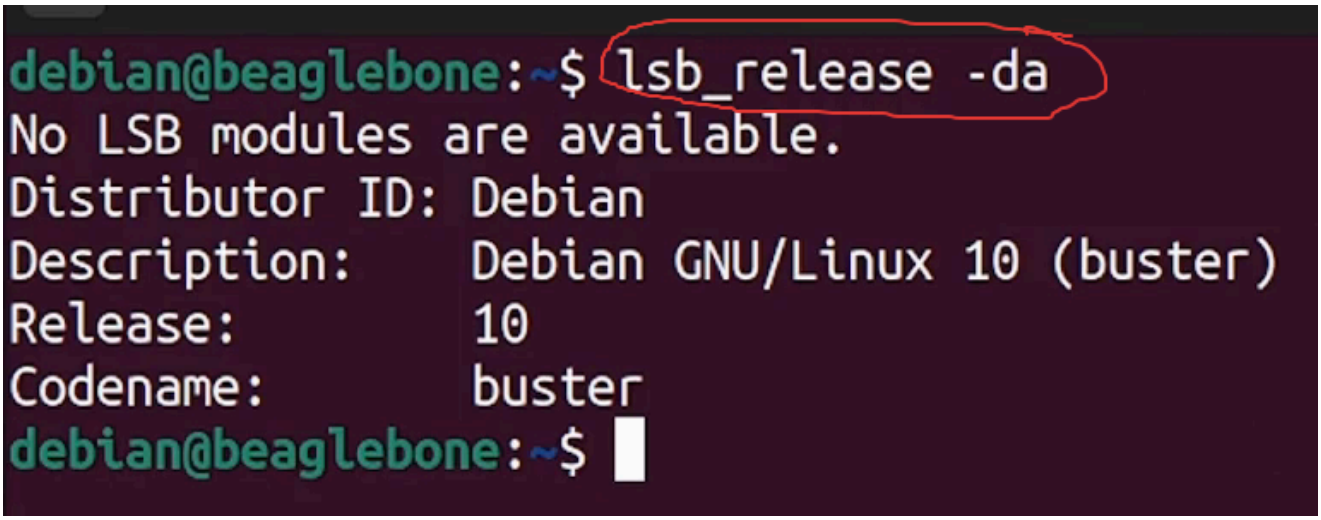
> **Not**
>
> Şifre yazarken ekranda hiçbir karakter görünmez. Bu normaldir, yazmaya devam edip Enter'a bas.

> **Önemli**
>
> `temppwd` şifresi kartla gelen **eski image'larda** geçerlidir. En güncel image'larda varsayılan şifre yoktur. Kullanıcı adı ve şifre, SD karta yazdıktan sonra FAT bölümündeki `sysconf.txt` dosyasından ilk açılıştan önce ayarlanmalıdır (5.4.4).

İlk bağlantıda `Are you sure you want to continue connecting (yes/no)?` diye sorarsa `yes` yaz.

### 5.4.3 Karttaki Debian sürümünü kontrol et

```
lsb_release -da
```

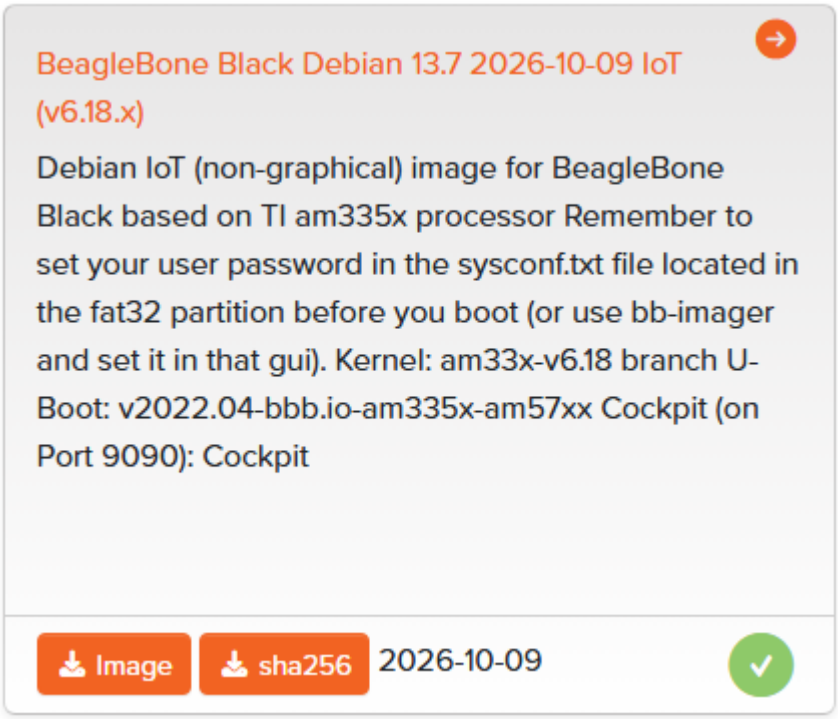



Bu örnekte kartta **Debian 10 (buster)** yüklü.

> **İpucu**
>
> Kartında zaten en güncel Debian image'ı yüklüyse bu bölümün geri kalanını atlayabilirsin.

### 5.4.4 En güncel image'ı indir

Güncel image'lar: https://beagleboard.org/latest-images




Bu notun yazıldığı tarihte en güncel sürüm **Debian 13.7 (2026-10-09, IoT)** idi. Karttaki Debian 10'dan çok daha yeni olduğu için eMMC'yi güncellemek gerekiyor.

- **Image** butonu ile `.img.xz` dosyasını indir.
- İstersen **sha256** ile indirdiğin dosyanın bozulmadığını kontrol edebilirsin:

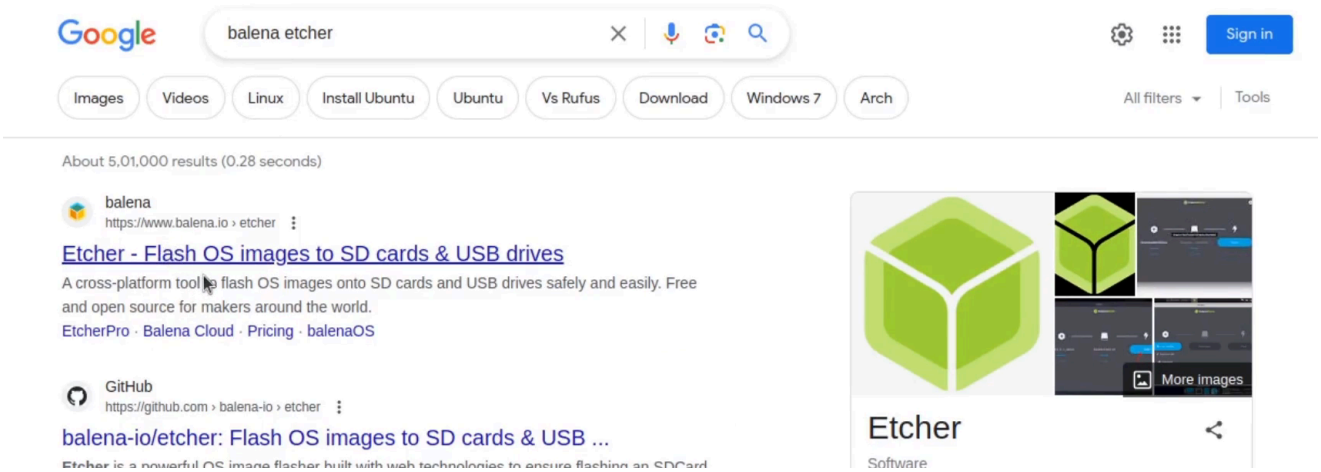
  ```
  sha256sum <indirilen-dosya>.img.xz
  ```

  Çıkan değer sitedeki sha256 değeriyle aynı olmalı.

> **Önemli**
>
> Sitedeki açıklamada da yazdığı gibi, bu image'da kullanıcı şifresi **ilk açılıştan önce** FAT32 bölümündeki `sysconf.txt` dosyasından ayarlanmalıdır.

### 5.4.5 Image'ı SD karta yaz (balenaEtcher)

Bu image doğrudan eMMC'ye yazılamaz. Önce **SD karta** yazılır, ardından kart SD karttan açılıp eMMC'ye aktarılır (5.3). SD karta yazmak için bir programa ihtiyacımız var: **balenaEtcher** (https://etcher.balena.io).



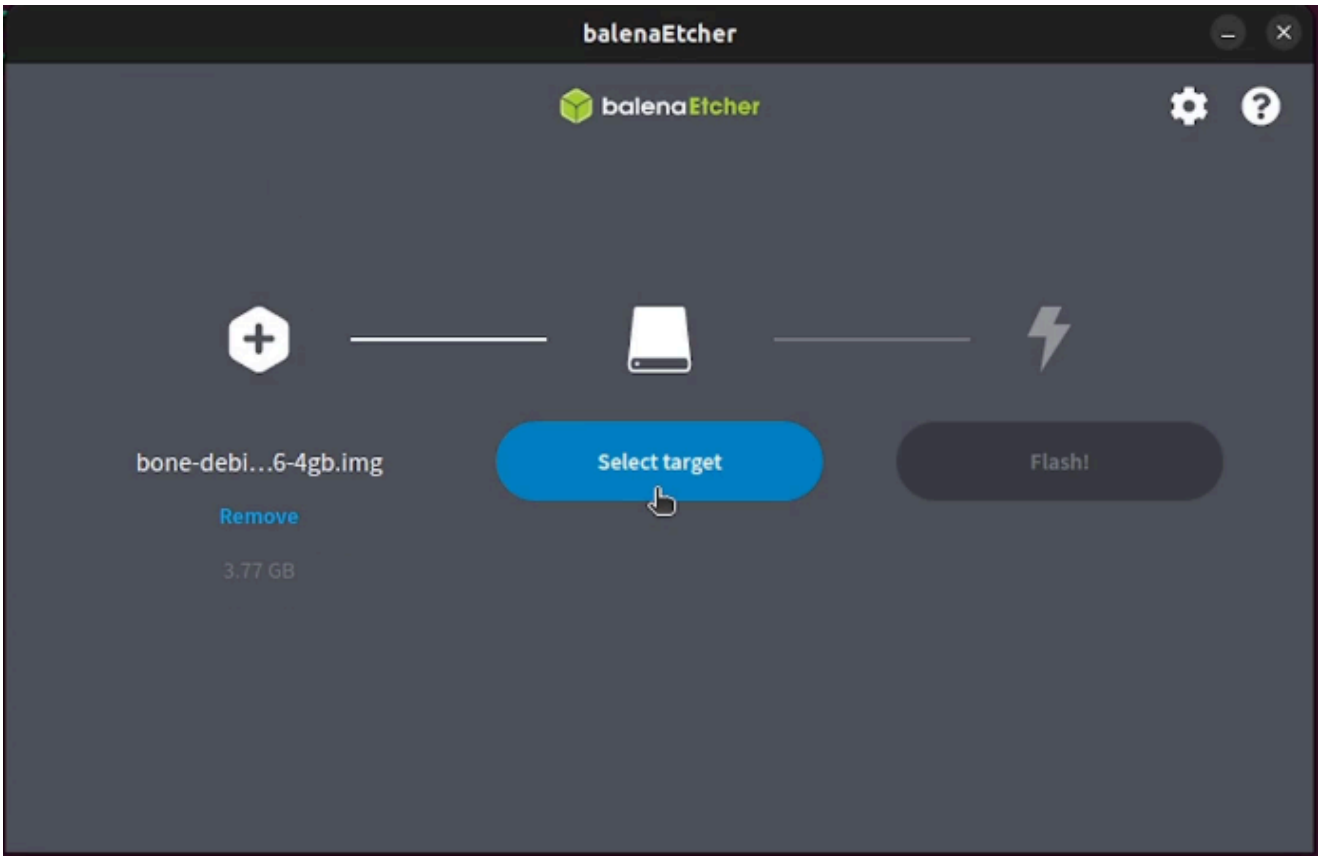

**Neden dosyayı SD karta kopyalamak yetmiyor?**

İndirilen `.img.xz` dosyası normal bir dosya değil, **bütün bir diskin birebir kopyasıdır**:

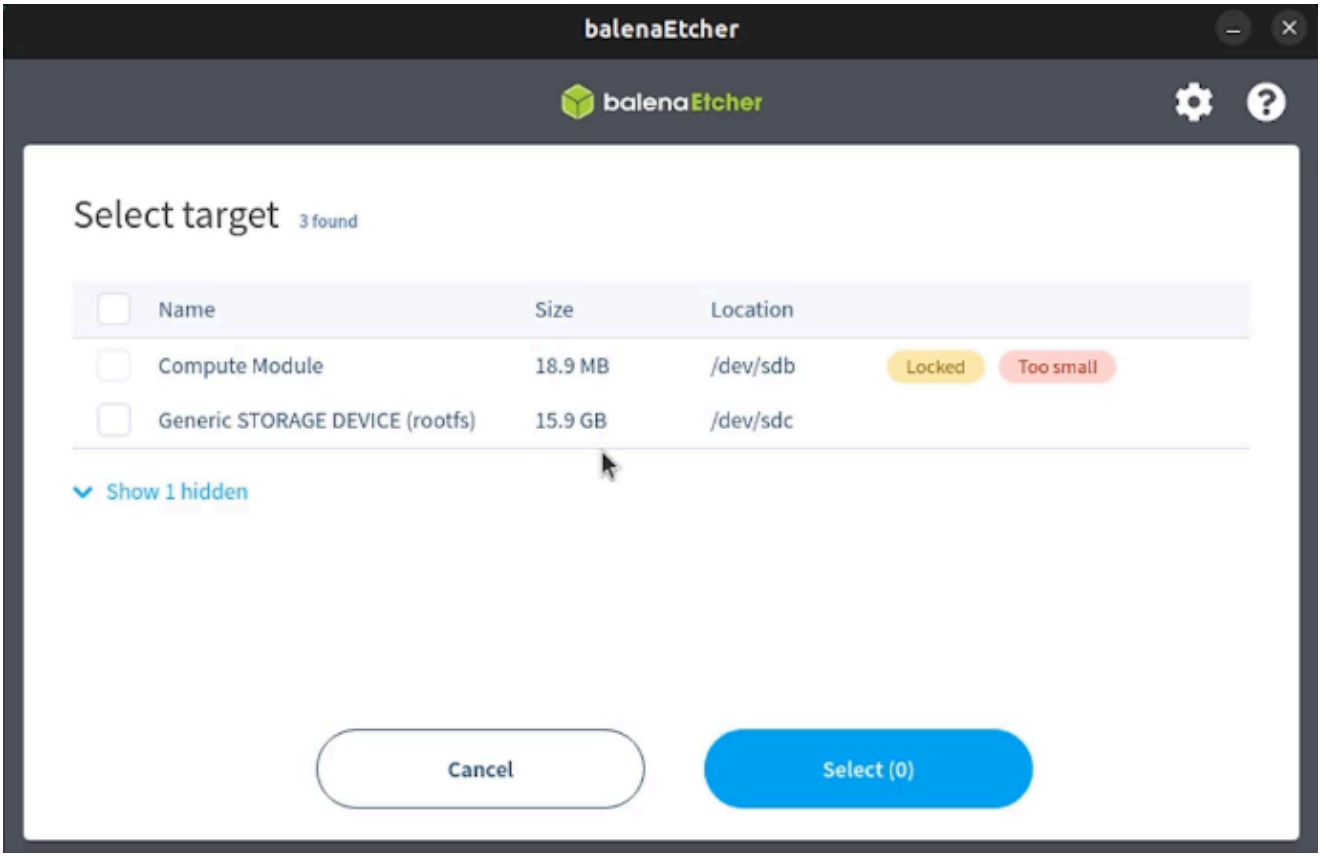
```
Debian image'ı (.img)
├── Bölüm tablosu (partition table)   → diskin hangi kısmı hangi bölüm
├── BOOT bölümü (FAT)                  → MLO, u-boot.img, sysconf.txt ...
└── rootfs bölümü (EXT4)               → Debian'ın tamamı
```

Dosyayı sürükle-bırak ile kopyalarsan SD kartta sadece tek bir büyük `.img` dosyası olur ve kart açılmaz. Etcher ise image'ı **diskin en başından itibaren ham olarak (byte byte) yazar**. Böylece SD kart image'daki bölümlere birebir sahip olur ve boot edilebilir hale gelir.

**Etcher ile yazma adımları**




1. **Flash from file** ile indirdiğin image dosyasını seç (kursta: `bone-debian-10.3-iot-armhf-2020-04-06-4gb.img.xz`).
2. **Select target** ile SD kartı seç. Kart okuyucu bilgisayarda bir USB depolama aygıtı olarak görünür.
3. **Flash!** butonuna bas. Yazma ve doğrulama (verify) bitene kadar bekle.




Bu örnekte iki aygıt görünüyor:

| Aygıt | Boyut | Konum | Açıklama |
|---|---|---|---|
| Compute Module | 18.9 MB | `/dev/sdb` | **Locked / Too small**, seçilemez |
| **Generic STORAGE DEVICE** | **15.9 GB** | `/dev/sdc` | **SD kart** → bunu seç |

**Hangisi SD kart? (`dmesg`)**

Listede birden fazla aygıt çıkarsa ve hangisinin SD kart olduğundan emin değilsen, SD kartı taktıktan hemen sonra terminalde şunu çalıştır:

```
sudo dmesg | tail -20
```

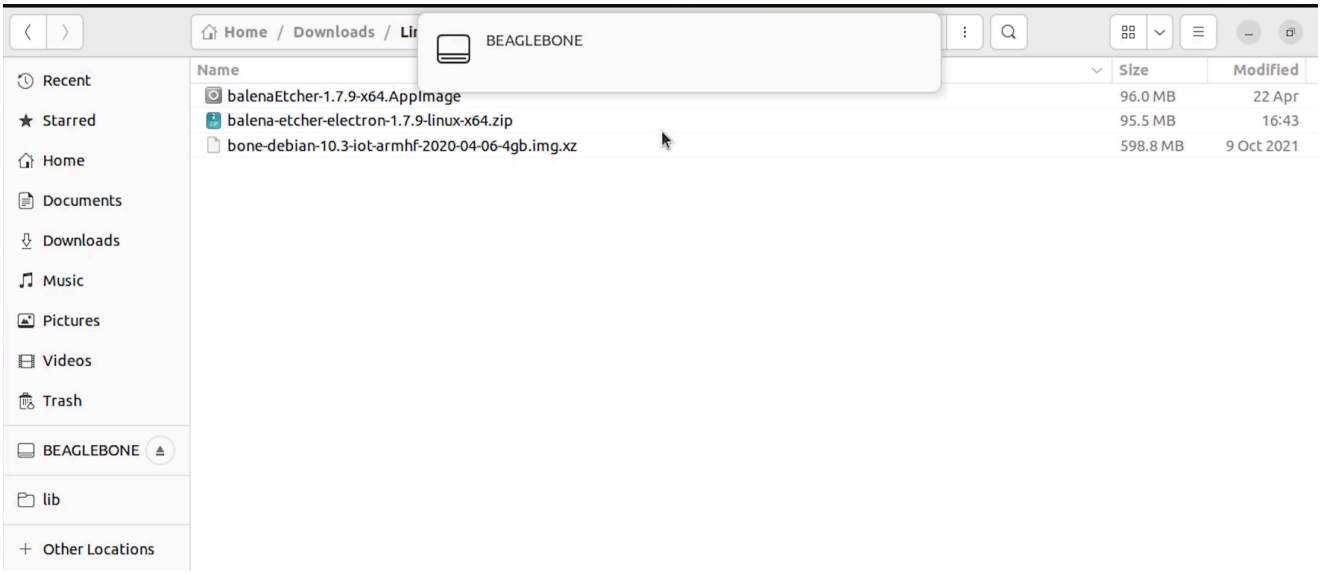
```
[ 5072.989201] rtw_8822be 0000:02:00.0: timed out to flush queue 1
[ 5073.129223] rtw_8822be 0000:02:00.0: timed out to flush queue 1
[ 5093.550258] usb 1-3: new high-speed USB device number 6 using xhci_hcd
[ 5093.886370] usb 1-3: device descriptor read/64, error -71
[ 5094.149894] usb 1-3: New USB device found, idVendor=05e3, idProduct=0751, bcdDevice=14.04
[ 5094.149911] usb 1-3: New USB device strings: Mfr=3, Product=4, SerialNumber=0
[ 5094.149921] usb 1-3: Product: USB Storage
[ 5094.149928] usb 1-3: Manufacturer: USB Storage
[ 5094.159962] usb-storage 1-3:1.0: USB Mass Storage device detected
[ 5094.160580] scsi host4: usb-storage 1-3:1.0
[ 5095.168034] scsi 4:0:0:0: Direct-Access     Generic  STORAGE DEVICE   1404 PQ: 0 ANSI: 6
[ 5095.168770] sd 4:0:0:0: Attached scsi generic sg2 type 0
[ 5095.356429] sd 4:0:0:0: [sdc] 31116288 512-byte logical blocks: (15.9 GB/14.8 GiB)
[ 5095.357604] sd 4:0:0:0: [sdc] Write Protect is off
[ 5095.357614] sd 4:0:0:0: [sdc] Mode Sense: 21 00 00 00
[ 5095.358923] sd 4:0:0:0: [sdc] Write cache: disabled, read cache: enabled, doesn't support DPO or FUA
[ 5095.372154]  sdc: sdc1
[ 5095.375795] sd 4:0:0:0: [sdc] Attached SCSI removable disk
[ 5095.843566] EXT4-fs (sdc1): mounted filesystem with ordered data mode. Opts: errors=remount-ro. Quota m
ode: none.
```

Son satırlarda yeni takılan aygıtın adı ve boyutu görünür. Bu örnekte `[sdc] ... (15.9 GB/14.8 GiB)` satırı, SD kartın `/dev/sdc` olduğunu gösteriyor.

> **İpucu**
>
> `lsblk` komutu da aygıtları boyutlarıyla listeler. Harf ( `sdb` , `sdc` ...) bilgisayardan bilgisayara değişir. Her seferinde boyuta bakarak kontrol et.

### 5.4.6 Kartı SD karttan boot et

1. Kartın **gücünü kes**.
2. SD kartı BeagleBone'a **tak**.
3. Güç kablosunu tak.
4. **Power** butonuna LED'ler sönene kadar basılı tut, LED'ler sönünce bırak. Kart artık kapalı.
5. **S2** butonunu basılı tut ve bu sırada **Power** butonuna bir kez basıp bırak.
6. S2'yi **2-3 saniye** daha basılı tut, sonra bırak.
7. Kart SD karttan açılmaya başlar. Bu sırada LED'ler hızlı hızlı yanıp söner.



Bilgisayarında **BEAGLEBONE** adında bir sürücü göründüyse kart açılmıştır. Şimdi 5.4.2'deki SSH komutuyla tekrar bağlan:

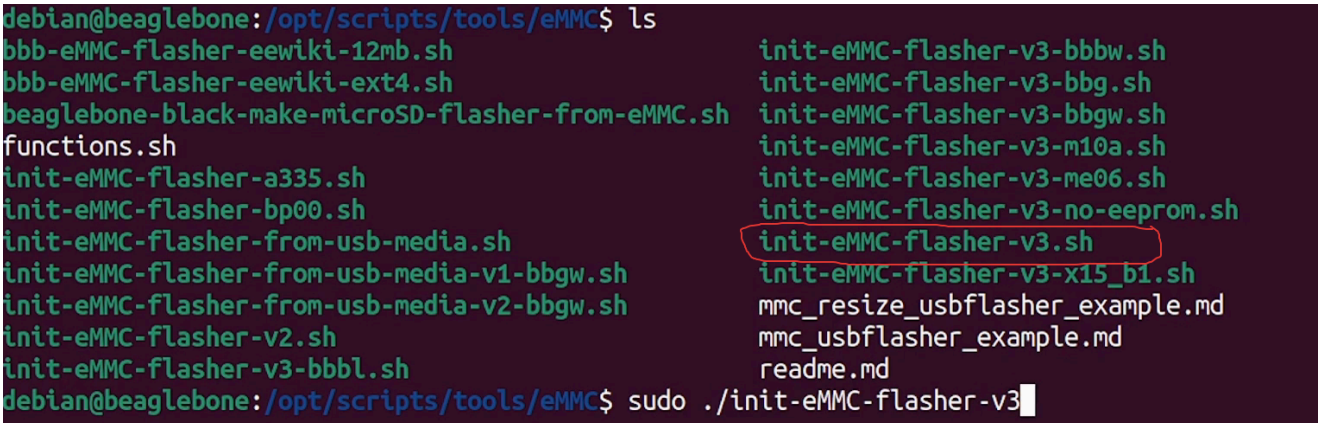
```
ssh debian@192.168.6.2
```

> **Not**
>
> Bağlanırken `REMOTE HOST IDENTIFICATION HAS CHANGED!` uyarısı alırsan bu normaldir. Kart farklı bir sistemden (SD karttan) açıldığı için SSH anahtarı değişmiştir. Eski kaydı silip tekrar bağlan:
>
> ```
> ssh-keygen -R 192.168.6.2
> ```

### 5.4.7 eMMC flasher script'ini çalıştır

Flasher script'leri `/opt/scripts/tools/eMMC` klasöründe bulunur:

```
cd /opt/scripts/tools/eMMC
ls
```



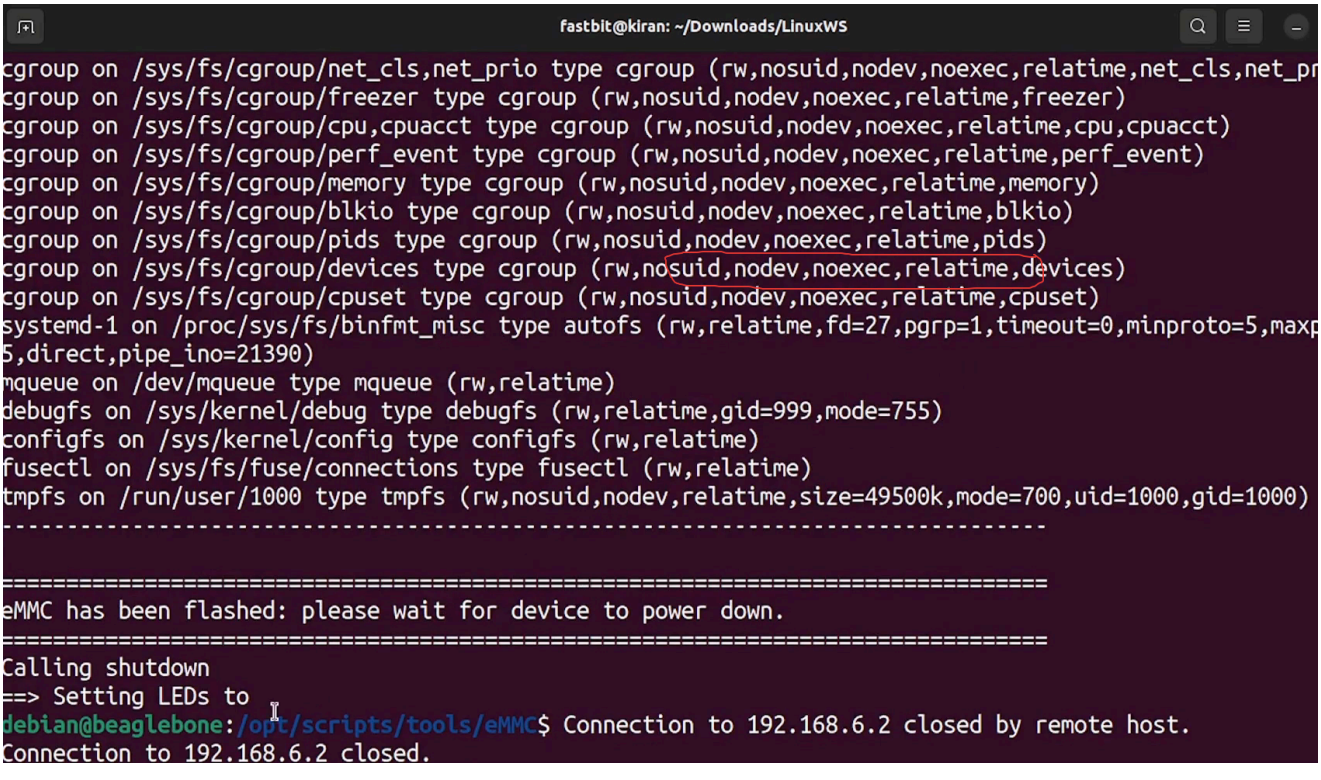

Listede birçok script var. BeagleBone Black için kullanılan **`init-eMMC-flasher-v3.sh`**:

```
sudo ./init-eMMC-flasher-v3.sh
```

> **Dikkat**
>
> Bu script eMMC'deki **her şeyi siler** ve yerine SD karttaki sistemi yazar. eMMC'de saklamak istediğin dosya varsa önce yedekle.

> **Not**
>
> Bu klasör ve script adı kursta kullanılan **Debian 10.3** image'ına göredir. Daha yeni image'larda flasher farklı bir yerde veya farklı bir adla olabilir. Klasör yoksa image'ın açıklamasına bakarak flasher yöntemini kontrol et.

Script çalışırken 5.3'te anlatılan adımları sırayla yapar: bölümlendirme → formatlama → kopyalama. Bu birkaç dakika sürer.




Flash işlemi başarılıysa şu mesajı görürsün:

```
eMMC has been flashed: please wait for device to power down.
```

Ardından kart kendini kapatır ve SSH bağlantısı kopar (`Connection to 192.168.6.2 closed by remote host.`). Bu normaldir.

### 5.4.8 eMMC'den boot et

1. SD kartı BeagleBone'dan **çıkar**. Artık ona ihtiyacın yok.
2. **Power** butonuna bir kez bas (S2'ye **basmadan**). Kart artık eMMC'den açılır.

3. SSH ile tekrar bağlan:

```
ssh debian@192.168.6.2
```

4. Yeni sürümün yüklendiğini kontrol et:

```
lsb_release -da
```

---

## 5.5 Özet

- eMMC kartın kalıcı, dahili deposudur (MMC1). DDR ise geçici çalışma belleğidir.
- eMMC'ye doğrudan yazılamaz: imaj önce **balenaEtcher** ile SD karta yazılır, kart **S2** ile SD karttan açılır, **flasher script** SD karttaki sistemi eMMC'ye kopyalar.
- USB üzerinden kartın adresi Linux'ta `192.168.6.2`, Windows'ta `192.168.7.2`. Bağlantı: `ssh debian@192.168.6.2`.

## 5.6 Karşılaşılabilecek Problemler

| Belirti | Sebep | Çözüm |
|---|---|---|
| `ping 192.168.6.2` cevap vermiyor | Kart henüz açılmadı ya da Windows'tasın | 1–2 dakika bekle. Windows'ta adres `192.168.7.2`. `beaglebone.local` da denenebilir. |
| `REMOTE HOST IDENTIFICATION HAS CHANGED!` | Kart farklı bir sistemden açıldı | `ssh-keygen -R 192.168.6.2` |
| Şifre kabul edilmiyor | Yeni image'da varsayılan şifre yok | SD karttaki `sysconf.txt` ile kullanıcı/şifre belirle (5.4.4) |
| `/opt/scripts/tools/eMMC` yok | Image sürümü farklı | Image'ın açıklamasındaki flasher yöntemini kullan |
| Kart flash'tan sonra yine eski sistemle açılıyor | SD kart hâlâ takılı ve S2'ye basıldı / flash yarıda kaldı | SD kartı çıkar, S2'ye basmadan aç. Gerekirse flash'ı tekrarla. |

# Kısım III
# U-Boot ile Çalışmak

# Bölüm 6 — U-Boot Komut Satırı ve uEnv.txt

> **Bu bölümde:**
>
> - `uEnv.txt` dosyasının **satır satır ne yaptığını** anlayacaksın.
> - Bu dosyayı SD karta kopyalamadan, **seri port (UART) üzerinden** doğrudan U-Boot'a göndereceksin ( `loady` , `env import` ).
> - Kursun challenge'ı: boot imajlarını eMMC yerine **SD karttan (MMC0)** yükleyeceksin. Ek alıştırmada da tersini yapıp rootfs'i **eMMC'den** bağlayacaksın.
> - **ROOTFS** bölümünde hangi klasör ve dosyaların olduğunu ve bunlara **hangi komutlarla** ulaşacağını göreceksin.
>
> **Ön koşul:** Bölüm 4 — SD karttan boot, Bölüm 5 — eMMC'deki Debian.

> **Not**
>
> Bu bölümdeki terminal görüntüleri temsilidir. Senin kartında U-Boot sürümü, dosya boyutları ve bölüm boyutları farklı görünebilir.

---

## 6.1 Temel Kavramlar

| Kavram | Ne işe yarar? |
|---|---|
| `uEnv.txt` | U-Boot ortam değişkenlerini **metin olarak** tanımlayan dosya. Her satır `isim=değer` biçimindedir. U-Boot'un bu dosyayı açılışta otomatik okuyup okumaması, U-Boot'un yapılandırmasına bağlıdır (BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt` 'yi arar). |
| **Ortam değişkeni (environment)** | U-Boot'un ayarları ve makroları. `printenv` ile görülür, `setenv` ile değiştirilir, `saveenv` ile kalıcı kaydedilir. |
| **Minicom** | Bilgisayardan seri port (UART) üzerinden karta bağlanmayı sağlayan terminal programı (Bölüm 2). |
| `loady` | U-Boot'ta **YMODEM** protokolüyle seri porttan dosya alıp RAM'e yazan komut. |
| `env import -t <adres> <boyut>` | RAM'deki `isim=değer` metnini okuyup U-Boot'un **aktif** ortamına ekler. `-t` "düz metin" demektir, `<boyut>` metnin bayt cinsinden uzunluğudur. |

Bütün akış şöyle:

```
Bilgisayar (uEnv.txt) ──YMODEM / UART──► RAM 0x82000000 ──env import──► U-Boot ortamı ──boot──► Linux
```

---

## 6.2 `uEnv.txt` Satır Satır

Kullanacağımız dosya bu:

```
mypcip=setenv serverip 192.168.1.2
ipaddr=192.168.27.1
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000 uImage;load mmc
0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
```

### `mypcip` ve `ipaddr` — ağ ayarları

| Satır | Anlamı |
|---|---|
| `mypcip=setenv serverip 192.168.1.2` | Bir **makro**. Kendi başına hiçbir şey yapmaz; `run mypcip` yazınca içindeki komut çalışır ve `serverip` (TFTP/NFS sunucusunun, yani bilgisayarın IP'si) `192.168.1.2` olur. |
| `ipaddr=192.168.27.1` | Kartın kendi statik IP adresi. |

> **Uyarı**
>
> Bu örnekte kart `192.168.27.x`, sunucu `192.168.1.x` ağında. Ağ geçidi ( `gatewayip` ) tanımlanmadığı sürece TFTP/NFS çalışmaz. Ağdan boot yapacaksan iki adresi **aynı alt ağda** seç (Bölüm 8). SD karttan boot için bu iki satır kullanılmaz.

### `bootargs` — kernel'e verilen parametreler

`bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw`

| Parça | Anlamı |
|---|---|
| `console=ttyO0,115200` | Linux mesajları UART0'a (J1 header) 115200 baud ile yazılsın. `ttyO0` eski OMAP kernel'lerindeki addır; yeni kernel'lerde `ttyS0` olur. |
| `root=/dev/mmcblk0p2` | Root filesystem: MMC aygıtı **0**, bölüm **2** ( `p2` ). |
| `rw` | Rootfs okuma-yazma modunda bağlansın. |

### `bootcmd` — otomatik açılış komutu

`bootcmd` , `;` ile ayrılmış komutlardan oluşur. U-Boot bunları sırayla çalıştırır:

| Sıra | Komut | Ne yapar? |
|---|---|---|
| 1 | `echo "****Booting from memory****"` | Ekrana mesaj basar (hangi yoldan açıldığını görmek için). |

| Sıra | Komut | Ne yapar? |
|---|---|---|
| 2 | `load mmc 0:1 0x82000000 uImage` | `mmc 0:1` = SD kart, 1. bölüm (BOOT). Oradaki `uImage` dosyasını RAM'de `0x82000000` adresine yükler. |
| 3 | `load mmc 0:1 0x88000000 am335x-boneblack.dtb` | Aynı bölümdeki device tree dosyasını (DTB) `0x88000000` adresine yükler. |
| 4 | `bootm 0x82000000 - 0x88000000` | Kernel'i başlatır. Ortadaki `-` "initramfs yok" demektir, son adres DTB'nin adresidir. |

```
RAM (DDR, 0x80000000'dan başlar)
├── 0x82000000  →  uImage   (kernel)
└── 0x88000000  →  am335x-boneblack.dtb
```

> **Not**
>
> `mmc 0` ↔ `/dev/mmcblk0` eşleşmesi çoğu kurulumda doğrudur ama **garanti değildir**. U-Boot tarafında `mmc list`, Linux tarafında `lsblk` ile kontrol et (6.5).

---

## 6.3 Uygulama: `uEnv.txt`'yi Seri Porttan Göndermek

Normalde `uEnv.txt` SD kartın BOOT bölümüne kopyalanır. Her değişiklikte SD kartı çıkar-tak yapmak yerine dosyayı **seri porttan** gönderip anında deneyebiliriz.

### Adım 1 — Bilgisayarda dosyayı hazırla ve Minicom'u aç

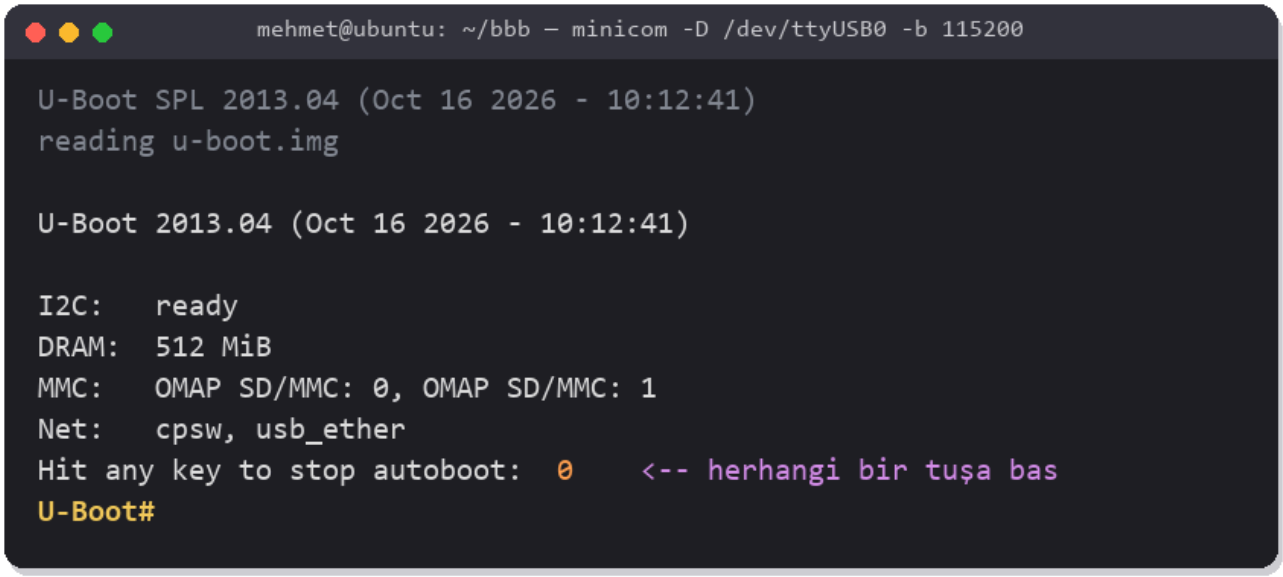
Dosyanın içeriğini ve **boyutunu** kontrol et, sonra Minicom ile karta bağlan:

```
mehmet@ubuntu: ~/bbb
mehmet@ubuntu:~/bbb$ cat uEnv.txt
mypcip=setenv serverip 192.168.1.2
ipaddr=192.168.27.1
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000
uImage;load mmc 0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
mehmet@ubuntu:~/bbb$ ls -l uEnv.txt
-rw-rw-r-- 1 mehmet mehmet 269 Eki 16 17:05 uEnv.txt
mehmet@ubuntu:~/bbb$ sudo minicom -D /dev/ttyUSB0 -b 115200
```

> **İpucu**
>
> Windows'ta port adı `/dev/ttyUSB0` yerine `COM3` gibi olur (2.6).

### Adım 2 — U-Boot'u durdur

Kartı aç. `Hit any key to stop autoboot` yazısı çıkınca **bir tuşa bas**. `U-Boot#` komut satırı gelmeli:




### Adım 3 — `loady` ile dosyayı RAM'e al

U-Boot'ta YMODEM alımını başlat:

```
U-Boot# loady 0x82000000
```

U-Boot `C` karakterleri basarak beklemeye başlar. Şimdi Minicom'da:

1. `Ctrl-A` → `S` (Send files)
2. Protokol olarak **ymodem** seç
3. `uEnv.txt` dosyasını seçip gönder

Transfer bitince U-Boot alınan dosyanın boyutunu yazar:

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# loady 0x82000000
## Ready for binary (ymodem) download to 0x82000000 at 115200 bps...
C
+-----------[ YMODEM upload - Press CTRL-C to quit ]-----------+
| Sending: uEnv.txt                                            |
| Bytes Sent:    269   BPS:189                                 |
| Sending:                                                     |
| Ymodem sectors/kbytes sent:   0/ 0k                          |
| Transfer complete                                            |
+--------------------------------------------------------------+
## Total Size      = 0x0000010D = 269 Bytes
U-Boot#
```

> **Not**
>
> `## Total Size = 0x0000010D = 269 Bytes` satırı önemli: Bu değer otomatik olarak **`filesize`** değişkenine yazılır. Bir sonraki adımda boyutu elle yazmak yerine `${filesize}` kullanabiliriz.

### Adım 4 — `env import` ile ortama aktar ve kontrol et

```
U-Boot# env import -t 0x82000000 ${filesize}
U-Boot# printenv bootargs
U-Boot# printenv bootcmd
```

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# env import -t 0x82000000 ${filesize}
U-Boot# printenv bootargs
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
U-Boot# printenv bootcmd
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000
uImage;load mmc 0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
U-Boot# run mypcip
U-Boot# printenv serverip
serverip=192.168.1.2
U-Boot#
```

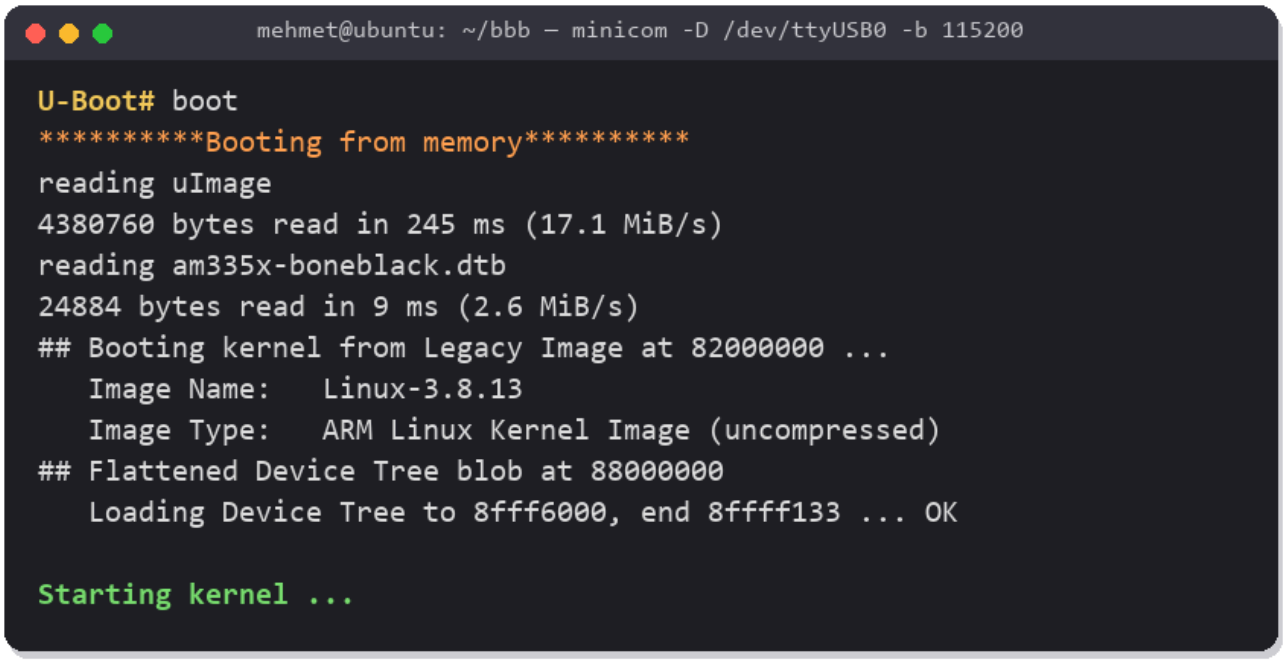
`printenv` çıktısı `uEnv.txt` 'deki değerlerle aynıysa aktarım başarılıdır. `run mypcip` ile makronun da çalıştığını görebilirsin.

> **Uyarı**
>
> `${filesize}` yerine boyutu elle yazarsan **hex** olarak yaz ( `0x10d` veya `10d` ). U-Boot sayıları varsayılan olarak hex kabul eder; `269` yazarsan `0x269` = 617 bayt okur ve dosyanın sonundaki çöp veriyi de içe aktarmaya çalışır.

### Adım 5 — Boot et

```
U-Boot# boot
```

`boot` komutu `bootcmd` 'yi çalıştırır. Ekrana bizim `echo` mesajımız gelmeli:




### Dikkat edilecekler

| Konu | Açıklama |
|---|---|
| **Adres çakışması** | `uEnv.txt` `0x82000000` 'a yüklendi; `bootcmd` de kernel'i aynı adrese yüklüyor. Sorun değil, çünkü `env import` dosyayı önce okuyup bitiriyor, kernel **sonra** üzerine yazılıyor. Sıra ters olursa kernel bozulur. |
| **Geçicilik** | `env import` sadece RAM'deki aktif ortamı değiştirir. Kart yeniden açılınca değişiklikler kaybolur. |
| **Kalıcı yapmak** | Her şeyin doğru olduğundan emin olduktan sonra `saveenv` ile kaydedilebilir. Bu komut kalıcı U-Boot ortamının üzerine yazar; geri dönmek için `env default -a` + `saveenv` gerekir. |

| Konu | Açıklama |
|---|---|
| **Otomatik okumada `bootcmd`** | BBB'nin varsayılan U-Boot'u SD karttaki `uEnv.txt`'yi kendi `bootcmd`'si **çalışırken** okur ve `uenvcmd` değişkenini arar. Bu yüzden SD karttaki `uEnv.txt`'de `bootcmd` tanımlamak o açılışı etkilemez; orada `uenvcmd` kullanılır (4.8'deki örnek gibi). Seri porttan yüklemede ise `boot` komutunu biz verdiğimiz için `bootcmd` çalışır. |

---

## 6.4 Challenge: Boot İmajlarını MMC0'dan (SD Kart) Yüklemek

### Görev

Kursun bu bölümdeki challenge'ı:

> *Derste boot imajları (uImage ve DTB) **MMC1**'den (eMMC) yüklendi, dosya sistemi olarak **MMC0**'daki (microSD) rootfs kullanıldı. `uEnv.txt`'yi, boot imajları da **MMC0**'dan (microSD) yüklenecek şekilde değiştir.*

| | Önce | Sonra |
|---|---|---|
| Kernel + DTB | eMMC ( `mmc 1` ) | **SD kart ( `mmc 0` )** |
| Rootfs | SD kart, 2. bölüm ( `/dev/mmcblk0p2` ) | Değişmez |

### Çözüm

Değişen tek şey `load` komutlarındaki **aygıt numarası** (ve dosyanın o aygıttaki yeri). `bootargs` değişmez, çünkü rootfs zaten SD kartta.

**1) Dosyaları SD karta koy.** `uImage` ve `am335x-boneblack.dtb`'yi SD kartın BOOT bölümüne kopyala:

```
cp uImage am335x-boneblack.dtb /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT /media/$USER/ROOTFS
```

**2) `load` satırlarını `mmc 0:1`'e çevir.**

```
eski:  load mmc 1:2 0x82000000 /boot/uImage                  (eMMC, 2. bölüm)
yeni:  load mmc 0:1 0x82000000 uImage                        (SD kart, 1. bölüm: BOOT)

eski:  load mmc 1:2 0x88000000 /boot/am335x-boneblack.dtb
yeni:  load mmc 0:1 0x88000000 am335x-boneblack.dtb
```

6.2'deki dosyanın `bootcmd` satırı tam olarak bunu yapıyor:

```
bootcmd=echo "**********Booting from memory**********";load mmc 0:1 0x82000000 uImage;load mmc
0:1 0x88000000 am335x-boneblack.dtb;bootm 0x82000000 - 0x88000000
```

**3) Doğrula.** U-Boot'u durdurup dosyaların orada olduğunu gör, sonra boot et:

```
=> ls mmc 0:1
=> boot
```

Ekranda `**********Booting from memory**********` yazısından sonra `reading uImage` görüyorsan imajlar SD karttan geliyor.

> **İpucu**
>
> SD kart dosyayı U-Boot'a FAT bölümünden de ext bölümünden de verebilir. `uImage` 'ı BOOT (FAT) bölümüne koymak, dosyayı Windows'tan bile kopyalayabilmeni sağlar. ROOTFS bölümündeki `/boot` klasöründe tutmak istersen `load mmc 0:2 ... /boot/uImage` yaz.

---

## 6.5 Ek Alıştırma: Kernel ve DTB SD Karttan, Rootfs eMMC'den

### Görev

Challenge'ın tersi: kernel ve DTB SD karttan gelsin, ama rootfs eMMC'deki Debian olsun.

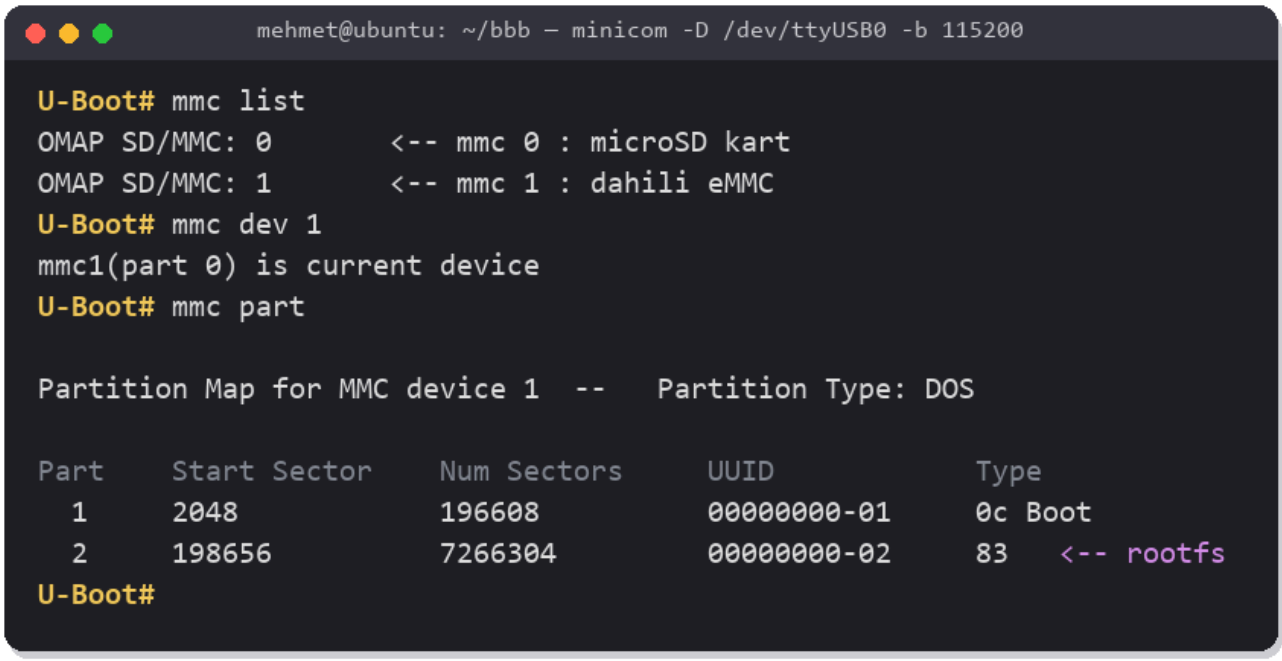
| Ne? | Nereden? |
|---|---|
| Kernel ( `uImage` ) | **MMC0** — microSD kart, 1. bölüm |
| Device tree ( `.dtb` ) | **MMC0** — microSD kart, 1. bölüm |
| Root filesystem | **MMC1** — dahili eMMC, 2. bölüm |

```
microSD (mmc 0)                      eMMC (mmc 1)
├── p1 BOOT  → uImage, .dtb  ──┐     ├── p1 BOOT
└── p2 ROOTFS                  │     └── p2 ROOTFS  ◄── root=/dev/mmcblk1p2
                               ▼
                    U-Boot RAM'e yükler → kernel başlar → rootfs'i eMMC'den bağlar
```

### Adım 1 — Aygıt numaralarını doğrula

Önce varsayımımızı kontrol edelim: `mmc 0` gerçekten SD kart, `mmc 1` gerçekten eMMC mi ve eMMC'de 2. bölüm var mı?

```
U-Boot# mmc list
U-Boot# mmc dev 1
U-Boot# mmc part
```




eMMC'de `83` (Linux) tipinde 2. bölüm görünüyorsa rootfs için doğru yerdeyiz.

### Adım 2 — Sadece `bootargs`'ı değiştir

Kernel ve DTB hâlâ SD karttan yükleneceği için `bootcmd` **aynı kalır**. Değişen tek şey, kernel'e "rootfs nerede?" bilgisini veren `bootargs`:

| | **Değer** |
|---|---|
| **Eski** — rootfs SD kartta | `bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw` |
| **Yeni** — rootfs eMMC'de | `bootargs=console=ttyO0,115200 root=/dev/mmcblk1p2 rw` |

Bunu iki yolla yapabilirsin:

- **a)** `uEnv.txt`'deki satırı değiştirip 6.3'teki gibi tekrar gönder.
- **b)** Doğrudan U-Boot'ta `setenv` ile değiştir:

```
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# boot
```

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# printenv bootargs
bootargs=console=ttyO0,115200 root=/dev/mmcblk0p2 rw
U-Boot# setenv bootargs console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# printenv bootargs
bootargs=console=ttyO0,115200 root=/dev/mmcblk1p2 rw
U-Boot# boot
**********Booting from memory**********
reading uImage
...
Starting kernel ...
```

### Adım 3 — Linux'ta sonucu doğrula

Kart açılınca rootfs'in gerçekten eMMC'den bağlandığını kontrol et:

```
cat /proc/cmdline   # kernel'e giden bootargs
lsblk               # hangi bölüm / olarak bağlı?
findmnt /           # rootfs'in kaynağı
```

```
root@beaglebone: ~ (seri konsol)

root@beaglebone:~# cat /proc/cmdline
console=ttyO0,115200 root=/dev/mmcblk1p2 rw
root@beaglebone:~# lsblk
NAME         MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
mmcblk0      179:0    0  7.4G  0 disk
|-mmcblk0p1  179:1    0   96M  0 part
`-mmcblk0p2  179:2    0  7.3G  0 part
mmcblk1      179:8    0  3.6G  0 disk
|-mmcblk1p1  179:9    0   96M  0 part
`-mmcblk1p2  179:10   0  3.5G  0 part /          <-- rootfs eMMC'de
root@beaglebone:~# findmnt /
TARGET SOURCE         FSTYPE OPTIONS
/      /dev/mmcblk1p2 ext4   rw,relatime
```

`/` bağlama noktası `mmcblk1p2`'nin yanında görünüyorsa challenge tamam.

### Sonuç

| Bileşen | Durum |
|---|---|
| **Kernel ve DTB** | microSD'nin 1. bölümünden RAM'e yüklenir. |

| Bileşen | Durum |
|---|---|
| **Rootfs** | Kernel açıldıktan sonra eMMC'nin 2. bölümünden bağlanır. |
| `bootcmd` | Değişmez; yükleme kaynağı hâlâ SD kart. |
| `bootargs` | `root=/dev/mmcblk1p2` olarak güncellenir. |

> **Önemli**
>
> Bunun çalışması için:
>
> - eMMC'nin 2. bölümünde gerçekten bir Linux rootfs olmalı.
> - Kernel'de MMC sürücüsü ve rootfs'in dosya sistemi (ör. ext4) desteği **built-in** olmalı (veya bir initramfs'ten yüklenmeli).
> - Linux'taki numaralandırma U-Boot'unkiyle aynı olmalı. Değilse kernel `VFS: Unable to mount root fs` hatasıyla durur; bu durumda `root=/dev/mmcblk0p2` dene veya `root=PARTUUID=...` kullan.

---

## 6.6 ROOTFS Bölümünde Neler Var?

**Root filesystem (rootfs)**, Linux'un `/` (kök) dizinidir. Kernel açıldıktan sonra `root=` ile gösterilen bölümü `/` olarak bağlar ve bundan sonra çalışan **her program, her ayar dosyası ve her kütüphane** buradan okunur. Kernel tek başına bir işe yaramaz; kullanıcıya kabuk (shell), komutlar ve servisler rootfs ile gelir.

Rootfs'teki klasör düzeni rastgele değildir; **FHS (Filesystem Hierarchy Standard)** denen ortak bir standarda uyar. Bu yüzden Ubuntu'da, BeagleBone'daki Ångström'de veya eMMC'deki Debian'da aynı klasörleri görürüz.

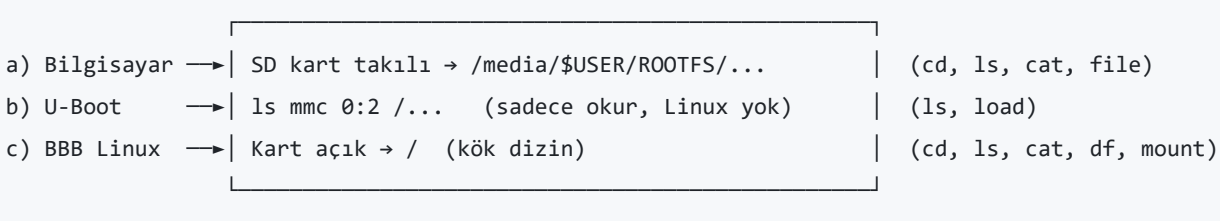
### 6.6.1 Klasörler ve görevleri

| Klasör | İçinde ne var? | BBB'de örnek | Bakmak için |
|---|---|---|---|
| `/bin` | Temel kullanıcı komutları ( `ls` , `cp` , `cat` , `sh` ...) | `bash` , `busybox` | `ls /bin` |
| `/sbin` | Sistem yönetimi komutları (genelde root için) | `init` , `mount` , `ifconfig` , `reboot` | `ls /sbin` |
| `/boot` | Kernel ve device tree dosyaları | `uImage` , `am335x-boneblack.dtb` | `ls -l /boot` |
| `/etc` | Sistem **ayar** dosyaları (metin) | `hostname` , `fstab` , `passwd` , `os-release` | `cat /etc/hostname` |
| `/lib` | Paylaşılan kütüphaneler ( `.so` ) ve kernel modülleri | `libc.so.6` , `modules/3.8.13/` | `ls /lib/modules` |

| Klasör | İçinde ne var? | BBB'de örnek | Bakmak için |
|---|---|---|---|
| `/usr` | Ek programlar, kütüphaneler, belgeler (rootfs'in en büyük kısmı) | `/usr/bin`, `/usr/lib` | `du -sh /usr` |
| `/home` | Kullanıcıların kişisel klasörleri | `/home/root` (Ångström'de root'un ev dizini) | `ls /home` |
| `/root` | root kullanıcısının ev dizini (Debian/Ubuntu'da) | eMMC'deki Debian'da kullanılır | `ls /root` |
| `/var` | Zamanla değişen veriler: loglar, kuyruklar, önbellek | `/var/log` | `ls /var/log` |
| `/tmp` | Geçici dosyalar (açılışta temizlenir, genelde RAM'de) | | `ls /tmp` |
| `/media`, `/mnt` | Harici diskleri (USB, SD) bağlama noktaları | `mount /dev/mmcblk1p1 /mnt` | `ls /mnt` |
| `/dev` *(sanal)* | Donanım aygıt dosyaları | `mmcblk0p2`, `ttyO0` | `ls /dev` |
| `/proc` *(sanal)* | Kernel ve süreç bilgileri | `cpuinfo`, `meminfo`, `cmdline` | `cat /proc/cpuinfo` |
| `/sys` *(sanal)* | Aygıt ve sürücü ayarları (sysfs) | `/sys/class/leds/...` | `ls /sys/class` |
| `/run` *(sanal)* | Çalışma anı bilgileri (PID dosyaları, soketler) | | `ls /run` |

> **Önemli**
>
> *(sanal)* işaretli klasörler **sanal (virtual)** dosya sistemleridir. Diskte gerçek dosya içermezler; kernel çalışırken içlerini **RAM'de** doldurur ( `devtmpfs`, `proc`, `sysfs`, `tmpfs` ). Bu yüzden SD kartı bilgisayara takıp baktığında bu klasörler **boş** görünür. Bu bir hata değildir.

### 6.6.2 ROOTFS'e üç yerden ulaşabiliriz



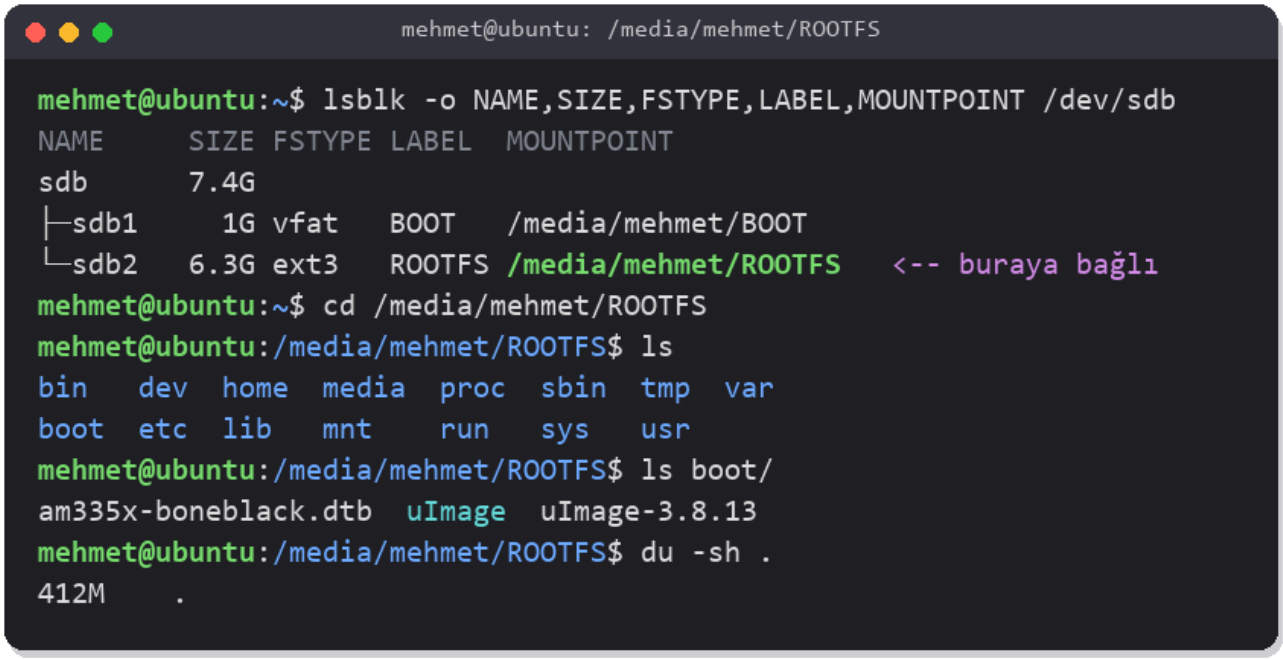

| Nereden? | Ne zaman? | Rootfs'in yolu |
|---|---|---|
| **a) Bilgisayar** | SD kart okuyucuya takılıyken, kartı açmadan dosya kopyalamak / düzeltmek için | `/media/$USER/ROOTFS` |

| Nereden? | Ne zaman? | Rootfs'in yolu |
|---|---|---|
| **b) U-Boot** | Linux açılmadan önce, dosya gerçekten orada mı diye bakmak için | `mmc 0:2` (SD) veya `mmc 1:2` (eMMC) |
| **c) BBB Linux** | Kart açıldıktan sonra, seri konsol (Minicom) veya SSH üzerinden | `/` |

### 6.6.3 a) Bilgisayardan ulaşmak

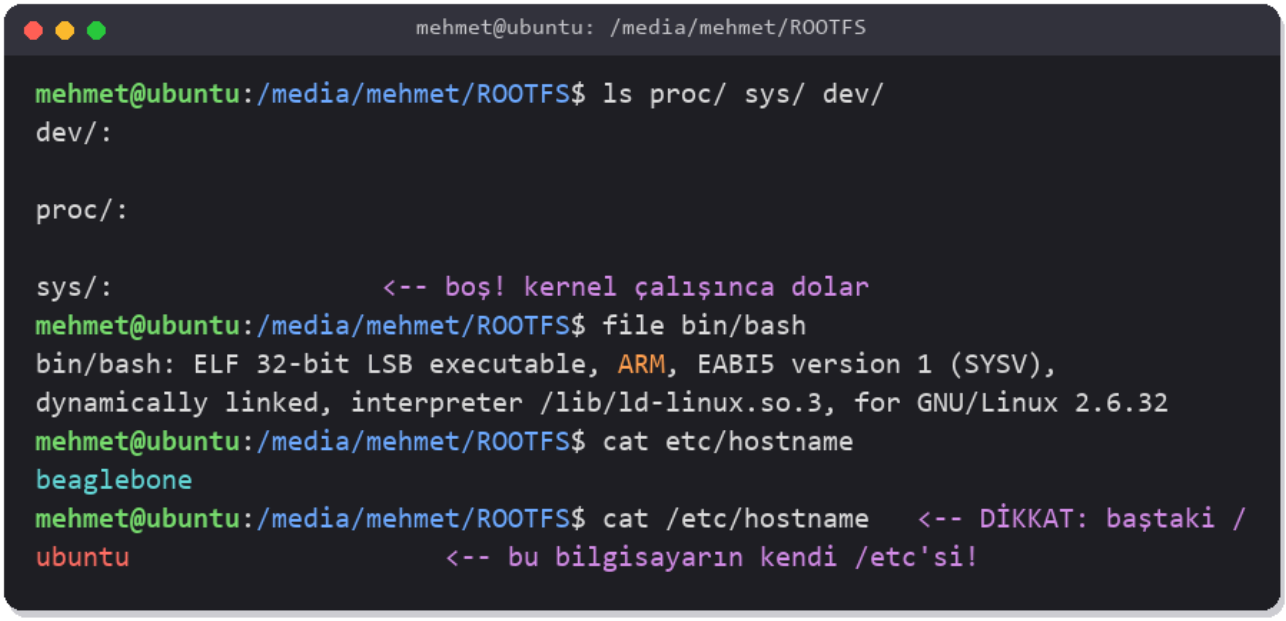
SD kartı bilgisayara tak. Ubuntu bölümleri **etiket (label)** adlarıyla otomatik bağlar. Önce kartın nereye bağlandığını bul, sonra klasöre gir:

```
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINT   # SD kart hangi aygıt, nereye bağlı?
cd /media/$USER/ROOTFS                       # rootfs'in içine gir
ls                                           # kök klasörleri listele
ls boot/                                     # kernel ve dtb burada
du -sh .                                     # rootfs'in toplam boyutu
```




Burada iki şeyi gözlemle: sanal klasörler boş, programlar ise bilgisayarın değil **ARM** işlemcinin komut setiyle derlenmiş:

```
ls proc/ sys/ dev/      # boş görünür
file bin/bash           # "ARM" yazısını gör
cat etc/hostname        # BBB'nin hostname'i (başta / YOK)
```




> **Uyarı**
>
> Bilgisayardayken yolun başına `/` **koyma**. `cat /etc/hostname` bilgisayarın **kendi** dosyasını okur; BBB'ninki `etc/hostname` (ROOTFS klasöründeyken) veya `/media/$USER/ROOTFS/etc/hostname` yolundadır. Bu karışıklıkla bilgisayarın `/etc/fstab` gibi dosyalarını yanlışlıkla bozmak mümkündür. Rootfs'e yazmak için `sudo` gerekir ve işin bitince kartı çıkarmadan önce `sync` + `umount` yap.
>
> `file bin/bash` çıktısında `ARM` yazdığı için bu programları bilgisayarda **çalıştıramazsın** ( `./bin/bash` → `Exec format error` ). Sadece okuyabilir/kopyalayabilirsin. Bunun sebebini Bölüm 9'da (cross compile) göreceğiz.

### 6.6.4 b) U-Boot'tan ulaşmak

U-Boot dosya sistemini **okuyabilir** (ext2/3/4 ve FAT). Linux açılmadan önce kernel'in gerçekten orada olup olmadığını görmek için kullanışlıdır:

```
U-Boot# ls mmc 0:2 /          # SD kartın 2. bölümü (ROOTFS) kökü
U-Boot# ls mmc 0:2 /boot      # kernel ve dtb
U-Boot# ls mmc 1:2 /          # eMMC'nin 2. bölümü (Debian rootfs)
```

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

U-Boot# ls mmc 0:2 /
<DIR>       4096 .
<DIR>       4096 ..
<DIR>       4096 bin
<DIR>       4096 boot
<DIR>       4096 dev
<DIR>       4096 etc
<DIR>       4096 home
<DIR>       4096 lib
    ...
<DIR>       4096 usr
<DIR>       4096 var
U-Boot# ls mmc 0:2 /boot
            24884 am335x-boneblack.dtb
          4380760 uImage
U-Boot# ls mmc 1:2 /          # eMMC'deki rootfs (Debian)
<DIR>       4096 bin
    ...
```

| Komut | Anlamı |
|---|---|
| `ls mmc <aygıt>:<bölüm> <yol>` | Bölümdeki klasörü listeler |
| `load mmc 0:2 0x82000000 /boot/uImage` | Dosyayı RAM'e yükler (4.10.2) |
| `ext4ls` / `ext2ls` / `fatls` | Eski U-Boot sürümlerinde dosya sistemine özel `ls` komutları |

> **Not**
>
> U-Boot'ta `cd` yoktur ve dosya içeriğini `cat` ile göremezsin; her komutta tam yolu yazarsın. U-Boot rootfs'e normalde **yazmaz**, sadece okur.

### 6.6.5 c) BBB Linux'tan ulaşmak

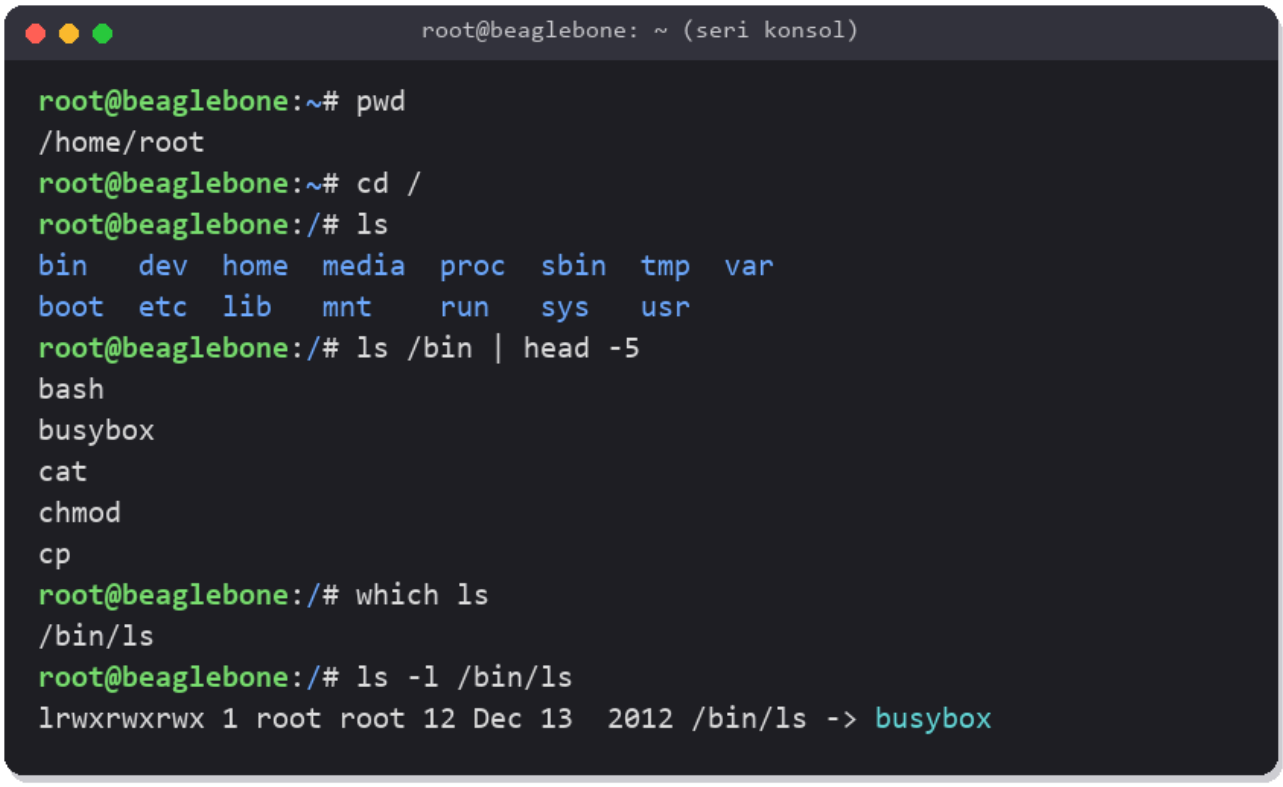
Kart açılıp `root` olarak giriş yaptıktan sonra rootfs artık `/` dizinidir. Dolaşmak için temel komutlar:

| Komut | Görevi |
|---|---|
| `pwd` | Şu an hangi klasördeyim? |
| `cd /` · `cd /etc` · `cd ..` · `cd ~` | Kök dizine / bir klasöre / bir üste / ev dizinine git |
| `ls` · `ls -l` · `ls -a` | Listele / ayrıntılı (izin, boyut, tarih) / gizli dosyalarla |
| `cat <dosya>` · `head -n 5` · `less` | Metin dosyasını göster / ilk satırları / sayfa sayfa |

| Komut | Görevi |
|---|---|
| `which <komut>` | Bir komutun hangi dosya olduğunu bul |
| `find / -name "*.dtb" 2>/dev/null` | Rootfs'te dosya ara |
| `df -h /` · `du -sh <klasör>` | Rootfs'in doluluk oranı / klasör boyutu |
| `mount` | Hangi bölüm nereye bağlı? |

### Kök dizin ve `/bin`

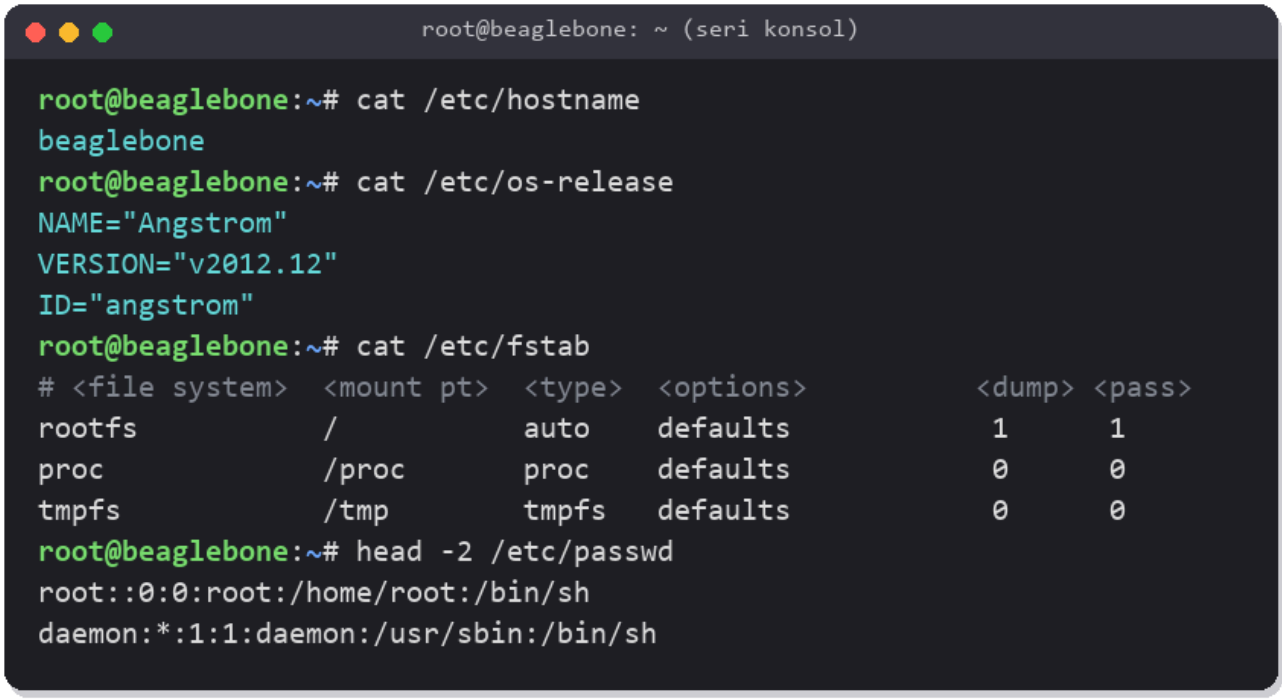
```
pwd                # /home/root  (Ångström'de root'un ev dizini)
cd /
ls
ls /bin | head -5
which ls
ls -l /bin/ls      # Ångström'de birçok komut busybox'a link
```




> **İpucu**
>
> **BusyBox**, `ls`, `cp`, `mount` gibi yüzlerce komutu **tek bir küçük programda** toplayan araçtır. Gömülü sistemlerde yer kazanmak için yaygın kullanılır; `/bin/ls` gibi dosyalar aslında `busybox`'a giden **sembolik linklerdir**.

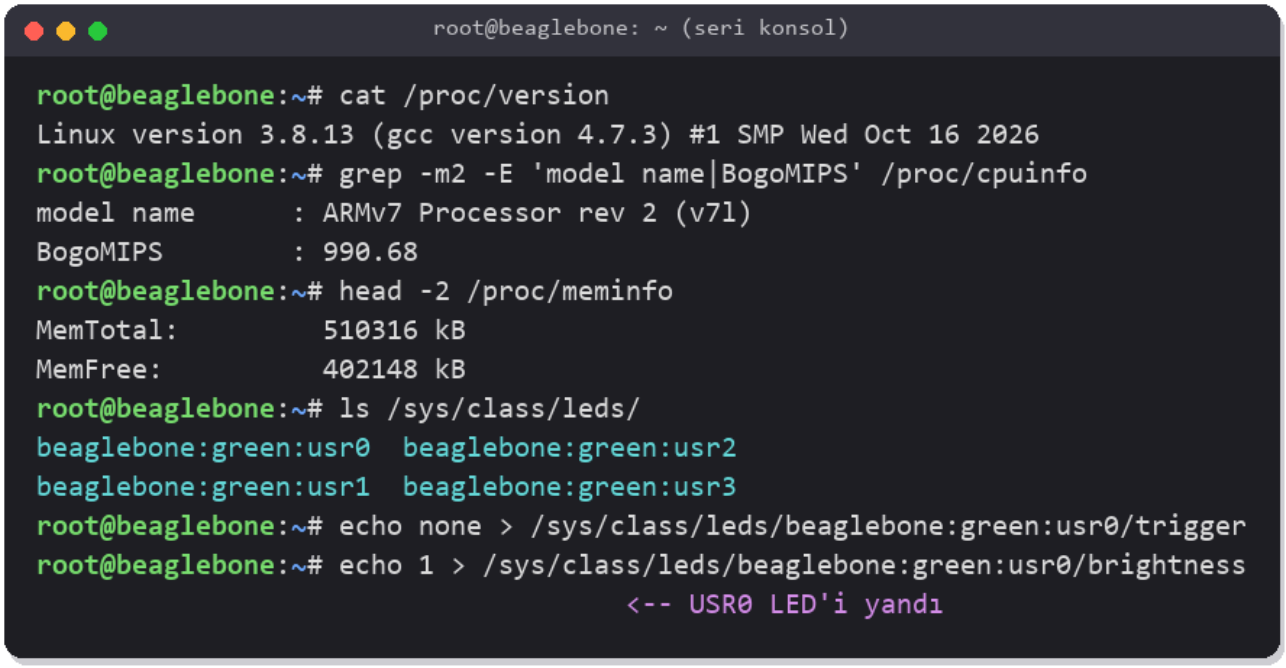
### `/etc` — ayar dosyaları

```
cat /etc/hostname      # kartın adı
cat /etc/os-release    # hangi dağıtım, hangi sürüm
cat /etc/fstab         # açılışta hangi bölüm nereye bağlanacak
head -2 /etc/passwd    # kullanıcı listesi
```




### `/proc` ve `/sys` — kernel ile konuşmak

```
cat /proc/version                                  # kernel sürümü
cat /proc/cmdline                                  # U-Boot'un verdiği bootargs
grep -m2 -E 'model name|BogoMIPS' /proc/cpuinfo    # işlemci bilgisi
head -2 /proc/meminfo                              # RAM bilgisi
ls /sys/class/leds/                                # kartın USR LED'leri
echo none > /sys/class/leds/beaglebone:green:usr0/trigger
echo 1    > /sys/class/leds/beaglebone:green:usr0/brightness   # LED'i yak (0 = söndür)
```




`/sys` altındaki dosyalara yazmak, sürücüye doğrudan komut vermek demektir. `trigger` dosyasını `none` yapmadan `brightness`'a yazarsan LED'in varsayılan görevi (ör. `heartbeat`) senin değerini hemen ezer.

### `/dev`, `/boot` ve disk kullanımı

```
ls /dev/mmcblk* /dev/ttyO*   # MMC bölümleri ve seri portlar
ls -l /boot                  # kernel ve dtb
df -h /                      # rootfs ne kadar dolu, hangi bölümden geliyor
du -sh /usr /lib /etc        # en çok yer kaplayan klasörler
mount | grep -E ' / | /proc | /sys | /dev '   # gerçek ve sanal bağlamalar
```

```
root@beaglebone: ~ (seri konsol)
root@beaglebone:~# ls /dev/mmcblk* /dev/ttyO*
/dev/mmcblk0    /dev/mmcblk0p2  /dev/mmcblk1p1  /dev/ttyO0
/dev/mmcblk0p1  /dev/mmcblk1    /dev/mmcblk1p2
root@beaglebone:~# ls -l /boot
-rw-r--r-- 1 root root   24884 Dec 13  2012 am335x-boneblack.dtb
lrwxrwxrwx 1 root root      14 Dec 13  2012 uImage -> uImage-3.8.13
-rw-r--r-- 1 root root 4380760 Dec 13  2012 uImage-3.8.13
root@beaglebone:~# df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/mmcblk0p2  6.2G  412M  5.5G   7% /
root@beaglebone:~# du -sh /usr /lib /etc 2>/dev/null
287M    /usr
61M     /lib
3.1M    /etc
root@beaglebone:~# mount | grep -E ' / | /proc | /sys | /dev '
/dev/mmcblk0p2 on / type ext3 (rw,relatime)
devtmpfs on /dev type devtmpfs (rw,relatime)
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
```

`mount` çıktısında `/` gerçek bir diskten ( `/dev/mmcblk0p2` , `ext3` ) gelirken `/dev` , `/proc` ve `/sys` 'in tipi `devtmpfs` , `proc` , `sysfs` olarak görünür. Bu, 6.6.1'deki sanal klasörlerin kanıtıdır. Rootfs'i eMMC'den açtıysan (6.5) `df -h /` satırında `/dev/mmcblk1p2` görürsün.

---

## 6.7 Özet

| Komut / Değişken | Görevi |
|---|---|
| `uEnv.txt` | U-Boot ortam değişkenlerini metin olarak tanımlar |
| `loady <adres>` | YMODEM ile seri porttan RAM'e dosya alır, boyutu `filesize` 'a yazar |
| `env import -t <adres> ${filesize}` | RAM'deki metin değişkenlerini aktif ortama aktarır |
| `printenv <isim>` | Değişkenin değerini gösterir |
| `setenv <isim> <değer>` | Değişkeni değiştirir (geçici) |
| `run <makro>` | Değişkendeki komutları çalıştırır (ör. `run mypcip` ) |
| `boot` | `bootcmd` 'yi çalıştırır |
| `bootcmd` | Otomatik açılışta çalışan komut dizisi |
| `bootargs` | Linux kernel'ine verilen açılış parametreleri |
| `root=/dev/mmcblk1p2` | Rootfs'in eMMC'nin 2. bölümünde olduğunu söyler |

| Komut / Değişken | Görevi |
|---|---|
| `mmc list` / `mmc part` | U-Boot'un gördüğü MMC aygıtlarını ve bölümleri listeler |
| `saveenv` | Aktif ortamı kalıcı olarak kaydeder (dikkatli kullan) |
| `/media/$USER/ROOTFS` | SD kart bilgisayara takılıyken rootfs'in yolu |
| `ls mmc 0:2 /boot` | U-Boot'tan rootfs'teki klasörü listeler |
| `/bin`, `/sbin`, `/usr` | Programlar |
| `/etc` | Ayar dosyaları ( `hostname`, `fstab`, `passwd` ) |
| `/boot` | Kernel ( `uImage` ) ve device tree ( `.dtb` ) |
| `/lib` | Kütüphaneler ve kernel modülleri |
| `/dev`, `/proc`, `/sys` | Sanal klasörler; kernel çalışırken dolar |
| `cat /proc/cmdline` | Kernel'e giden bootargs |
| `df -h /` · `mount` | Rootfs hangi bölümden geliyor? |

## 6.8 Karşılaşılabilecek Problemler

| Belirti | Sebep | Çözüm |
|---|---|---|
| `loady` sonrası transfer başlamıyor | Minicom'da xmodem seçildi ya da `lrzsz` yok | Protokol **ymodem** olmalı; `sudo apt install lrzsz` |
| `env import` sonrası değişkenler bozuk / fazladan satırlar | Boyut ondalık yazıldı | `${filesize}` kullan ya da boyutu hex yaz |
| `## Error: "mypcip" not defined` | `env import` yapılmadı ya da kart yeniden açıldı | Adım 3–4'ü tekrarla; değişiklikler kalıcı değildir |
| `VFS: Unable to mount root fs` (challenge) | Linux'ta eMMC'nin numarası farklı | `root=/dev/mmcblk0p2` dene; Linux'ta `lsblk` ile kontrol et |
| Açılışta `uEnv.txt`'deki `bootcmd` çalışmıyor | Otomatik okumada `bootcmd` değil `uenvcmd` çalıştırılır | SD karttaki dosyada komutları `uenvcmd`'ye yaz |

# Bölüm 7 — Seri Porttan (UART) Boot

> **Bu bölümde:**
>
> - BeagleBone Black'i **SD kart ve eMMC kullanmadan**, bütün boot dosyalarını bilgisayardan **seri port (UART0)** üzerinden göndererek açacaksın.
> - Xmodem ve Ymodem protokollerinin farkını ve ROM Code'un UART boot davranışını öğreneceksin.
> - Kernel, DTB ve initramfs'i RAM'de nereye yükleyeceğini ve `bootm` ile nasıl başlatacağını göreceksin.
>
> **Ön koşul:** Bölüm 2 — Minicom, Bölüm 3 — boot sırası, Bölüm 6 — `loady` ile dosya gönderme.

---

## 7.1 Dosya Transfer Protokolleri

Bilgisayardan karta dosya göndermenin iki yolu var:

| Bağlantı | Protokoller | Ne zaman kullanılır? |
|---|---|---|
| **IP ağı (Ethernet)** | TFTP, HTTP, FTP, SMTP... | Kartta çalışan bir ağ yığını (U-Boot veya Linux) varsa. Hızlıdır (Bölüm 8). |
| **UART (seri port)** | Xmodem, Ymodem, Zmodem, Kermit... | Ağ yokken veya kartta henüz hiçbir yazılım yokken. Yavaştır ama çok basittir. |

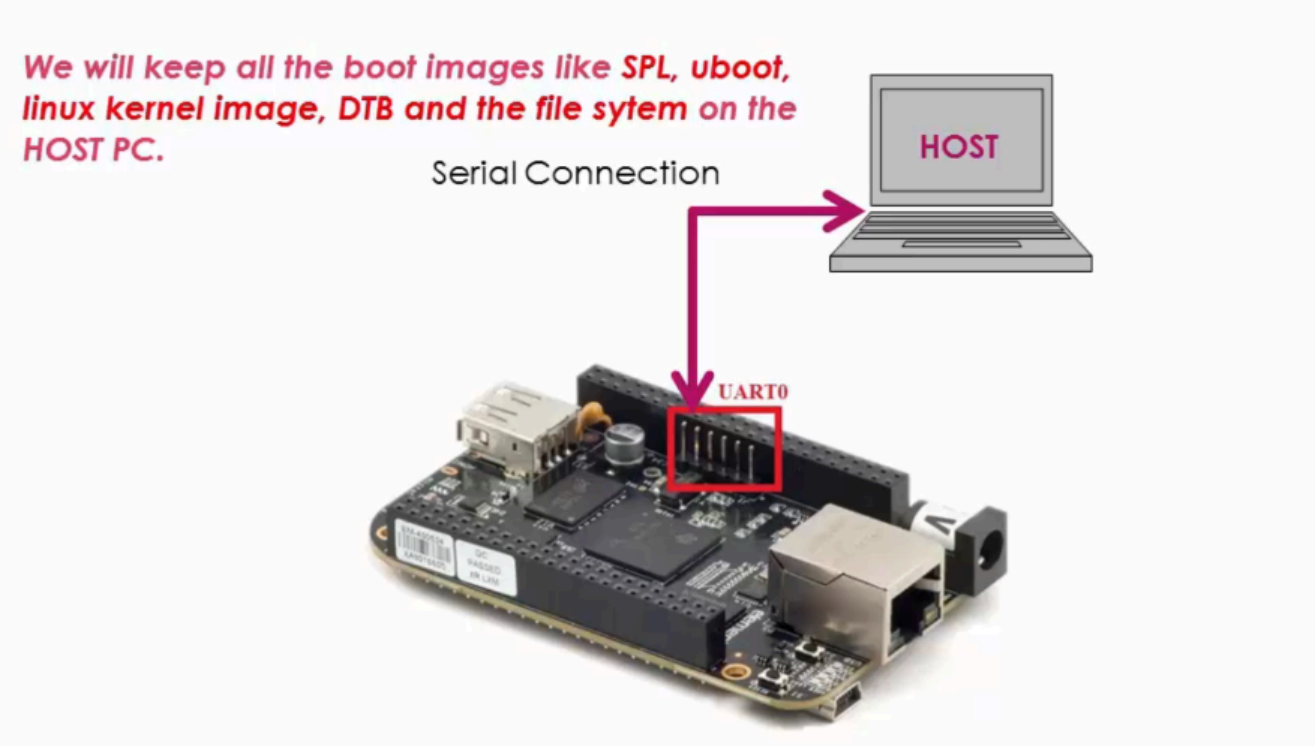
UART sadece ham baytları taşır; "dosya nerede başlıyor, nerede bitiyor, veri bozuldu mu?" gibi bilgileri taşımaz. **Xmodem / Ymodem** gibi protokoller veriyi paketlere bölüp her pakete kontrol bilgisi (checksum / CRC) ekler, bozuk paketi tekrar istetir.

| Protokol | Özellik |
|---|---|
| **Xmodem** | 128 bayt (Xmodem-1K'da 1024 bayt) paketler. Dosya adı ve boyutu göndermez. AM335x'in **ROM kodu** bunu kullanır. |

| Protokol | Özellik |
|---|---|
| **Ymodem** | Xmodem'in geliştirilmiş hali: dosya adını ve boyutunu da gönderir. U-Boot'ta `loady` komutu. |

---

## 7.2 Seri Boot Fikri



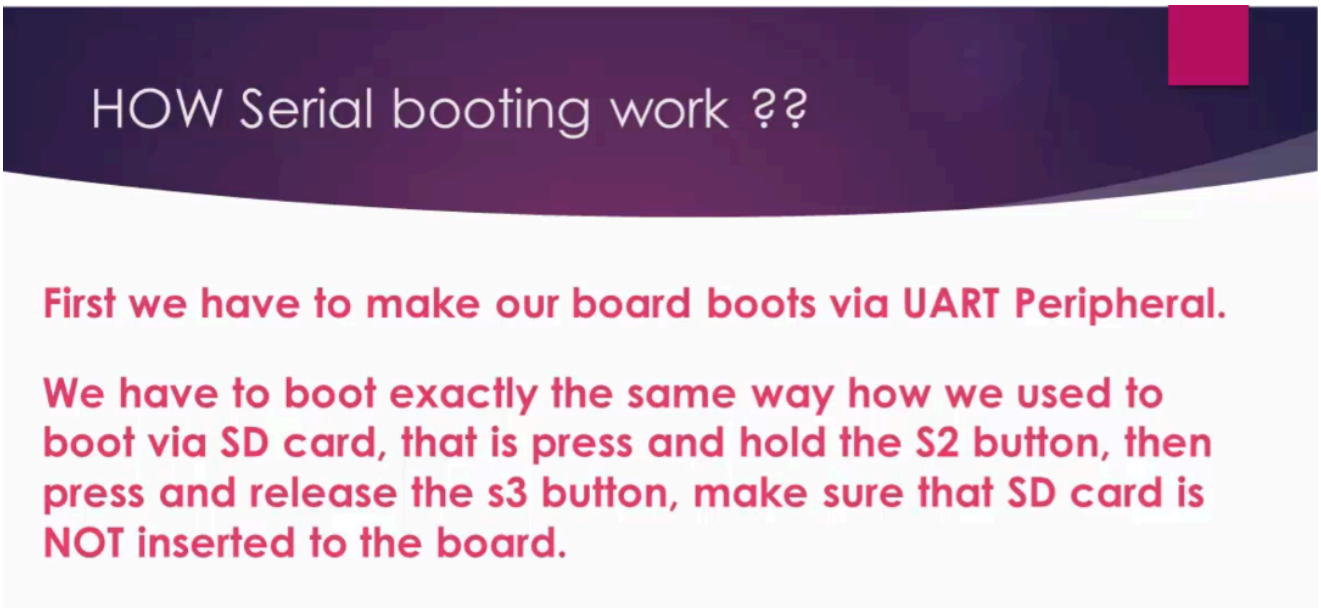

Bütün boot dosyaları — **SPL, U-Boot, Linux kernel (uImage), DTB ve dosya sistemi** — kartta değil, **bilgisayarda (HOST)** durur. Kart ile bilgisayar arasındaki tek bağlantı, J1 header'daki **UART0** pinlerine bağlı USB-TTL seri kablodur. Kart açılırken her dosyayı bu kablo üzerinden sırayla göndeririz.

Bu yöntem şu durumlarda işe yarar:

- SD kart ve eMMC boşsa veya içindeki bootloader bozulduysa (kartı "kurtarmak" için).
- Yeni derlenmiş bir SPL / U-Boot'u karta yazmadan denemek için.

---

## 7.3 Kartı UART Boot Moduna Sokmak

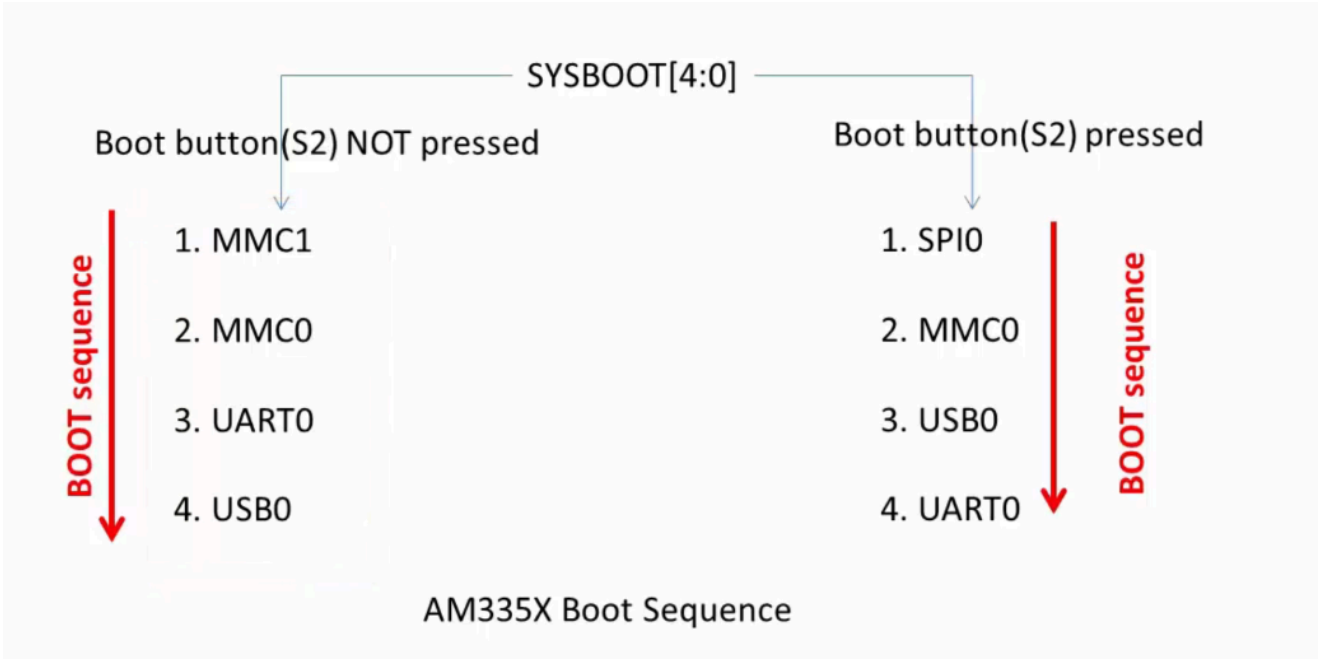



Kartı UART'tan boot ettirmek için SD karttan boot ederken yaptığımızın aynısını yaparız:

1. **S2 (BOOT)** butonuna basılı tut,
2. **S3 (POWER)** butonuna basıp bırak (veya beslemeyi tak),
3. Boot başladıktan sonra S2'yi bırak.

Tek fark: **SD kart takılı olmamalı.** Takılıysa ROM kodu boot dosyalarını SD karttan bulur ve UART'a hiç sıra gelmez.

### AM335x boot sırası




AM335x'in içindeki ROM kodu, açılışta `SYSBOOT[4:0]` pinlerine bakarak hangi çevre birimlerini hangi sırayla deneyeceğine karar verir (3.5). BBB'de S2 butonu bu pinlerden birinin değerini değiştirir:

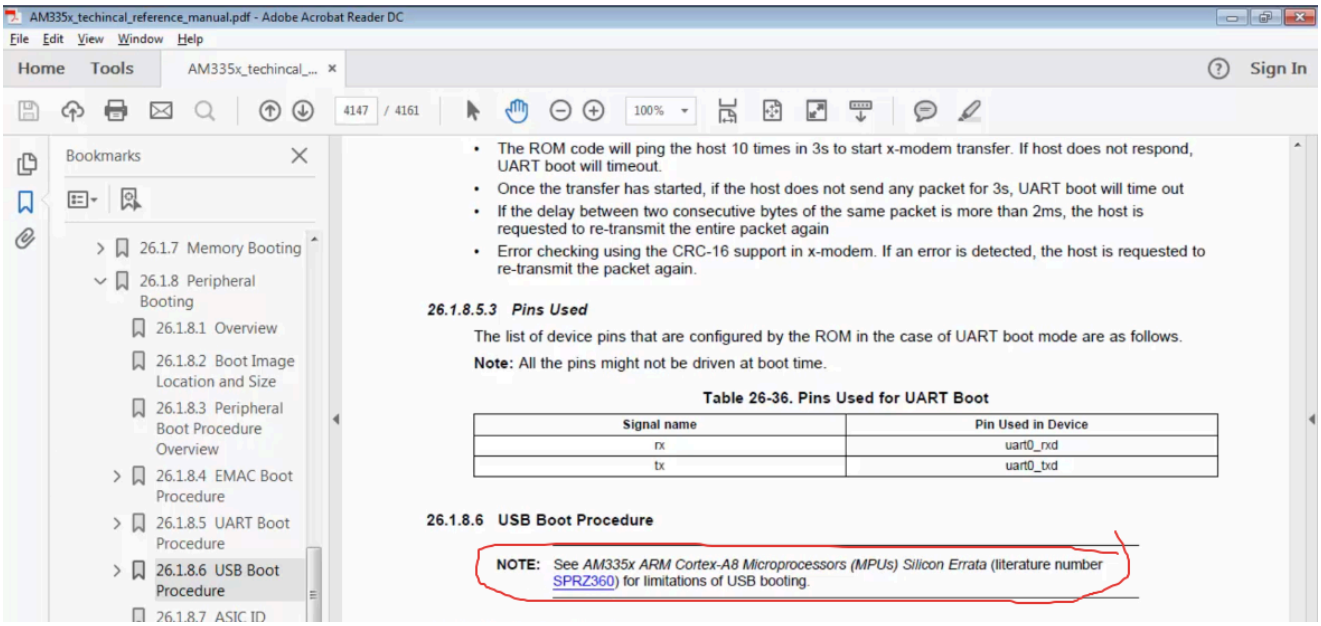
| Sıra | S2 basılı **değil** | S2 **basılı** |
|---|---|---|
| 1 | MMC1 (eMMC) | SPI0 |
| 2 | MMC0 (SD kart) | MMC0 (SD kart) |
| 3 | UART0 | **USB0** |
| 4 | USB0 | **UART0** |

S2 basılı bir şekilde boot işlemi yaparsak ROM kodu sırayla her çevre biriminde geçerli bir boot image'ı olup olmadığını kontrol eder:

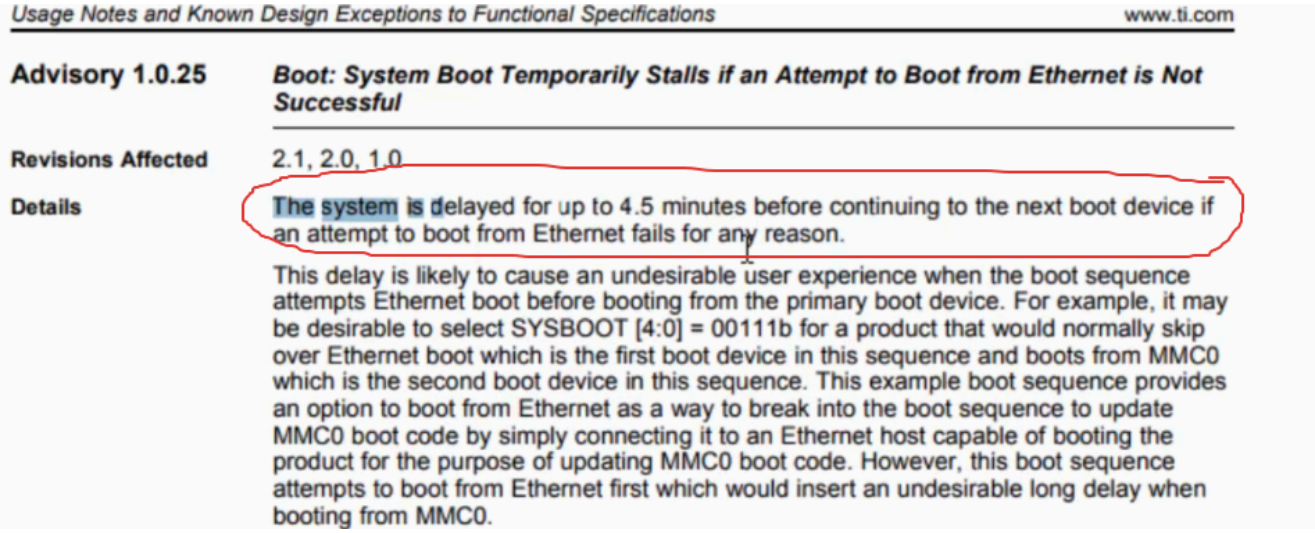
- **SPI0**: BBB'de SPI flash yok → hemen geçer.
- **MMC0**: SD kart takılı değil → hemen geçer.
- **USB0**: Burada farklı bir durum oluşuyor (aşağıda).
- **UART0**: Bizim istediğimiz yer.

## USB0 sorunu: 4.5 dakikalık bekleme

Eğer USB kablosu bu sırada karta takılıysa ve image dosyası USB üzerinden gönderilmiyorsa, ROM kodu bir sonraki boot aygıtı olan UART0'a geçmeden önce **4.5 dakikaya kadar** bekliyor. Bu süre dolduktan sonra UART0'a geçiyor.




AM335x Technical Reference Manual'ın (TRM) **26.1.8.6 USB Boot Procedure** bölümünde, USB boot'un kısıtlamaları için **Silicon Errata (SPRZ360)** belgesine bakılması söyleniyor:



*Usage Notes and Known Design Exceptions to Functional Specifications* www.ti.com

**Advisory 1.0.25** ***Boot: System Boot Temporarily Stalls if an Attempt to Boot from Ethernet is Not Successful***

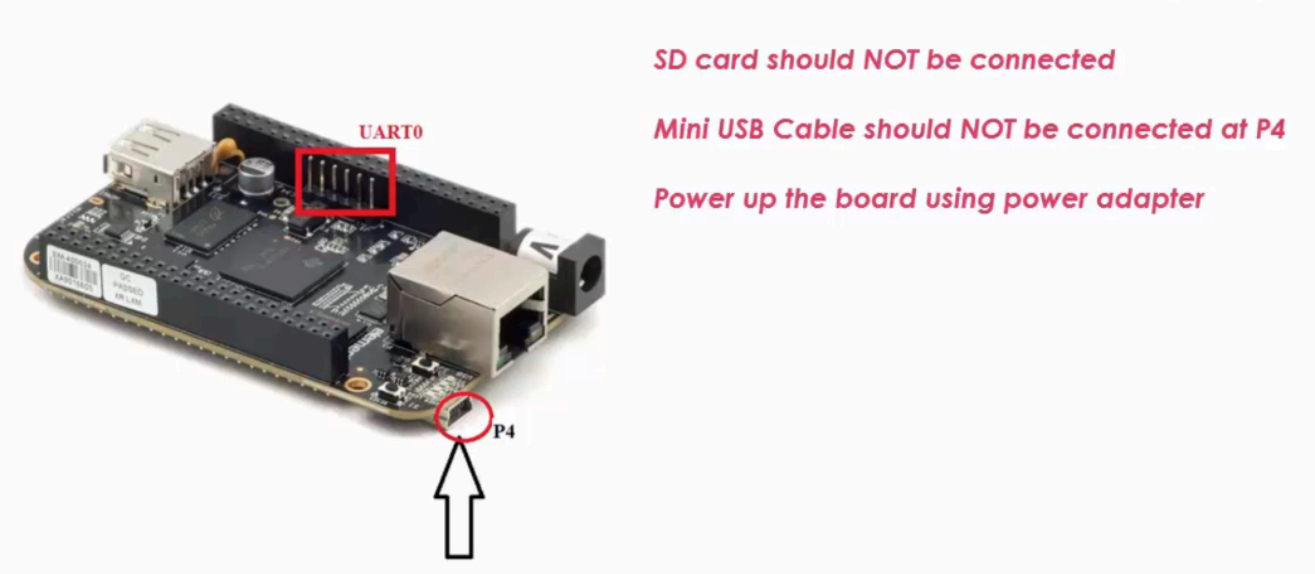
**Revisions Affected** 2.1, 2.0, 1.0

**Details** The system is delayed for up to 4.5 minutes before continuing to the next boot device if an attempt to boot from Ethernet fails for any reason.

This delay is likely to cause an undesirable user experience when the boot sequence attempts Ethernet boot before booting from the primary boot device. For example, it may be desirable to select SYSBOOT [4:0] = 00111b for a product that would normally skip over Ethernet boot which is the first boot device in this sequence and boots from MMC0 which is the second boot device in this sequence. This example boot sequence provides an option to boot from Ethernet as a way to break into the boot sequence to update MMC0 boot code by simply connecting it to an Ethernet host capable of booting the product for the purpose of updating MMC0 boot code. However, this boot sequence attempts to boot from Ethernet first which would insert an undesirable long delay when booting from MMC0.

Errata'daki **Advisory 1.0.25**'te Ethernet denilmiş, ancak bu durum USB için de geçerli. Çünkü AM335x'in ROM kodu USB boot'u, USB kablosu üzerinden **sanal bir Ethernet bağlantısı (RNDIS)** kurarak yapar. Yani USB boot aslında "USB üzerinden Ethernet boot"tur ve aynı 4.5 dakikalık beklemeye takılır.

## Çözüm: Kartı harici adaptörle beslemek



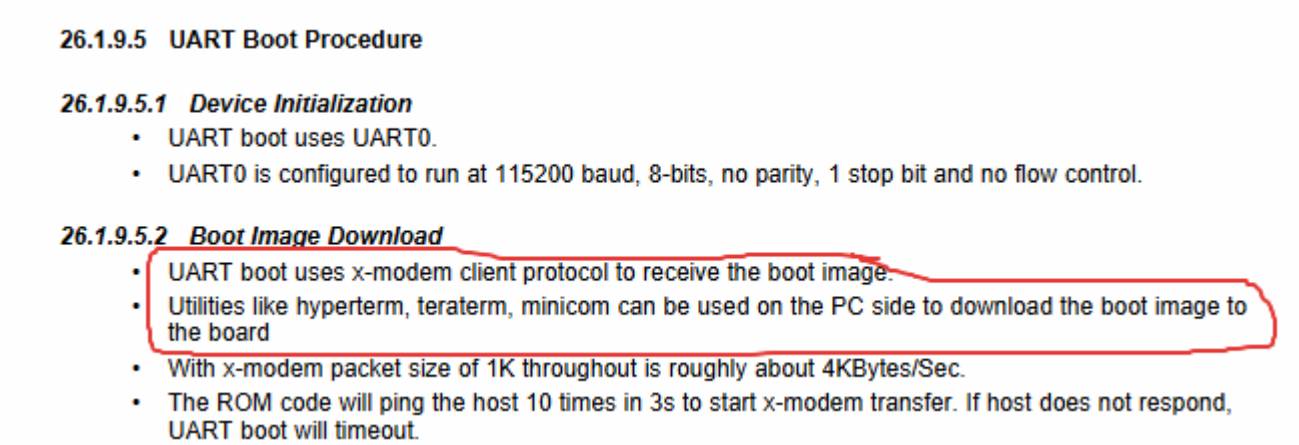

Peki bu bekleme süresinden nasıl kurtulabiliriz? Beslemeyi **P4 mini USB portundan değil**, harici bir **5V - 2A adaptör** yardımıyla (DC jack) yapacağız. USB kablosu takılı olmayınca ROM kodu USB0'da hiçbir şey bulamaz, hiç beklemeden UART0 sekansına geçer ve image yüklemesini oradan yaparız.

| Koşul | Neden? |
|---|---|
| **SD kart takılı olmamalı** | ROM, UART0'dan önce MMC0'ı dener; SD kartta MLO varsa oradan açılır. |
| **P4 (mini USB) kablosu takılı olmamalı** | Takılıysa USB0 boot denenir ve 4.5 dakika beklenir. |
| **Kart DC jack'ten 5V adaptörle beslenmeli** | USB kablosu çıkınca kartın beslemesi başka yerden gelmeli. |
| **UART0 seri kablosu bağlı olmalı** | Bütün dosyalar buradan gidecek. |

> **Uyarı**
>
> Seri kablonun (USB-TTL) **VCC / 5V** ucunu karta bağlama. J1 header'da sadece **GND, RX ve TX** bağlanır; kart zaten adaptörden besleniyor.

---

## 7.4 ROM Kodunun UART Boot Davranışı



**26.1.9.5 UART Boot Procedure**

***26.1.9.5.1 Device Initialization***

- UART boot uses UART0.
- UART0 is configured to run at 115200 baud, 8-bits, no parity, 1 stop bit and no flow control.

***26.1.9.5.2 Boot Image Download***

- UART boot uses x-modem client protocol to receive the boot image.
- Utilities like hyperterm, teraterm, minicom can be used on the PC side to download the boot image to the board
- With x-modem packet size of 1K throughout is roughly about 4KBytes/Sec.
- The ROM code will ping the host 10 times in 3s to start x-modem transfer. If host does not respond, UART boot will timeout.

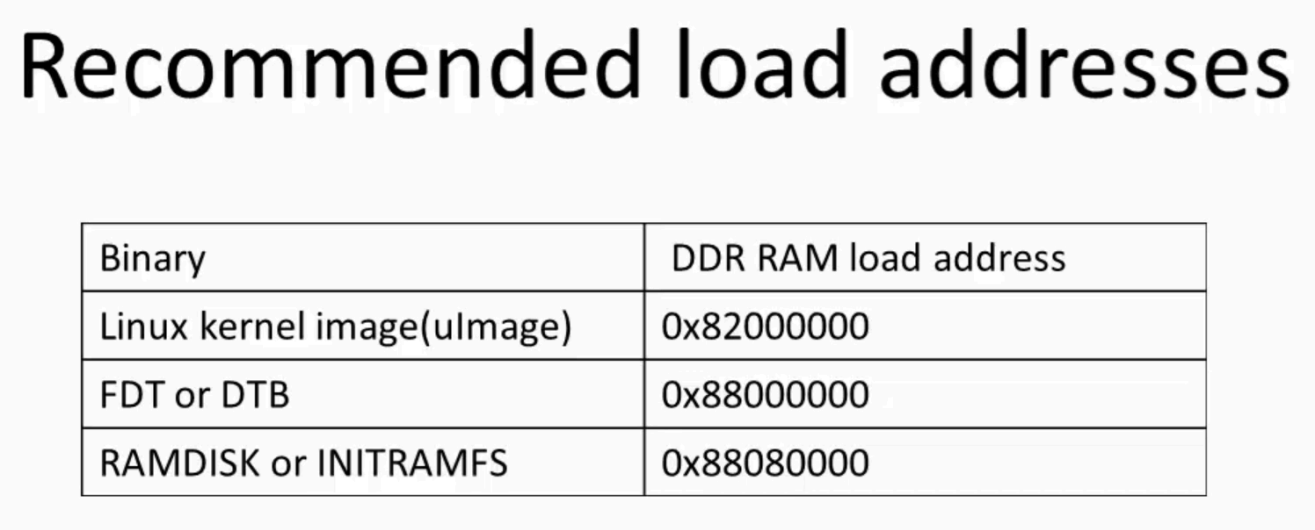
TRM'nin **26.1.9.5 UART Boot Procedure** bölümüne göre:

| Konu | Değer |
|---|---|
| Kullanılan port | **UART0** (pinler: `uart0_rxd`, `uart0_txd`) |
| Ayarlar | **115200 baud, 8 bit, parity yok, 1 stop bit, flow control yok** (115200 8N1) |
| Protokol | **Xmodem** (ROM kodu Xmodem istemcisi olarak dosyayı alır) |
| Bilgisayar tarafı | HyperTerminal, Tera Term, **Minicom** gibi programlar kullanılabilir |
| Hız | 1K paketlerle yaklaşık **4 KB/s** |
| Zaman aşımı | ROM, 3 saniye içinde 10 kez bilgisayarı "dürter" (ekrana `C` karakteri basar). Cevap gelmezse veya transfer başladıktan sonra 3 saniye paket gelmezse UART boot iptal olur. |
| Hata kontrolü | Xmodem'in CRC-16 desteği; bozuk paket tekrar istenir |

> **İpucu**
>
> Minicom ekranında art arda `CCCC...` görmek, kartın UART boot moduna girip **dosya beklediği** anlamına gelir. 3 saniyelik süre kısa olduğu için Minicom'da gönderme menüsünü ( `Ctrl-A → S →` **xmodem**) önceden hazırla.

---

## 7.5 Önerilen RAM Yükleme Adresleri



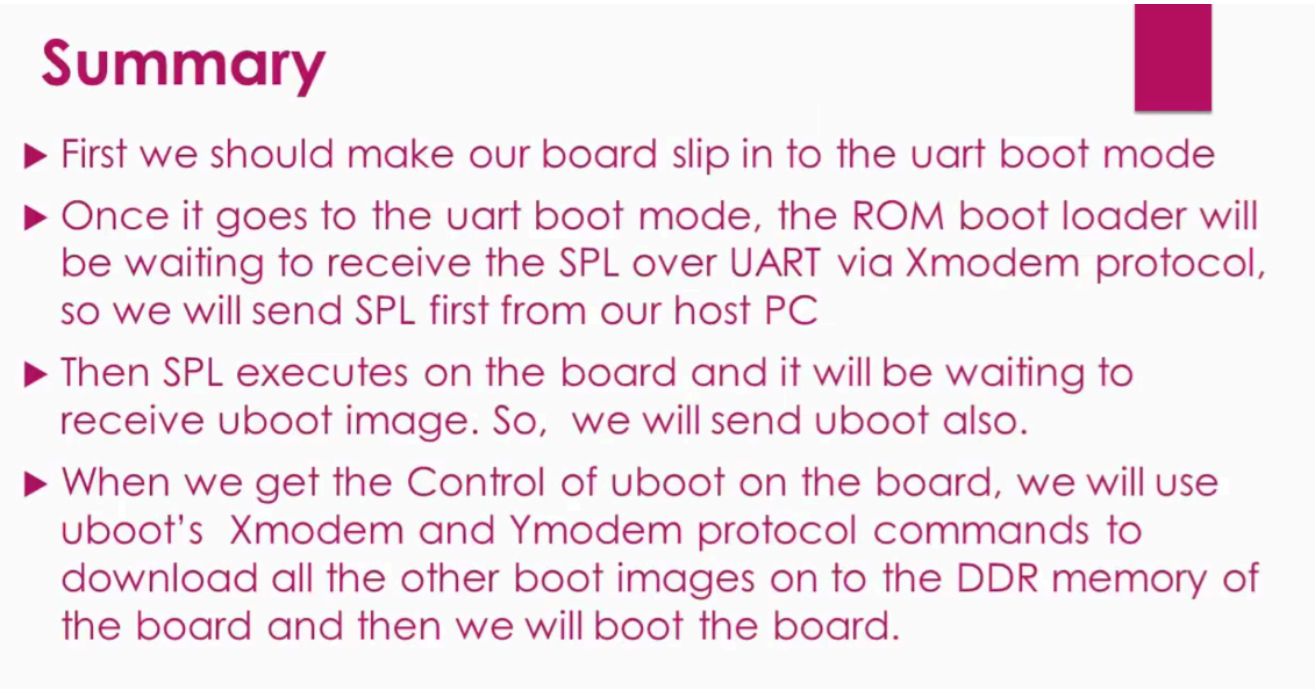

U-Boot'a geldikten sonra kernel, DTB ve dosya sistemini RAM'e (DDR, `0x80000000`'dan başlar) yükleyeceğiz. Dosyaların birbirinin üzerine yazmaması için bu adresler kullanılır:

| **Dosya** | **Adres** |
|---|---|
| Kernel ( `uImage` ) | `0x82000000` |
| Device tree ( `.dtb` ) | `0x88000000` |
| initramfs | `0x88080000` |

Seri boot'ta SD kart ve eMMC kullanılmadığı için rootfs'i bağlayacak bir disk yoktur. Bu yüzden dosya sistemi de RAM'e yüklenir: **RAMDISK / INITRAMFS**, RAM'de duran küçük bir rootfs'tir. DTB'ye ayrılan alan `0x88000000` – `0x8807FFFF` arası (512 KB) olduğu için initramfs hemen arkasından, `0x88080000`'dan başlar.

---

## 7.6 Özet: Seri Boot Adımları




```
Bilgisayar (HOST)                          BeagleBone Black
_________________                          ________________________________________

                                           ROM kodu (S2 basılı, SD kart yok) → UART0
u-boot-spl.bin ——— Xmodem ———————————→     SPL çalışır (DDR'ı hazırlar)
u-boot.img     ——— Ymodem ———————————→     U-Boot çalışır → => komut satırı komut satırı
uImage         ——— loadx / loady ————→     0x82000000
.dtb           ——— loadx / loady ————→     0x88000000
initramfs      ——— loadx / loady ————→     0x88080000
                                           bootm → Linux açılır
```

1. Önce kartı **UART boot moduna** sokarız (SD kart yok, USB kablosu yok, S2 basılı, adaptörle besleme).
2. UART boot moduna girince ROM kodu, **SPL**'i UART üzerinden **Xmodem** protokolüyle bekler. Bu yüzden bilgisayardan önce SPL'i göndeririz.
3. SPL kartta çalışır (DDR RAM'i hazırlar) ve bu sefer **U-Boot** image'ını bekler. U-Boot'u da göndeririz.
4. U-Boot'un kontrolü ele geçince, U-Boot'un **Xmodem / Ymodem** komutlarıyla (`loadx` / `loady`) diğer bütün boot image'larını (kernel, DTB, initramfs) kartın DDR belleğine indirir ve kartı boot ederiz.

> **Not**
>
> ROM kodu SD karttaki `MLO` dosyasından farklı olarak, UART'tan gelen SPL'in başında **GP header** beklemez. Bu yüzden UART boot'ta `MLO` değil, U-Boot derlemesinin ürettiği ham `u-boot-spl.bin` gönderilir.

---

## 7.7 Elimizdeki Boot Dosyaları Nedir?

Seri boot için bilgisayarda (HOST) şu 5 dosya hazır olmalı. Her biri boot zincirinin bir halkasıdır; her dosyanın işi bir sonrakini yükleyip çalıştırmaktır:

| Dosya | Nedir? | Kim yükler? | Nereye yüklenir? | Protokol |
|---|---|---|---|---|
| `u-boot-spl.bin` | **SPL** (Secondary Program Loader). U-Boot'un küçük ilk aşaması. Saatleri, pin ayarlarını, UART'ı ve en önemlisi **DDR RAM**'i hazırlar. | ROM kodu | AM335x'in iç SRAM'i ( `0x402F0400` , en fazla ~109 KB) | Xmodem |
| `u-boot.img` | Asıl **U-Boot**. Başında 64 baytlık U-Boot (mkimage) başlığı vardır. Komut satırını ( `=>` ) bize o verir. | SPL | DDR ( `0x80800000` ) | Ymodem |
| `uImage` | **Linux kernel**. `zImage` 'in başına U-Boot başlığı (yükleme adresi, giriş noktası, CRC) eklenmiş hali (4.9). | U-Boot ( `loadx` ) | `0x82000000` | Xmodem |
| `am335x-boneblack.dtb` | **Device Tree Blob**. Kernel'e kartta hangi donanımın olduğunu anlatan derlenmiş dosya (Bölüm 11). | U-Boot ( `loadx` ) | `0x88000000` | Xmodem |
| `initramfs` | RAM'de çalışan küçük **kök dosya sistemi** (genellikle BusyBox). `cpio` arşivi + gzip + U-Boot başlığı. SD/eMMC olmadığı için rootfs'i buradan alırız. | U-Boot ( `loadx` ) | `0x88080000` | Xmodem |

### Bu dosyaları nereden bulurum?

Bu dosyaları **kendin yazmazsın**. Hepsi açık kaynak projelerden (U-Boot, Linux kernel, BusyBox) çıkar; senin işin kaynak kodu indirip ayarlamak ve derlemektir. Üç yol vardır:

| Yol | Ne zaman? | Artısı / eksisi |
|---|---|---|
| **1. Hazır (prebuilt) dosyalar** | İlk deneme, "kart çalışıyor mu?" testi | En hızlısı; ama içinde ne olduğunu sen seçmezsin. |
| **2. Kaynak koddan elle derleme** | Öğrenirken (kursta yapılan) | Her adımı görürsün; daha uzun sürer. |
| **3. Buildroot / Yocto** | Gerçek projeler | Hepsini tek komutla üretir; ilk derleme uzun sürer. |

#### Yol 1 — Hazır dosyalar

- **Kursun kaynakları:** Kurslar bu dosyaları genelde derlenmiş olarak verir (Udemy ekleri / kursun GitHub reposu). Kursla birebir uyumlu oldukları için ilk seri boot denemesinde en kolayı bunlar.
- **TI Processor SDK (AM335x):** TI'ın sitesinden indirilir. `board-support/prebuilt-images/` klasöründe genellikle `MLO` , `u-boot-spl.bin` , `u-boot.img` , `zImage` , `.dtb` dosyaları ve bir rootfs hazır gelir.

- **BeagleBoard.org Debian imajı:** Bootloader, kernel ve DTB içerir ama hazır initramfs yoktur; seri boot için daha az uygundur.

**Yol 2 — Kaynak koddan derleme**

Cross toolchain kurulumu ve SPL/U-Boot derlemesi Bölüm 9'da adım adım anlatılıyor. `mkimage` aracı için `sudo apt install u-boot-tools` yeterli.

| Dosya | Kaynak kod | Çıktının yeri |
|---|---|---|
| `u-boot-spl.bin`, `u-boot.img` | **U-Boot** (Bölüm 9) | `spl/u-boot-spl.bin`, `u-boot.img` |
| `uImage` | **Linux kernel** (BBB için genelde `github.com/beagleboard/linux`) | `arch/arm/boot/uImage` |
| `am335x-boneblack.dtb` | Aynı kernel kaynağı | `arch/arm/boot/dts/am335x-boneblack.dtb` |
| `initramfs` | **BusyBox** (`busybox.net`) ile hazırlanan rootfs klasörü | Senin oluşturduğun dosya |

```
# U-Boot  -> spl/u-boot-spl.bin ve u-boot.img  (ayrıntılar: Bölüm 9)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4

# Kernel + DTB  -> arch/arm/boot/uImage ve .../dts/am335x-boneblack.dtb
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs LOADADDR=0x80008000

# BusyBox  -> _install/ klasörü (rootfs'in temeli)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=../rootfs install

# initramfs: rootfs klasöründen cpio arşivi + U-Boot başlığı
cd rootfs
find . | cpio -H newc -o | gzip -9 > ../initramfs.cpio.gz
cd ..
mkimage -A arm -O linux -T ramdisk -C gzip -n initramfs -d initramfs.cpio.gz initramfs
```

Son satırdaki `mkimage` önemli: `bootm` komutu ramdisk'in başında **U-Boot başlığı** görmek ister. Ham `initramfs.cpio.gz` gönderirsen `Wrong Ramdisk Image Format` hatası alırsın (7.9 (g)).

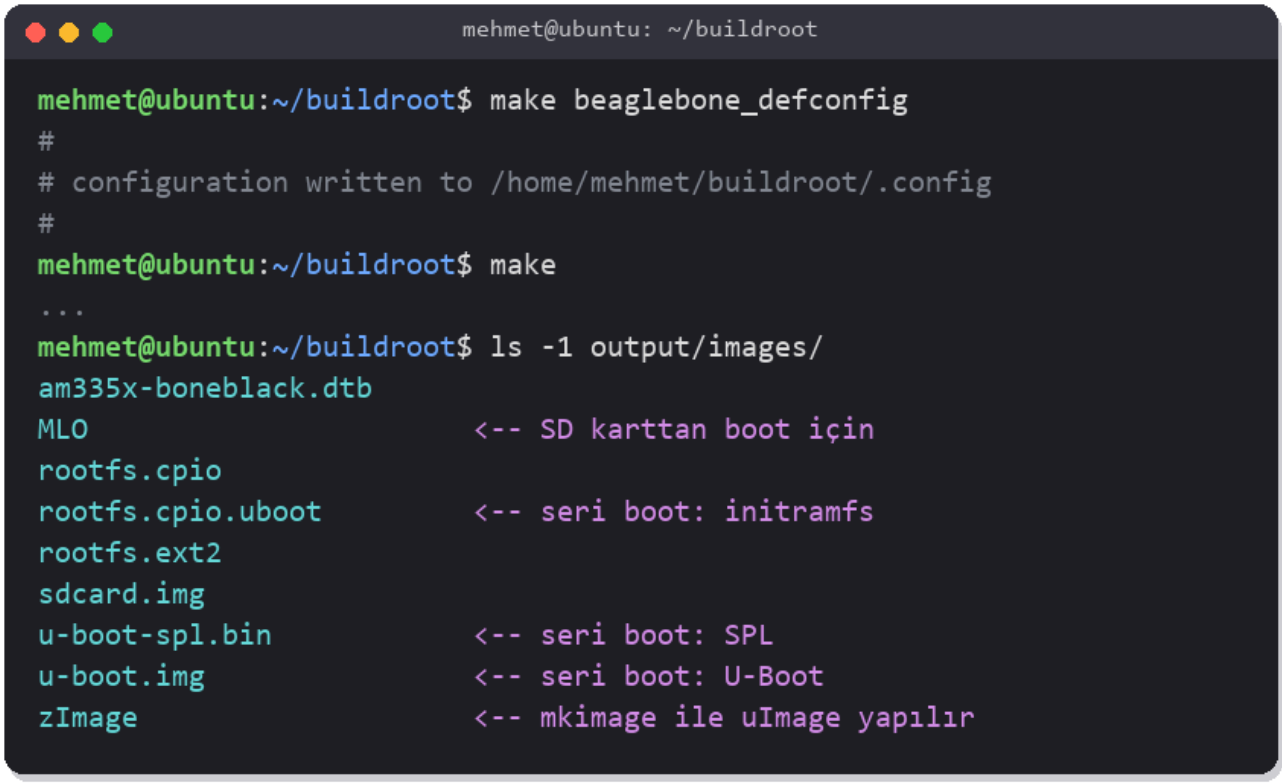
**Yol 3 — Buildroot ile hepsini tek seferde**

Buildroot toolchain'i, U-Boot'u, kernel'i, DTB'yi ve rootfs'i senin yerine indirip derler:

```
git clone https://gitlab.com/buildroot.org/buildroot.git
cd buildroot
make beaglebone_defconfig
make menuconfig
make                    # ilk derleme 30 dk - 1 saat sürebilir
ls -1 output/images/
```

`make menuconfig`'te seri boot için iki ayar yap:

| Menü | Ayar | Neden? |
|---|---|---|
| Bootloaders → U-Boot → U-Boot SPL binary name | `MLO spl/u-boot-spl.bin` | Varsayılan sadece `MLO`'yu kopyalar; seri boot için ham SPL gerekir. |
| Filesystem images → cpio the root filesystem → **Create U-Boot image of the root filesystem** | Aç | `rootfs.cpio.uboot` üretir: başında U-Boot başlığı olan initramfs. |




Buildroot kernel'i `zImage` olarak verir. `bootm` için `uImage`'e çevir (ya da `uImage` yerine `zImage` gönderip `bootm` yerine `bootz` kullan):

```
mkimage -A arm -O linux -T kernel -C none -a 0x80008000 -e 0x80008000 -n Linux -d zImage uImage
```

### Her projede baştan mı derleyeceğim?

Hayır. Hangi dosyanın ne zaman yeniden üretildiği:

| Dosya | Ne zaman yeniden derlenir? |
|---|---|
| SPL + U-Boot | Neredeyse hiç. Bir kez derlenir, uzun süre aynısı kullanılır. |
| Kernel ( `uImage` ) | Yeni bir sürücü eklendiğinde veya kernel ayarı değiştiğinde. |
| DTB | Donanım ayarı (pin, I2C/SPI cihazı vb.) değiştiğinde; sadece `make dtbs` yeterli. |
| initramfs / rootfs | En sık değişen kısım; kendi uygulamanı buraya koyarsın. Sadece `cpio` + `mkimage` adımı tekrarlanır. |

Gerçekten **senin yazdığın** kısım: kendi uygulaman, gerekirse bir kernel sürücüsü ve DTS'teki küçük değişiklikler. Geri kalanı hazır projelerin derlenmesidir.

> **İpucu**
>
> Önce kursun hazır dosyalarıyla seri boot'u bir kez çalıştır. Sonra dosyaları **teker teker** kendi derlediklerinle değiştir. Bir şey bozulursa hangi dosyada olduğunu hemen anlarsın.

---

## 7.8 Adım Adım Seri Porttan Boot

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; sürüm numaraları, tarih, boyut ve hız değerleri senin derlemene göre farklı olacaktır.

### Adım 0 — Bilgisayarda hazırlık

Bütün dosyaları tek bir klasörde topla (örnekte `~/bbb` ) ve U-Boot başlıklarını `mkimage -l` ile kontrol et:

```
mehmet@ubuntu: ~/bbb
mehmet@ubuntu:~/bbb$ ls -l
-rw-r--r-- 1 mehmet mehmet   24884 Eki 17 10:02 am335x-boneblack.dtb
-rw-r--r-- 1 mehmet mehmet 1867312 Eki 17 10:05 initramfs
-rw-r--r-- 1 mehmet mehmet  403524 Eki 17 09:58 u-boot.img
-rw-r--r-- 1 mehmet mehmet   92612 Eki 17 09:58 u-boot-spl.bin   <-- < 109 KB olmalı
-rw-r--r-- 1 mehmet mehmet 4380760 Eki 17 10:01 uImage
mehmet@ubuntu:~/bbb$ mkimage -l uImage
Image Name:   Linux-3.8.13
Image Type:   ARM Linux Kernel Image (uncompressed)
Data Size:    4380696 Bytes = 4278.02 KiB = 4.18 MiB
Load Address: 80008000
Entry Point:  80008000
mehmet@ubuntu:~/bbb$ mkimage -l initramfs
Image Name:   initramfs
Image Type:   ARM Linux RAMDisk Image (gzip compressed)  <-- U-Boot başlığı var
Data Size:    1867248 Bytes = 1823.48 KiB = 1.78 MiB
```

```
cd ~/bbb
ls -l
mkimage -l uImage
mkimage -l initramfs
```

`u-boot-spl.bin` **109 KB'tan küçük** olmalı; ROM kodu onu iç SRAM'e koyar ve SRAM'de daha fazla yer yoktur.

### Adım 1 — Seri kabloyu bağla, Minicom'u hazırla

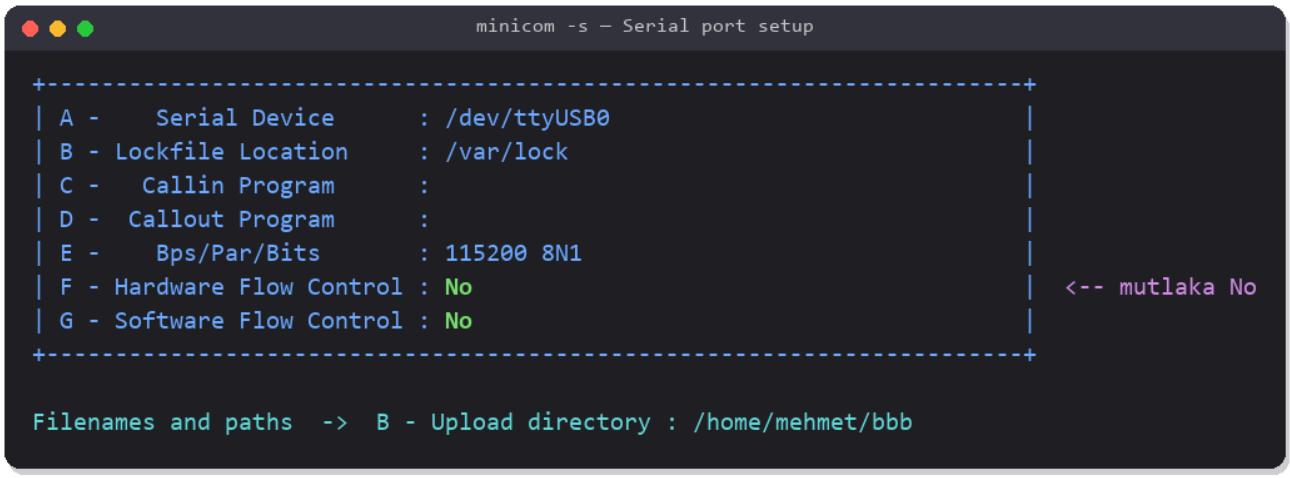
USB-TTL kablosunu J1 header'a bağla (**GND, RX, TX**; kablonun RX'i kartın TX'ine, TX'i kartın RX'ine). Port adını bulma, `dialout` grubu ve Minicom ayarları Bölüm 2'de anlatıldı. Kısaca:

```
mehmet@ubuntu: ~/bbb
mehmet@ubuntu:~/bbb$ sudo dmesg | tail -3
[ 5121.33] usb 1-2: FTDI USB Serial Device converter detected
[ 5121.34] usb 1-2: FTDI USB Serial Device converter now attached to ttyUSB0  <--
mehmet@ubuntu:~/bbb$ ls -l /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 0 Eki 17 10:10 /dev/ttyUSB0
mehmet@ubuntu:~/bbb$ sudo usermod -aG dialout $USER     # bir kez; sonra oturumu kapat-aç
mehmet@ubuntu:~/bbb$ sudo apt install minicom lrzsz      # lrzsz = sx / sb / rz programları
mehmet@ubuntu:~/bbb$ which sx sb
/usr/bin/sx
/usr/bin/sb
mehmet@ubuntu:~/bbb$ cd ~/bbb && minicom -D /dev/ttyUSB0 -b 115200
```

```
sudo dmesg | tail -3
sudo usermod -aG dialout $USER      # bir kez; sonra oturumu kapatıp aç
sudo apt install minicom lrzsz
minicom -s                          # ayarlar (aşağıda)
cd ~/bbb && minicom -D /dev/ttyUSB0 -b 115200
```

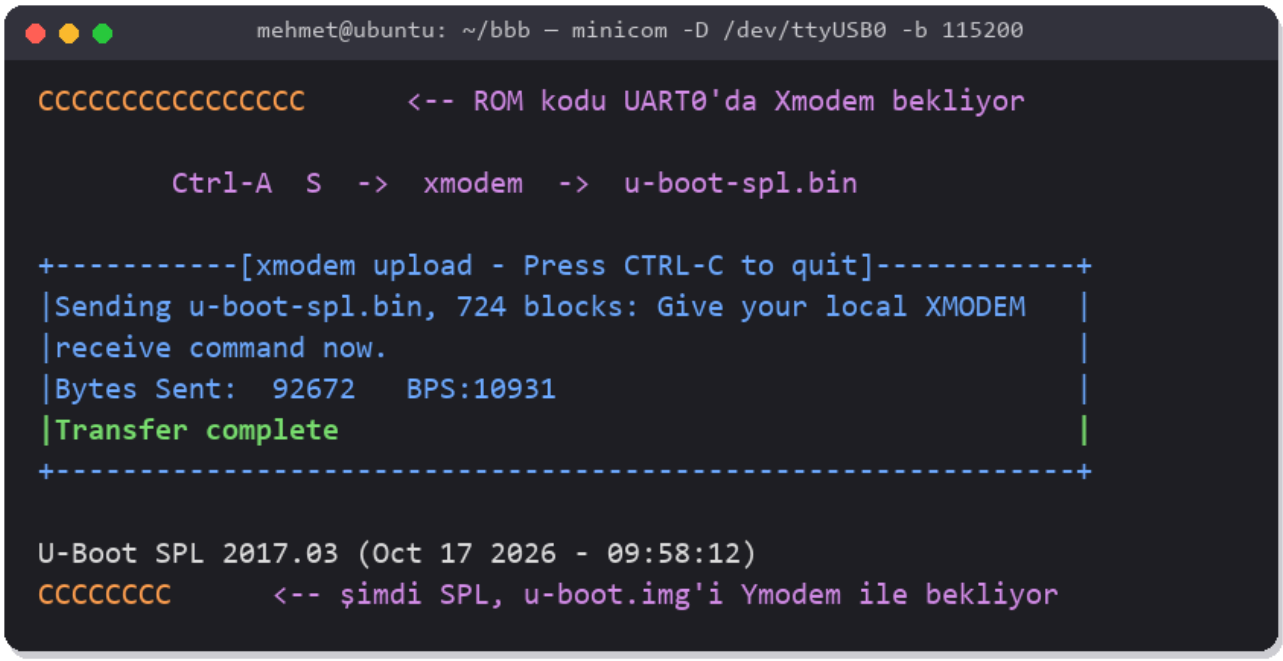
> **Önemli**
>
> Minicom Xmodem/Ymodem gönderirken işi kendisi yapmaz, **`lrzsz`** paketindeki `sx` (Xmodem) ve `sb` (Ymodem) programlarını çalıştırır. `lrzsz` kurulu değilse dosya gönderilemez.

`minicom -s` → **Serial port setup** ekranında ayarlar şöyle olmalı. **Filenames and paths** menüsünde upload klasörünü `~/bbb` yaparsan dosya seçerken her seferinde klasör aramazsın:




## Adım 2 — Kartı UART boot modunda aç ve SPL'i gönder (Xmodem)

1. SD kartı çıkar, P4 mini USB kablosunu çıkar, 5V adaptörü hazırla (7.3).
2. Minicom açıkken **S2**'ye basılı tut, adaptörü tak / **S3**'e bas, sonra S2'yi bırak.
3. Ekranda `CCCC...` akmaya başlar: ROM kodu UART0'da Xmodem ile dosya bekliyor.
4. Minicom'da `Ctrl-A` → `S` → **xmodem** → **`u-boot-spl.bin`** seç, gönder.

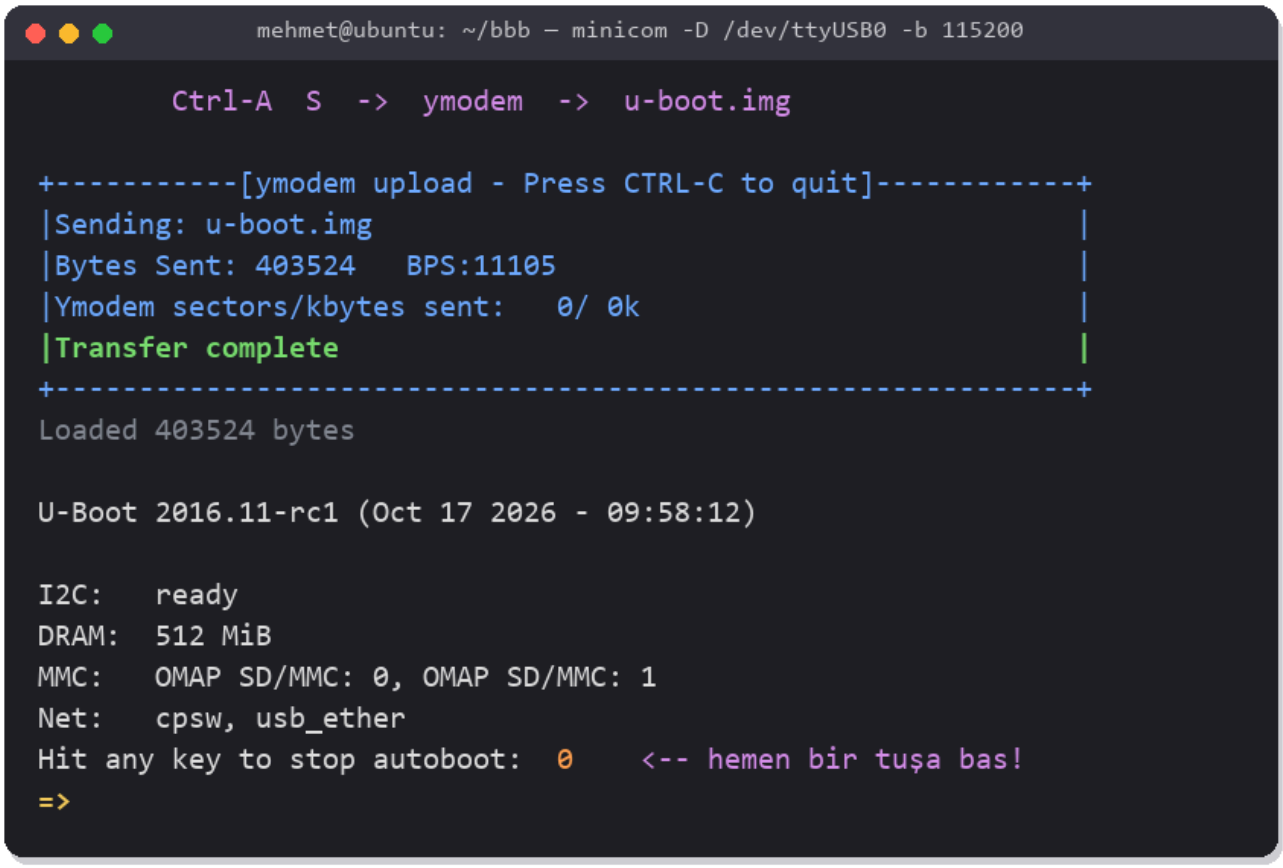



Transfer bitince SPL çalışır, DDR'ı hazırlar, `U-Boot SPL ...` yazısını basar ve tekrar `CCCC` basmaya başlar. Bu sefer bekleyen **SPL**'dir ve **Ymodem** ister.

### Adım 3 — U-Boot'u gönder (Ymodem)

> **Not**
>
> Kursun `serial-boot` klasöründeki dosyalar: SPL **2017.03**, U-Boot **2016.11-rc1**. Bu sürümlerde U-Boot'un komut istemi `=>` 'dir. Bölüm 4'teki eski Ångström U-Boot'u (2013.04) ise `U-Boot#` gösterir. Komutlar ikisinde de aynıdır.

1. `Ctrl-A` → `S` → **ymodem** → `u-boot.img` seç, gönder.
2. U-Boot açılır. `Hit any key to stop autoboot` çıkınca **hemen bir tuşa bas.**




> **Uyarı**
>
> Autoboot'u durdurmazsan U-Boot kendi `bootcmd` 'ini çalıştırır ve kartın **eMMC**'sindeki Linux'u (ör. Debian) açar. O zaman seri boot'un amacı boşa gider; kartı kapatıp Adım 2'den başla.

### Adım 4 — Kernel'i RAM'e al: `loadx 0x82000000`

```
=> loadx 0x82000000
```

U-Boot Xmodem ile beklemeye başlar. Minicom'da `Ctrl-A` → `S` → **xmodem** → `uImage`.

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

=> loadx 0x82000000
## Ready for binary (xmodem) download to 0x82000000 at 115200 bps...
C
        Ctrl-A  S  ->  xmodem  ->  uImage
+-----------[xmodem upload - Press CTRL-C to quit]------------+
|Sending uImage, 34225 blocks: Give your local XMODEM receive |
|command now.                                                 |
|Bytes Sent:4380800   BPS:11012                               |  <-- ~6.5 dk sürer
|Transfer complete                                            |
+-------------------------------------------------------------+
## Total Size      = 0x0042d880 = 4380800 Bytes
## Start Addr      = 0x82000000
=>
```

> **Not**
>
> 115200 baud'da hız en fazla ~11 KB/s'dir. 4 MB'lık bir `uImage` yaklaşık **6–7 dakika** sürer; bu normaldir. Xmodem son paketi 128 bayta tamamladığı için `Total Size` dosya boyutundan biraz büyük görünür, sorun değil.

### Adım 5 — DTB ve initramfs'i RAM'e al

```
=> loadx 0x88000000        (Minicom: xmodem -> am335x-boneblack.dtb)
=> loadx 0x88080000        (Minicom: xmodem -> initramfs)
```

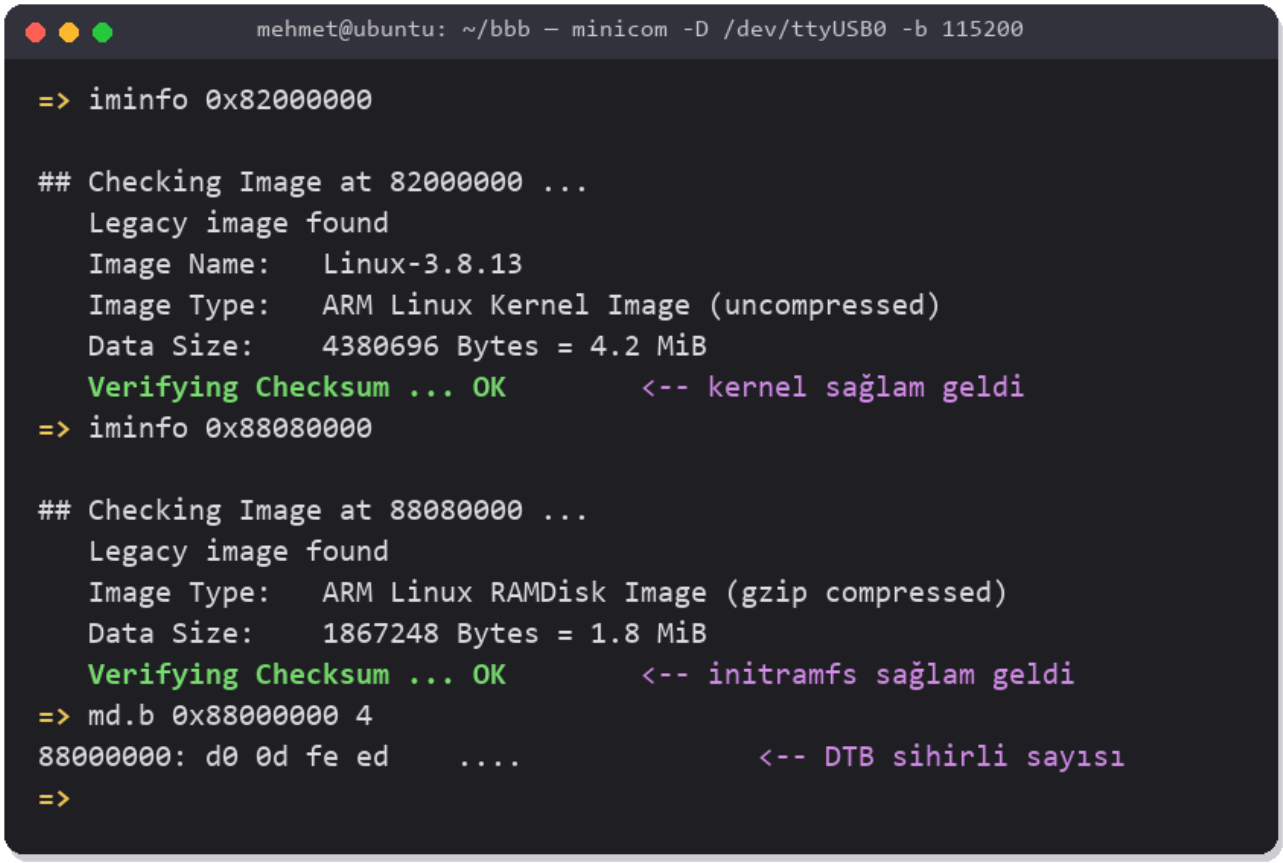
```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

=> loadx 0x88000000
## Ready for binary (xmodem) download to 0x88000000 at 115200 bps...
        Ctrl-A  S  ->  xmodem  ->  am335x-boneblack.dtb
|Transfer complete                                            |
## Total Size      = 0x00006180 = 24960 Bytes
## Start Addr      = 0x88000000
=> loadx 0x88080000
## Ready for binary (xmodem) download to 0x88080000 at 115200 bps...
        Ctrl-A  S  ->  xmodem  ->  initramfs
|Transfer complete                                            |
## Total Size      = 0x001c7e80 = 1867392 Bytes
## Start Addr      = 0x88080000
=>
```

### Adım 6 — Yüklenenleri kontrol et (isteğe bağlı ama önerilir)

Boot etmeden önce dosyaların sağlam geldiğini görmek, sonradan "neden açılmadı?" diye uğraşmaktan iyidir:

```
=> iminfo 0x82000000
=> iminfo 0x88080000
=> md.b 0x88000000 4
```




- `iminfo`, U-Boot başlığındaki CRC'yi RAM'deki veriyle karşılaştırır. `Verifying Checksum ... OK` görmelisin.
- DTB'nin U-Boot başlığı yoktur; ilk 4 baytı her zaman `d0 0d fe ed` olmalıdır.

### Adım 7 — `bootargs` ayarla ve `bootm` ile boot et

```
=> setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
=> bootm 0x82000000 0x88080000 0x88000000
```

Boot etmeden önce kernel'e **rootfs'in nerede olduğunu** `bootargs` ile bildiriyoruz:

| Parametre | Anlamı |
|---|---|
| `console=ttyO0,115200` | Kernel mesajlarını UART0'a, 115200 baud ile bas (kursun kernel'i 3.8.13 için `ttyO0` ; 7.9 (h)). |

| Parametre | Anlamı |
|---|---|
| `root=/dev/ram0` | Kök dosya sistemi (rootfs) bir disk değil, **RAM**'de duruyor. |
| `rw` | Rootfs okunur-yazılır bağlansın. |
| `initrd=0x88080000` | initramfs'i RAM'de **`0x88080000`** adresine yüklediğimizi kernel'e bildirir (Adım 5'teki `loadx` adresiyle aynı olmalı). |
| `bootm <kernel> <initramfs> <dtb>` | Sıra önemli: **kernel – ramdisk – DTB**. Ramdisk yoksa ortaya `-` yazılır ( `bootm 0x82000000 - 0x88000000` ). |

> **Not**
>
> `bootm` 'in ikinci parametresi ( `0x88080000` ) de initramfs'in adresini ve boyutunu kernel'e DTB'nin `/chosen` düğümü üzerinden bildirir. Yani adres iki yerden veriliyor; `bootargs` 'taki `initrd=` ile `bootm` 'deki adresin **aynı** olduğuna dikkat et. `printenv bootargs` ile yazdığını kontrol edebilirsin.

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200
=> setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
=> bootm 0x82000000 0x88080000 0x88000000
## Booting kernel from Legacy Image at 82000000 ...
   Image Name:   Linux-3.8.13
   Verifying Checksum ... OK
## Loading init Ramdisk from Legacy Image at 88080000 ...
   Image Type:   ARM Linux RAMDisk Image (gzip compressed)
   Verifying Checksum ... OK
## Flattened Device Tree blob at 88000000
   Booting using the fdt blob at 0x88000000
   Loading Kernel Image ... OK
   Loading Ramdisk to 8fe38000, end 8ffffdf0 ... OK
   Loading Device Tree to 8fe2e000, end 8fe37133 ... OK

Starting kernel ...

[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 3.8.13 (mehmet@ubuntu) (gcc 4.9) #1 SMP
[    0.000000] Kernel command line: console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
[    1.732050] Trying to unpack rootfs image as initramfs...  <-- initramfs
[    2.104512] Freeing unused kernel memory: 380K
```

## Adım 8 — Linux açıldı

Kernel initramfs'i açar, içindeki `/init` (BusyBox) çalışır ve seri konsoldan kabuk gelir:

BusyBox (initramfs) — seri konsol

```
Please press Enter to activate this console.
/ # cat /proc/cmdline
console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
/ # mount
rootfs on / type rootfs (rw)        <-- rootfs RAM'de
proc on /proc type proc (rw,relatime)
sysfs on /sys type sysfs (rw,relatime)
/ # uname -a
Linux (none) 3.8.13 #1 SMP Sat Oct 17 10:01:44 2026 armv7l GNU/Linux
/ # free -m
             total       used       free     shared    buffers
Mem:           497         14        483          0          0
```

`mount` çıktısında `/` 'nin **`rootfs`** olduğunu görürsün: Kart ne SD karta ne eMMC'ye dokunmadan, tamamen seri porttan gelen dosyalarla RAM'den çalışıyor.

> **İpucu**
>
> RAM'deki her şey kart kapanınca silinir. Bir sonraki açılışta bütün adımları (SPL'den itibaren) tekrar yapman gerekir.

### Komutların kısa özeti

```
[Kart: S2 basılı + güç, ekranda CCCC]
Minicom: Ctrl-A S -> xmodem -> u-boot-spl.bin
Minicom: Ctrl-A S -> ymodem -> u-boot.img
[Hit any key -> tuşa bas]
=> loadx 0x82000000          -> xmodem -> uImage
=> loadx 0x88000000          -> xmodem -> am335x-boneblack.dtb
=> loadx 0x88080000          -> xmodem -> initramfs
=> setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
=> bootm 0x82000000 0x88080000 0x88000000
```

---

## 7.9 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~/bbb — minicom -D /dev/ttyUSB0 -b 115200

=> bootm 0x82000000 0x88080000 0x88000000
## Loading init Ramdisk from Legacy Image at 88080000 ...
Wrong Ramdisk Image Format            <-- initramfs'te U-Boot başlığı yok
Ramdisk image is corrupt or invalid
=> iminfo 0x82000000
## Checking Image at 82000000 ...
   Verifying Checksum ... Bad Data CRC  <-- transfer bozuk / adres çakışması

Starting kernel ...
    <-- sonrası boş: console= yanlış ya da DTB uyumsuz

[    2.11] VFS: Cannot open root device "ram0" or unknown-block(1,0)
[    2.11] Kernel panic - not syncing: VFS: Unable to mount root fs  <-- /init yok

[    2.20] Failed to execute /init.  Attempting defaults...    <-- ARM değil / kütüphane eksik
```

### a) Ekranda hiç `cccc` çıkmıyor

| Olası neden | Çözüm |
|---|---|
| SD kart takılı | ROM önce MMC0'ı dener, SD'de MLO varsa oradan açılır. SD kartı çıkar. |
| P4 mini USB kablosu takılı | ROM USB0'da 4.5 dakika bekler. Kabloyu çıkar, kartı 5V adaptörle besle (7.3). |
| S2 basılı değildi | Kart eMMC'den açılır (ekranda eMMC'deki U-Boot/Linux çıkar). Kartı kapat, S2 basılıyken tekrar aç. |
| TX / RX ters bağlı ya da GND bağlı değil | Kablonun TX'i → kartın RX'i, RX'i → kartın TX'i. GND mutlaka bağlı olmalı. |
| Yanlış port / baud | `dmesg` ile doğru `/dev/ttyUSBx`'i bul; ayar 115200 8N1 olmalı. |
| Minicom'u kart açıldıktan sonra açtın | `C`'leri kaçırmış olabilirsin. Önce Minicom'u aç, sonra kartı aç. |

### b) `cccc` var ama transfer başlamıyor / yarıda kesiliyor

| Olası neden | Çözüm |
|---|---|
| **Hardware Flow Control: Yes** | Minicom karta veri göndermez. `minicom -s` ile **No** yap (en sık yapılan hata!). |
| `lrzsz` kurulu değil (`sx: not found`) | `sudo apt install lrzsz` |
| Dosya seçmek çok uzun sürdü, ROM vazgeçti | Upload klasörünü önceden ayarla, `C` görünür görünmez gönder. `C` akışı durduysa kartın gücünü kesip S2 ile baştan aç. |
| Yanlış protokol | ROM ve `loadx` → **xmodem**; SPL'in `u-boot.img` için beklediği ve `loady` → **ymodem**. Karıştırılırsa transfer hiç başlamaz veya `Transfer incomplete` olur. |

| Olası neden | Çözüm |
|---|---|
| Transfer sırasında klavyeye basıldı | Gelen tuşlar Xmodem paketlerini bozar. Transfer bitene kadar dokunma. |

### c) SPL gönderildi ama hiçbir şey olmuyor

| Olası neden | Çözüm |
|---|---|
| `u-boot-spl.bin` yerine `MLO` gönderildi | MLO'nun başında GP header var; ROM onu kod sanıp çalıştırır ve kilitlenir. UART boot'ta **ham** `u-boot-spl.bin` gönderilir. |
| SPL 109 KB'tan büyük | ROM iç SRAM'e sığdıramaz. SPL'deki gereksiz özellikleri kapatıp yeniden derle. |
| SPL farklı bir kart/işlemci için derlenmiş | BBB için uygun defconfig ile yeniden derle (9.3). |

### d) SPL çalışıyor ama `u-boot.img` yüklenmiyor

| Olası neden | Çözüm |
|---|---|
| SPL, UART'tan Ymodem alma desteğiyle derlenmemiş ( `CONFIG_SPL_YMODEM_SUPPORT` ) | SPL başka bir aygıt (MMC) arar ve hata verir. Config'te YMODEM desteğini aç, yeniden derle. |
| `u-boot.img` Xmodem ile gönderildi | SPL Ymodem bekler; **ymodem** seç. |
| `u-boot.img` yerine `u-boot.bin` gönderildi | SPL, başında mkimage başlığı olan `u-boot.img` 'i bekler. |

### e) U-Boot açıldı ama kendi kendine eMMC'den boot etti

`Hit any key to stop autoboot` sırasında tuşa basılmadı. Kartı kapatıp baştan başla; geri sayım başlar başlamaz bir tuşa bas.

> **Dikkat**
>
> Seri boot sırasında U-Boot'ta `saveenv` **yapma**. Değişkenler eMMC'deki ortama yazılır ve kartın normal açılışını bozabilir.

### f) Transfer çok yavaş

115200 baud'da ~11 KB/s normaldir; `uImage` 6–7 dakika, initramfs birkaç dakika sürer. Hızlandırmak için:

- **Baud yükseltme:** `loadx` / `loady` ikinci parametre olarak baud alır: `loady 0x82000000 460800` . U-Boot hızı değiştirmeni ister; Minicom'da `Ctrl-A` → `P` ile aynı hıza geç, Enter'a bas, gönder. Bitince U-Boot 115200'e dönmeni ister; Minicom'u geri al ve **ESC**'ye bas. (Her USB-TTL kablo yüksek hızlarda güvenilir çalışmaz; hata alırsan 115200'e dön.)
- **Ymodem:** `loady` 1 KB'lık paketlerle çalıştığı için `loadx` 'ten biraz daha hızlıdır.

- **TFTP:** U-Boot açıldıktan sonra Ethernet kablosu varsa `tftpboot` ile saniyeler içinde yüklenir (Bölüm 8).

### g) `bootm` hataları

| Hata | Neden | Çözüm |
|---|---|---|
| `Wrong Ramdisk Image Format` / `Ramdisk image is corrupt or invalid` | initramfs'in başında U-Boot başlığı yok veya yanlış adrese yüklendi | Bilgisayarda `mkimage -T ramdisk ...` ile başlık ekle (7.7). `iminfo 0x88080000` ile kontrol et. |
| `Bad Data CRC` | Transfer sırasında veri bozuldu ya da bir dosya diğerinin üstüne yazıldı | Dosyayı tekrar gönder. Adresleri 7.5'teki gibi kullan; DTB 512 KB'ı aşmamalı. |
| `Bad Magic Number` / `Wrong Image Format for bootm command` | `uImage` yerine `zImage` gönderildi veya adres yanlış | Kernel'i `make uImage LOADADDR=0x80008000` ile derle ya da `zImage` için `bootz` kullan. |
| DTB bulunamadı / `FDT_ERR_BADMAGIC` | DTB bozuk ya da yanlış adrese yüklendi | `md.b 0x88000000 4` → `d0 0d fe ed` görmelisin. Görmüyorsan DTB'yi tekrar gönder. |

### h) `Starting kernel ...` yazıp donuyor

| Olası neden | Çözüm |
|---|---|
| `console=` yanlış | Eski kernel'ler (3.x, TI omap-serial sürücüsü) → `ttyO0`. Daha yeni (4.x ve sonrası) kernel'lerde genellikle `ttyS0`. Kernel aslında açılmış olabilir, sadece mesajları başka yere basıyordur. |
| DTB, kernel sürümüyle uyumsuz | DTB'yi **aynı kernel kaynağından** derlenmiş olanla değiştir; BBB için `am335x-boneblack.dtb` ( `am335x-bone.dtb` değil). |
| `bootm` parametre sırası yanlış | Sıra: `bootm <kernel> <ramdisk> <dtb>`. |

### i) Kernel açılıyor ama panic veriyor

| Hata | Neden | Çözüm |
|---|---|---|
| `VFS: Unable to mount root fs on unknown-block(1,0)` | initramfs içinde `/init` yok; kernel `root=/dev/ram0`'ı bağlamaya çalışıyor ve başarısız oluyor | rootfs klasöründe `ln -s sbin/init init` yapıp initramfs'i yeniden oluştur, ya da `bootargs`'a `rdinit=/sbin/init` ekle. |
| `VFS: Unable to mount root fs on unknown-block(0,0)` | Kernel ramdisk'i hiç almamış veya `CONFIG_BLK_DEV_INITRD` kapalı | `bootm`'in ikinci parametresini kontrol et; kernel config'te initramfs/initrd desteğini aç. |

| Hata | Neden | Çözüm |
|---|---|---|
| `Failed to execute /init` / `No working init found` | initramfs'teki programlar ARM için derlenmemiş veya kütüphaneler eksik | BusyBox'ı `CROSS_COMPILE=arm-linux-gnueabihf-` ile ve tercihen **static** derle; dinamik derlediysen `lib/` klasörüne gerekli kütüphaneleri koy. `file bin/busybox` → `ARM` yazmalı. |
| `RAMDISK: incomplete write` | initramfs yerine ext2 **initrd** imajı kullanılıyor ve kernel'in varsayılan ramdisk boyutundan büyük | `bootargs`'a `ramdisk_size=<KB>` ekle. |

### j) Ekranda bozuk / anlamsız karakterler

Baud rate uyuşmuyor. Minicom 115200 8N1 olmalı. Baud yükseltme (f) denediysen Minicom'u tekrar 115200'e al.

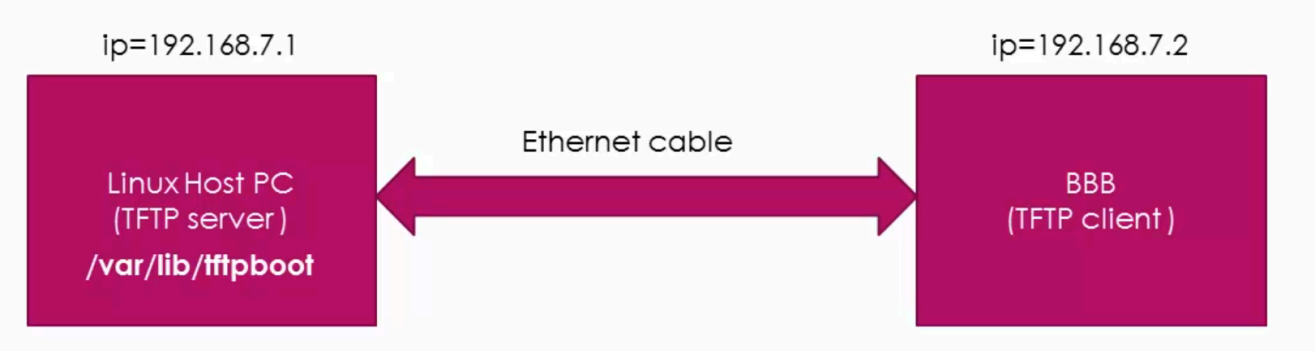
# Bölüm 8 — TFTP ile Ağdan Boot

> **Bu bölümde:**
>
> - Bilgisayarına bir TFTP sunucusu ( `tftpd-hpa` ) kuracaksın.
> - Kernel, DTB ve initramfs'i her açılışta **Ethernet üzerinden** host'tan çekerek kartı açacaksın: önce komutları elle yazarak, sonra `uEnv.txt` ile otomatik olarak.
> - Neden bazı dosyaların yine de SD kartta durmak zorunda olduğunu anlayacaksın.
> - Challenge: eMMC → SD → TFTP → NFS sırasıyla deneyen genel bir `uEnv.txt` yazacaksın.
>
> **Ön koşul:** Bölüm 4 — SD kart, Bölüm 6 — `uEnv.txt` ve U-Boot komutları, Bölüm 7 — RAM adresleri ve `bootm` .

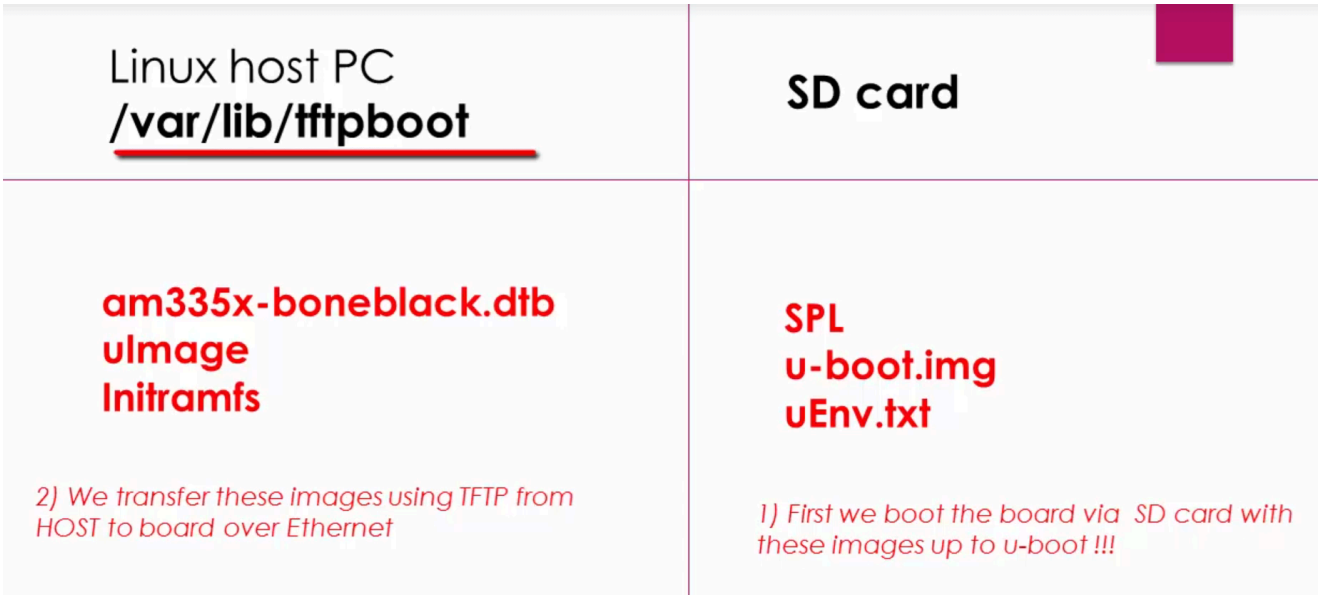
---

## 8.1 Genel Bakış

Host bilgisayar **TFTP sunucusu**, BeagleBone Black ise **TFTP istemcisi** olur. İkisi Ethernet kablosuyla bağlanır:

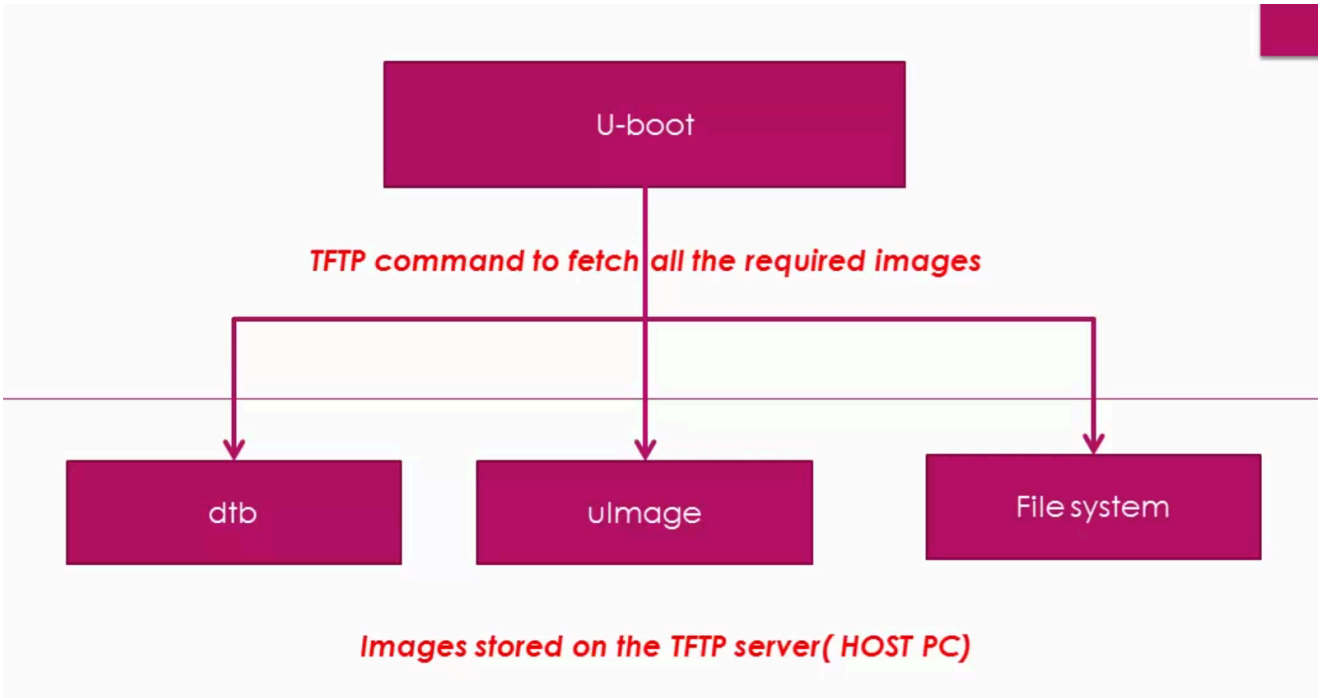


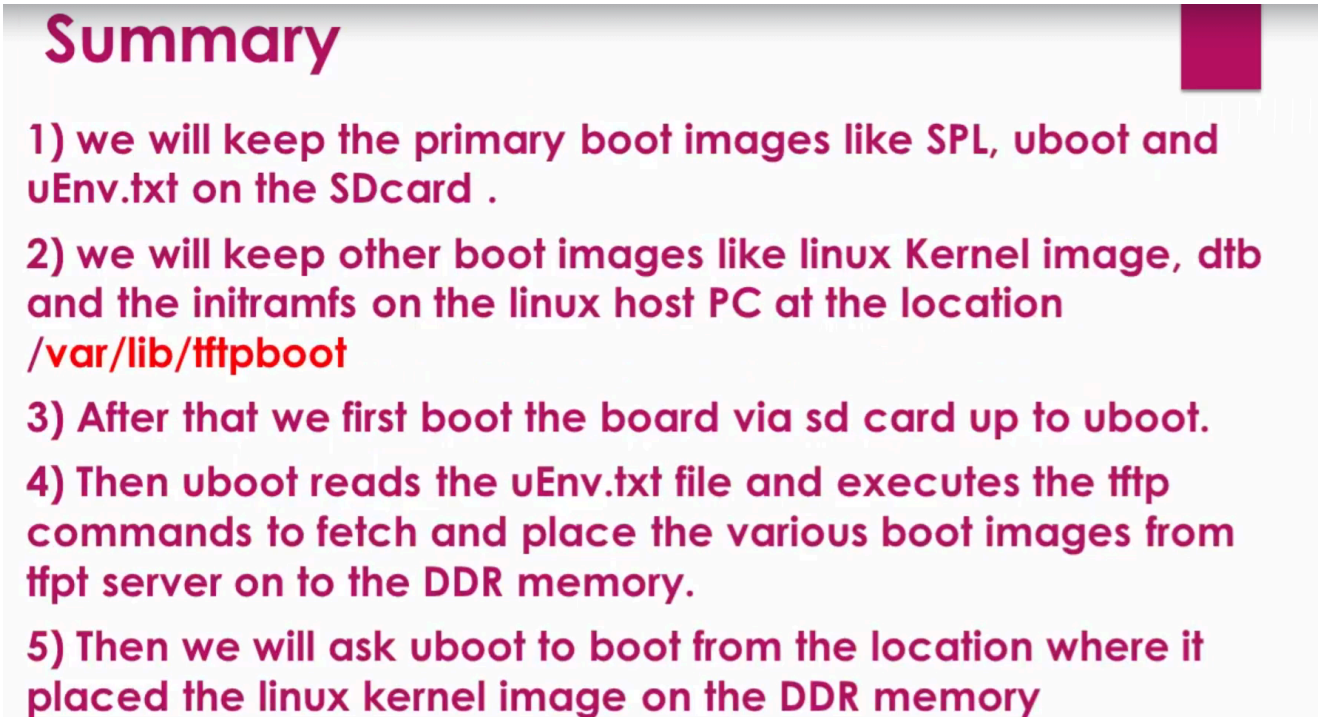

Dosyalar iki gruba ayrılır:



| Nerede? | Dosyalar |
|---|---|
| **SD kart** | SPL (MLO), u-boot.img, uEnv.txt |
| **Host PC** `/var/lib/tftpboot` | am335x-boneblack.dtb, uImage, initramfs |

Kart önce SD karttan U-Boot'a kadar açılır. Sonra U-Boot, uEnv.txt'deki `tftp` komutlarıyla diğer dosyaları host'tan çekip DDR'a yerleştirir:




---

## 8.2 Neden Bütün Dosyalar TFTP ile Gönderilmiyor?

Kısaca: **TFTP isteğini yapacak kod U-Boot'un içinde.** U-Boot ağdan gelecek olsaydı, onu ağdan isteyecek bir program olmazdı. Bu bir tavuk-yumurta problemi.

### Boot zinciri

```
1. ROM Code (çipin içinde, değiştirilemez)
   │  SD karttan MLO/SPL'i bulur, dahili SRAM'e (~109 KB) yükler
   ▼
2. SPL / MLO
   │  saatleri ve pinleri ayarlar, DDR RAM'i başlatır,
   │  u-boot.img'yi DDR'a yükler
   ▼
3. u-boot.img  (tam U-Boot)
   │  Ethernet + MMC + UART sürücülerini başlatır,
   │  uEnv.txt'yi okur, TFTP istemcisini çalıştırır
   ▼
TFTP ile host'tan:  uImage, .dtb, initramfs  →  DDR  →  bootm
```

- **SPL**, DDR RAM'i başlatır. DDR hazır olmadan büyük dosyalar (U-Boot, kernel) bir yere yüklenemez.
- **u-boot.img**, U-Boot'un kendisidir. Ethernet sürücüsü ve TFTP istemcisi buradadır.
- **uEnv.txt**, U-Boot'a `ipaddr`, `serverip` ve hangi dosyayı hangi adrese çekeceğini söyler. U-Boot bu bilgiyi okuyabilmek için dosyanın yerel bir depolamada durması gerekir.
- **TFTP istemci/sunucu protokolüdür.** Host dosyayı kendiliğinden göndermez, kartın istemesi gerekir. İsteyecek kod (U-Boot) çalışmadan kart ağdan hiçbir şey alamaz.
- **ROM Code'un okuyabileceği yerler sabittir.** BBB'nin boot listesinde Ethernet yoktur (3.5). AM335x'in ROM'u aslında Ethernet'ten boot'u destekler, ama BBB'de bu seçeneği açmak için donanımda değişiklik gerekir.

> **Not**
>
> U-Boot'un başlattığı sürücüler **kalıcı değildir**. U-Boot sadece dosya yükleyip kernel'i çalıştırmak için gereken sürücüleri başlatır. Kernel açılınca bütün donanımı kendi sürücüleriyle **yeniden** başlatır.

### Bunun amacı: geliştirmeyi hızlandırmak

| Dosya | Ne sıklıkla değişir? | Nerede durur? |
|---|---|---|
| SPL, u-boot.img, uEnv.txt | Neredeyse hiç | SD kart (bir kere kopyalanır) |
| uImage, .dtb, initramfs | Sürekli (kernel derle, DT düzenle, rootfs'e ekle) | Host PC, `/var/lib/tftpboot` |

- Kernel ya da device tree değişince SD kartı çıkarıp takmaya gerek kalmaz. Yeni dosyayı `/var/lib/tftpboot` içine kopyalayıp kartı resetlemek yeterli.

- SD kartın yıpranma ve bozulma riski azalır.
- Sektörde de böyle çalışılır: bootloader kartta sabit durur, kernel (ve sonra NFS ile rootfs) ağdan gelir.

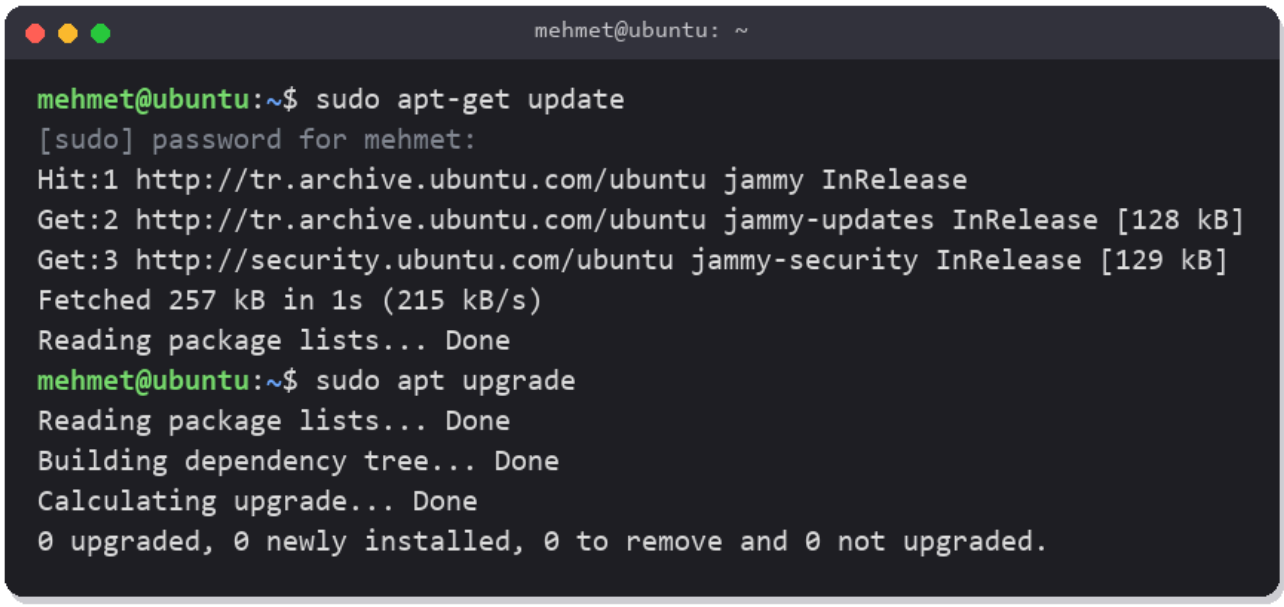
> **İpucu**
>
> Tek cümleyle: SD karttaki SPL + U-Boot kartı **"ağdan dosya isteyebilir"** hale getirir. Ondan sonra gelen her şey TFTP ile host'tan alınır.

---

## 8.3 TFTP Host'u Hazırlama

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; sürüm numaraları, tarih ve PID değerleri senin bilgisayarında farklı olacaktır.

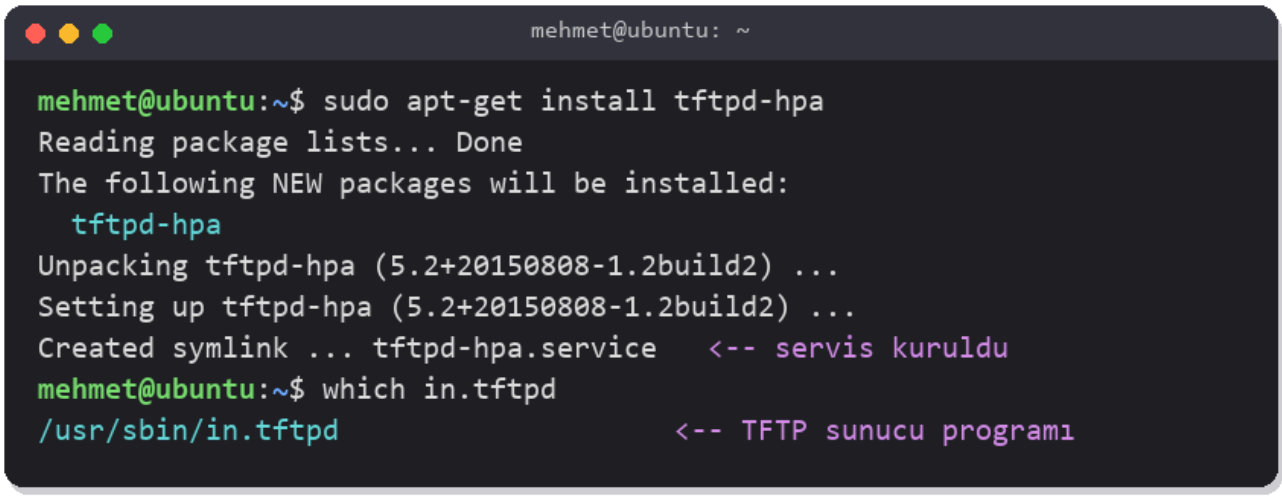
### Adım 1 — Paket listesini güncelle




```
sudo apt-get update
sudo apt upgrade
```

### Adım 2 — TFTP sunucusunu kur

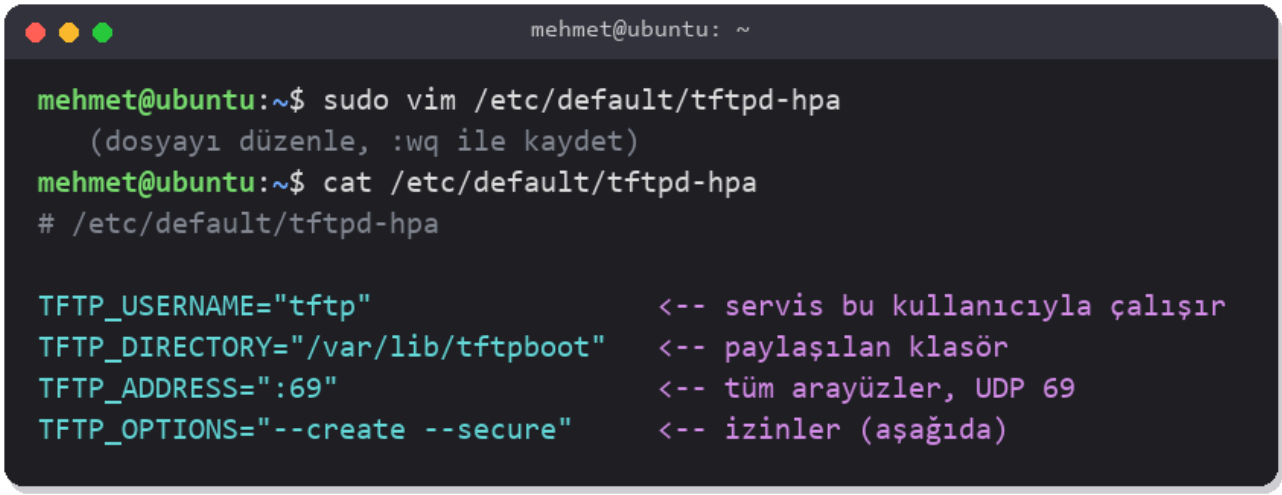
`tftpd-hpa`, Trivial File Transfer Protocol için bir sunucu (daemon) programıdır.




```
sudo apt-get install tftpd-hpa
```

### Adım 3 — Ayar dosyasını düzenle

`/etc/default/tftpd-hpa` dosyasını aç ve içeriği aşağıdaki gibi yap:




```
sudo vim /etc/default/tftpd-hpa
```

```
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/var/lib/tftpboot"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="--create --secure"
```

| Ayar | Anlamı |
|---|---|
| `TFTP_USERNAME` | Servis bu kullanıcının yetkileriyle çalışır |
| `TFTP_DIRECTORY` | Kartın dosya çekeceği klasör |

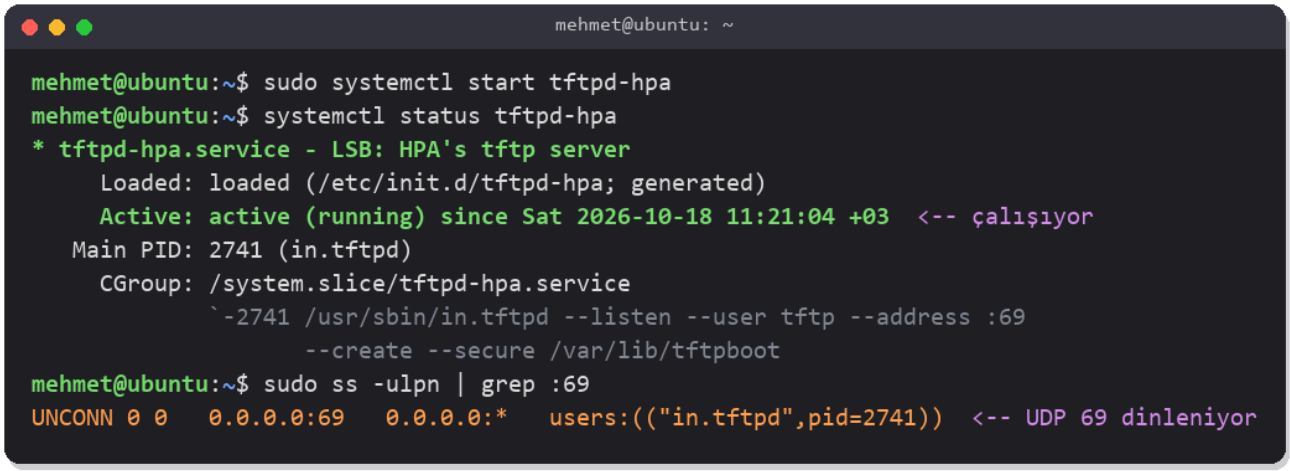
| Ayar | Anlamı |
|---|---|
| `TFTP_ADDRESS=":69"` | Bütün ağ arayüzlerinde UDP 69 portunu dinle |
| `--secure` | Dosya yolları bu klasöre göre yazılır ( `tftp ... uImage` yeterli, tam yol gerekmez) |
| `--create` | İstemcinin klasöre yeni dosya yüklemesine izin verir |

> **Uyarı**
>
> Ayar dosyasında portu **69** olarak bırak. `TFTP_ADDRESS="0.0.0.0:6900"` gibi farklı bir port yazarsan U-Boot'un `tftpboot` komutu sunucuyu bulamaz, çünkü varsayılan olarak 69'a bağlanır. `TFTP_DIRECTORY` için `/home/<user>/tftp` gibi başka bir klasör de kullanabilirsin, ama bu kitapta her yerde `/var/lib/tftpboot` geçiyor.

## Adım 4 — tftpboot klasörünü oluştur

```
sudo mkdir -p /var/lib/tftpboot
sudo chown tftp:tftp /var/lib/tftpboot
sudo chmod -R 777 /var/lib/tftpboot
```

## Adım 5 — TFTP servisini başlat ve kontrol et



```
sudo systemctl start tftpd-hpa
systemctl status tftpd-hpa       # "active (running)" görmelisin
sudo ss -ulpn | grep :69         # UDP 69 dinleniyor mu?
```

> **İpucu**
>
> Ayar dosyasını sonradan değiştirirsen servisi yeniden başlat: `sudo systemctl restart tftpd-hpa`

---

## 8.4 Bağlantılar

| **Kablo** | **Görevi** |
|---|---|
| 5V adaptör (barrel jack) | Güç. Bilgisayarın USB'si 500 mA ile sınırlı, kart boot sırasında resetlenebilir. |
| USB-TTL kablo (J1 header) | Seri konsol (minicom): U-Boot'u durdurmak ve mesajları görmek |
| **Ethernet kablosu** (host ↔ BBB) | **TFTP dosya aktarımı** |

Bu bölümde kullanılan adresler:

| **Cihaz** | **IP** | **U-Boot değişkeni** |
|---|---|---|
| Host PC (TFTP sunucu) | `192.168.27.1` | `serverip` |
| BBB (TFTP istemci) | `192.168.27.2` | `ipaddr` |

İki adres **aynı alt ağda** ( `192.168.27.x` ) olmalı. Kursun şemasındaki `192.168.7.x` adreslerini de kullanabilirsin; önemli olan host ile kartın aynı ağda olması.

---

## 8.5 Yol 1 — Komutları Elle Yazarak TFTP ile Boot

İki yol var:

| | **Yol 1 — Elle (bu başlık)** | **Yol 2 — uEnv.txt ile otomatik (8.6)** |
|---|---|---|
| Nasıl? | Her açılışta U-Boot'u durdurup komutları minicom'a yazarsın | Komutlar SD karttaki `uEnv.txt` 'de durur, U-Boot kendisi çalıştırır |
| Ne zaman? | İlk deneme, hata ayıklama: hangi komutun nerede takıldığını görürsün | Kurulum bir kez çalıştıktan sonra, günlük geliştirmede |

Önce Yol 1 ile bir kere çalıştır, sonra Yol 2'ye geç. Yol 2'deki `uEnv.txt` aynı komutları içerdiği için bir hata olursa Yol 1'e dönüp tek tek denersin.

**Hangi dosyalar?**

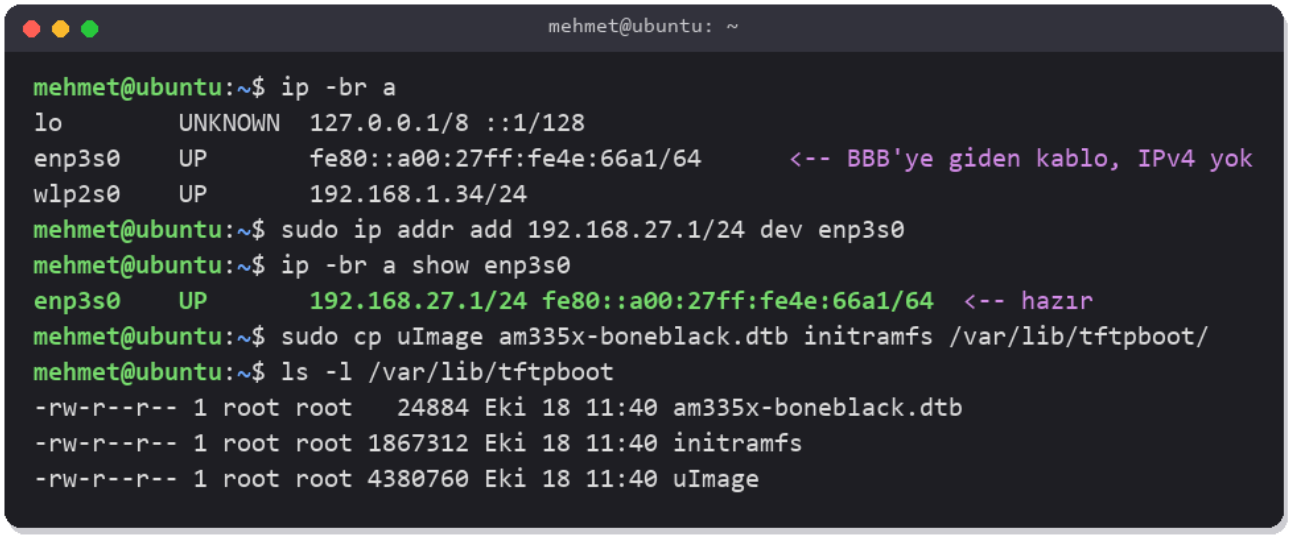
| Nerede? | Dosya | Kaynak |
|---|---|---|
| SD kart (BOOT) | `MLO`, `u-boot.img` | Kursun güncel `SD-boot` imajları (U-Boot 2022.04) ya da Bölüm 9'da kendi derlediklerin |
| Host (`/var/lib/tftpboot`) | `uImage`, `am335x-boneblack.dtb`, `initramfs` | Kursun `tftp-boot` klasörü (Ångström kernel 3.8.13) ya da kendi derlediklerin |

> **Uyarı**
>
> Kursun güncel `SD-boot` U-Boot'unda `bootdelay=0`. Yani `Hit any key to stop autoboot` geri sayımı yok denecek kadar kısa. U-Boot'u durdurmak için kart açılırken **Space tuşuna basılı tut** ya da art arda bas. Bekleme süresini uzatmak 9.4'teki challenge'ın konusu.

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; arayüz adı (`enp3s0`), sürüm, tarih, boyut ve hız değerleri senin ortamında farklı olacaktır.

### Adım 1 — Host'un Ethernet arayüzüne IP ver, dosyaları kopyala

Önce kartın bağlı olduğu Ethernet arayüzünün adını bul. Sonra o arayüze statik IP ver ve boot dosyalarını TFTP klasörüne kopyala:



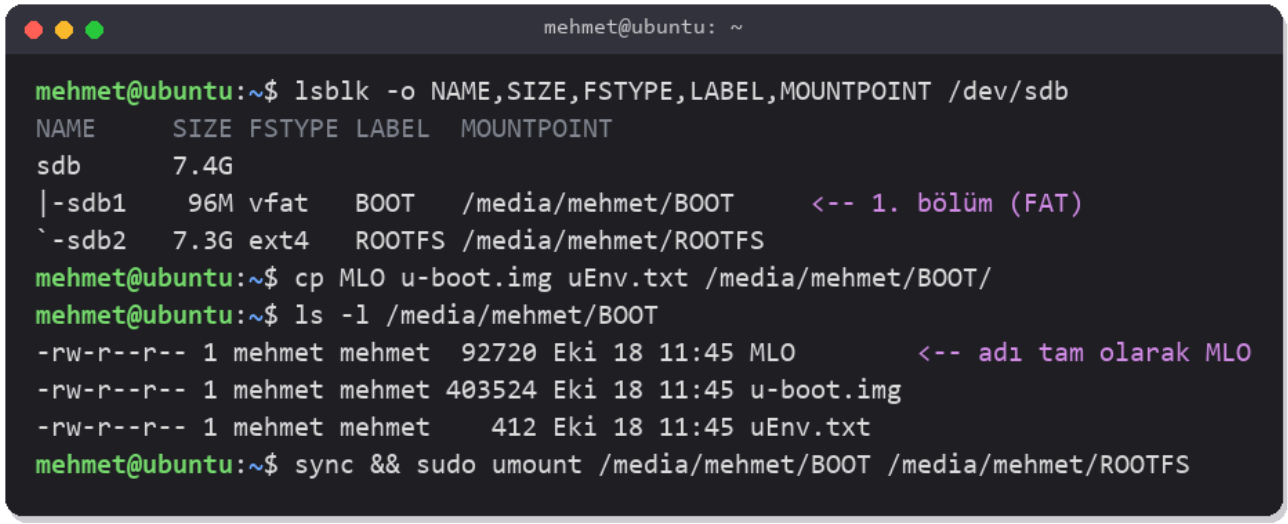

```
ip -br a                                                    # arayüz adını bul (örn. enp3s0, eth0)
sudo ip addr add 192.168.27.1/24 dev enp3s0       # geçici: yeniden başlatınca silinir
sudo cp uImage am335x-boneblack.dtb initramfs /var/lib/tftpboot/
ls -l /var/lib/tftpboot
```

> **İpucu**
>
> `ifconfig` **eski bir komut.** Yeni Ubuntu'larda kurulu bile gelmez ( `net-tools` paketi gerekir). Yerine `ip a` / `ip -br a` kullan.
>
> IP adresinin kalıcı olmasını istersen NetworkManager ile bir profil oluştur:
>
> ```
> nmcli con add type ethernet ifname enp3s0 con-name bbb ipv4.method manual ipv4.addresses
> 192.168.27.1/24
> nmcli con up bbb
> ```

> **Önemli**
>
> **Ubuntu sanal makinedeyse** (VirtualBox/VMware), VM'in ağ ayarını **Bridged** yap ve fiziksel Ethernet kartını seç. NAT modunda kart host'a ulaşamaz.
>
> Güvenlik duvarı açıksa UDP 69'a izin ver: `sudo ufw allow 69/udp`

## Adım 2 — SD kartı hazırla (1. bölüm: BOOT)

Dosyalar SD kartın **1. bölümüne (BOOT, FAT)** gider. ROM Code ve U-Boot sadece bu FAT bölümünü okur. 2. bölüm (ROOTFS, ext4) bu yöntemde kullanılmaz, çünkü kök dosya sistemi initramfs olarak ağdan geliyor.




```
lsblk -o NAME,SIZE,FSTYPE,LABEL,MOUNTPOINT
cp MLO u-boot.img uEnv.txt /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT
```

> **Önemli**
>
> SD karttaki SPL dosyasının adı tam olarak `MLO` olmalı. Seri boot'ta gönderdiğimiz `u-boot-spl.bin` SD kart için uygun değil, çünkü ROM Code SD karttan okurken başında GP başlığı olan `MLO` dosyasını arar. İkisi de U-Boot derlemesinde birlikte çıkar.

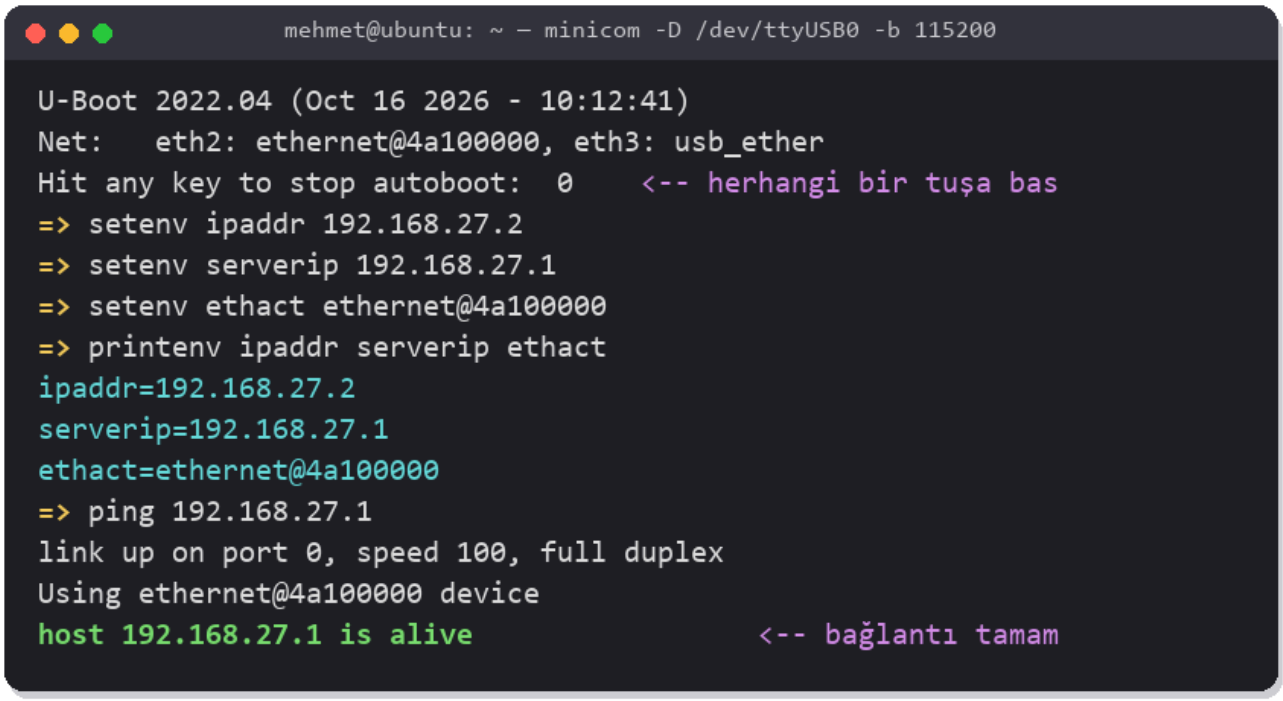
`uEnv.txt` 'nin içeriği için iki seçenek var:

- **Elle deneyeceksen (Adım 3–5):** Önceki bölümlerdeki `uEnv.txt` 'yi ya hiç koyma ya da U-Boot açılırken bir tuşa basıp autoboot'u durdur. Aksi halde kart kendi kendine SD/eMMC'den boot eder.
- **Otomatik boot için:** 8.6'daki `uEnv.txt` 'yi koy.

Kartı SD karttan açmak için **S2 butonuna basılı tutarak** güç ver. Basmazsan kart eMMC'den açılır.

## Adım 3 — U-Boot'u durdur, ağ ayarlarını yap, ping at

Minicom açıkken kartı S2'ye basılı tutarak başlat, `Hit any key to stop autoboot` yazısı çıkınca bir tuşa bas:



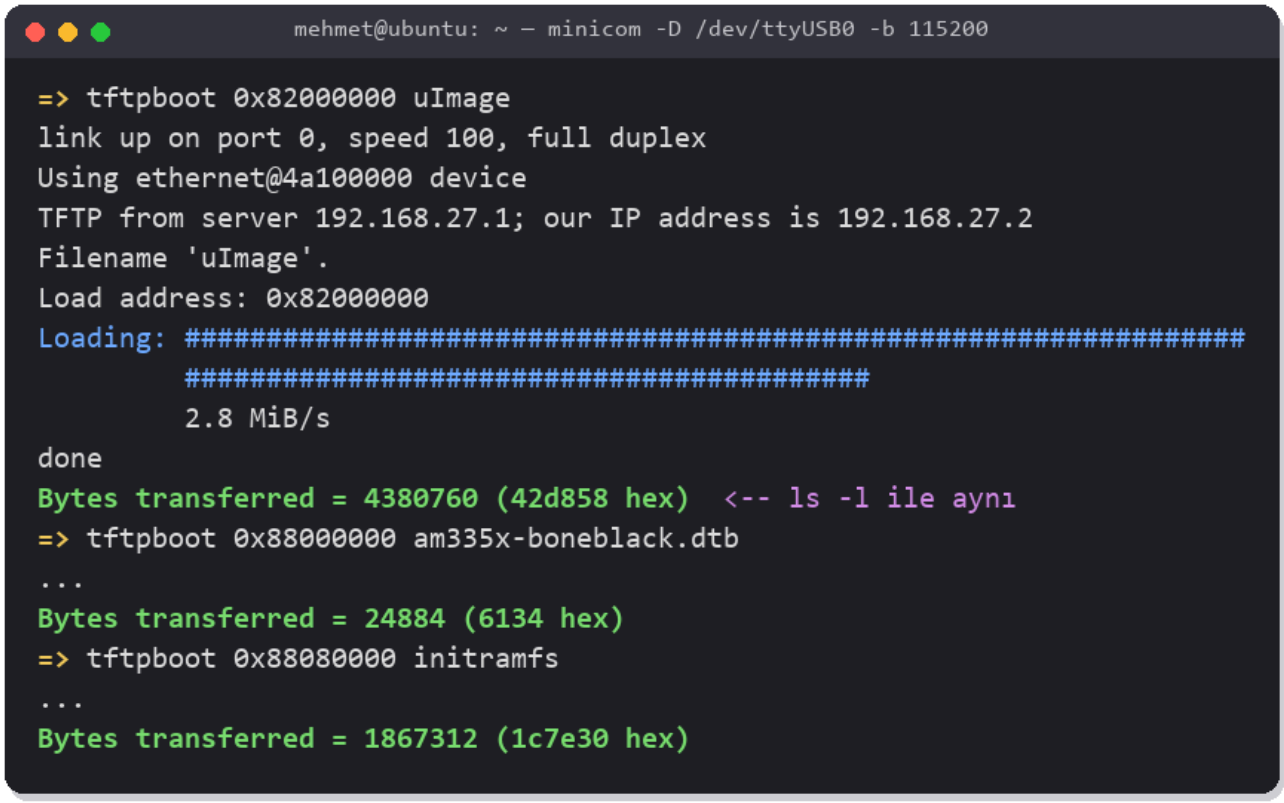

```
=> setenv ipaddr 192.168.27.2
=> setenv serverip 192.168.27.1
=> setenv ethact ethernet@4a100000
=> printenv ipaddr serverip ethact
=> ping 192.168.27.1
```

| Komut | Açıklama |
|---|---|
| `setenv ipaddr ...` | Kartın (istemci) IP adresi |

| Komut | Açıklama |
|---|---|
| `setenv serverip ...` | TFTP sunucusunun (host) IP adresi. `tftpboot` dosyayı bu adresten ister. |
| `setenv ethact ethernet@4a100000` | Kullanılacak ağ aygıtı. BBB'nin U-Boot'unda iki aygıt var: Ethernet portu ve USB üzerinden ağ ( `usb_ether` ). Adlarını açılıştaki `Net:` satırında görürsün. U-Boot 2022.04'te Ethernet'in adı `ethernet@4a100000` , 2016–2017 öncesi sürümlerde `cpsw` 'dir. Çoğu zaman zaten varsayılandır, ama `Using usb_ether device` görürsen bu satır gerekir. |
| `ping ...` | Kablo ve IP ayarlarını test eder. `is alive` görmelisin. |

> **Not**
>
> U-Boot sadece **ping atabilir**, gelen ping'e cevap vermez. Host'tan karta `ping 192.168.27.2` atarsan cevap gelmez, bu normaldir. Testi her zaman kart tarafından yap.

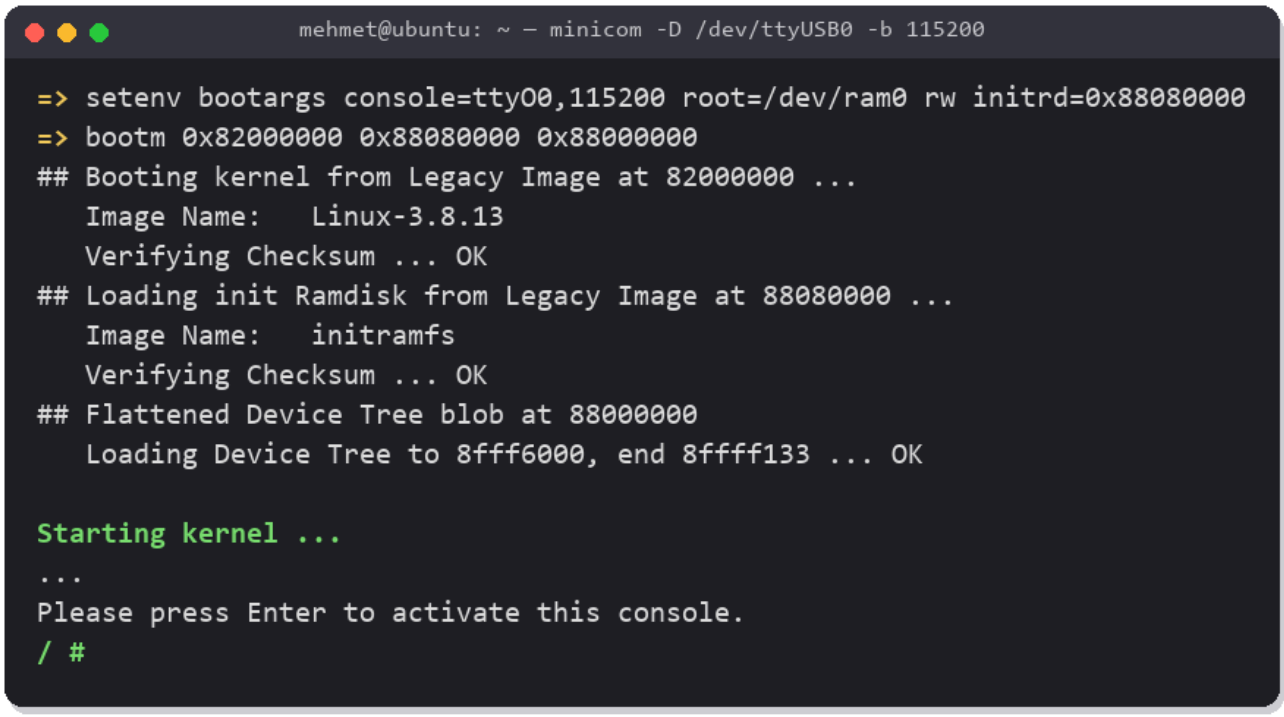
### Adım 4 — Dosyaları TFTP ile RAM'e çek




```
=> tftpboot 0x82000000 uImage
=> tftpboot 0x88000000 am335x-boneblack.dtb
=> tftpboot 0x88080000 initramfs
```

Adresler seri boot bölümündekiyle aynı (7.5): kernel `0x82000000` , DTB `0x88000000` , initramfs `0x88080000` . `Bytes transferred` değeri host'taki `ls -l` boyutuyla aynı olmalı. ( `tftp` ve `tftpboot` U-Boot'ta aynı komuttur.)

### Adım 5 — `bootargs` ayarla ve boot et




```
=> setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
=> bootm 0x82000000 0x88080000 0x88000000
```

`bootm` argüman sırası: **kernel – initramfs – DTB**. Komuta sadece adresler yazılır. `bootargs` 'ın ayrıntıları: 7.8 Adım 7.

### Komutların kısa özeti

```
# Host
sudo ip addr add 192.168.27.1/24 dev enp3s0
sudo cp uImage am335x-boneblack.dtb initramfs /var/lib/tftpboot/

# U-Boot (minicom)
setenv ipaddr 192.168.27.2
setenv serverip 192.168.27.1
ping 192.168.27.1
tftpboot 0x82000000 uImage
tftpboot 0x88000000 am335x-boneblack.dtb
tftpboot 0x88080000 initramfs
setenv bootargs console=ttyO0,115200 root=/dev/ram0 rw initrd=0x88080000
bootm 0x82000000 0x88080000 0x88000000
```

---

## 8.6 Yol 2 — uEnv.txt ile Otomatik TFTP Boot

Yol 1'deki U-Boot komutlarının hepsi `uEnv.txt`'ye yazılır. Kart açılınca U-Boot bu dosyayı SD kartın BOOT bölümünden okur ve `uenvcmd` içindeki komutları kendisi çalıştırır. Böylece **minicom'a hiçbir şey yazmana gerek kalmaz**; minicom'u sadece açılışı izlemek için kullanırsın.

Host tarafındaki hazırlık (IP verme, dosyaları `/var/lib/tftpboot`'a kopyalama) iki yolda da aynı. Yol 2 sadece U-Boot'ta elle yazdığın kısmı (Yol 1, Adım 3–5) otomatik yapar.

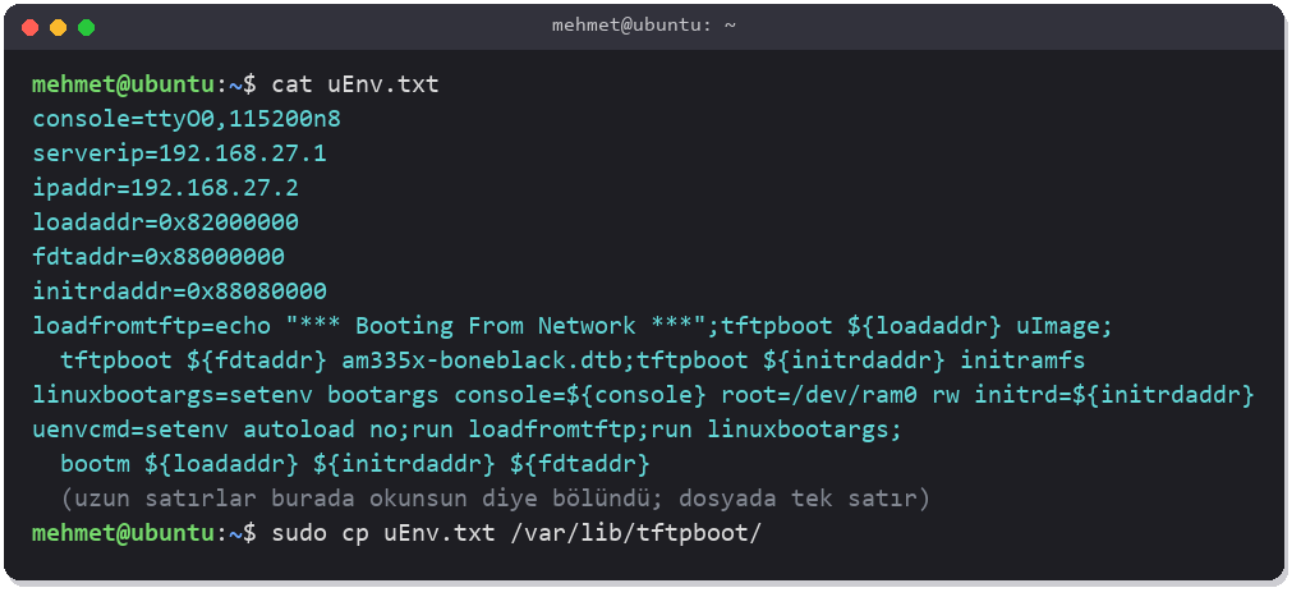
### Adım 1 — Host'u hazırla

Yol 1'in Adım 1'i ile aynı: Ethernet arayüzüne `192.168.27.1` ver, `uImage`, `am335x-boneblack.dtb`, `initramfs` dosyalarını `/var/lib/tftpboot`'a kopyala ve `systemctl status tftpd-hpa` ile servisin çalıştığını kontrol et.

### Adım 2 — `uEnv.txt` dosyasını yaz

Host'ta aşağıdaki içerikle bir `uEnv.txt` oluştur. Her değişken **tek satırda** olmalı; satırları bölme.

```
console=ttyO0,115200n8
serverip=192.168.27.1
ipaddr=192.168.27.2
loadaddr=0x82000000
fdtaddr=0x88000000
initrdaddr=0x88080000
loadfromtftp=echo "*** Booting From Network ***";tftpboot ${loadaddr} uImage;tftpboot ${fdtaddr}
am335x-boneblack.dtb;tftpboot ${initrdaddr} initramfs
linuxbootargs=setenv bootargs console=${console} root=/dev/ram0 rw initrd=${initrdaddr}
uenvcmd=setenv autoload no;run loadfromtftp;run linuxbootargs;bootm ${loadaddr} ${initrdaddr}
${fdtaddr}
```



| Satır | Açıklama | Yol 1'deki karşılığı |
|---|---|---|
| `console=ttyO0,115200n8` | Kendi tanımladığımız bir değişken; aşağıda `${console}` ile kullanılıyor. `n8`: parity yok, 8 bit. | `bootargs` içindeki `console=...` |
| `serverip=...` | TFTP sunucusunun (host) IP'si | `setenv serverip 192.168.27.1` |
| `ipaddr=...` | Kartın IP'si | `setenv ipaddr 192.168.27.2` |
| `loadaddr`, `fdtaddr`, `initrdaddr` | Kernel, DTB ve initramfs'in RAM adresleri. Adresleri bir kere yazıp aşağıda `${...}` ile kullanıyoruz; değiştirmek gerekirse tek yerden değişir. | `0x82000000`, `0x88000000`, `0x88080000` |
| `loadfromtftp=...` | Bir **makro**: ekrana mesaj basar, sonra 3 dosyayı TFTP ile RAM'e çeker. `run loadfromtftp` yazılınca çalışır. | 3 adet `tftpboot` komutu |
| `linuxbootargs=...` | Bir makro: `bootargs`'ı ayarlar | `setenv bootargs ...` |
| `uenvcmd=...` | U-Boot'un `uEnv.txt`'yi okuduktan sonra **otomatik çalıştırdığı** komut. Sırayla: makroları çalıştır, sonra `bootm`. | Yazdığın bütün komutların sırası |
| `setenv autoload no` | `dhcp` / `bootp` komutlarının IP aldıktan sonra kendiliğinden TFTP ile dosya çekmesini engeller. Statik IP kullandığımız için şart değil, ama zararı yok ve ileride `dhcp` eklersen sürpriz yaşatmaz. | — |

> **Uyarı**
>
> En sık yapılan iki hata:
>
> - `setenv booargs ...` gibi bir yazım hatası. U-Boot hata vermez, `booargs` adında yeni bir değişken oluşturur. Kernel'e gerçek `bootargs` gitmez, `Starting kernel ...` sonrası ekran boş kalabilir veya kernel rootfs bulamadan panic verir.
> - Makroyu bir adla tanımlayıp başka bir adla çağırmak (ör. `loadftp=...` tanımlayıp `run loadfromtftp` yazmak). U-Boot `## Error: "loadfromtftp" not defined` der ve dosyalar hiç çekilmez. Tanımladığın ad ile çağırdığın ad **birebir aynı** olmalı.

> **Önemli**
>
> Komutu `bootcmd` içine değil **`uenvcmd`** içine yaz. BBB'nin U-Boot'u `uEnv.txt`'yi kendi `bootcmd`'si çalışırken okur ve ardından `uenvcmd`'yi çalıştırır. Bu yüzden dosyadaki `bootcmd` o açılışı etkilemez (6.3).

### Adım 3 — `uEnv.txt`'yi SD kartın BOOT bölümüne koy

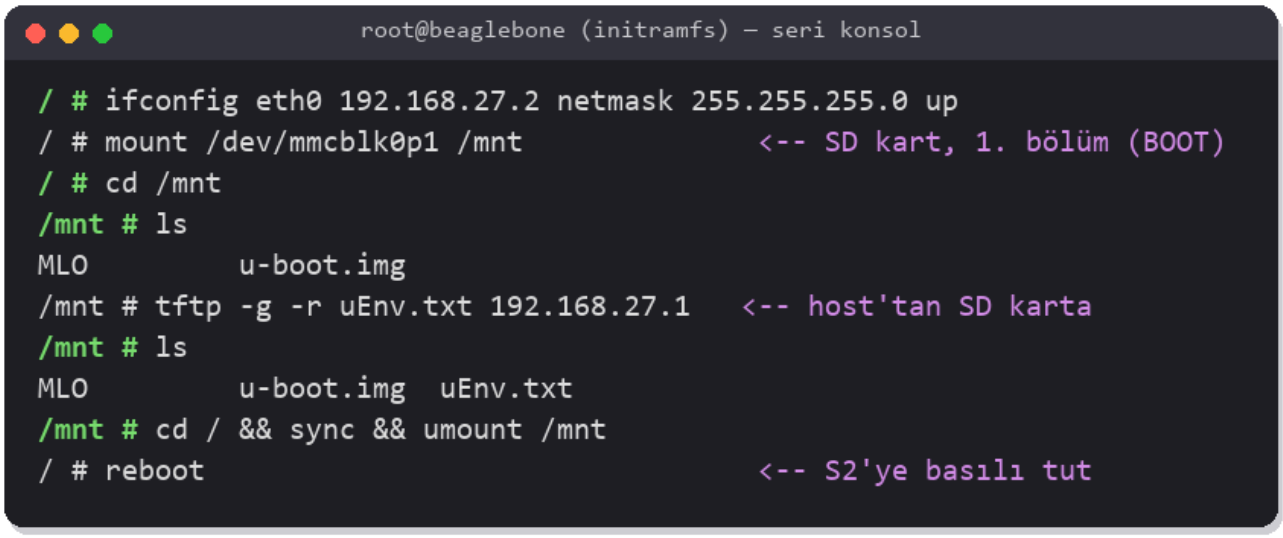
İki seçenek var.

**a) Kart okuyucuyla (en basiti):** SD kartı bilgisayara tak ve dosyayı 1. bölüme kopyala.

```
cp uEnv.txt /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT
```

**b) SD kartı çıkarmadan, TFTP ile:** Kart Linux'a açıkken (örneğin Yol 1 ile açtıktan sonra) dosyayı host'tan doğrudan SD karta çekebilirsin. Bunun için BusyBox'taki `tftp -g -r uEnv.txt 192.168.27.1` komutu kullanılır (8.7).

Önce host'ta dosyayı TFTP klasörüne koy ( `sudo cp uEnv.txt /var/lib/tftpboot/` ). Sonra kartta:




```
ifconfig eth0 192.168.27.2 netmask 255.255.255.0 up
mount /dev/mmcblk0p1 /mnt
cd /mnt
tftp -g -r uEnv.txt 192.168.27.1
ls
cd / && sync && umount /mnt
reboot
```

> **Önemli**
>
> `tftp` dosyayı **bulunduğun klasöre** kaydeder. Önce SD kartın BOOT bölümünü bağlayıp ( `mount` ) o klasöre girmezsen dosya initramfs'e, yani RAM'e yazılır ve yeniden başlatınca kaybolur. `mmcblk0` SD kart, `mmcblk1` eMMC'dir; `p1` 1. bölüm demek.

### Adım 4 — Kartı başlat ve hiçbir tuşa basma

S2'ye basılı tutarak güç ver (veya `reboot` sırasında basılı tut). Bu sefer `Hit any key to stop autoboot` çıkınca **dokunma**; U-Boot `uEnv.txt`'yi okur ve `uenvcmd`'yi çalıştırır:

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200

U-Boot 2022.04 (Oct 16 2026 - 10:12:41)
Hit any key to stop autoboot:  0
mmc0 is current device
reading uEnv.txt                          <-- SD kart, 1. bölüm
412 bytes read in 4 ms (100.6 KiB/s)
Importing environment from mmc ...
Running uenvcmd ...                       <-- bizim komutumuz
*** Booting From Network ***
Using ethernet@4a100000 device
TFTP from server 192.168.27.1; our IP address is 192.168.27.2
Filename 'uImage'.
Bytes transferred = 4380760 (42d858 hex)
Filename 'am335x-boneblack.dtb'.
Bytes transferred = 24884 (6134 hex)
Filename 'initramfs'.
Bytes transferred = 1867312 (1c7e30 hex)
## Booting kernel from Legacy Image at 82000000 ...
Starting kernel ...
```

Ekranda `Running uenvcmd ...`, ardından `*** Booting From Network ***` ve üç tane `Bytes transferred` görmelisin.

### Adım 5 — Doğrula

Linux açılınca `bootargs`'ın gerçekten kernel'e gittiğini kontrol et:

```
root@beaglebone (initramfs) — seri konsol

/ # cat /proc/cmdline
console=ttyO0,115200n8 root=/dev/ram0 rw initrd=0x88080000  <-- linuxbootargs çalışmış
```

```
cat /proc/cmdline
```

Burada `console=ttyO0,115200n8 root=/dev/ram0 ...` görmüyorsan `linuxbootargs` çalışmamış demektir (değişken adını kontrol et).

### Adım 6 — S2'ye basmadan açılış (Challenge)

Kursun bu bölümdeki challenge'ı: *TFTP boot'u `uEnv.txt` ile otomatikleştir; kart güç alınca ya da reset'e basılınca imajlar kendiliğinden insin ve **S2'ye basmadan** açılsın.*

S2'ye basmadan güç verince ROM Code **eMMC**'deki MLO/U-Boot'u çalıştırır (3.5). Çözüm, eMMC'deki U-Boot'un `uEnv.txt`'yi nerede aradığına bağlı:

1. **Önce kontrol et.** U-Boot'u durdur ve ortamına bak:

   ```
   => printenv mmcdev bootenvfile bootenv
   => printenv bootcmd
   ```

   `mmcdev=0` ve `bootcmd` içinde `loadbootenv` / `envboot` gibi bir adım varsa, U-Boot açılışta önce **SD kartın 1. bölümündeki** `uEnv.txt`'yi okur (`mmc 0` = SD kart).

2. **Bu durumda yapman gereken tek şey** Adım 3'teki `uEnv.txt`'yi SD kartın BOOT bölümüne koymak. Kart eMMC'den açılsa bile SD karttaki dosyayı okur ve `uenvcmd` ile TFTP boot'u başlatır. S2'ye basmana gerek kalmaz.

3. eMMC'deki U-Boot dosyayı başka bir yerde arıyorsa (BeagleBoard'un yeni Debian imajları gibi, ör. eMMC'nin rootfs'indeki `/boot/uEnv.txt`), iki seçenek var: - Komutları U-Boot'un okuduğu dosyaya yaz, ya da - eMMC'deki `MLO`'yu devre dışı bırak. ROM Code eMMC'de geçerli MLO bulamayınca SD karta geçer (3.1).

> **Dikkat**
>
> eMMC'deki `MLO`'yu silmek, eMMC'deki sistemin kendi başına açılmamasına yol açar. Geri dönmek için Bölüm 5'teki gibi eMMC'yi yeniden flash'lamak gerekir. Önce 1. ve 2. maddeyi dene.

### Yol 2'nin özeti

Artık geliştirme döngüsü şu kadar:

```
# Host: kernel'i / DTB'yi yeniden derledin
sudo cp uImage am335x-boneblack.dtb /var/lib/tftpboot/
# Kartın reset butonuna bas → U-Boot yeni dosyaları kendisi çeker ve boot eder
```

> **İpucu**
>
> **Yol 2'yi kullanınca Yol 1'deki komutlara gerek kalmaz mı?** Her açılışta yazmana gerek kalmaz, ama komutları bilmek yine de gerekli. Bir şey bozulduğunda U-Boot'u durdurup `run loadfromtftp`, `printenv bootargs` gibi komutları tek tek çalıştırarak hangi adımın bozulduğunu bulursun. `uEnv.txt` sadece aynı komutları senin yerine yazar.

---

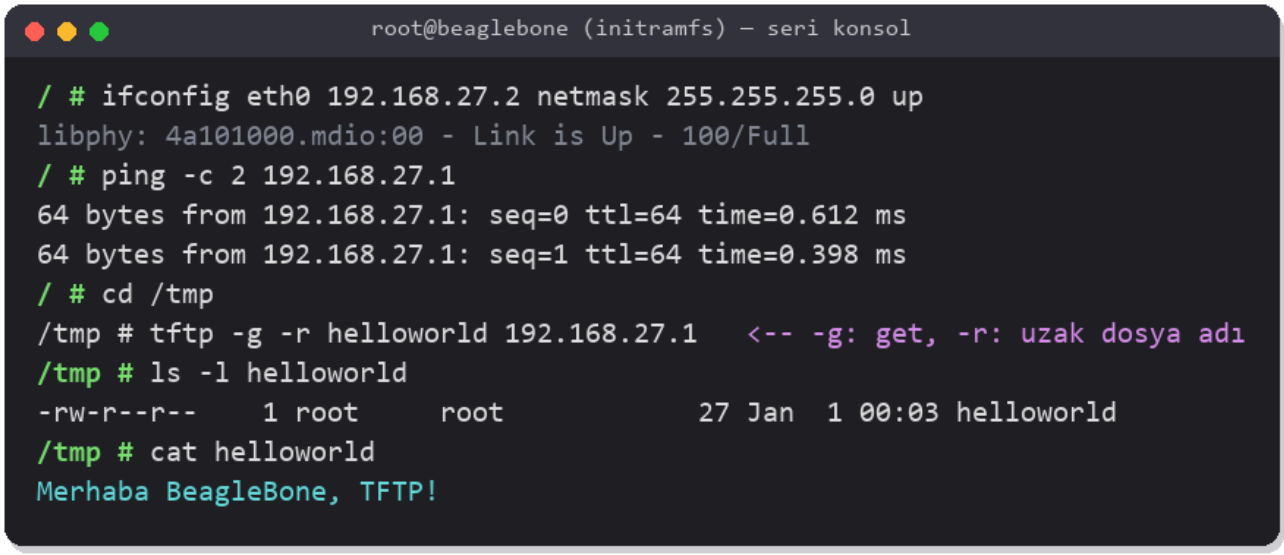
## 8.7 Linux Açıldıktan Sonra TFTP ile Dosya Çekme

Kart Linux'a açıldıktan sonra BusyBox'taki `tftp` istemcisiyle host'tan dosya alabilirsin.

**Host PC'de:**

```
echo "Merhaba BeagleBone, TFTP!" > helloworld     # veya: vi helloworld
sudo cp helloworld /var/lib/tftpboot/
```

**BeagleBone'da (seri konsol):**




```
ifconfig eth0 192.168.27.2 netmask 255.255.255.0 up
ping -c 2 192.168.27.1
cd /tmp
tftp -g -r helloworld 192.168.27.1
cat helloworld
```

| Komut / seçenek | Açıklama |
|---|---|
| `ifconfig eth0 ... up` | U-Boot'un IP ayarı kernel'e **geçmez**. Kernel ağ kartını sıfırdan başlatır, bu yüzden Linux'ta IP'yi her açılışta tekrar vermek gerekir. BusyBox'ta `ifconfig` vardır; `ip addr add 192.168.27.2/24 dev eth0` ve `ip link set eth0 up` da çalışır. |
| `tftp` | Tek başına yazarsan kullanım yardımını gösterir. |
| `-g` | get: sunucudan dosya al |
| `-r helloworld` | remote: sunucudaki dosya adı |
| `-l isim` | local: kartta farklı bir adla kaydet (isteğe bağlı) |
| `192.168.27.1` | TFTP sunucusunun IP'si |

| Komut / seçenek | Açıklama |
| --- | --- |
| `-p -l dosya 192.168.27.1` | put: karttan host'a dosya gönder ( `--create` ayarı sayesinde) |

> **Not**
>
> initramfs RAM'de çalışır. Kartta oluşturduğun veya indirdiğin dosyalar **yeniden başlatınca kaybolur**.

---

## 8.8 Challenge: Her Kaynaktan Boot Edebilen Genel uEnv.txt

### Görev

Tek bir `uEnv.txt` yazacağız. Bu dosya kaynakları sırayla deneyecek ve ilk çalışan kaynaktan boot edecek: **eMMC → SD → TFTP → NFS**. eMMC'de boot dosyaları yoksa SD karta, SD kart yoksa TFTP'ye geçecek.

### Çözüm fikri

U-Boot'un komut satırı (hush) `if ... then ... fi` ve `&&` destekler. Komutların başarılı olup olmadığına göre karar verebiliriz:

- `load` dosyayı bulamazsa hata döner.
- `mmc rescan` kart yoksa hata döner.
- `tftpboot` dosya yoksa ya da sunucu cevap vermezse hata döner.
- `bootm` başarılı olursa **geri dönmez**, kernel çalışır. Başarısız olursa (bozuk imaj vb.) bir sonraki satıra geçilir.

Her kaynak için bir `try_...` makrosu yazıyoruz. Makro önce dosyaları yüklemeyi dener; yükleme başarılıysa `bootargs`'ı ayarlayıp `bootm` çalıştırır. Başarısızsa hiçbir şey yapmaz ve `uenvcmd` sıradaki makroya geçer:

```
uenvcmd
  ├─ try_emmc : eMMC'de uImage + dtb var mı? ─ evet → bootm (rootfs: eMMC p2)
  │                                          └ hayır ↓
  ├─ try_sd   : SD kart takılı ve imajlar var mı? ─ evet → bootm (rootfs: SD p2)
  │                                               └ hayır ↓
  ├─ try_tftp : TFTP'den uImage + dtb + initramfs ─ evet → bootm (rootfs: RAM)
  │                                               └ hayır ↓
  ├─ try_nfs  : TFTP'den uImage + dtb ─ evet → bootm (rootfs: NFS, host'ta)
  │                                   └ hayır ↓
  └─ "Hicbir kaynaktan boot edilemedi"
```

### Çözüm: `uEnv.txt`

Her değişken **tek satırda** olmalı. Bu çözümde varsayılan dosya düzeni şu: her MMC aygıtında 1. bölüm (FAT, BOOT) içinde `uImage` ve `am335x-boneblack.dtb`, 2. bölüm (ext4) rootfs.

```
console=ttyO0,115200n8
ipaddr=192.168.27.2
serverip=192.168.27.1
netretry=once
loadaddr=0x82000000
fdtaddr=0x88000000
initrdaddr=0x88080000
nfspath=/srv/nfs/bbb,nolock,wsize=1024,rsize=1024
mmc_load=mmc dev ${mmcnum} && mmc rescan && load mmc ${mmcnum}:1 ${loadaddr} uImage && load mmc
${mmcnum}:1 ${fdtaddr} am335x-boneblack.dtb
mmc_args=setenv bootargs console=${console} root=${mmcroot} rw rootwait
emmc_root=if mmc dev 0 && mmc rescan; then setenv mmcroot /dev/mmcblk1p2; else setenv mmcroot
/dev/mmcblk0p2; fi
try_emmc=echo "[1/4] eMMC deneniyor ..."; setenv mmcnum 1; if run mmc_load; then run emmc_root;
run mmc_args; bootm ${loadaddr} - ${fdtaddr}; fi
try_sd=echo "[2/4] SD kart deneniyor ..."; setenv mmcnum 0; if run mmc_load; then setenv mmcroot
/dev/mmcblk0p2; run mmc_args; bootm ${loadaddr} - ${fdtaddr}; fi
net_load=tftpboot ${loadaddr} uImage && tftpboot ${fdtaddr} am335x-boneblack.dtb
try_tftp=echo "[3/4] TFTP (initramfs) deneniyor ..."; if run net_load && tftpboot ${initrdaddr}
initramfs; then setenv bootargs console=${console} root=/dev/ram0 rw initrd=${initrdaddr}; bootm
${loadaddr} ${initrdaddr} ${fdtaddr}; fi
try_nfs=echo "[4/4] TFTP + NFS deneniyor ..."; if run net_load; then setenv bootargs
console=${console} root=/dev/nfs rw nfsroot=${serverip}:${nfspath}
ip=${ipaddr}:${serverip}::255.255.255.0::eth0:off rootwait; bootm ${loadaddr} - ${fdtaddr}; fi
uenvcmd=if test "${generic_tried}" != "yes"; then setenv generic_tried yes; setenv autoload no;
run try_emmc; run try_sd; run try_tftp; run try_nfs; echo "!!! Hicbir kaynaktan boot edilemedi";
fi
```

### Satırların açıklaması

| Satır | Açıklama |
|---|---|
| `console`, `ipaddr`, `serverip`, `*addr` | Yol 2'deki ile aynı: konsol, IP'ler ve RAM adresleri |
| `netretry=once` | **Önemli.** Varsayılan olarak `tftpboot` sunucu cevap vermezse **sonsuza kadar** tekrar dener (`Retry count exceeded; starting again`). O zaman NFS adımına hiç geçilmez. `once` ile bir kere dener, olmazsa hata döner. |
| `nfspath` | NFS rootfs'in host'taki yolu ve NFS seçenekleri: `nolock` (kartta kilit servisi yok), `wsize/rsize=1024` (küçük paket boyutu, yavaş ama güvenilir) |

| Satır | Açıklama |
|---|---|
| `mmc_load` | Ortak MMC yükleyici. `${mmcnum}` hangi aygıt olduğunu söyler ( `1` = eMMC, `0` = SD). Dört komut `&&` ile bağlı: biri başarısız olursa geri kalanlar çalışmaz ve makro hata döner. |
| `mmc_args` | MMC için `bootargs` . Rootfs, `${mmcroot}` ile belirtilen bölüm. |
| `emmc_root` | Kernel'de eMMC'nin adını bulur (aşağıdaki uyarıya bak). SD kart varsa eMMC `mmcblk1` , yoksa `mmcblk0` olur. |
| `try_emmc` , `try_sd` | `mmc_load` başarılıysa → `bootargs` ayarla → `bootm` . `-` : initramfs yok. |
| `net_load` | TFTP ile kernel + DTB. TFTP ve NFS adımları ortak kullanır. |
| `try_tftp` | Kernel + DTB + **initramfs**. Rootfs RAM'de (Yol 2 ile aynı). |
| `try_nfs` | Kernel + DTB TFTP'den gelir, rootfs NFS ile host'tan bağlanır. `ip=...` kernel'e kendi IP'sini verir; U-Boot'un IP'si kernel'e geçmediği için NFS'te şart. Bu adım, host'ta `initramfs` dosyası yoksa devreye girer. |
| `uenvcmd` | Makroları sırayla çağırır. `generic_tried` bir **koruma bayrağı**: U-Boot sürümüne göre varsayılan `bootcmd` önce SD'de, sonra eMMC'de `uEnv.txt` arayabilir, yani `uenvcmd` iki kez çalışabilir. Bayrak sayesinde zincir sadece bir kez denenir. |

> **Uyarı**
>
> **eMMC'nin kernel'deki adı değişebilir.** U-Boot'ta eMMC her zaman `mmc 1` 'dir. Ama eski kernel'lerde (3.8.13 gibi) SD kart **takılı değilse** eMMC `mmcblk0` olarak görünür, takılıysa `mmcblk1` olarak görünür. `emmc_root` makrosu SD kartın olup olmadığına bakıp doğru adı seçer. Yeni kernel'lerde DTB'deki `aliases` ile bu sabitlenebilir.

## Dosya nereye konur?

Burada da bir tavuk-yumurta durumu var: `uEnv.txt` 'nin kendisi yerel bir depolamada olmalı. U-Boot onu önce SD'de ( `mmcdev 0` ), bulamazsa eMMC'de ( `mmcdev 1` ) arar.

| Senaryo | `uEnv.txt` nerede olmalı? |
|---|---|
| SD kart her zaman takılı | SD kartın 1. bölümü (BOOT) yeterli |
| "SD kart yoksa TFTP'ye geç" durumu da çalışsın | **eMMC'nin 1. bölümüne de** aynı dosyayı koy |

En güvenlisi aynı `uEnv.txt` 'yi hem SD'nin hem eMMC'nin BOOT bölümüne koymak.

### Örnek açılış: eMMC'de imaj yok, SD kart takılı değil

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200 (SD kart takılı değil)

U-Boot 2022.04 (Oct 16 2026 - 10:12:41)
Card did not respond to voltage select!     <-- mmcdev 0 (SD) yok
SD/MMC found on device 1                    <-- eMMC'ye geçti
reading uEnv.txt
Running uenvcmd ...
[1/4] eMMC deneniyor ...
** Unable to read file uImage **           <-- eMMC'de imaj yok
[2/4] SD kart deneniyor ...
Card did not respond to voltage select!     <-- SD yok
[3/4] TFTP (initramfs) deneniyor ...
Using ethernet@4a100000 device
TFTP from server 192.168.27.1; our IP address is 192.168.27.2
Bytes transferred = 4380760 (42d858 hex)
Bytes transferred = 24884 (6134 hex)
Bytes transferred = 1867312 (1c7e30 hex)
## Booting kernel from Legacy Image at 82000000 ...
Starting kernel ...                         <-- TFTP'den açıldı
```

### Nasıl test edilir?

Her satırda tek bir şeyi değiştirip kartı yeniden başlat. Hangi `[n/4]` satırından sonra `Starting kernel ...` geldiğine bak.

| Test | Ne yaparsın? | Beklenen |
|---|---|---|
| 1 | Her şey yerinde | `[1/4] eMMC` → boot |
| 2 | eMMC'deki `uImage`'ın adını değiştir (`mv uImage uImage.bak`) | eMMC başarısız → `[2/4] SD` → boot |
| 3 | SD kartı çıkar (`uEnv.txt` eMMC'de olmalı) | eMMC, SD başarısız → `[3/4] TFTP` → boot |
| 4 | Host'ta `/var/lib/tftpboot/initramfs`'i taşı | … → `[4/4] NFS` → boot (NFS sunucusu kurulu olmalı) |
| 5 | Ethernet kablosunu da çıkar | `!!! Hicbir kaynaktan boot edilemedi`, U-Boot komut satırı |

Bir adımı elle denemek istersen U-Boot'u durdurup `uEnv.txt`'yi yükle (`run loadbootenv; run importbootenv`), sonra makroları tek tek çalıştır: `run try_sd`, `printenv bootargs` gibi.

### Sınırlamalar

- Bu zincir sadece **U-Boot aşamasındaki** hataları yakalar (dosya yok, kart yok, sunucu yok). Kernel açılıp sonra panic verirse (yanlış rootfs vb.) U-Boot'a geri dönülmez.
- `MLO` ve `u-boot.img` yine eMMC'de ya da SD'de olmalı. ROM Code BBB'de ağdan boot edemez (8.2).
- `uEnv.txt` içindeki U-Boot `echo` mesajlarında Türkçe karakter kullanma; seri konsolda bozuk görünebilir.

---

## 8.9 Özet

- Host: `tftpd-hpa` kur, `/var/lib/tftpboot`'u hazırla, Ethernet arayüzüne `192.168.27.1` ver, dosyaları kopyala.
- SD kart: sadece `MLO`, `u-boot.img`, `uEnv.txt`. Kernel, DTB ve initramfs ağdan gelir.
- U-Boot: `setenv ipaddr` / `serverip` → `ping` → `tftpboot` (3 dosya) → `setenv bootargs` → `bootm kernel initrd dtb`.
- Bu komutları `uEnv.txt`'deki `uenvcmd`'ye yazarsan açılış otomatik olur; geliştirme döngüsü "dosyayı kopyala, reset'e bas"a iner.
- U-Boot'taki IP ayarı kernel'e geçmez; Linux'ta `eth0`'a IP'yi yeniden ver.

## 8.10 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200 (temsili hatalar)

=> tftpboot 0x82000000 uImage
*** ERROR: `serverip' not set              <-- setenv serverip ...
=> tftpboot 0x82000000 uImage
Using usb_ether device                     <-- setenv ethact ...
=> tftpboot 0x82000000 uImage
TFTP from server 192.168.27.1; our IP address is 192.168.27.2
Loading: T T T T T T T T T T
Retry count exceeded; starting again       <-- IP/kablo/firewall/VM ağı
=> tftpboot 0x82000000 uimage
TFTP error: 'File not found' (1)           <-- ad/klasör yanlış (büyük-küçük harf!)
=> ping 192.168.27.1
ping failed; host 192.168.27.1 is not alive <-- host IP'si yok / kablo
```

| Hata | Sebep | Çözüm |
|---|---|---|
| `*** ERROR: 'serverip' not set` | Sunucu IP'si tanımlı değil | `setenv serverip 192.168.27.1` |
| `Using usb_ether device` | U-Boot USB ağını kullanmaya çalışıyor | `setenv ethact ethernet@4a100000` (eski U-Boot'larda `setenv ethact cpsw`) |

| Hata | Sebep | Çözüm |
|---|---|---|
| `Loading: T T T T ...` / `Retry count exceeded` | Paket gidiyor ama cevap gelmiyor | Host IP'si ve alt ağ doğru mu? `systemctl status tftpd-hpa` çalışıyor mu? Firewall (`sudo ufw allow 69/udp`)? VM ise Bridged mi? Sunucu port 69'da mı? |
| `TFTP error: 'File not found' (1)` | Dosya `/var/lib/tftpboot` içinde değil ya da adı yanlış | `ls -l /var/lib/tftpboot`. Linux'ta büyük-küçük harf farklıdır: `uImage` ≠ `uimage` |
| `TFTP error: 'Permission denied'` | Dosya okunamıyor | `sudo chmod -R 777 /var/lib/tftpboot` |
| `ping failed; host ... is not alive` | Host'un o arayüzde IP'si yok / kablo takılı değil | `ip -br a` ile kontrol et, Ethernet ışıklarına bak |
| Kart U-Boot'a gelmeden eMMC'den açılıyor | S2'ye basılmadı | Güç verirken S2'yi basılı tut |
| Kart SD'den hiç açılmıyor | SPL dosyasının adı yanlış / BOOT bölümü FAT değil | Dosya adı `MLO` olmalı; 1. bölüm FAT ve boot bayrağı açık olmalı |
| `bootm` sonrası `Wrong Ramdisk Image Format` | initramfs'te U-Boot başlığı yok | `mkimage -A arm -T ramdisk -C gzip -d initramfs.cpio.gz initramfs` (7.9 (g)) |
| `## Error: "loadfromtftp" not defined` | `uenvcmd` içinde çağrılan makro adı, tanımlanan adla aynı değil | `uEnv.txt`'de iki adı birebir aynı yaz |
| `Running uenvcmd ...` hiç çıkmıyor | `uEnv.txt` okunmadı: BOOT bölümünde değil, adı yanlış (`uenv.txt`) veya kart eMMC'den açıldı | Dosyayı 1. bölüme koy, S2'ye basılı tutarak aç |
| `Starting kernel ...` sonrası ekran boş | `bootargs` ayarlanmamış (ör. `booargs` yazım hatası) ya da `console` yanlış | U-Boot'ta `printenv bootargs` ile kontrol et |

# Bölüm 9 — Cross Toolchain, SPL ve U-Boot Derleme

> **Bu bölümde:**
>
> - Cross compile'ın ne olduğunu ve neden gerektiğini öğreneceksin.
> - Linaro cross toolchain'ini kurup `PATH`'e ekleyeceksin.
> - U-Boot 2022.04 kaynağını indirip BeagleBone Black için yapılandıracak ve derleyeceksin: `MLO`, `u-boot-spl.bin`, `u-boot.img`.
> - Kendi derlediğin U-Boot ile kartı açacaksın.
> - Challenge: U-Boot'un açılıştaki bekleme süresini kaynak koddan 10 saniyeye çıkaracaksın.
>
> **Ön koşul:** Bölüm 3 — SPL ve U-Boot'un görevleri, Bölüm 4 — SD karta dosya koyma.

---

## 9.1 Bu Bölümde Neden Toolchain Kuruyoruz?

Şimdiye kadar kullandığımız `MLO`, `u-boot.img`, `uImage` hazır dosyalardı. Bu bölümde **SPL ve U-Boot'u kaynak koddan kendimiz derleyeceğiz**. Bunun için ARM kodu üretebilen bir derleyiciye ihtiyacımız var.

| | **Host PC** | **BeagleBone Black** |
|---|---|---|
| İşlemci | x86_64 (Intel/AMD) | ARM Cortex-A8 (AM335x) |
| Komut seti | x86 makine kodu | ARM makine kodu |

Ubuntu'daki `gcc` x86 için kod üretir. Onunla derlenen U-Boot'u ROM Code yükler ama ARM işlemci çalıştıramaz.

> **Cross compile:** Bir mimaride (x86) çalışıp **başka bir mimari** (ARM) için çalıştırılabilir kod üretmek. Bunu yapan derleyici ve yardımcı araçların toplamına **cross toolchain** denir.

**Neden kartın üzerinde derlemiyoruz?**

- Kart yavaş ve RAM'i az. U-Boot dakikalar, kernel saatler sürer.
- Tavuk-yumurta: kartta derleme yapmak için önce çalışan bir Linux gerekir. O Linux'u açacak olan da derleyeceğimiz U-Boot.

### Toolchain'in içinde ne var?

| **Araç** | **Görevi** |
|---|---|
| `arm-linux-gnueabihf-gcc` | C kodunu ARM assembly'ye çevirir |

| Araç | Görevi |
|---|---|
| `arm-linux-gnueabihf-as` | Assembly'yi makine koduna çevirir (binutils) |
| `arm-linux-gnueabihf-ld` | Parçaları birleştirir; kodu linker script'e göre SRAM/DDR adreslerine yerleştirir |
| `arm-linux-gnueabihf-objcopy` | ELF dosyasından ham binary üretir ( `u-boot.bin` , `u-boot-spl.bin` ) |
| glibc ve kütüphaneler | Kernel ve kullanıcı programları için. U-Boot kullanmaz. |
| `arm-linux-gnueabihf-gdb` | Hata ayıklama |

**İsimdeki önek ( `arm-linux-gnueabihf-` ):**

| Parça | Anlamı |
|---|---|
| `arm` | Hedef mimari |
| `linux` | Hedef işletim sistemi: Linux üzerinde çalışacak programlar |
| `gnueabi` | GNU ABI (Application Binary Interface) |
| `hf` | **h**ard-**f**loat: ondalıklı sayı işlemleri donanımdaki FPU ile yapılır (Cortex-A8'de FPU var) |

### Neden U-Boot bölümünde anlatılıyor?

U-Boot, kitapta **derlediğimiz ilk parça**. Toolchain burada bir kere kuruluyor ve sonraki her aşamada aynısı kullanılıyor:

```
cross toolchain ─┬── SPL + U-Boot          (bu bölüm)
                 ├── Linux kernel + DTB     (uImage, am335x-boneblack.dtb)
                 ├── BusyBox / rootfs / initramfs
                 └── kendi uygulamaların ve sürücülerin
```

> **Not**
>
> U-Boot "bare-metal" çalışır: altında işletim sistemi yoktur ve glibc kullanmaz. Teknik olarak `arm-none-eabi-` (işletim sistemi olmayan) toolchain ile de derlenebilir. `arm-linux-gnueabihf-` seçilmesinin sebebi, aynı toolchain'in kernel ve kullanıcı programları için de gerekli olması. Tek toolchain her şeye yetiyor.

---

## 9.2 Sürüm Uyumu: Hangi Toolchain?

En çok sorun çıkaran konu bu. **Eski U-Boot/kernel kaynağını yeni GCC ile derlemek çoğu zaman hata verir.** Toolchain'i derleyeceğin kaynağın yaşına göre seç:

| Derlenecek kaynak | Uygun toolchain | Nasıl kurulur? |
|---|---|---|
| U-Boot 2013.x, kernel 3.8 | GCC 4.x (ör. Linaro 4.9-2014.09) | Linaro arşivinden indir |
| U-Boot 2016.x – 2022.x, kernel 4.x | GCC 6 / 7 (ör. Linaro 6.5, 7.5) | Linaro'dan indir (aşağıda) |
| U-Boot 2023+, kernel 5.x / 6.x | Güncel GCC | `sudo apt install gcc-arm-linux-gnueabihf` |

> **Önemli**
>
> Bu bölümde kursun videolarında kullanılan **U-Boot 2022.04 + Linaro GCC 7.5.0** kullanılıyor (9.3 Adım 4'teki menuconfig ekranında görünüyor). Başka sürüm kullanırsan komutlarda sadece sürüm numaraları değişir, adımlar aynıdır.

> **Not**
>
> Kursun GitHub reposundaki eski derleme notları ( `notes/compilation_commands` ) Linaro GCC 6.3.1 ve `am335x_boneblack_defconfig` kullanıyor. Bu, kursun 2017 sürümüne ait; güncellenen videolarda U-Boot 2022.04 kullanılıyor. Bu kitap videolardaki güncel sürümü takip ediyor.

İndirme adresleri:

| Dosya | Adres |
|---|---|
| Linaro GCC 7.5 | `https://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/` |
| Linaro GCC 6.5 | `https://releases.linaro.org/components/toolchain/binaries/6.5-2018.12/arm-linux-gnueabihf/` |
| Linaro GCC 4.9 | `https://releases.linaro.org/archive/14.09/components/toolchain/binaries/` |
| U-Boot kaynakları | `https://ftp.denx.de/pub/u-boot/` |

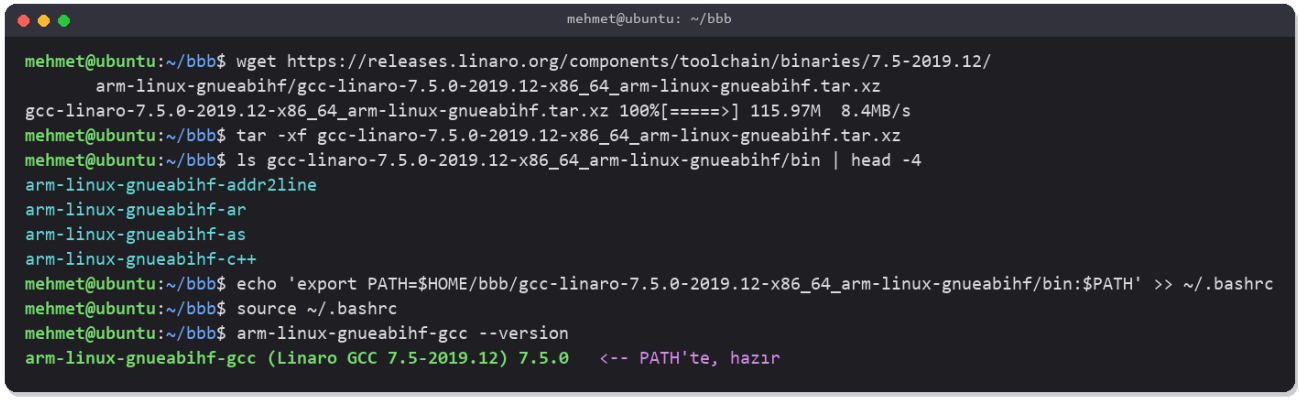
---

## 9.3 Adım Adım: Toolchain Kurulumu ve U-Boot Derleme

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**; sürüm, tarih ve boyut değerleri senin ortamında farklı olacaktır. Örneklerde çalışma klasörü `~/bbb` .

## Adım 1 — Cross toolchain'i kur

Linaro toolchain hazır derlenmiş bir arşiv olarak gelir. İndirip açman ve `bin` klasörünü `PATH`'e eklemen yeterli:



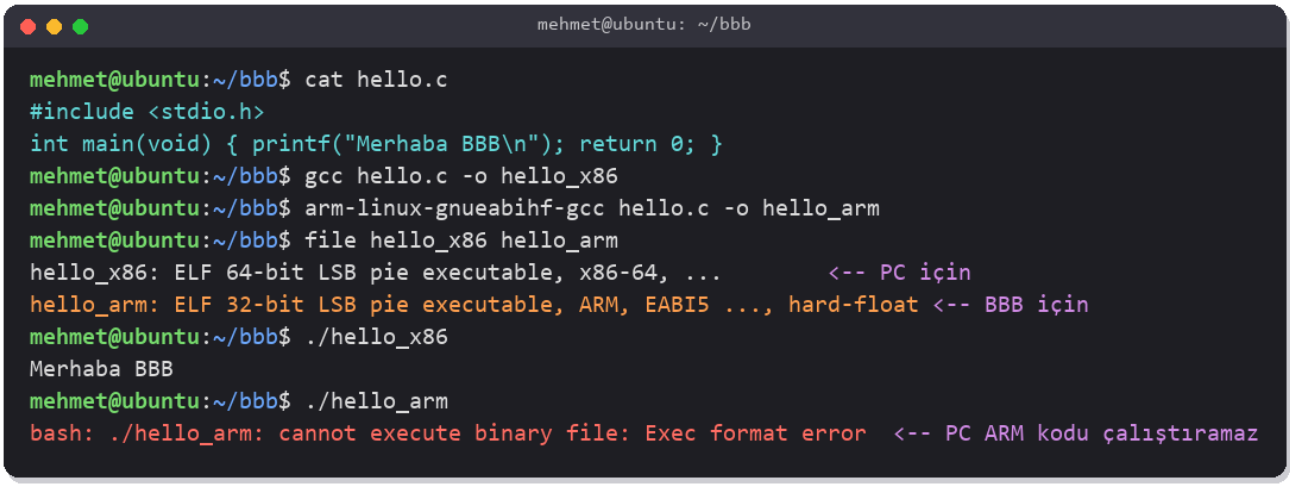

```
cd ~/bbb
wget https://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
tar -xf gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
echo 'export PATH=$HOME/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin:$PATH' >> ~/.bashrc
source ~/.bashrc
arm-linux-gnueabihf-gcc --version
```

`~/.bashrc`'ye eklediğimiz için her yeni terminalde `PATH` hazır olur.

> **İpucu**
>
> Güncel U-Boot (2023+) derleyeceksen indirmeye gerek yok: `sudo apt install gcc-arm-linux-gnueabihf` yeterli. Bu paket `/usr/bin`'e kurulur, `PATH` ayarı gerekmez.

## Adım 2 — (İsteğe bağlı) Cross compile'ı kendi gözünle gör

Aynı C dosyasını iki derleyiciyle derleyip `file` ile karşılaştır:

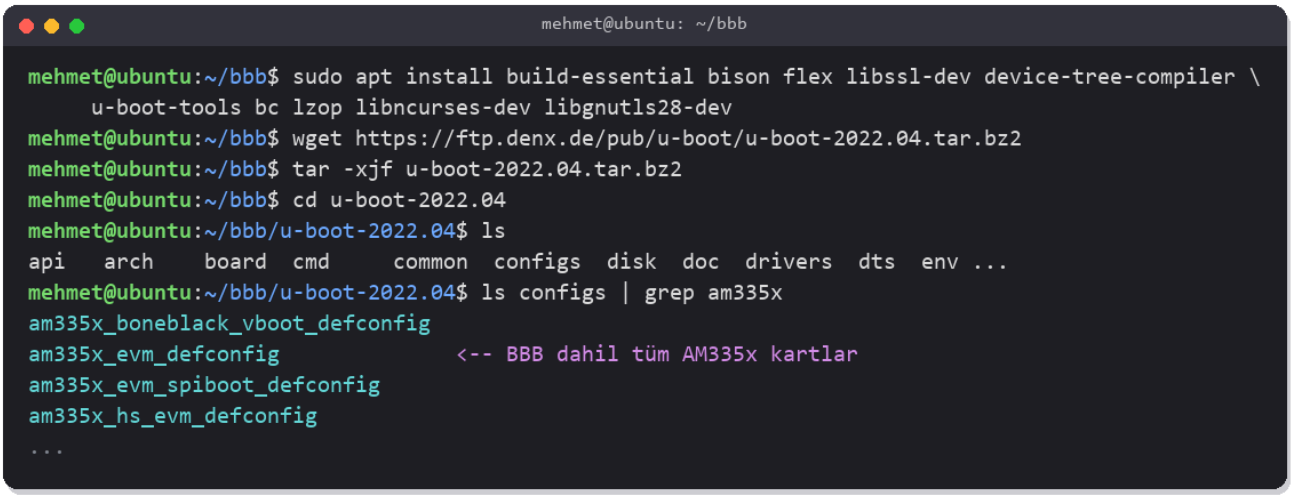



```
gcc hello.c -o hello_x86
arm-linux-gnueabihf-gcc hello.c -o hello_arm
file hello_x86 hello_arm
./hello_arm          # Exec format error: PC ARM kodu çalıştıramaz
```

`hello_arm` dosyasını karta kopyalarsan orada çalışır. U-Boot da aynı şekilde: PC'de derlenir, kartta çalışır. 6.6.3'te rootfs'teki programların PC'de çalışmamasının sebebi de budur.

### Adım 3 — Host paketlerini kur ve U-Boot kaynağını indir

U-Boot derlenirken bazı yardımcı programlar (`mkimage`, `kconfig` vb.) **PC için** derlenip çalıştırılır. Bu yüzden host'ta da bazı paketler gerekir:



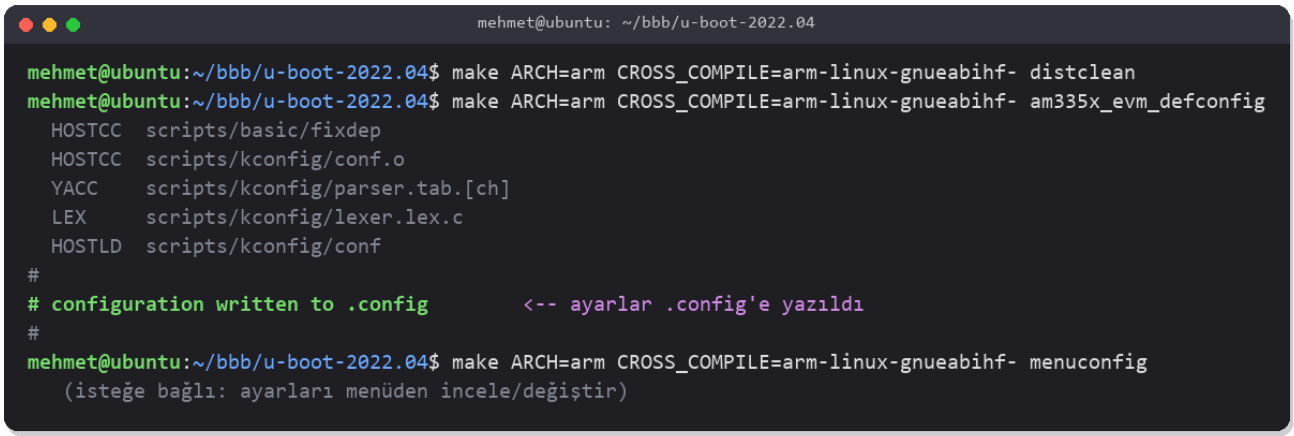

```
sudo apt install build-essential bison flex libssl-dev device-tree-compiler \
     u-boot-tools bc lzop libncurses-dev libgnutls28-dev
wget https://ftp.denx.de/pub/u-boot/u-boot-2022.04.tar.bz2
tar -xjf u-boot-2022.04.tar.bz2
cd u-boot-2022.04
ls configs | grep am335x
```

| Paket | Neden? |
|---|---|
| `build-essential` | Host'un kendi `gcc`'si ve `make` (yardımcı araçlar için) |
| `bison`, `flex` | Kconfig ve dtc ayrıştırıcıları |
| `libssl-dev`, `libgnutls28-dev` | `mkimage` ve diğer host araçlarının imza/özet özellikleri |
| `device-tree-compiler` | `dtc`: U-Boot'un kendi device tree'si için |
| `libncurses-dev` | `make menuconfig` menüsü |
| `u-boot-tools` | Hazır `mkimage` (kernel/initramfs'e başlık eklemek için) |

> **İpucu**
>
> `git clone https://source.denx.de/u-boot/u-boot.git` ve ardından `git checkout v2022.04` ile de alabilirsin. Kaynağı değiştirip deneyeceksen git daha kullanışlı, çünkü `git diff` ile ne değiştirdiğini görürsün.

## Adım 4 — BBB için yapılandır

`configs/` klasöründe her kart için hazır bir ayar dosyası (`defconfig`) bulunur. Önce eski derleme kalıntılarını temizle, sonra BBB'nin ayarını yükle:




```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig     # isteğe bağlı
```

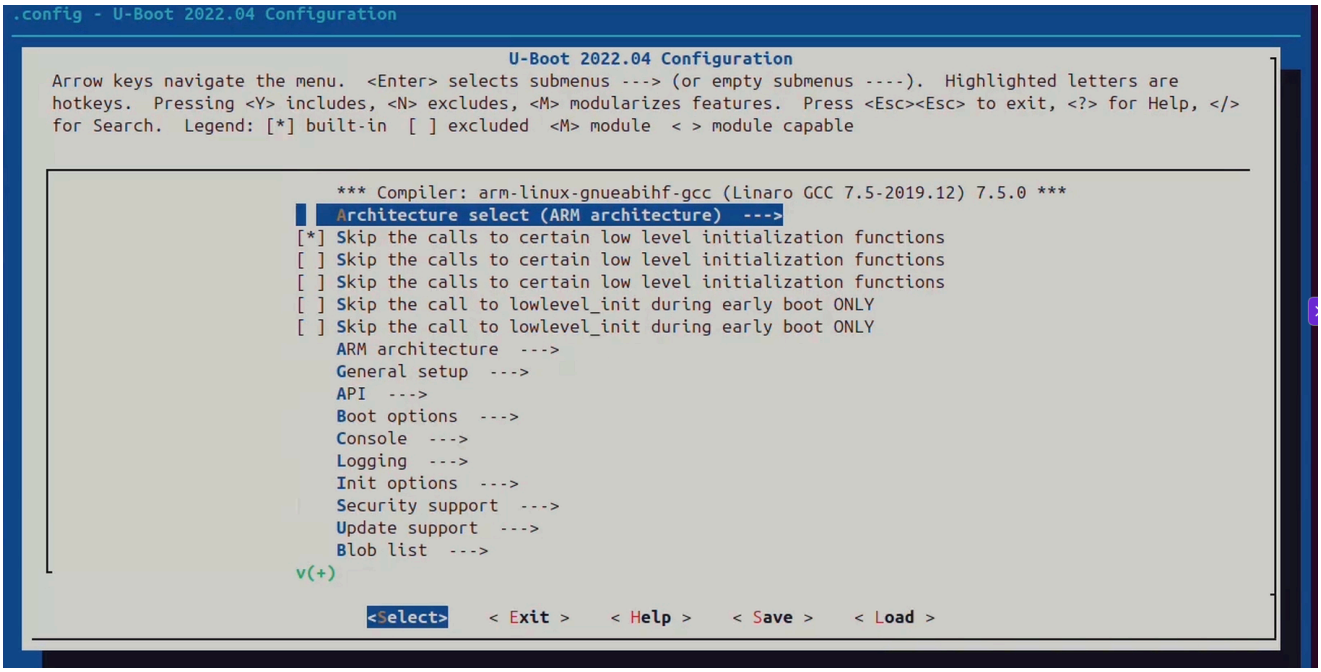
| Parça | Anlamı |
|---|---|
| `ARCH=arm` | Hedef mimari: `arch/arm/` altındaki kodlar kullanılır |
| `CROSS_COMPILE=arm-linux-gnueabihf-` | Makefile'daki `gcc`, `ld`, `objcopy` komutlarının **başına eklenir**. `gcc` yerine `arm-linux-gnueabihf-gcc` çalışır. Sondaki `-` önemli. |
| `distclean` | Önceki derlemeden kalan her şeyi (`.config` dahil) siler |

| Parça | Anlamı |
| --- | --- |
| `am335x_evm_defconfig` | BBB'nin hazır ayarlarını `.config` dosyasına yazar |
| `menuconfig` | `.config` 'i menüden düzenler (ör. `bootdelay` , komutlar) |

> **Not**
>
> Ayar adı sürüme göre değişir: U-Boot 2013.10'da `am335x_evm_config` , 2017–2019 arası `am335x_boneblack_defconfig` , 2020 ve sonrası `am335x_evm_defconfig` . Bu sonuncusu tek bir SPL/U-Boot ile BBB, Bone, Bone Green, PocketBeagle ve EVM kartlarını kapsar; U-Boot açılırken kartın EEPROM'unu okuyup doğru DTB'yi seçer. Doğru adı `ls configs | grep am335x` ile bul.

### `menuconfig` ekranı

`make ... menuconfig` komutu aşağıdaki menüyü açar (kursun ekran görüntüsü). İsteğe bağlı bir adım; hiçbir şeyi değiştirmeden `Exit` ile çıkabilirsin.

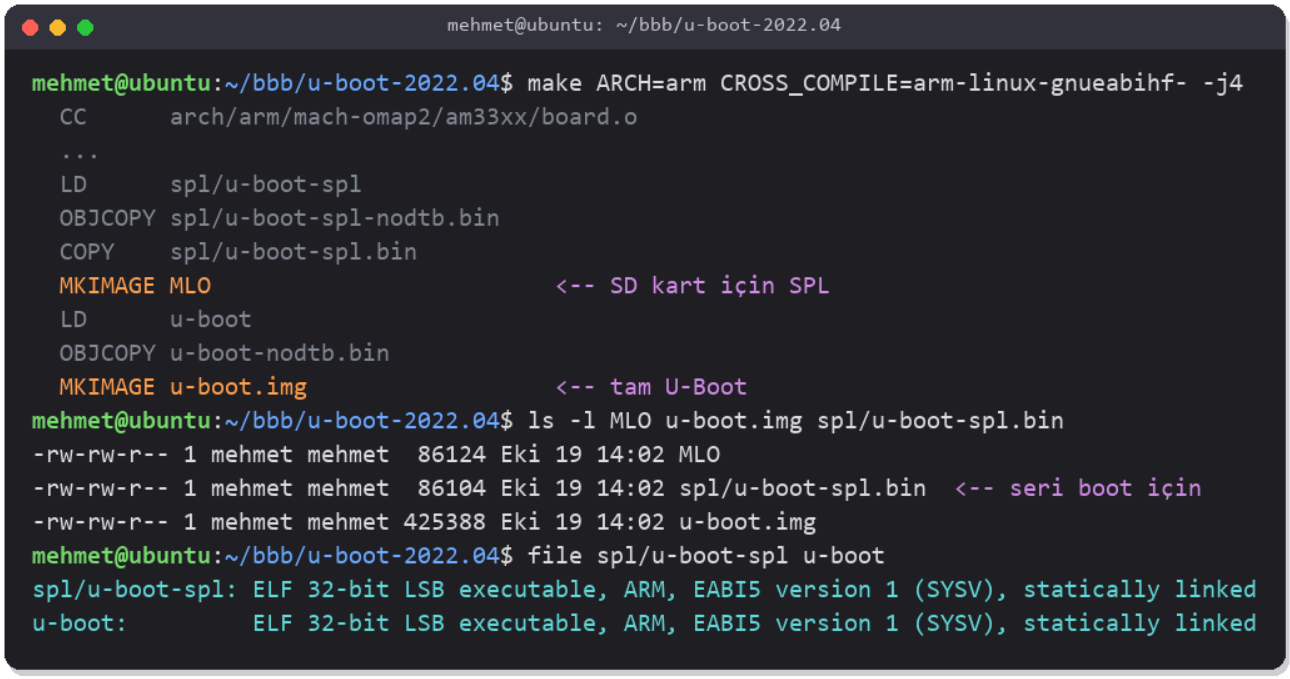



| Ekranda | Anlamı |
| --- | --- |
| `U-Boot 2022.04 Configuration` | Derlenen U-Boot sürümü. Başka sürüm indirdiysen burada o yazar. |
| `*** Compiler: arm-linux-gnueabihf-gcc (Linaro GCC 7.5-2019.12) 7.5.0 ***` | `CROSS_COMPILE` ile verdiğin derleyici. **Toolchain'in doğru bulunduğunu buradan anlarsın.** Burada Ubuntu'nun `gcc` 'si görünüyorsa `CROSS_COMPILE` yazılmamış ya da `PATH` yanlış demektir. |
| `Architecture select (ARM architecture)` | `ARCH=arm` ve defconfig'in seçtiği mimari |
| `[*]` / `[ ]` | Seçili (U-Boot'a dahil) / seçili değil. **Boşluk** tuşu ile değiştirilir. |
| `--->` | Alt menü, **Enter** ile girilir |

| Ekranda | Anlamı |
|---|---|
| `v(+)` | Listenin devamı var, aşağı ok ile kaydır |
| `<Save>` / `<Exit>` | Değişiklikleri `.config`'e yazar / çıkar (değişiklik varsa kaydetmek ister) |
| `/` | Ayar arama (ör. `BOOTDELAY`) |

Kursta sık bakılan yerler: **Boot options** (`bootdelay`, `bootcmd`), **Command line interface** (hangi U-Boot komutlarının derleneceği), **Environment** (ortam değişkenlerinin nereye kaydedileceği).

## Adım 5 — Derle




```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
ls -l MLO u-boot.img spl/u-boot-spl.bin
file spl/u-boot-spl u-boot
```

`-j4`: 4 iş parçacığıyla paralel derle (çekirdek sayına göre artırabilirsin, `nproc` ile öğren).

Derleme sonunda çıkan dosyalar:

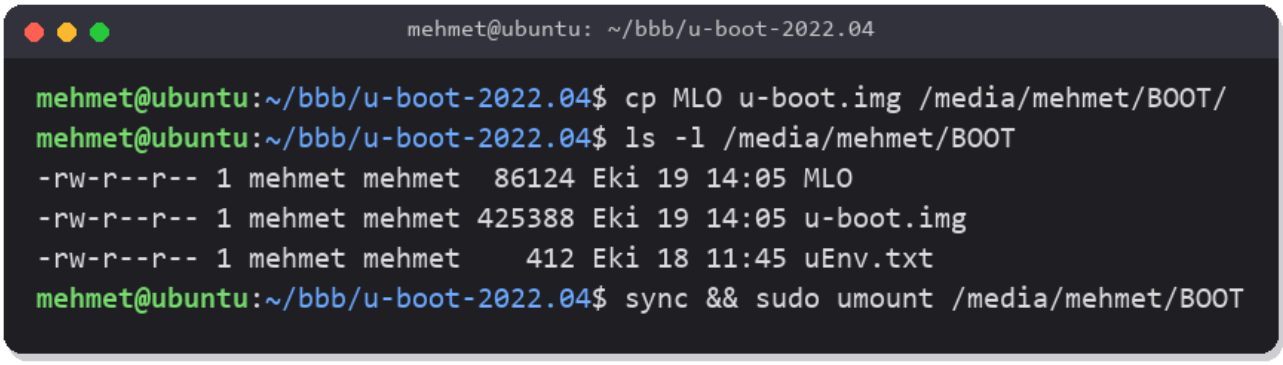
| Dosya | Ne işe yarar? | Nerede kullandık? |
|---|---|---|
| `MLO` | SPL + başında **GP başlığı** (ROM Code'un okuyacağı yükleme adresi ve boyut) | SD kart / eMMC BOOT bölümü (Bölüm 4) |
| `spl/u-boot-spl.bin` | Başlıksız ham SPL | Seri (UART) boot, Xmodem ile (Bölüm 7) |
| `u-boot.img` | Tam U-Boot + U-Boot başlığı (SPL bunu okur) | SD kart, seri boot (Ymodem) |

| Dosya | Ne işe yarar? | Nerede kullandık? |
|---|---|---|
| `u-boot` , `spl/u-boot-spl` | ELF dosyaları (sembollü) | Hata ayıklama ( `gdb` ) |

`file` çıktısındaki **ARM, EABI5** ifadesi dosyanın gerçekten ARM için derlendiğini gösterir.

> **Önemli**
>
> `MLO` **109 KB'tan küçük** olmalı. ROM Code onu AM335x'in dahili SRAM'ine koyar ve SRAM'de daha fazla yer yok. U-Boot'a çok özellik eklersen SPL büyüyebilir; o zaman SPL ayarlarından gereksiz özellikleri kapatmak gerekir.

## Adım 6 — SD karta kopyala

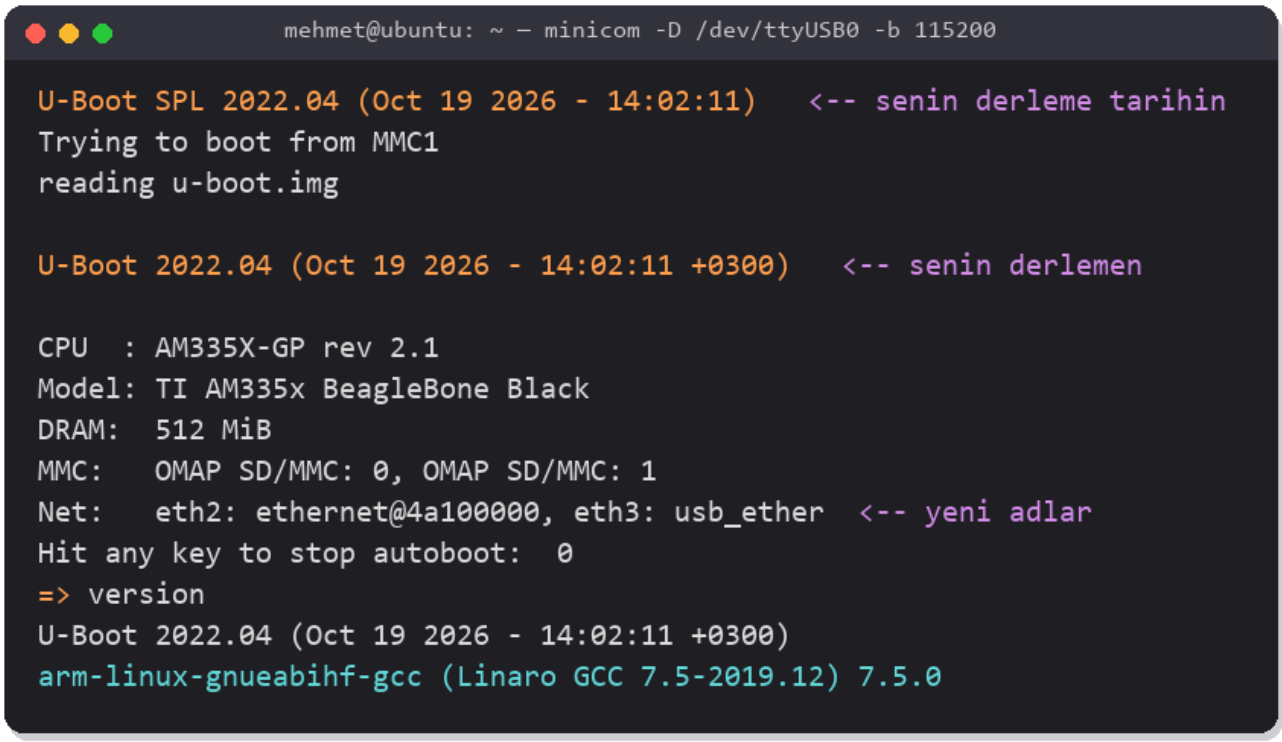



```
cp MLO u-boot.img /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT
```

Seri boot için ise `spl/u-boot-spl.bin` ve `u-boot.img` dosyalarını kullan (Bölüm 7).

## Adım 7 — Kartta doğrula

Kartı S2'ye basılı tutarak aç. İlk satırlardaki **tarih ve saat senin derleme zamanın** olmalı:




```
=> version
```

> **Not**
>
> Yeni U-Boot sürümlerinde komut satırı `U-Boot#` yerine `=>` olarak görünür. Önceki bölümlerdeki komutlar aynı şekilde çalışır.
>
> Bir fark var: Ethernet aygıtının adı değişti. Kursun hazır U-Boot'unda (2013.04) `cpsw` olan aygıt 2022.04'te `ethernet@4a100000` olarak görünür (`Net:` satırına bak). TFP bölümündeki `setenv ethact cpsw` yerine `setenv ethact ethernet@4a100000` yazman gerekir (8.5). Çoğu zaman varsayılan zaten doğrudur, emin olmak için `printenv ethact` ile bak.

### Komutların kısa özeti

```
# Toolchain (bir kere)
wget https://releases.linaro.org/components/toolchain/binaries/7.5-2019.12/arm-linux-gnueabihf/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
tar -xf gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf.tar.xz
echo 'export PATH=$HOME/bbb/gcc-linaro-7.5.0-2019.12-x86_64_arm-linux-gnueabihf/bin:$PATH' >> ~/.bashrc
source ~/.bashrc

# U-Boot
sudo apt install build-essential bison flex libssl-dev device-tree-compiler u-boot-tools bc lzop libncurses-dev libgnutls28-dev
wget https://ftp.denx.de/pub/u-boot/u-boot-2022.04.tar.bz2 && tar -xjf u-boot-2022.04.tar.bz2 && cd u-boot-2022.04
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- am335x_evm_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
cp MLO u-boot.img /media/$USER/BOOT/ && sync
```

---

## 9.4 Challenge: Autoboot Bekleme Süresini 10 Saniyeye Çıkarmak

### Görev

Kursun bu bölümdeki challenge'ı:

> *U-Boot açılınca otomatik açılışa (uEnv.txt'yi okuma, uImage'ı yükleme...) geçmeden önce birkaç saniye bekler. Bu sürede bir tuşa basarsan U-Boot komut satırına düşersin. Bu bekleme süresini **10 saniyeye** çıkar ve kartta test et. Bunun için U-Boot **kaynak kodunu** değiştirip yeniden derlemen gerekiyor.*

### Bekleme süresi nereden geliyor?

Geri sayımı `bootdelay` ortam değişkeni belirler. Bu değişkenin **varsayılan** değeri derleme sırasında `CONFIG_BOOTDELAY` ayarından gelir ve U-Boot'un içine gömülür:

| Kaynak | Değer |
|---|---|
| U-Boot 2022.04 varsayılanı ( `am335x_evm_defconfig` değiştirmiyor) | 2 saniye |
| Kursun güncel `SD-boot` U-Boot'u | 0 (geri sayım yok, tuşa basılı tutmak gerekir) |
| Kursun eski videolarındaki U-Boot | 5 saniye |

> **Not**
>
> `setenv bootdelay 10 + saveenv` ile de aynı sonuç alınır. Ama bu sadece o kartın kayıtlı ortamını değiştirir; challenge, ayarın **U-Boot imajının kendisinde** değişmesini istiyor. Böylece imajı hangi karta koyarsan koy 10 saniye bekler.

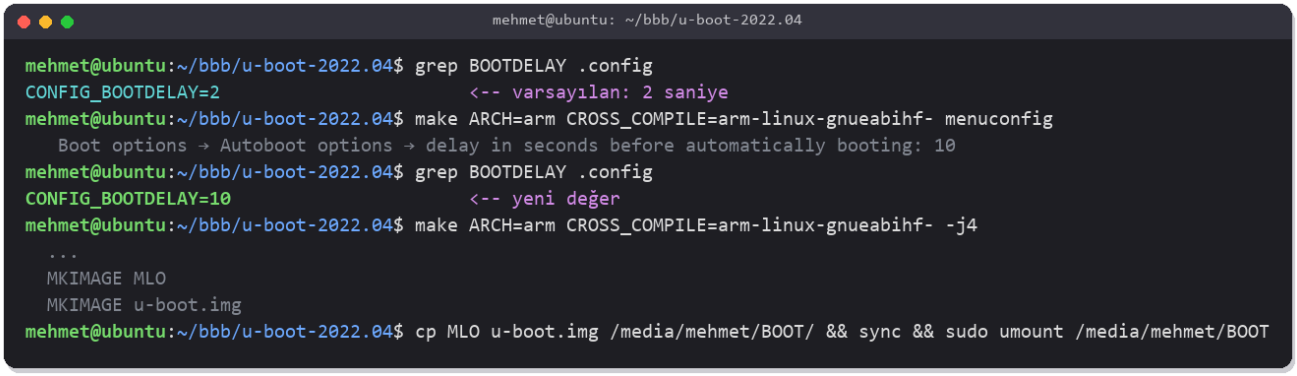
## Çözüm

**1) Ayarı değiştir.** 9.3'teki U-Boot kaynak klasöründe `menuconfig`'i aç ve şu ayarı `10` yap:

```
Boot options  --->
    Autoboot options  --->
        (10) delay in seconds before automatically booting
```

Ayarı bulmak için menüde `/` tuşuna basıp `BOOTDELAY` aramak daha hızlı. Kaydedip çık.

**2) Kontrol et, derle, SD karta kopyala.**




```
grep BOOTDELAY .config                                  # CONFIG_BOOTDELAY=10 görmelisin
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j4
cp MLO u-boot.img /media/$USER/BOOT/ && sync && sudo umount /media/$USER/BOOT
```

**3) Kartta test et.** Kartı S2'ye basılı tutarak aç. Geri sayım 10'dan başlamalı:

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200
U-Boot 2022.04 (Oct 19 2026 - 15:20:44 +0300)

CPU  : AM335X-GP rev 2.1
Model: TI AM335x BeagleBone Black
DRAM:  512 MiB
Net:   eth2: ethernet@4a100000, eth3: usb_ether
Hit any key to stop autoboot: 10     <-- 10'dan geri sayar
Hit any key to stop autoboot:  7     <-- bir tuşa basınca durur
=> printenv bootdelay
bootdelay=10
```

> **Önemli**
>
> Geri sayım hâlâ eski değerdeyse, kartta daha önce `saveenv` ile kaydedilmiş bir ortam vardır. Kayıtlı ortam derlemedeki varsayılanı ezer. U-Boot'ta `env default -a` ile varsayılanlara dön ve `saveenv` ile kaydet. Ayrıca `uEnv.txt` 'de `bootdelay=` satırı olmadığından emin ol.

> **İpucu**
>
> Değişikliğin kaynak kodda kalıcı olmasını istersen `make savedefconfig` ile küçük bir `defconfig` dosyası üret ve onu `configs/` altına kendi adınla kaydet (ör. `configs/my_bbb_defconfig` ). Sonraki derlemelerde `make my_bbb_defconfig` yeterli olur.

## 9.5 Özet

- Cross toolchain, PC'de (x86) çalışıp ARM kodu üretir. Önek: `arm-linux-gnueabihf-` .
- Toolchain'i derlenecek kaynağın yaşına göre seç; eski kaynak + yeni GCC = derleme hatası.
- Derleme üç komut: `distclean` → `am335x_evm_defconfig` → `make -j4` . Hepsinde `ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-` .
- Çıktılar: `MLO` (SD/eMMC için), `spl/u-boot-spl.bin` (seri boot için), `u-boot.img` (ikisi için de).
- Kendi U-Boot'unun çalıştığını açılıştaki derleme tarihinden anlarsın.
- U-Boot'un davranışını değiştirmenin kalıcı yolu `menuconfig` ile `.config` 'i değiştirip yeniden derlemektir (ör. `CONFIG_BOOTDELAY` ).

## 9.6 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~/bbb/u-boot-2022.04 (temsili hatalar)
mehmet@ubuntu:~/bbb/u-boot-2022.04$ make CROSS_COMPILE=arm-linux-gnueabihf- -j4
/bin/sh: 1: arm-linux-gnueabihf-gcc: not found     <-- PATH ayarlı değil

include/linux/compiler-gcc.h:121:30: fatal error: linux/compiler-gcc11.h:
No such file or directory                          <-- GCC U-Boot'tan yeni

multiple definition of `yylloc'; ...dtc-lexer.lex.o: first defined here  <-- host GCC 10+

tools/../include/image.h:...: fatal error: openssl/evp.h: No such file  <-- libssl-dev

/bin/sh: 1: bison: not found                       <-- bison/flex eksik
```

| Hata | Sebep | Çözüm |
|---|---|---|
| `arm-linux-gnueabihf-gcc: not found` | Toolchain `PATH`'te değil | `echo $PATH` ile kontrol et, `source ~/.bashrc` çalıştır, yolu doğru yazdığından emin ol |
| `fatal error: linux/compiler-gccN.h: No such file or directory` | GCC, U-Boot/kernel kaynağından **daha yeni**. Eski kaynaklar sadece tanıdıkları GCC sürümleri için başlık dosyası içerir. | 9.2'deki tablodan kaynağa uygun toolchain'i kullan |
| `multiple definition of 'yylloc'` | Eski kaynaktaki `dtc` + host'ta GCC 10 veya üstü | `scripts/dtc/dtc-lexer.l` (veya `.lex.c_shipped`) içinde `YYLTYPE yylloc;` satırını `extern YYLTYPE yylloc;` yap |
| `openssl/evp.h: No such file` | `libssl-dev` kurulu değil | `sudo apt install libssl-dev` |
| `bison: not found` / `flex: not found` | Host paketleri eksik | `sudo apt install bison flex` |
| `Your dtc is too old` | Host'taki `dtc` eski | `sudo apt install device-tree-compiler` |
| `*** Can't find default configuration "..."` | Yanlış `defconfig` adı | `ls configs \| grep am335x` |
| 32-bit Linaro 4.x toolchain çalışmıyor: `arm-linux-gnueabihf-gcc: No such file or directory` (dosya varken) | Eski toolchain 32-bit, 64-bit Ubuntu'da 32-bit kütüphaneler eksik | `sudo dpkg --add-architecture i386 && sudo apt update && sudo apt install libc6:i386 libstdc++6:i386 zlib1g:i386` |
| Kart açılıyor ama ekranda eski U-Boot tarihi var | Kart eMMC'den açıldı ya da SD'ye kopyalanan dosyalar eski | S2'ye basılı tutarak aç, `ls -l /media/$USER/BOOT` ile tarihleri kontrol et |

| Hata | Sebep | Çözüm |
| --- | --- | --- |
| SPL hiç çalışmıyor | `MLO` yerine `u-boot-spl.bin` kopyalandı ya da `MLO` çok büyük | SD karta `MLO` koy, boyutunu kontrol et (< 109 KB) |

# Kısım IV
# Linux Kernel

# Bölüm 10 — Linux Kernel: Kaynak Ağacı ve Derleme

> **Bu bölümde:**
>
> - BeagleBone Black için hangi kernel kaynağının kullanıldığını ve kursun kernel sürüm geçmişini öğreneceksin.
> - Kernel kaynak ağacında gezinmeyi öğreneceksin: ARM mimarisi kodu, kart yapılandırma dosyaları ve SoC sürücüleri nerede durur?
> - Kernel'i `bb.org_defconfig` ile yapılandırıp `uImage`, `am335x-boneblack.dtb` ve modülleri derleyeceksin.
> - Kendi derlediğin kernel ile kartı açacaksın.
>
> **Ön koşul:** Bölüm 3 — boot zinciri, Bölüm 9 — cross toolchain kurulumu.

---

## 10.1 Hangi Kernel?

Linux kernel'in "resmi" kaynağı Linus Torvalds'ın yönettiği **mainline** ağaçtır (kernel.org). Kart üreticileri bunun üzerine kendi kartlarına özel yamaları ekleyip **vendor** ağaçları yayınlar. BeagleBone Black için iki seçenek var:

| Kaynak | Adres | Ne zaman? |
|---|---|---|
| **BeagleBoard kernel'i** (vendor) | `https://github.com/beagleboard/linux.git` | Kursta ve bu kitapta kullanılan. BBB'ye özel ayar ( `bb.org_defconfig` ), cape ve PRU desteği hazır gelir. |
| **Mainline** | `https://git.kernel.org` | En yeni özellikler. BBB'de temel donanım çalışır, bazı ekstralar eksik olabilir. |

Kursun kullandığı kernel sürümleri zamanla değişti:

| Nerede? | Kernel |
|---|---|
| Kursun ilk hazır imajları (Ångström) | 3.8.13 |
| Kursun eski derleme notları | 4.4 ( `bb.org_defconfig` ) ve 4.11 ( `omap2plus_defconfig` ) |
| **Kursun güncel imajları** | **5.10.168** |

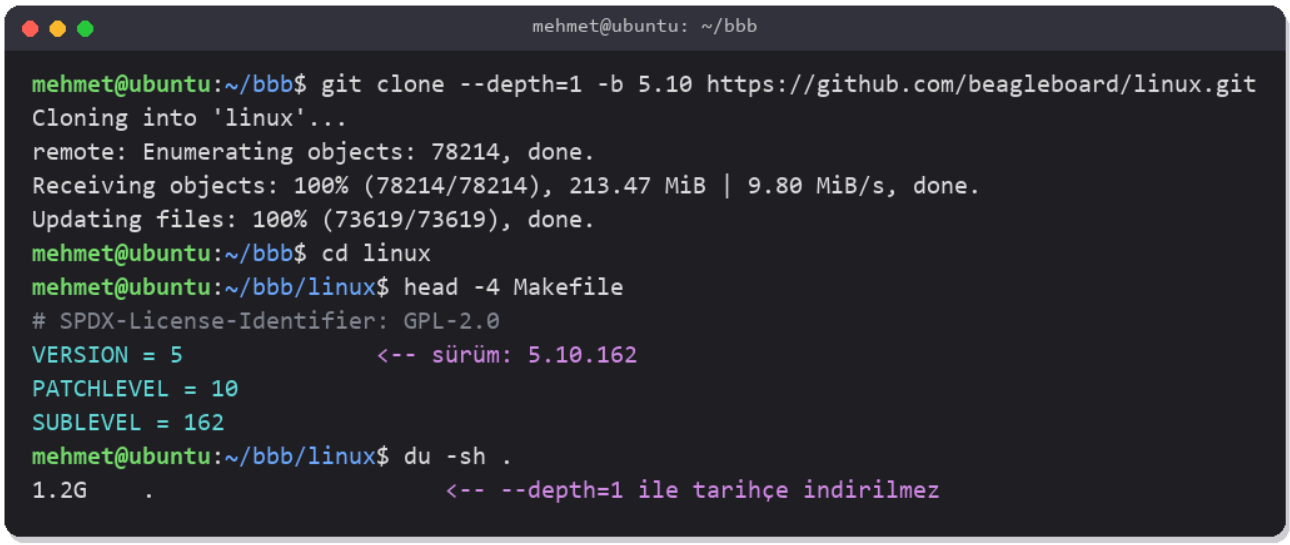
Kursun başındaki "Important Update on Linux Kernel Version" notu bu güncellemeyi anlatıyor. Bu kitap BeagleBoard'un **5.10** dalını kullanıyor. 5.10 bir **LTS** (Long Term Support, uzun süre desteklenen) sürümdür.

> **Not**
>
> Dal (branch) zamanla güncellenir. Bu kitap yazılırken `5.10` dalının ucu **5.10.162** sürümündeydi. Kursun hazır imajı 5.10.168. Aradaki fark sadece küçük düzeltmelerdir, anlatılan her şey ikisinde de aynıdır.

---

## 10.2 Kaynak Kodu İndirme

```
cd ~/bbb
git clone --depth=1 -b 5.10 https://github.com/beagleboard/linux.git
cd linux
head -4 Makefile
```




| Parça | Anlamı |
|---|---|
| `--depth=1` | Sadece son sürümü indir, binlerce commit'lik geçmişi indirme. İndirme birkaç GB yerine birkaç yüz MB olur. |
| `-b 5.10` | `5.10` dalını al |
| `Makefile` başı | `VERSION`, `PATCHLEVEL`, `SUBLEVEL` satırları indirdiğin sürümü gösterir |

> **İpucu**
>
> Kaynak kodda bir fonksiyonun ya da sürücünün nerede tanımlandığını bulmak için tarayıcıdan **Elixir**'i kullan: https://elixir.bootlin.com/linux/v5.10/source. Her sembole tıklayınca tanımına ve kullanıldığı yerlere gider. Bilgisayarda ise `grep -rn "aranan" drivers/` iş görür.

---

## 10.3 Kaynak Ağacının Genel Yapısı

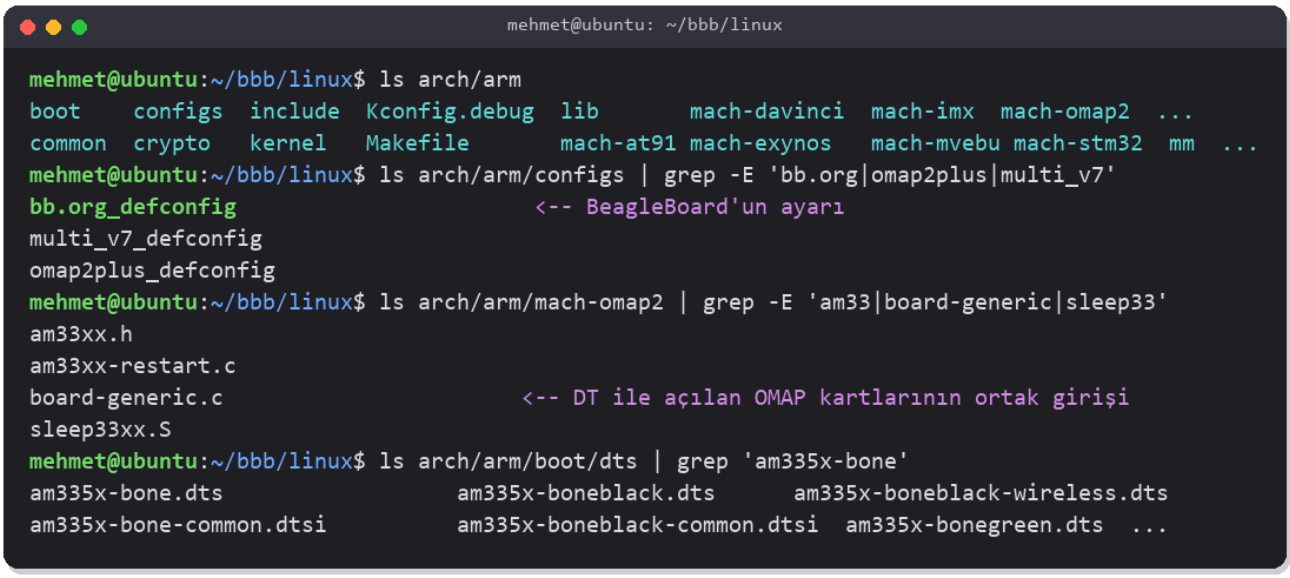
| Klasör | İçinde ne var? |
|---|---|
| **`arch/`** | Mimariye özel kod: `arch/arm`, `arch/arm64`, `arch/x86` ... Açılış kodu, bellek yönetiminin mimariye bağlı kısmı, **device tree** dosyaları ve **defconfig**'ler burada. |
| **`drivers/`** | Bütün aygıt sürücüleri: GPIO, I2C, SPI, UART, MMC, USB, ağ... Kaynağın yaklaşık yarısı. |
| `include/` | Ortak başlık dosyaları ( `include/linux/`, `include/dt-bindings/` ...) |
| `init/` | Kernel'in açılışı: `start_kernel()` ( `init/main.c` ) ve ilk kullanıcı programını ( `init` ) başlatan kod |
| `kernel/` | Çekirdeğin kalbi: zamanlayıcı (scheduler), süreç yönetimi, sinyaller, zamanlayıcılar |
| `mm/` | Bellek yönetimi |
| `fs/` | Dosya sistemleri: ext4, FAT, NFS, proc, sysfs... |
| `net/` | Ağ yığını: TCP/IP, soketler |
| `block/` | Blok aygıt katmanı (diskler, SD kart) |
| `lib/`, `crypto/`, `security/`, `sound/`, `ipc/` | Yardımcı kütüphaneler, şifreleme, güvenlik modülleri, ses, süreçler arası iletişim |
| `Documentation/` | Kernel'in kendi belgeleri. Device tree binding'leri `Documentation/devicetree/bindings/` altında (11.7). |
| `scripts/` | Derleme sistemi araçları ( `kconfig`, `dtc` ...) |
| `Makefile`, `Kconfig` | Derleme sisteminin girişi ve ayar menüsünün kökü |

Kernel'in kullanıcı programlarıyla ilişkisi:

```
Kullanıcı alanı (user space)     uygulamalar, BusyBox, kütüphaneler (glibc)
─────────────────────────────── sistem çağrıları (open, read, write, ioctl...)
Kernel alanı (kernel space)      kernel/ mm/ fs/ net/ ipc/   ← mimariden bağımsız çekirdek
                                 drivers/                    ← aygıt sürücüleri
                                 arch/arm/                   ← ARM'a özel kod
─────────────────────────────── 
Donanım                          AM335x: Cortex-A8, UART, GPIO, I2C, MMC, Ethernet...
```

---

## 10.4 ARM Mimarisi Kodunun Düzeni (`arch/arm`)




| Klasör | İçinde ne var? |
|---|---|
| `arch/arm/boot/` | Derlenen imajlar ( `Image` , `zImage` , `uImage` ) buraya çıkar. |
| `arch/arm/boot/compressed/` | `zImage` 'in başındaki, kernel'i RAM'de açan küçük program (4.9) |
| `arch/arm/boot/dts/` | Device tree kaynakları ( `am335x-boneblack.dts` ve `.dtsi` 'leri) |
| `arch/arm/configs/` | Hazır ayar dosyaları (**defconfig**) |
| `arch/arm/kernel/` | ARM'a özel çekirdek kodu. Kernel'in ilk çalışan komutları `arch/arm/kernel/head.S` 'tedir. |
| `arch/arm/mm/` | ARM'a özel bellek yönetimi (MMU, cache) |
| `arch/arm/mach-omap2/` | **TI OMAP/AM335x ailesine özel kod:** güç ve saat alanları, uyku modu ( `sleep33xx.S` ), DT ile açılan kartların ortak girişi ( `board-generic.c` ) |
| `arch/arm/mach-*/` | Diğer SoC aileleri: `mach-imx` (NXP i.MX), `mach-stm32` , `mach-at91` ... |

> **Not**
>
> Device tree'den önce her kart için `arch/arm/mach-omap2/board-xxx.c` gibi bir **board file** yazılırdı. Bugün `mach-omap2` içinde kart dosyası yoktur. Kartları device tree tarif eder, `board-generic.c` sadece "bu SoC ailesi DT ile açılıyor" der (11.1).

---

## 10.5 Kart Yapılandırma Dosyaları

Bir kart için kernel hazırlarken iki tür dosya kullanılır:

| Dosya türü | Ne tarif eder? | BBB için |
|---|---|---|
| **defconfig** ( `arch/arm/configs/` ) | **Kernel'e hangi özelliklerin ve sürücülerin derleneceğini** | `bb.org_defconfig` |
| **Device tree** ( `arch/arm/boot/dts/` ) | **Kartta hangi donanımın nerede olduğunu** | `am335x-boneblack.dts` |

Kullanılabilecek defconfig'ler:

| defconfig | Kapsamı |
|---|---|
| `bb.org_defconfig` | BeagleBoard'un BBB ve diğer Beagle kartları için hazırladığı ayar. Cape, PRU, USB gadget desteği açık. **Bu kitapta kullanılan.** |
| `omap2plus_defconfig` | Mainline'daki bütün OMAP2+ / AM335x / AM437x / DRA7 kartları için ortak ayar |
| `multi_v7_defconfig` | Bütün ARMv7 kartları (TI, NXP, ST, Rockchip...) için tek kernel. En büyük imaj. |

BBB ailesinin device tree dosyaları:

| DTS | Kart |
|---|---|
| `am335x-boneblack.dts` | BeagleBone Black |
| `am335x-boneblack-wireless.dts` | BeagleBone Black Wireless |
| `am335x-bonegreen.dts` | SeeedStudio BeagleBone Green |
| `am335x-bone.dts` | Eski BeagleBone (White) |
| `am335x-pocketbeagle.dts` | PocketBeagle |

Device tree'nin yapısı ve include zinciri Bölüm 11'de anlatılıyor.

---

## 10.6 SoC Sürücülerinin Düzeni ( `drivers/` )

Sürücüler mimariye göre değil, **aygıt türüne** göre klasörlenir. AM335x'in çevre birimlerini süren sürücüler:

| AM335x birimi | Sürücü dosyası | Kernel ayarı |
|---|---|---|
| UART | `drivers/tty/serial/8250/8250_omap.c` | `CONFIG_SERIAL_8250_OMAP` |
| GPIO | `drivers/gpio/gpio-omap.c` | `CONFIG_GPIO_OMAP` |
| I2C | `drivers/i2c/busses/i2c-omap.c` | `CONFIG_I2C_OMAP` |
| SPI (McSPI) | `drivers/spi/spi-omap2-mcspi.c` | `CONFIG_SPI_OMAP24XX` |
| MMC / SD / eMMC | `drivers/mmc/host/sdhci-omap.c` (eski: `omap_hsmmc.c` ) | `CONFIG_MMC_SDHCI_OMAP` |
| Ethernet (CPSW) | `drivers/net/ethernet/ti/cpsw.c` , `cpsw_new.c` | `CONFIG_TI_CPSW` |
| USB (MUSB) | `drivers/usb/musb/musb_dsps.c` | `CONFIG_USB_MUSB_DSPS` |
| USB gadget: Ethernet ( `g_ether` ) | `drivers/usb/gadget/legacy/ether.c` | `CONFIG_USB_ETH` |
| RTC | `drivers/rtc/rtc-omap.c` | `CONFIG_RTC_DRV_OMAP` |
| Watchdog | `drivers/watchdog/omap_wdt.c` | `CONFIG_OMAP_WATCHDOG` |
| PWM (EHRPWM) | `drivers/pwm/pwm-tiehrpwm.c` | `CONFIG_PWM_TIEHRPWM` |
| ADC | `drivers/iio/adc/ti_am335x_adc.c` | `CONFIG_TI_AM335X_ADC` |
| LCD / HDMI | `drivers/gpu/drm/tilcdc/` | `CONFIG_DRM_TILCDC` |
| PMIC (TPS65217) | `drivers/mfd/tps65217.c` | `CONFIG_MFD_TPS65217` |

Bir sürücünün hangi device tree düğümüyle eşleştiğini `compatible` dizisinden bulursun:

```
grep -n "compatible" drivers/gpio/gpio-omap.c          # "ti,omap4-gpio" ...
grep -rn "ti,omap4-gpio" arch/arm/boot/dts/am33xx*.dtsi # DT'deki karşılığı
```

> **Not**
>
> UART sürücüsü konsolun adını belirler: `8250_omap` → `/dev/ttyS0` , eski `omap-serial` → `/dev/ttyO0` . Kernel 5.10'da `8250_omap` kullanıldığı için `bootargs` 'ta **`console=ttyS0`** yazılır (3.4).

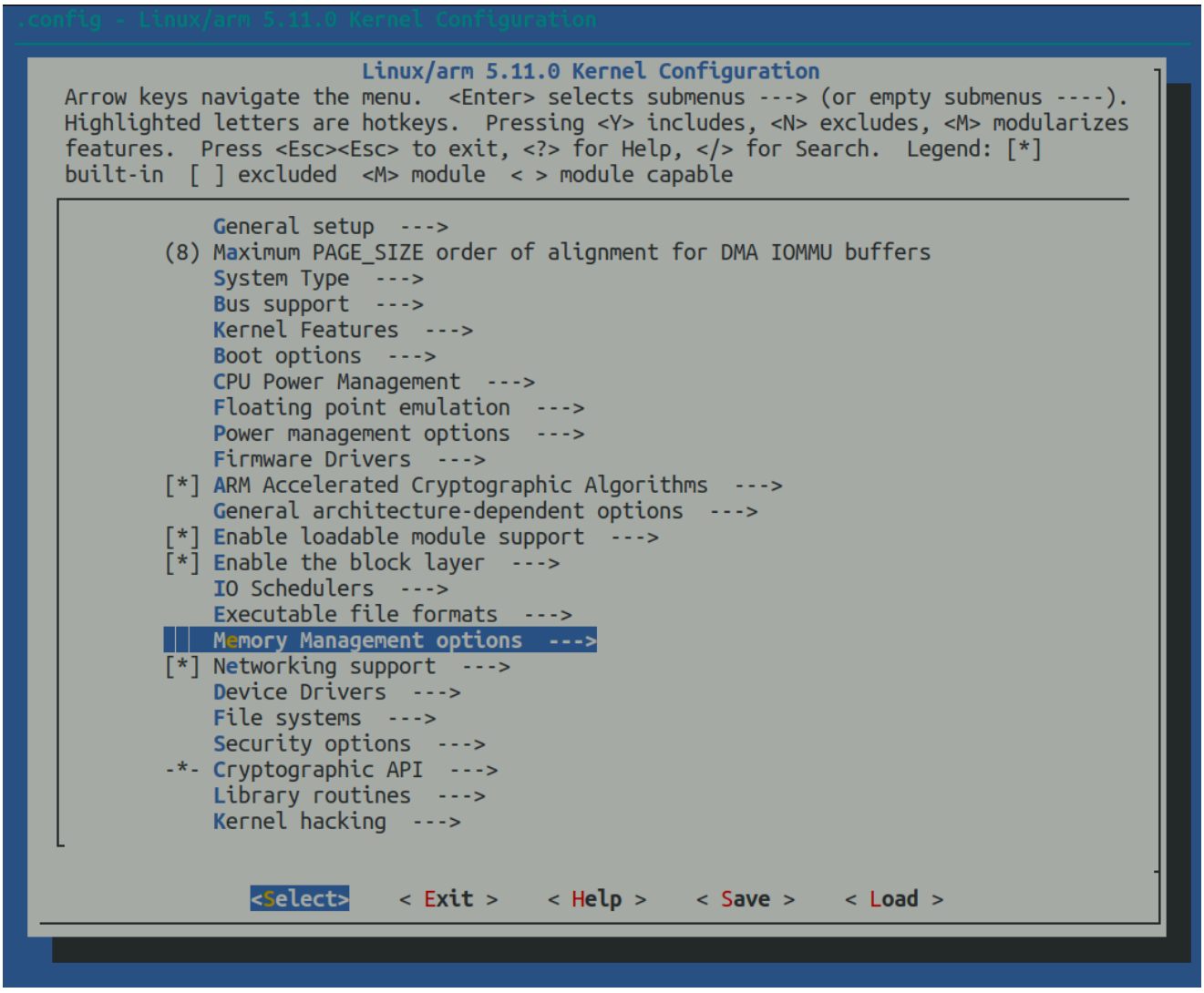
---

## 10.7 Kernel Yapılandırması: Kconfig, `.config` , menuconfig

Kernel'de binlerce seçenek var ve her biri `CONFIG_` ile başlayan bir isimle temsil edilir.

| Parça | Görevi |
|---|---|
| `Kconfig` dosyaları | Her klasörde bulunur. Seçenekleri, açıklamalarını ve birbirine bağımlılıklarını tanımlar. |
| `.config` | Derlemede kullanılan **gerçek** ayar dosyası. Binlerce satır, kaynak klasörünün kökünde. |

| Parça | Görevi |
|---|---|
| `defconfig` | `.config` 'in kısaltılmış hali: sadece varsayılandan farklı olan satırlar. `make xxx_defconfig` bunu açıp `.config` üretir. |
| `make menuconfig` | `.config` 'i menüden düzenler. U-Boot'taki menüyle aynı (9.3). |
| `make savedefconfig` | Değiştirdiğin `.config` 'ten tekrar küçük bir `defconfig` üretir. |




*Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)*

Her sürücü veya özellik için üç seçenek var:

| Değer | menuconfig'te | Anlamı |
|---|---|---|
| `y` | `[*]` / `<*>` | Kernel imajının **içine** derlenir (built-in). Kernel açılır açılmaz hazırdır. |
| `m` | `<M>` | **Modül** olarak ayrı bir `.ko` dosyasına derlenir. Rootfs'ten `modprobe` / `insmod` ile sonradan yüklenir. |
| `n` | `[ ]` / `< >` | Derlenmez |

> **Önemli**
>
> Rootfs'i bağlamak için gereken sürücüler (MMC, ext4, NFS için ağ sürücüsü) `y` olmalıdır. Kernel bunlar olmadan rootfs'i bağlayamaz, rootfs'i bağlamadan da modül yükleyemez. Modüller ( `m` ) ancak rootfs bağlandıktan sonra kullanılabilir.

---

## 10.8 Adım Adım: Kernel'i Derleme

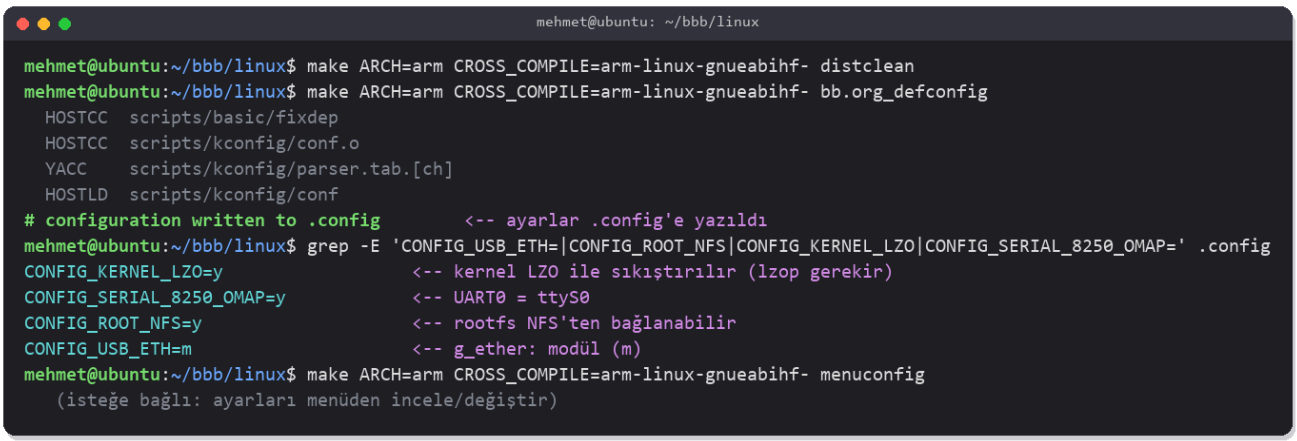
> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**. Örneklerde çalışma klasörü `~/bbb/linux` , toolchain Bölüm 9'da kurulan Linaro GCC 7.5.0.

### Adım 1 — Host paketlerini kur

```
sudo apt install build-essential bc bison flex libssl-dev libncurses-dev lzop u-boot-tools git
```

| Paket | Neden? |
|---|---|
| `bc` | Derleme sırasında bazı sabitler `bc` ile hesaplanır |
| `lzop` | `bb.org_defconfig` kernel'i **LZO** ile sıkıştırır ( `CONFIG_KERNEL_LZO=y` ) |
| `libssl-dev` | Modül imzalama ve sertifika araçları |
| `u-boot-tools` | `mkimage` : `uImage` üretmek için |
| `bison` , `flex` , `libncurses-dev` | Kconfig ve `menuconfig` |

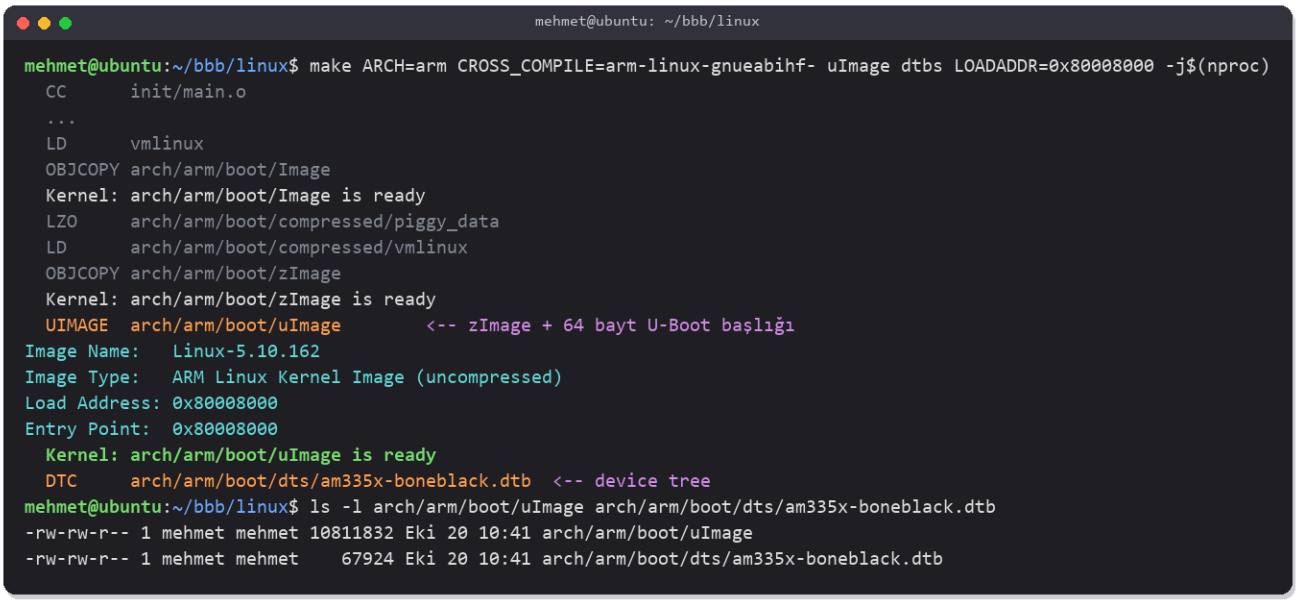
### Adım 2 — Yapılandır



```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig     # isteğe bağlı
```

Kernel'de U-Boot'tan farklı olarak `ARCH=arm` **her komutta zorunludur**. Yazmazsan kernel host'un mimarisini (x86) varsayar ve yanlış `.config` üretir.

### Adım 3 — Kernel imajını ve device tree'yi derle




```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs LOADADDR=0x80008000 -j$(nproc)
```

| Parça | Anlamı |
|---|---|
| `uImage` | `vmlinux` → `Image` → `zImage` → `uImage` zincirini üretir (4.9) |
| `dtbs` | Bütün `.dts` dosyalarını `.dtb`'ye derler |
| `LOADADDR=0x80008000` | `uImage` başlığına yazılan yükleme ve giriş adresi. AM335x'te DDR `0x80000000`'da başlar; kernel ilk 32 KB'ın ( `0x8000` ) arkasına yerleşir. |
| `-j$(nproc)` | İşlemcinin çekirdek sayısı kadar paralel derle |

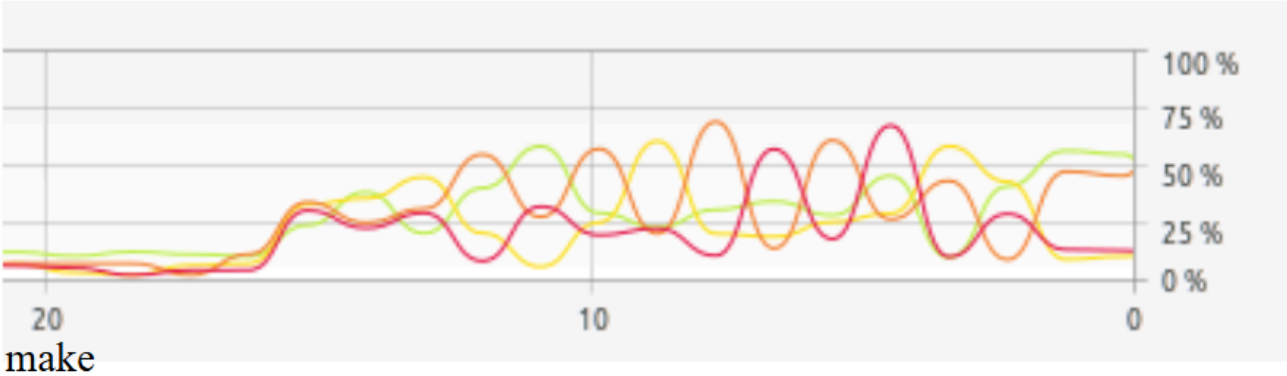
İlk derleme bilgisayarına göre 10–40 dakika sürer. Paralel derlemenin etkisi:

# Benefits of parallel compile jobs (make -j<n>)

Tests on Linux 5.11 on arm

make allnoconfig configuration

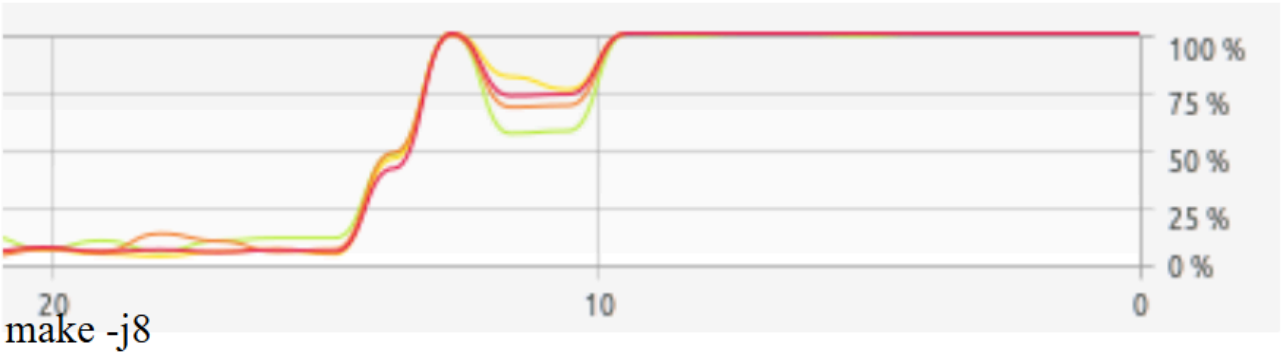
gnome-system-monitor showing the load on 4 threads / 2 CPUs




make

Command: make

Total time: 129 s




make -j8

Command: make -j8

Total time: 67 s

*Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)*

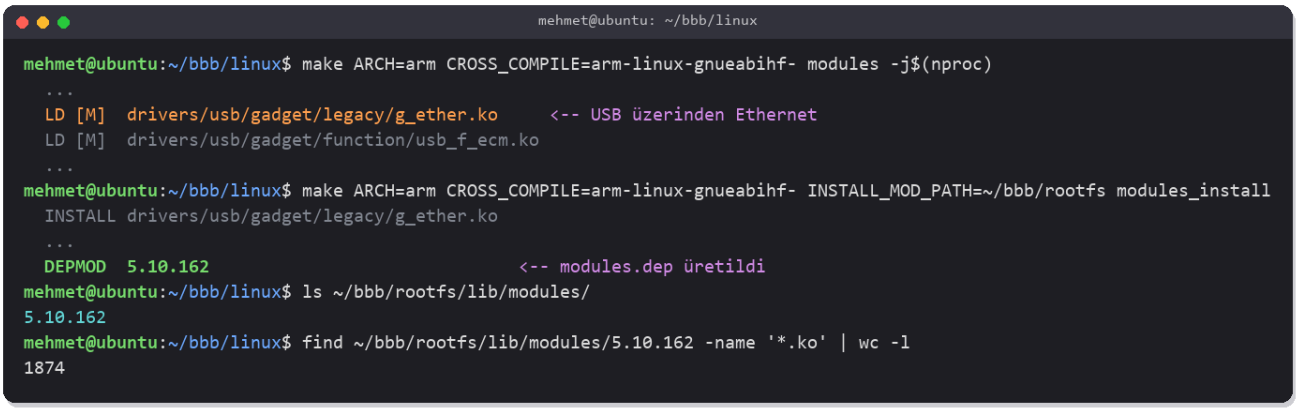
Derleme sonunda çıkan dosyalar:

| Dosya | Ne? |
|---|---|
| `vmlinux` | Derlenmiş kernel, ELF formatında (sembollü, hata ayıklama için) |
| `arch/arm/boot/Image` | Ham binary kernel |
| `arch/arm/boot/zImage` | Sıkıştırılmış, kendini açan kernel |

| Dosya | Ne? |
|---|---|
| `arch/arm/boot/uImage` | `zImage` + U-Boot başlığı. **Karta giden dosya.** |
| `arch/arm/boot/dts/am335x-boneblack.dtb` | BBB'nin device tree'si. **Karta giden dosya.** |
| `System.map` | Kernel sembollerinin adresleri |

## Adım 4 — Modülleri derle ve kur

`m` olarak seçilen sürücüler ayrı derlenir ve rootfs'e kurulur:




```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=~/bbb/rootfs modules_install
```

`modules_install`, modülleri `<INSTALL_MOD_PATH>/lib/modules/5.10.162/` altına kopyalar ve `modprobe`'un kullandığı `modules.dep` dosyasını üretir. Buradaki rootfs klasörünü Bölüm 12'de BusyBox ile dolduracağız.

> **Dikkat**
>
> `INSTALL_MOD_PATH`'i **unutma**. Yazmazsan `sudo make modules_install` modülleri **host bilgisayarının** `/lib/modules` klasörüne kurar; ARM modülleri Ubuntu'nun klasörüne karışır.

## Komutların kısa özeti

```
cd ~/bbb
git clone --depth=1 -b 5.10 https://github.com/beagleboard/linux.git
cd linux
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- distclean
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- bb.org_defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs LOADADDR=0x80008000 -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- modules -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=~/bbb/rootfs modules_install
```

---

## 10.9 Yeni Kernel'i Kartta Denemek

Kernel'i tam olarak denemek için bir rootfs gerekir. Kendi rootfs'imizi Bölüm 12'de hazırlayacağız. Kernel'in açıldığını görmek için şimdilik **eMMC'deki Debian'ın rootfs'ini** kullanabilirsin.

**1) Dosyaları SD kartın BOOT bölümüne kopyala:**

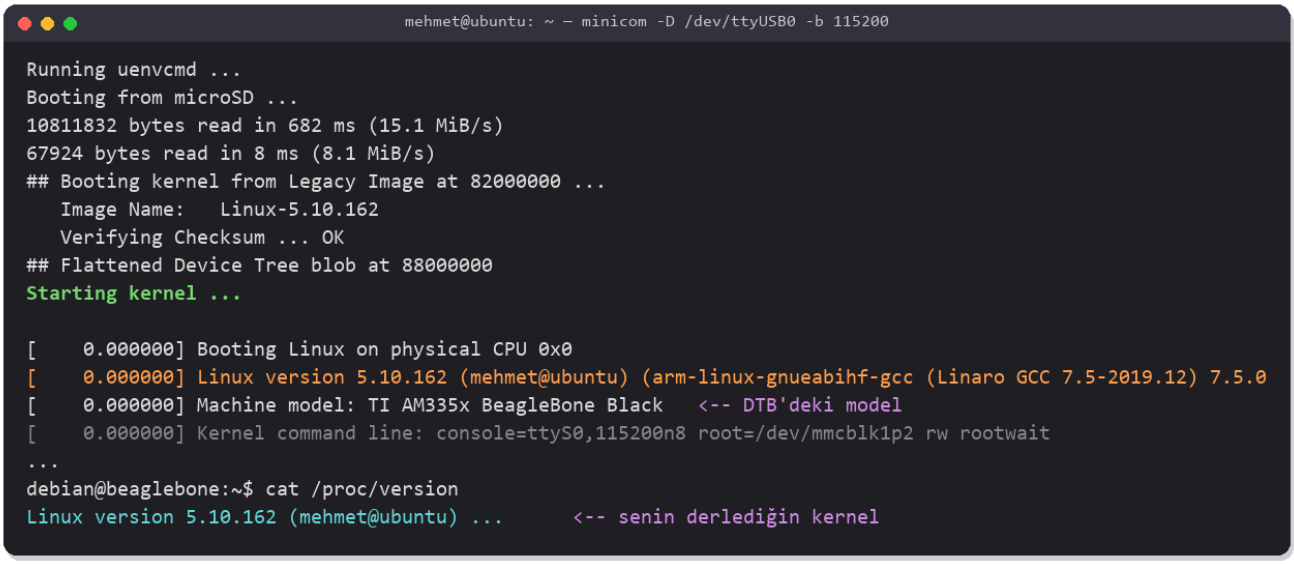
```
cp arch/arm/boot/uImage arch/arm/boot/dts/am335x-boneblack.dtb /media/$USER/BOOT/
```

**2) BOOT bölümündeki `uEnv.txt` 'yi yaz.** Kursun `uEnv-SD.txt` dosyası temel alındı; konsol `ttyS0`, rootfs eMMC:

```
console=ttyS0,115200n8
netargs=setenv bootargs console=${console} root=/dev/mmcblk1p2 rw rootwait
netboot=echo Booting from microSD ...; setenv autoload no; load mmc 0:1 ${loadaddr} uImage; load
mmc 0:1 ${fdtaddr} am335x-boneblack.dtb; run netargs; bootm ${loadaddr} - ${fdtaddr}
uenvcmd=run netboot
```

> **Önemli**
>
> eMMC'deki rootfs'in hangi bölümde olduğu Debian imajına göre değişir. Eski imajlarda 2. bölüm ( `mmcblk1p2` ), yenilerinde tek bölüm ( `mmcblk1p1` ) olabilir. U-Boot'ta `mmc dev 1` ve `mmc part` ile kontrol et (6.5).

**3) Kartı S2'ye basılı tutarak aç.** Açılış mesajlarında kendi kernel sürümünü, derleme makineni ve toolchain'ini görmelisin:




```
cat /proc/version
```

> **Not**
>
> eMMC'deki Debian kendi kernel modüllerini ( `/lib/modules/<eski sürüm>` ) içerir, senin kernel'inin modüllerini içermez. Kernel açılır ama bazı sürücüler (Wi-Fi, USB gadget...) yüklenmeyebilir. Bu bir sorun değil; amacımız sadece kernel'in açıldığını görmek. Modüller Bölüm 12'deki rootfs'e kurulacak.

---

## 10.10 Özet

- BeagleBone Black için BeagleBoard'un kernel ağacı kullanılıyor: `https://github.com/beagleboard/linux.git` , dal `5.10` .
- `arch/arm` mimariye özel kodu, `arch/arm/configs` defconfig'leri, `arch/arm/boot/dts` device tree'leri, `drivers/` sürücüleri tutar. AM335x'e özel SoC kodu `arch/arm/mach-omap2` 'de.
- `defconfig` → `.config` → `menuconfig` ile ayarlanır. `y` = kernel'in içinde, `m` = modül.
- Derleme: `distclean` → `bb.org_defconfig` → `uImage dtbs LOADADDR=0x80008000` → `modules` → `modules_install INSTALL_MOD_PATH=...` . Her komutta `ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-` .
- Karta giden dosyalar: `uImage` , `am335x-boneblack.dtb` ve rootfs'e kurulan modüller.

## 10.11 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~/bbb/linux (temsili hatalar)
mehmet@ubuntu:~/bbb/linux$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- uImage dtbs -j4
multiple (or no) load addresses:
This is incompatible with uImages
Specify LOADADDR on the commandline to build an uImage  <-- LOADADDR=... eksik

"mkimage" command not found - U-Boot images will not be built  <-- u-boot-tools

/bin/sh: 1: lzop: not found                              <-- sudo apt install lzop

scripts/extract-cert.c:21:10: fatal error: openssl/bio.h: No such file  <-- libssl-dev

/bin/sh: 1: bc: not found                                <-- sudo apt install bc
```

| Belirti | Sebep | Çözüm |
|---|---|---|
| `Specify LOADADDR on the commandline to build an uImage` | `uImage` için yükleme adresi verilmedi | Komuta `LOADADDR=0x80008000` ekle |
| `"mkimage" command not found` | `u-boot-tools` kurulu değil | `sudo apt install u-boot-tools` |
| `lzop: not found` | Kernel LZO ile sıkıştırılıyor | `sudo apt install lzop` |

| Belirti | Sebep | Çözüm |
| --- | --- | --- |
| `openssl/bio.h: No such file` | `libssl-dev` eksik | `sudo apt install libssl-dev` |
| `bc: not found` | `bc` eksik | `sudo apt install bc` |
| Derleme x86 için yapılıyor, `.config`'te ARM seçenekleri yok | Bir komutta `ARCH=arm` unutuldu | `distclean` ile temizleyip her komutta `ARCH=arm` yazarak baştan başla |
| `Starting kernel ...` sonrası hiçbir şey yok | `console=` yanlış ya da DTB uyumsuz | Kernel 5.10'da `console=ttyS0`. DTB'yi aynı kernel kaynağından derle. |
| `VFS: Unable to mount root fs on unknown-block(179,...)` | `root=` yanlış bölümü gösteriyor | U-Boot'ta `mmc part`, Linux'ta (açılabiliyorsa) `lsblk` ile doğru bölümü bul |
| Ubuntu'nun `/lib/modules` klasöründe ARM modülleri var | `modules_install` `INSTALL_MOD_PATH` olmadan `sudo` ile çalıştırıldı | O sürüm klasörünü `/lib/modules`'ten sil, komutu `INSTALL_MOD_PATH` ile tekrar çalıştır |

# Bölüm 11 — Linux Device Tree

> **Bu bölümde:**
>
> - Kernel'in, kendini tanıtmayan donanımı (UART, I2C, GPIO...) nereden öğrendiğini anlayacaksın.
> - DTS, DTSI, DTB, DTBO dosyalarını ve `dtc` derleyicisini tanıyacaksın.
> - Device tree söz dizimini, `compatible` ile driver eşleşmesini ve BeagleBone Black'in include zincirini göreceksin.
> - Çalışan sistemde device tree'yi `/proc/device-tree` altından inceleyeceksin.
>
> **Ön koşul:** Bölüm 3 — boot zinciri, Bölüm 4 — U-Boot'un DTB'yi RAM'e yüklemesi.

---

## 11.1 Problem: Kernel Donanımı Nereden Bilecek?

Kernel'in çalışabilmesi için kartta **hangi donanımların olduğunu, nerede durduklarını ve nasıl bağlandıklarını** bilmesi gerekir. Örneğin:

- UART denetleyicisinin register'ları hangi adreste?
- Kaç MB RAM var, hangi adresten başlıyor?
- I2C hattına hangi çip, hangi adresle bağlı?
- USR0 LED'i hangi GPIO pinine bağlı?
- Hangi cihaz hangi interrupt hattını kullanıyor?

### Keşfedilebilen ve keşfedilemeyen donanım

| | **Keşfedilebilir (discoverable)** | **Keşfedilemez (non-discoverable)** |
|---|---|---|
| **Örnek** | PCI, PCIe, USB | UART, I2C, SPI, GPIO, Ethernet MAC, NAND, SoC içindeki çoğu çevre birimi |
| **Nasıl bulunur?** | Cihaz kendini tanıtır (Vendor ID / Device ID). Kernel bus'ı tarayarak bulur. | Cihaz kendini tanıtmaz. Kernel'e **birinin söylemesi gerekir**. |

Masaüstü PC'lerde (x86) bu bilgi BIOS/UEFI tarafından **ACPI tabloları** ile verilir. Gömülü sistemlerde (ARM, RISC-V, PowerPC) ise donanımın büyük kısmı SoC'un içinde, belirli adreslerde duran ve kendini tanıtmayan bloklardır. Bu yüzden bu bilgiyi kernel'e bir şekilde aktarmak gerekir.

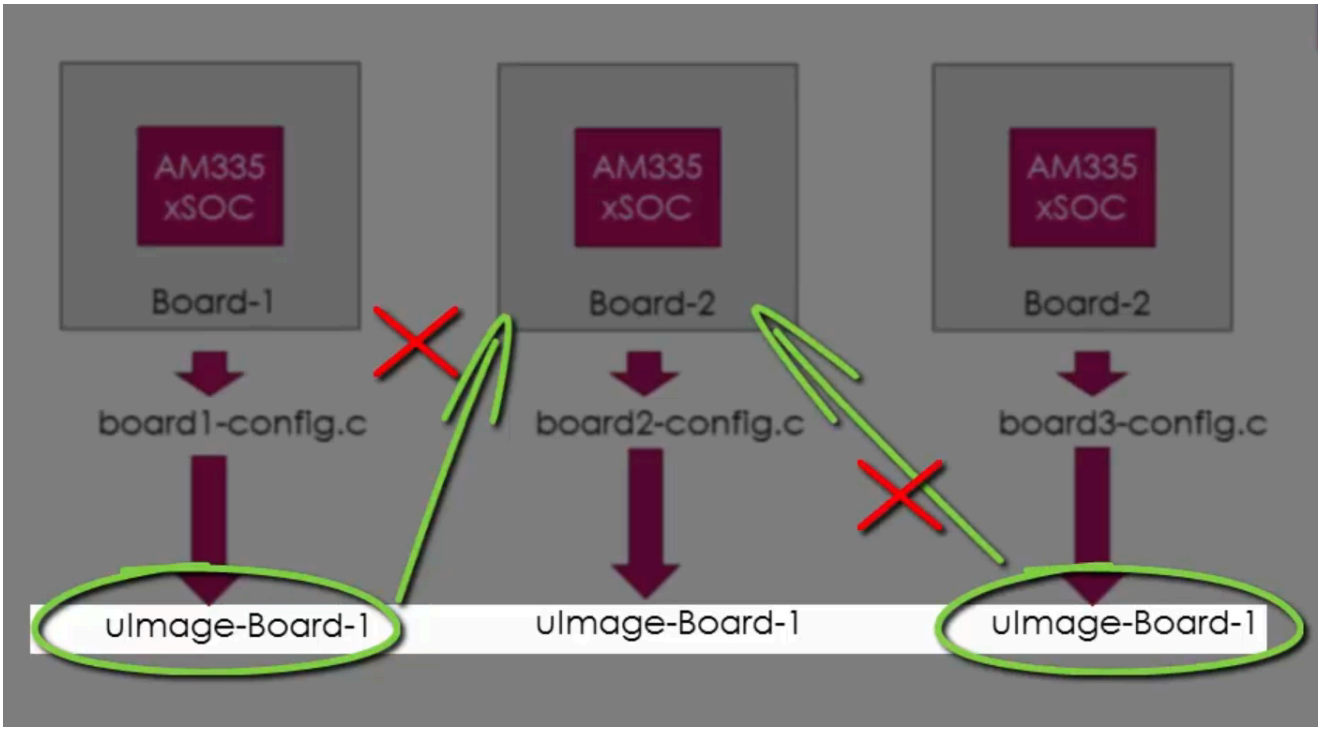
### Eski yöntem: board file'lar

Device Tree'den önce ARM'da her kart için kernel kaynak kodunda bir C dosyası (**board file**) yazılırdı (`arch/arm/mach-xxx/board-yyy.c`). Donanım bilgisi bu dosyalara sabit olarak gömülürdü:

```
/* Eski usül: donanım C koduna gömülü (temsili örnek) */
static struct resource uart0_resources[] = {
    { .start = 0x44E09000, .end = 0x44E09FFF, .flags = IORESOURCE_MEM },
    { .start = 72,         .end = 72,         .flags = IORESOURCE_IRQ },
};
```

Bu yöntemin sorunları:

- Her yeni kart = kernel'e yeni C kodu. `arch/arm` altında binlerce kart dosyası birikti.
- Kartta küçük bir değişiklik (ör. LED'in başka pine taşınması) bile **kernel'i yeniden derlemeyi** gerektiriyordu.
- Her kart için ayrı bir kernel image'ı gerekiyordu. **Tek bir kernel, birden fazla kartta çalışamıyordu.**




2011'de Linus Torvalds ARM tarafındaki bu karmaşadan açıkça şikayet etti. Bunun üzerine ARM dünyası, PowerPC'de zaten kullanılan **Device Tree**'ye geçti (kernel 3.x serisi). Bugün yeni ARM32 ve ARM64 kartları için Device Tree zorunludur.

> **Not**
>
> Temel fikir: **Donanım tarifi koddan ayrılır.** Kernel aynı kalır, kartı tarif eden küçük bir veri dosyası (DTB) değişir. Böylece aynı `zImage` hem BeagleBone Black'te hem de başka bir AM335x kartında çalışabilir. Sadece DTB farklıdır.

---

## 11.2 Device Tree Nedir?

**Device Tree (DT)**, bir sistemdeki donanımı tarif eden, **ağaç (tree) şeklinde bir veri yapısıdır**.

- Ağaç **node**'lardan (düğüm) oluşur. Her node bir donanım bloğunu temsil eder (CPU, bellek, UART, I2C denetleyicisi, EEPROM...).
- Her node'un **property**'leri (özellikleri) vardır: adres, interrupt numarası, uyumlu driver adı, durum vb.
- Node'lar iç içe olabilir. Bir I2C denetleyicisinin altında ona bağlı I2C cihazları alt node olarak yer alır. Bu, donanımdaki **fiziksel bağlantı hiyerarşisini** yansıtır.
- Kökeni Sun'ın **OpenFirmware** standardına dayanır. Bugün **devicetree.org** tarafından Devicetree Specification olarak standartlaştırılmıştır.
- **İşletim sisteminden bağımsızdır**: Linux, U-Boot, FreeBSD, Zephyr gibi projeler de kullanır.

> **Önemli**
>
> Device Tree **donanımın ne olduğunu** tarif eder, **nasıl kullanılacağını** değil. "Bu kartta 0x44E09000 adresinde bir UART var" bir DT bilgisidir. "Bu UART'ı 115200 baud ile konsol olarak kullan" ise bir yapılandırma tercihidir ve genelde kernel command line'a ( `bootargs` ) aittir.

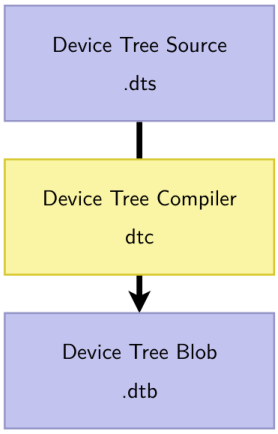
Device Tree'de olmayan bir şey kernel tarafından **bilinemez**. Driver kernel'e derlenmiş olsa bile, DT'de ona uyan bir node yoksa o driver hiç çalışmaz (probe edilmez).

---

## 11.3 Dosya Türleri: DTS, DTSI, DTC, DTB, DTBO

### Device Tree: from source to blob

- A tree data structure describing the hardware is written by a developer in a **Device Tree Source** file, `.dts`
- Processed by the **Device Tree Compiler**, `dtc`
- Produces a more efficient representation: **Device Tree Blob**, `.dtb`
- Additional C preprocessor pass
- `.dtb` → accurately describes the hardware platform in an **OS-agnostic** way.
- `.dtb`≈ few dozens of kilobytes
- DTB also called **FDT**, *Flattened Device Tree*, once loaded into memory.
  - `fdt` command in U-Boot
  - `fdt_` APIs






*Görsel: Bootlin, "Linux kernel and driver development" eğitim slaytları (CC BY-SA 3.0)*

| Kısaltma | Açılımı | Ne? |
|---|---|---|
| **DT** | Device Tree | Genel kavram: donanımı tarif eden ağaç yapısı |

| Kısaltma | Açılımı | Ne? |
|---|---|---|
| **.dts** | Device Tree **Source** | İnsanın okuyup yazabildiği **metin** kaynak dosyası. Bir kartı tarif eder. Örn: `am335x-boneblack.dts` |
| **.dtsi** | Device Tree Source **Include** | Başka dosyalara `#include` edilen ortak parça. Genelde SoC'u ya da bir kart ailesini tarif eder. Örn: `am33xx.dtsi`. C'deki `.h` dosyalarına benzer. |
| **dtc** | Device Tree **Compiler** | `.dts` dosyasını `.dtb`'ye derleyen (ve tersini yapabilen) araç |
| **.dtb** | Device Tree **Blob** (Binary) | Derlenmiş, **ikili (binary)** çıktı. Bootloader bunu RAM'e yükler ve kernel bunu okur. Örn: `am335x-boneblack.dtb` |
| **FDT** | Flattened Device Tree | DTB'nin bellekteki "düzleştirilmiş" formatının adı. U-Boot'taki `fdt` komutu ve `fdtaddr` değişkeni buradan gelir. |
| **.dtbo** | Device Tree Blob **Overlay** | Var olan bir DTB'ye çalışma/boot anında eklenen **yama**. Cape/HAT gibi eklenti kartlar için kullanılır. (11.12) |

## Derleme akışı

```
am33xx.dtsi ─┐
am335x-bone-common.dtsi ─┤  #include
am335x-boneblack-common.dtsi ─┤
                              ▼
            am335x-boneblack.dts
                    │
                    │  1) C preprocessor (cpp): #include, #define çözülür
                    │  2) dtc: metin → binary
                    ▼
            am335x-boneblack.dtb   (~60-70 KB)
                    │
                    │  U-Boot RAM'e yükler (fdtaddr)
                    ▼
                Linux kernel
```

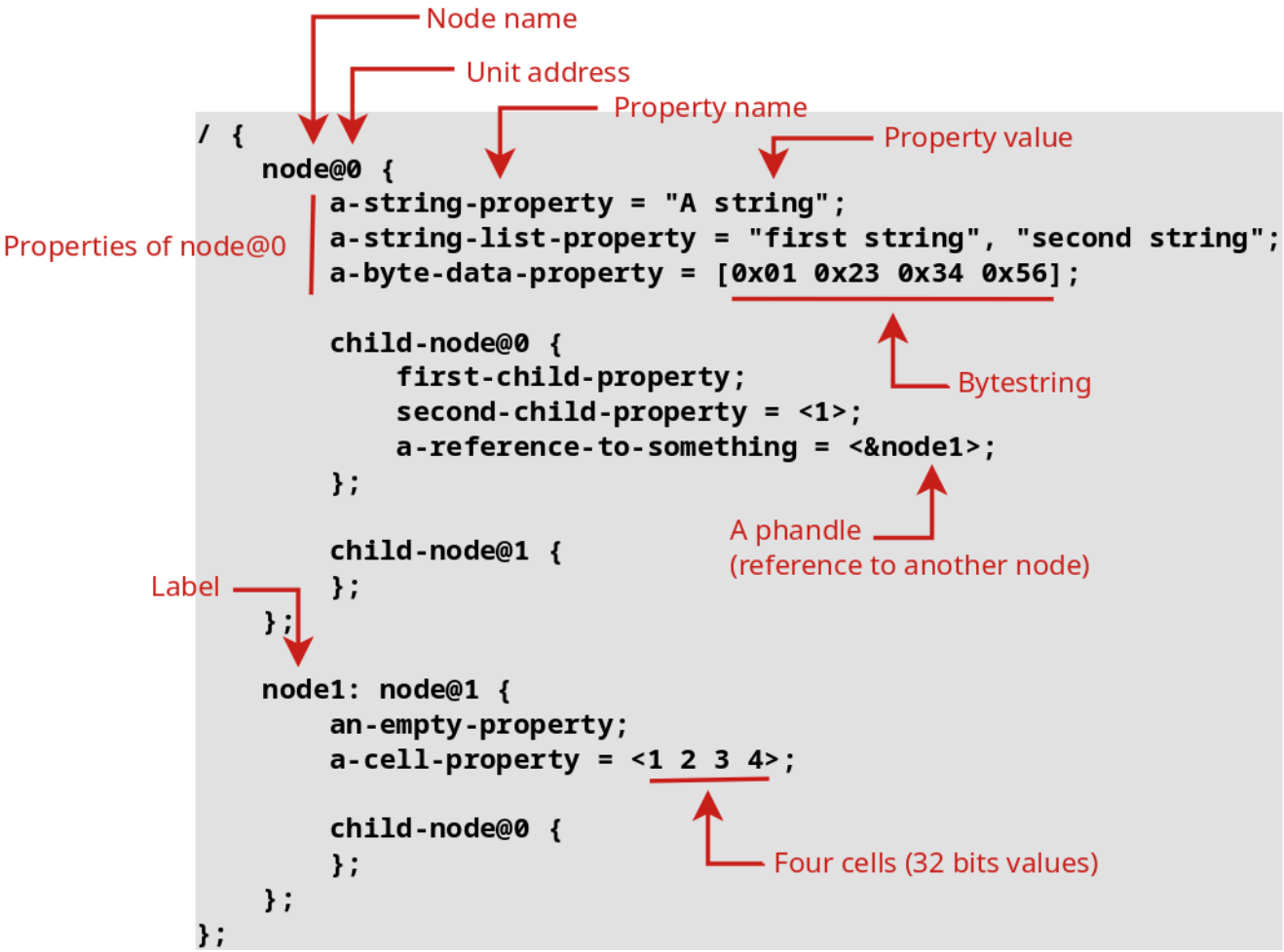
- Kernel kaynak ağacında `.dts` dosyaları önce **C ön işlemcisinden (cpp)** geçer. Bu sayede `#include <dt-bindings/gpio/gpio.h>` ve `GPIO_ACTIVE_HIGH` gibi sabitler kullanılabilir. Sonra `dtc` ile derlenir.
- DTB küçüktür (birkaç on KB) ve kernel'in hızlıca okuyabileceği bir formattadır.
- Her DTB dosyası `0xd00dfeed` sihirli sayısı (magic number) ile başlar. Bir dosyanın gerçekten DTB olup olmadığını böyle anlayabilirsin:

```
$ hexdump -C am335x-boneblack.dtb | head -1     # ilk 4 byte: d0 0d fe ed
00000000  d0 0d fe ed 00 01 0a 5c  00 00 00 38 00 00 f8 a8  |.......\...8....|
```

> **Not**
>
> Kernel tarafındaki `.dts` dosyaları kernel kaynak kodunun içindedir: ARM32 için `arch/arm/boot/dts/`, ARM64 için `arch/arm64/boot/dts/<vendor>/`. Kernel 6.5'ten itibaren ARM32 dosyaları da vendor klasörlerine taşındı. BeagleBone Black'in dosyası yeni kernel'lerde `arch/arm/boot/dts/ti/omap/am335x-boneblack.dts` konumundadır.

---

## 11.4 Söz Dizimi (Syntax)




*Görsel: Bootlin eğitim materyalleri (CC BY-SA 3.0)*

### Node

```
[label:] node-name[@unit-address] {
    [properties]
    [child nodes]
};
```

| Parça | Açıklama | Örnek |
|---|---|---|
| `/` | Kök (root) node. Her DT'de bir tane vardır ve tüm sistemi temsil eder. | `/ { ... };` |
| **node-name** | Cihazın **türünü** anlatan genel bir isim | `serial`, `i2c`, `gpio`, `memory`, `eeprom` |
| **@unit-address** | Node'un adresi. Genelde `reg` property'sinin ilk değeriyle aynıdır. | `serial@44e09000`, `eeprom@50` |
| **label** | Node'a verilen takma ad. Başka yerlerden `&label` diye referans verilir. | `uart0: serial@44e09000` |
| `;` | Her node ve property `;` ile biter. Unutmak en sık yapılan derleme hatasıdır. | |

Bir node'un tam yolu (path) dosya sistemi yoluna benzer: `/ocp/interconnect@44c00000/.../serial@0`.

## Property

Property'ler `isim = değer;` şeklindedir. Değer türleri:

| Tür | Yazım | Örnek |
|---|---|---|
| **Boş (boolean)** | Sadece isim. Varsa "true", yoksa "false" demektir. | `gpio-controller;` |
| **String** | Çift tırnak | `status = "okay";` |
| **String listesi** | Virgülle ayrılmış string'ler | `compatible = "ti,am3352-uart", "ti,omap3-uart";` |
| **Cell (u32) listesi** | `< >` içinde 32-bit sayılar | `reg = <0x44e09000 0x2000>;` |
| **Byte dizisi** | `[ ]` içinde hex byte'lar | `local-mac-address = [00 11 22 33 44 55];` |
| **phandle (referans)** | `< >` içinde `&label` | `gpios = <&gpio1 21 GPIO_ACTIVE_HIGH>;` |

> **İpucu**
>
> **phandle**, bir node'un başka bir node'a "işaretçi" (pointer) gibi referans vermesidir. Yukarıdaki `gpios` örneği şunu söyler: "Bu LED, `gpio1` denetleyicisinin **21.** pinine bağlı ve **active-high**." dtc derleme sırasında `&gpio1` 'i o node'a ait benzersiz bir sayıya (phandle) çevirir.

---

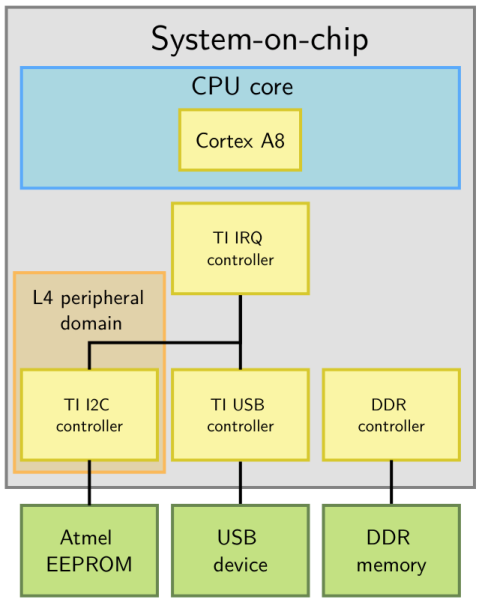
## 11.5 Bir Device Tree'nin Genel Yapısı

DT overall structure: simplified example

```
/ {
  cpus { ... };
  memory@0x80000000 { ... };
  chosen { ... };
  ocp {
    intc: interrupt-controller@48200000 { ... };
    usb0: usb@47401300 { ... };
    l4_per: interconnect@44c00000 {

      i2c0: i2c@40012000 {
        compatible = "ti,omap4-i2c";
        #address-cells = <1>;
        #size-cells = <0>;
        reg = <0x0 0x1000>;
        interrupts = <70>;
        status = "okay";
        pinctrl-names = "default";
        pinctrl-0 = <&i2c0_pins>;
        clock-frequency = <400000>;

        baseboard_eeprom: eeprom@50 {
          compatible = "atmel,24c256";
          reg = <0x50>;
        };
      };
    };
  };
};
```






*Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0). Soldaki ağaç, sağdaki AM335x tabanlı donanımı tarif ediyor.*

Görseldeki eşleşmeye dikkat et: DT'deki iç içe yapı **donanımdaki bağlantıyı** birebir yansıtır. EEPROM, I2C denetleyicisine bağlı olduğu için DT'de de `i2c@40012000` node'unun **altındadır**.

Hemen her DT'de bulunan standart node'lar:

| Node | Görevi |
|---|---|
| `/` | Kök. `model` (kartın adı) ve `compatible` (kartın kimliği) burada tanımlanır. |
| `cpus` | İşlemci çekirdekleri (BBB'de tek bir Cortex-A8) |
| `memory@...` | RAM'in başlangıç adresi ve boyutu |
| `chosen` | Donanım değil, bootloader ile kernel arasındaki bilgiler: `bootargs`, `stdout-path` (konsol) |
| `aliases` | Kısa isimler. Örn: `serial0 = &uart0;` sayesinde `uart0` → `/dev/ttyS0` olur. |
| `ocp` / `soc` | SoC içindeki çevre birimlerinin bulunduğu bus. AM335x'te adı `ocp`'dir (On-Chip Peripherals). |

### BeagleBone Black'ten gerçek bir örnek

BBB'nin ana DTS dosyası aslında çok kısadır. Asıl iş include ettiği dosyalardadır:

```
/* am335x-boneblack.dts (kısaltılmış) */
/dts-v1/;

#include "am33xx.dtsi"                    /* AM335x SoC tanımı */
#include "am335x-bone-common.dtsi"        /* Tüm BeagleBone'larda ortak */
#include "am335x-boneblack-common.dtsi"   /* BBB ve BBB Wireless'ta ortak */
#include "am335x-boneblack-hdmi.dtsi"     /* HDMI desteği */

/ {
    model = "TI AM335x BeagleBone Black";
    compatible = "ti,am335x-bone-black", "ti,am335x-bone", "ti,am33xx";
};
```

`am335x-bone-common.dtsi` içinden USR LED tanımı (kısaltılmış):

```
leds {
    pinctrl-names = "default";
    pinctrl-0 = <&user_leds_s0>;      /* pin mux ayarı */
    compatible = "gpio-leds";         /* hangi driver? → leds-gpio */

    led2 {
        label = "beaglebone:green:usr0";
        gpios = <&gpio1 21 GPIO_ACTIVE_HIGH>;    /* GPIO1_21 pini */
        linux,default-trigger = "heartbeat";    /* kalp atışı gibi yanıp söner */
        default-state = "off";
    };
    /* usr1, usr2, usr3 ... */
};
```

> **İpucu**
>
> Kart açıldığında USR0 LED'inin "kalp atışı" gibi yanıp sönmesinin sebebi tam olarak bu satırdır: `linux,default-trigger = "heartbeat";` . Yani bu davranış kernel kodunda değil, **Device Tree'de** tanımlıdır.

---

## 11.6 Önemli Property'ler

| Property | Anlamı | Örnek |
|---|---|---|
| `compatible` | **En önemli property.** Cihazın ne olduğunu ve hangi driver'ın onu yöneteceğini belirler. Format: `"üretici,model"` . En özelden en genele doğru sıralanır. | `"ti,am3352-uart"`, `"ti,omap3-uart"` |

| Property | Anlamı | Örnek |
|---|---|---|
| `reg` | Cihazın adresi. Memory-mapped cihazlarda **(başlangıç adresi, boyut)** çifti. I2C/SPI cihazlarında bus üzerindeki adres. | `reg = <0x44e09000 0x2000>;` `reg = <0x50>;` |
| `#address-cells` | Alt node'ların `reg` değerindeki **adresin** kaç adet 32-bit cell olduğu | `<1>` |
| `#size-cells` | Alt node'ların `reg` değerindeki **boyutun** kaç cell olduğu. I2C'de boyut yoktur, bu yüzden `<0>`. | `<1>` veya `<0>` |
| `status` | Cihaz kullanılıyor mu? `"okay"` → evet, `"disabled"` → hayır (driver probe edilmez). | `status = "okay";` |
| `interrupts` | Cihazın kullandığı interrupt hattı | `interrupts = <72>;` |
| `interrupt-parent` | Interrupt'ın hangi interrupt denetleyicisine gittiği (phandle) | `<&intc>` |
| `clocks` | Cihazın beslendiği saat (clock) kaynakları | `clocks = <&l4ls_gclk>;` |
| `pinctrl-0`, `pinctrl-names` | Pin multiplexing: SoC pinlerinin bu cihaz için hangi moda alınacağı | `pinctrl-0 = <&uart0_pins>;` |
| `gpios`, `*-gpios` | Kullanılan GPIO pinleri (phandle + pin no + flag) | `<&gpio1 21 GPIO_ACTIVE_HIGH>` |

## reg, #address-cells ve #size-cells ilişkisi

`reg`'in nasıl okunacağını **parent node'daki** `#address-cells` ve `#size-cells` belirler:

```
ocp {
    #address-cells = <1>;   /* alt node'larda adres = 1 cell */
    #size-cells = <1>;      /* alt node'larda boyut = 1 cell */

    serial@44e09000 {
        reg = <0x44e09000 0x2000>;   /* adres=0x44E09000, boyut=8 KB */
    };
};

i2c0: i2c@44e0b000 {
    #address-cells = <1>;
    #size-cells = <0>;      /* I2C cihazlarının "boyutu" yok */

    baseboard_eeprom: eeprom@50 {
        compatible = "atmel,24c256";
        reg = <0x50>;       /* I2C adresi 0x50 */
    };
};
```

> **Not**
>
> BBB'nin üzerindeki **EEPROM** (kart adını ve seri numarasını tutan çip) gerçekten de I2C0 hattında **0x50** adresindedir. U-Boot bu EEPROM'u okuyarak kartın BeagleBone Black olduğunu anlar ve doğru DTB'yi ( `am335x-boneblack.dtb` ) seçer.

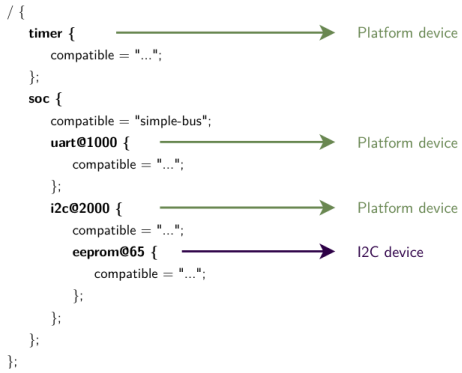
---

## 11.7 compatible ile Driver Eşleşmesi

Kernel DTB'yi okuduğunda her node için bir **device** oluşturur. Sonra her device için `compatible` string'ine bakar ve **aynı string'i destekleyen bir driver** arar. Eşleşme bulunursa driver'ın `probe()` fonksiyonu çağrılır ve cihaz o driver'a **bağlanır (bind)**.

### `compatible` property and Linux kernel drivers

- Linux identifies as **platform devices**:
  - Top-level DT nodes with a `compatible` string
  - Sub-nodes of `simple-bus`
    - Instantiated automatically at boot time
- Sub-nodes of I2C controllers → *I2C devices*
- Sub-nodes of SPI controllers → *SPI devices*
- Each Linux driver has a table of compatible strings it supports
  - struct of_device_id[]
- When a DT node compatible string matches a given driver, the device is *bound* to that driver.






*Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)*

- Kök node'un hemen altındaki ve `simple-bus` altındaki node'lar → **platform device** olur.
- I2C denetleyicisinin altındaki node'lar → **I2C device** olur.
- SPI denetleyicisinin altındaki node'lar → **SPI device** olur.

Driver tarafında bu eşleşme `of_device_id` tablosu ile yapılır ( `of` = OpenFirmware):

```
/* drivers/i2c/busses/i2c-omap.c (kısaltılmış) */
static const struct of_device_id omap_i2c_of_match[] = {
    { .compatible = "ti,omap4-i2c", .data = &omap4_pdata },
    { .compatible = "ti,omap3-i2c", .data = &omap3_pdata },
    { },
};
MODULE_DEVICE_TABLE(of, omap_i2c_of_match);

static struct platform_driver omap_i2c_driver = {
    .probe  = omap_i2c_probe,
    .remove = omap_i2c_remove,
    .driver = {
        .name           = "omap_i2c",
        .of_match_table = omap_i2c_of_match,   /* DT eşleşme tablosu */
    },
};
```

```
DT node:   compatible = "ti,omap4-i2c"
                    │
                    │  string eşleşmesi
                    ▼
Driver:    { .compatible = "ti,omap4-i2c" }  →  omap_i2c_probe() çağrılır
```

Driver `probe()` içinde `reg`, `interrupts`, `clocks` gibi property'leri okuyarak cihazı başlatır (`platform_get_resource()`, `of_property_read_u32()` vb.).

> **İpucu**
>
> `compatible` listesinde birden fazla string olmasının sebebi **geriye dönük uyumluluktur**. `"ti,am3352-uart", "ti,omap3-uart"` şunu söyler: "Ben aslında AM3352 UART'ıyım. Bana özel driver yoksa OMAP3 UART driver'ı da beni sürebilir." Kernel listeyi soldan sağa dener.

> **Not**
>
> Bir cihazın property'lerinin nasıl yazılacağı (hangi property zorunlu, hangi değerler geçerli) **binding** denen belgelerle tanımlanır. Bunlar kernel'de `Documentation/devicetree/bindings/` altındadır ve günümüzde YAML formatındadır. Yeni bir cihaz eklerken ilk bakılacak yer burasıdır.

---

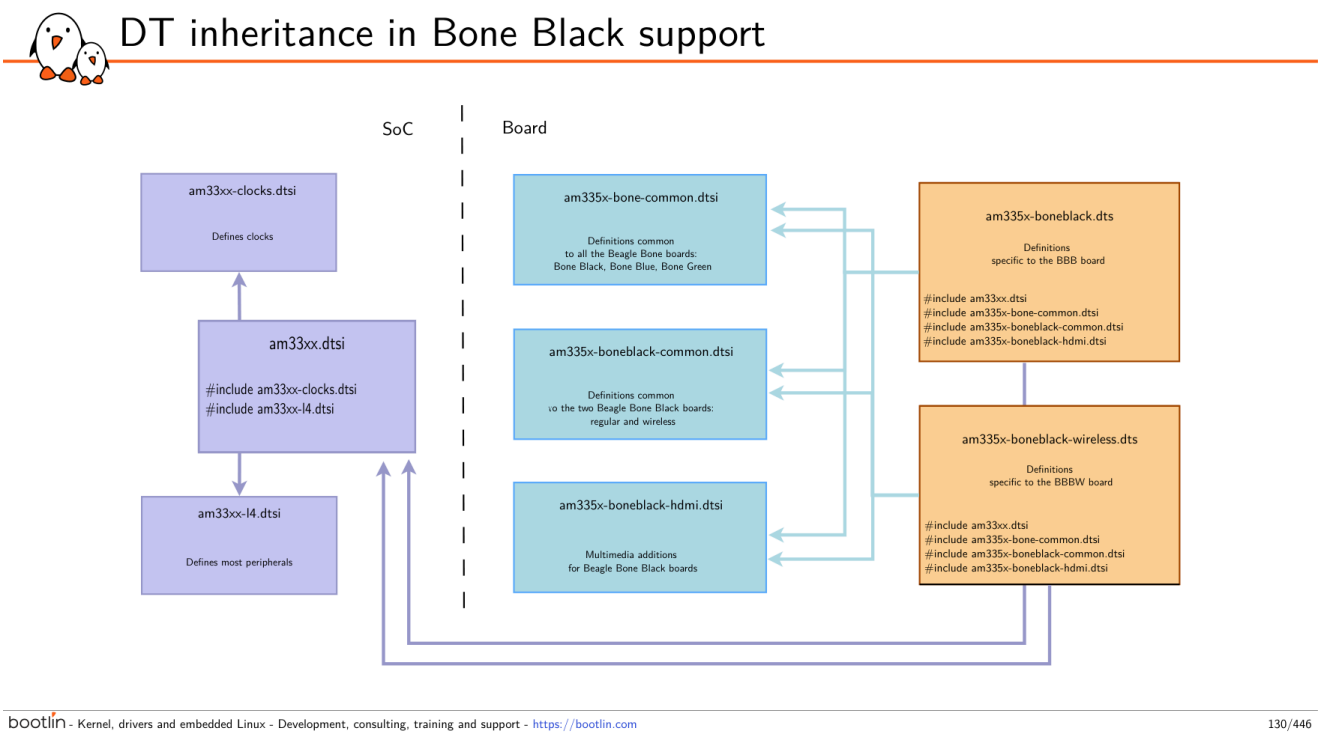
## 11.8 Kalıtım: .dtsi, include ve Label ile Override

Aynı SoC'u kullanan onlarca kart vardır. Her kart için SoC'un tamamını baştan yazmak yerine:

1. SoC'un tüm çevre birimleri bir `.dtsi` dosyasında tanımlanır. Çoğu `status = "disabled"` olarak bırakılır.
2. Kartın `.dts` dosyası bu `.dtsi`'yi include eder. Kartta kullanılan birimleri `status = "okay"` yaparak açar ve karta özel cihazları (EEPROM, LED, PMIC...) ekler.

Aynı node birden fazla kez tanımlanırsa dtc bunları **birleştirir**. Aynı property iki yerde varsa **sonra gelen kazanır**.

### BeagleBone Black'in include zinciri




*Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)*

| Dosya | Seviye | İçerik |
|---|---|---|
| `am33xx.dtsi` | SoC | AM335x işlemcinin genel tanımı. `am33xx-l4.dtsi` (çevre birimleri) ve `am33xx-clocks.dtsi` (saatler) dosyalarını include eder. |
| `am335x-bone-common.dtsi` | Kart ailesi | Tüm BeagleBone'larda ortak olanlar: LED'ler, EEPROM, PMIC (TPS65217), 256 MB RAM |
| `am335x-boneblack-common.dtsi` | Kart ailesi | BBB ve BBB Wireless'ta ortak olanlar: eMMC, 512 MB RAM |
| `am335x-boneblack-hdmi.dtsi` | Ek özellik | HDMI çıkışı (TDA19988 çipi) |
| `am335x-boneblack.dts` | Kart | Sadece BBB'ye özel olanlar: `model`, `compatible` |

Override'a güzel bir örnek RAM boyutudur. `am335x-bone-common.dtsi` RAM'i 256 MB tanımlar (eski BeagleBone White'ta 256 MB vardı). `am335x-boneblack-common.dtsi` aynı node'u tekrar yazarak bunu 512 MB'a çıkarır:

```
/* am335x-bone-common.dtsi */
memory@80000000 {
    device_type = "memory";
    reg = <0x80000000 0x10000000>;   /* 256 MB */
};

/* am335x-boneblack-common.dtsi (sonra include edildiği için kazanır) */
memory@80000000 {
    device_type = "memory";
    reg = <0x80000000 0x20000000>;   /* 512 MB */
};
```

## Label ile override (tercih edilen yöntem)

Bir node'u değiştirmek için tam yolunu tekrar yazmak yerine, **label**'ı ile doğrudan ona erişmek tercih edilir:

### Inheritance and labels

Doing:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

/ {
  ocp {
    serial@0 {
      status = "okay";
    };
  };
};
```

Is exactly equivalent to:

soc.dtsi

```
/ {
  ocp {
    uart0: serial@0 {
      compatible = "ti,am3352-uart";
      reg = <0x0 0x1000>;
      status = "disabled";
    };
  };
};
```

board.dts

```
#include "soc.dtsi"

&uart0 {
  status = "okay";
};
```

→ this solution is preferred



*Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0). Soldaki ve sağdaki board.dts tamamen aynı sonucu üretir. Sağdaki `&uart0 { ... };` yazımı tercih edilir.*

```
/* Kart dosyasında: SoC'ta "disabled" olan I2C2'yi aç ve hızını ayarla */
&i2c2 {
    status = "okay";
    clock-frequency = <100000>;
};
```

---

## 11.9 dtc Kullanımı

### dtc example

```
$ cat foo.dts
/dts-v1/;

/ {
        welcome = <0xBADCAFE>;
        bootlin {
                webinar = "great";
                demo = <1>, <2>, <3>;
        };
};
```

```
$ dtc -I dtb -O dts foo.dtb
/dts-v1/;

/ {
        welcome = <0xbadcafe>;

        bootlin {
                webinar = "great";
                demo = <0x01 0x02 0x03>;
        };
};
```

```
$ dtc -I dts -O dtb -o foo.dtb foo.dts
$ ls -l foo.dt*
-rw-r--r-- 1 thomas thomas 169 ... foo.dtb
-rw-r--r-- 1 thomas thomas 102 ... foo.dts
```



*Görsel: Bootlin eğitim slaytları (CC BY-SA 3.0)*

### Kurulum

```
sudo apt install device-tree-compiler
```

### Temel komutlar

```
# DTS → DTB (derleme)
dtc -I dts -O dtb -o foo.dtb foo.dts

# DTB → DTS (geri derleme / decompile)
dtc -I dtb -O dts -o foo_geri.dts foo.dtb

# Çalışan sistemin canlı device tree'sini DTS olarak dök
dtc -I fs -O dts /proc/device-tree
```

| Parametre | Anlamı |
|---|---|
| `-I` | Girdi formatı: `dts`, `dtb` veya `fs` (dosya sistemi, ör. `/proc/device-tree`) |
| `-O` | Çıktı formatı: `dts`, `dtb` veya `asm` |
| `-o` | Çıktı dosyası |
| `-@` | Label'ları (`__symbols__`) DTB'ye ekle. Overlay kullanılacaksa gereklidir. |

> **İpucu**
>
> Geri derleme ( `-I dtb -O dts` ) çok işe yarar: Elinde sadece bir `.dtb` varsa (ör. hazır bir image'dan çıkan) içinde ne olduğunu görebilirsin. Ancak çıkan DTS'te yorumlar, `#include` 'lar, `#define` isimleri ve label'lar kaybolmuştur. Sayılar ham hex olarak görünür ( `GPIO_ACTIVE_HIGH` yerine `0x00` gibi).

### Kernel kaynak ağacında DTB derleme

Kernel'in içindeki `.dts` dosyaları elle `dtc` çağırmak yerine kernel'in build sistemiyle derlenir. Bu yöntem cpp ön işleme adımını da otomatik yapar:

```
# Tüm DTB'leri derle (kernel kaynak klasöründe)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- dtbs

# Sadece BBB'nin DTB'sini derle (kernel 6.5+ yol yapısı)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- ti/omap/am335x-boneblack.dtb

# Çıktı:
#   arch/arm/boot/dts/ti/omap/am335x-boneblack.dtb
```

> **Not**
>
> Kernel 6.5'ten eski sürümlerde hedef sadece `am335x-boneblack.dtb` 'dir ve çıktı `arch/arm/boot/dts/am335x-boneblack.dtb` konumuna yazılır.

---

## 11.10 Boot Sürecinde Device Tree

Linux boot sürecinde gördüğümüz gibi U-Boot, kernel'i başlatmadan önce **hem kernel image'ını hem de DTB'yi** RAM'e yükler:

```
eMMC / SD kart
  /boot/zImage, /boot/am335x-boneblack.dtb
        │  U-Boot: load
        ▼
DDR RAM
  loadaddr = 0x82000000 → kernel
  fdtaddr  = 0x88000000 → DTB
        │  bootz / bootm   (r2 = DTB adresi)
        ▼
Linux kernel
  DTB'yi okur (unflatten), device'ları oluşturur
        │
        ▼
Driver'lar compatible ile eşleşir ve probe edilir
```

SD karttan boot bölümündeki `uEnv.txt` ile birebir aynı mantık:

```
fdtaddr=0x88000000                                  # DTB'nin RAM'deki adresi
load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb  # DTB'yi RAM'e yükle
bootm ${loadaddr} - ${fdtaddr}                      # kernel'i başlat ve DTB adresini ver
```

`bootm` / `bootz` komutunun üç parametresi:

| Parametre | Anlamı |
|---|---|
| `${loadaddr}` | Kernel image'ının RAM adresi |
| `-` | initramfs adresi (`-` = yok) |
| `${fdtaddr}` | **DTB'nin RAM adresi** |

ARM32'de U-Boot kernel'e atlarken DTB'nin fiziksel adresini `r2` **register'ında** verir. Kernel açılışta bu adresteki blob'u okur, `0xd00dfeed` magic number'ını kontrol eder ve DTB'yi bellekte gezilebilir bir ağaca dönüştürür (**unflatten**).

> **Not**
>
> U-Boot, kernel'e vermeden önce DTB'yi **değiştirebilir** de. Örneğin `bootargs` değişkenini DTB'deki `/chosen` node'una yazar ve RAM boyutunu `/memory` node'una işler. Kernel command line'ının kernel'e ulaşma yollarından biri budur.

## U-Boot'ta DTB'yi incelemek

U-Boot'un `fdt` komutuyla RAM'deki DTB'ye bakılabilir:

```
=> load mmc 0:2 ${fdtaddr} /boot/am335x-boneblack.dtb
=> fdt addr ${fdtaddr}           # hangi adresteki DTB ile çalışılacak
=> fdt print /                   # tüm ağacı yazdır
=> fdt print /leds               # sadece leds node'u
=> fdt list /ocp                 # sadece alt node isimleri
```

---

## 11.11 Çalışan Sistemde Device Tree'yi İncelemek

Kernel, okuduğu device tree'yi bir dosya sistemi olarak dışarı açar:

```
/sys/firmware/devicetree/base/     # asıl konum
/proc/device-tree                  # → yukarıdakine sembolik link
```

Burada **her node bir klasör, her property bir dosyadır**:

```
# Kartın modeli
$ cat /proc/device-tree/model
TI AM335x BeagleBone Black

# Kök node'un compatible listesi (string'ler \0 ile ayrılmıştır)
$ cat /proc/device-tree/compatible | tr '\0' '\n'
ti,am335x-bone-black
ti,am335x-bone
ti,am33xx

# LED node'larına bak
$ ls /proc/device-tree/leds/

# Bootloader'ın kernel'e verdiği bootargs
$ cat /proc/device-tree/chosen/bootargs

# Sayısal property'leri hex olarak gör (ör. RAM adresi ve boyutu)
$ hexdump -C /proc/device-tree/memory@80000000/reg
00000000  80 00 00 00 20 00 00 00                           |.... ...|
#         adres=0x80000000  boyut=0x20000000 (512 MB)
```

Tüm ağacı tek seferde okunabilir DTS olarak görmek için:

```
sudo apt install device-tree-compiler
dtc -I fs -O dts /proc/device-tree > canli.dts 2>/dev/null
less canli.dts
```

Bir cihazın hangi driver'a bağlandığını görmek için:

```
ls /sys/bus/platform/devices/          # DT'den oluşturulan platform device'lar
ls -l /sys/bus/platform/devices/44e09000.serial/driver
#  → .../drivers/omap8250   (UART0'ı süren driver, kernel config'e göre serial-omap da olabilir)
```

BBB'nin Debian image'larında DTB dosyaları `/boot/dtbs/<kernel-sürümü>/` klasöründedir:

```
ls /boot/dtbs/$(uname -r)/ | grep boneblack
```

> **İpucu**
>
> Bir donanım çalışmıyorsa ilk kontrol: `/proc/device-tree` altında o cihazın node'u var mı ve `status` değeri `okay` mi? Node yoksa ya da `disabled` ise driver hiç devreye girmez. `dmesg | grep -i <cihaz>` ile de probe hatalarına bakılabilir.

---

## 11.12 Device Tree Overlay (.dtbo)

BeagleBone'a **cape** (P8/P9 header'larına takılan eklenti kart) takıldığında donanım değişir. Her cape kombinasyonu için ayrı bir `.dts` yazmak mantıklı değildir. Bunun çözümü **overlay**'dir.

- Overlay, ana DTB'ye **sonradan uygulanan bir yamadır**. Node ekleyebilir veya mevcut node'ları değiştirebilir.
- `.dts` kaynağından `dtc -@` ile `.dtbo` olarak derlenir.
- Ana DTB'nin de `-@` ile derlenmiş olması gerekir (label'ların bulunabilmesi için).
- Genellikle U-Boot tarafından boot sırasında uygulanır.

Basit bir overlay örneği (P9 header'ındaki I2C2'yi açar):

```
/dts-v1/;
/plugin/;                 /* bu bir overlay */

&i2c2 {                   /* ana DTB'deki i2c2 label'ına yama */
    status = "okay";
    clock-frequency = <100000>;
};
```

```
dtc -@ -I dts -O dtb -o BB-I2C2-ORNEK.dtbo BB-I2C2-ORNEK.dts
```

BBB'nin resmi Debian image'larında overlay'ler `/boot/uEnv.txt` dosyasından yönetilir. Hazır overlay'ler `/lib/firmware/` klasöründedir:

```
# /boot/uEnv.txt (ilgili satırlar)
enable_uboot_overlays=1
uboot_overlay_addr0=/lib/firmware/BB-I2C2-00A0.dtbo
#disable_uboot_overlay_video=1      # HDMI'ı kapatıp o pinleri serbest bırakır
```

> **Not**
>
> Raspberry Pi'deki `config.txt` içindeki `dtoverlay=...` satırları da aynı mekanizmadır. Overlay kavramı BeagleBone'a özel değildir.

---

## 11.13 Özet

| Kavram | Bir cümlede |
|---|---|
| **Device Tree** | Keşfedilemeyen donanımı kernel'e tarif eden, ağaç yapısındaki veri |
| **.dts** | Bir kartın insan tarafından okunabilir kaynak tarifi |
| **.dtsi** | Birden fazla `.dts` tarafından include edilen ortak (genelde SoC) tarif |
| **dtc** | DTS ↔ DTB dönüşümü yapan derleyici |
| **.dtb** | Derlenmiş binary. Bootloader RAM'e yükler, kernel okur. |
| **.dtbo** | DTB'ye sonradan uygulanan yama (cape/HAT için) |
| **compatible** | Node ile driver'ı eşleştiren string |
| **reg** | Cihazın adresi (ve memory-mapped ise boyutu) |
| **status** | `"okay"` → kullan, `"disabled"` → kullanma |
| **label / phandle** | Node'a takma ad ve başka node'dan ona referans ( `&label` ) |
| **/proc/device-tree** | Çalışan kernel'in gördüğü device tree |

BeagleBone Black için akış:

```
am33xx.dtsi + am335x-bone*.dtsi + am335x-boneblack.dts
        │  (cpp + dtc  /  make dtbs)
        ▼
am335x-boneblack.dtb  ──►  SD/eMMC: /boot/...
        │  U-Boot: load → fdtaddr (0x88000000)
        ▼
bootz/bootm ${loadaddr} - ${fdtaddr}   (r2 = DTB adresi)
        │
        ▼
Kernel DTB'yi okur → device'ları oluşturur → compatible ile driver'ları probe eder
```

---

## Kaynaklar

- Görseller: Bootlin - Linux kernel and driver development training slaytları ve bootlin/training-materials deposu. Lisans: CC BY-SA 3.0
- Devicetree Specification - devicetree.org
- Linux kernel dokümantasyonu: Devicetree
- Bootlin - Device Tree 101 webinar
- eLinux.org - Device Tree Usage

# Kısım V
# Root Filesystem

# Bölüm 12 — BusyBox ile Minimal Root Filesystem

> **Bu bölümde:**
>
> - BusyBox'ın ne olduğunu ve neden gömülü sistemlerde bu kadar yaygın kullanıldığını öğreneceksin.
> - BusyBox 1.36.1'i ARM için **statik** derleyip kendi root filesystem'ini sıfırdan kuracaksın.
> - Kernel modüllerini rootfs'e kurup kartı kendi kernel'in ve kendi rootfs'inle açacaksın.
> - BusyBox `init`'ini, `inittab`'ı ve `rcS` başlangıç betiklerini anlayacaksın.
> - USB üzerinden Ethernet sürücüsünü ( `g_ether` ) açılışta otomatik yükleyeceksin.
> - Rootfs'i NFS ile bilgisayardan bağlamayı göreceksin.
> - İki challenge: SSH ve TFTP desteği eklemek, BusyBox'ı dinamik derlemek.
>
> **Ön koşul:** Bölüm 6 — rootfs klasörleri, Bölüm 9 — cross toolchain, Bölüm 10 — kernel ve modüller.

---

## 12.1 Root Filesystem'de Neler Olmalı?

Kernel açılışın sonunda `root=` ile gösterilen dosya sistemini `/` olarak bağlar ve içindeki `/sbin/init`'i çalıştırır (3.4). Bundan sonra olan her şey rootfs'in işidir. Klasörlerin görevlerini 6.6.1'de görmüştük. Çalışan en küçük bir Linux için gerekenler şunlar:

| Gereken | Neden? | Bu bölümde nereden? |
|---|---|---|
| `/sbin/init` | Kernel'in çalıştırdığı ilk program (PID 1) | BusyBox |
| Bir kabuk ( `/bin/sh` ) ve temel komutlar | Sisteme komut verebilmek | BusyBox |
| `/dev` , `/proc` , `/sys` klasörleri | Kernel'in sanal dosya sistemleri buraya bağlanır | `mkdir` |
| `/etc/inittab` , `/etc/init.d/rcS` | `init`'in açılışta ne yapacağı | Elle yazacağız |
| `/lib/modules/<sürüm>/` | Modül olarak derlenen sürücüler | Bölüm 10'daki `modules_install` |
| `/lib/*.so` (sadece dinamik derlemede) | Paylaşılan kütüphaneler | 12.11 |

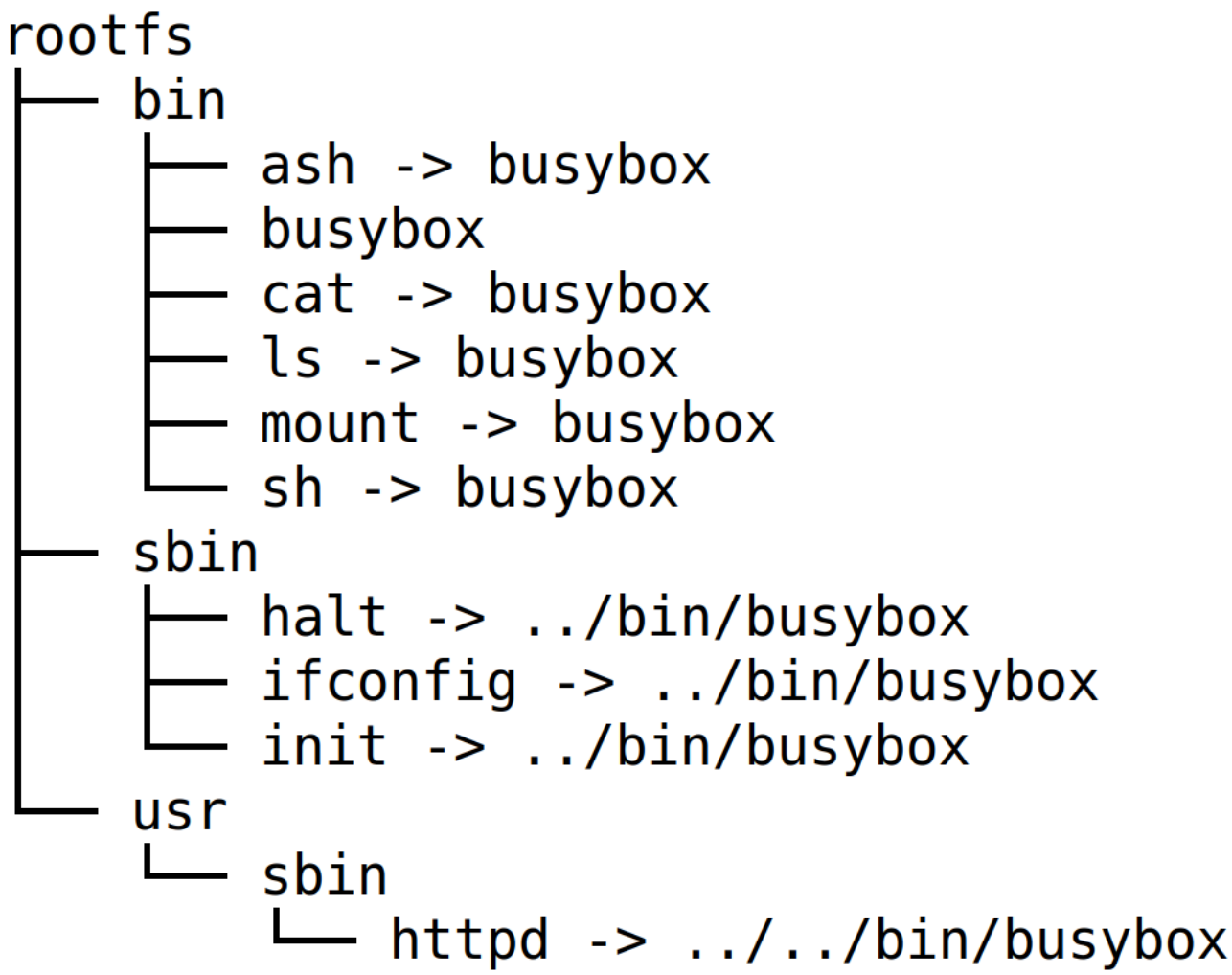
> **Not**
>
> Kursun "About RFS" ve "About initramfs" makaleleri bu konunun teorisini anlatıyor. initramfs, bu bölümde kuracağımız klasör yapısının sıkıştırılıp RAM'e yüklenen halidir. Bölüm 7'de kullandığımız `initramfs` dosyası da böyle bir BusyBox rootfs'idir.

---

## 12.2 BusyBox Nedir?

**BusyBox**, `ls`, `cp`, `mount`, `sh`, `init`, `ifconfig`, `vi`, `tftp` gibi yüzlerce Unix komutunu **tek bir küçük programda** toplayan bir projedir. Her komuta **applet** denir.

- Tek bir `busybox` dosyası vardır (birkaç yüz KB ile 2 MB arası).
- `/bin/ls`, `/sbin/init` gibi komutlar bu dosyaya giden **sembolik linklerdir**.
- `ls` olarak çağrılınca BusyBox, çağrıldığı adı (`argv[0]`) okur ve `ls` gibi davranır. `busybox ls` yazmak da aynı işi yapar.




*Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)*

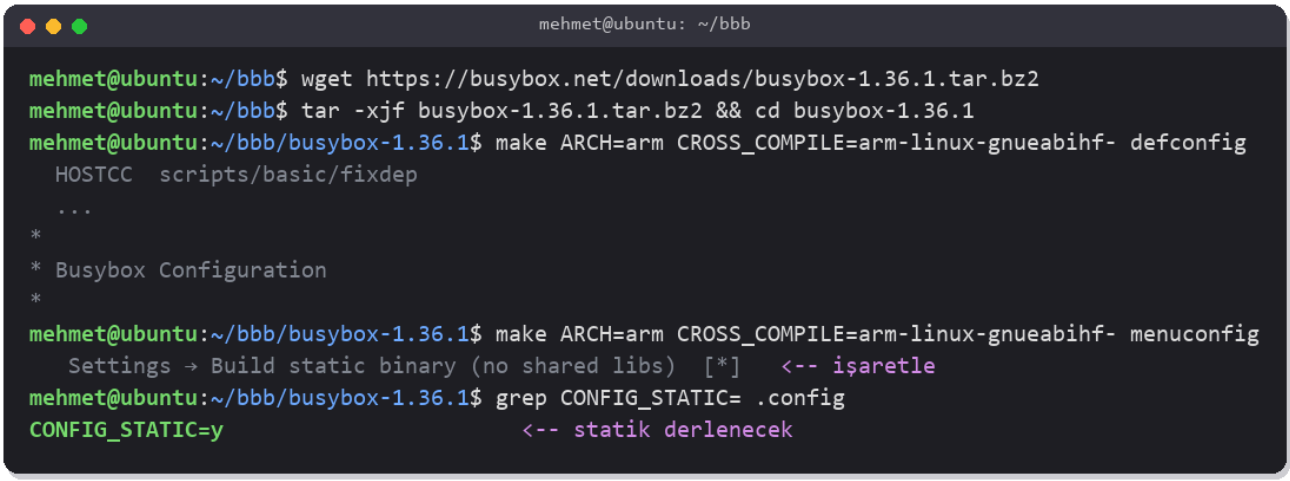
| | Masaüstü Linux (GNU coreutils vb.) | BusyBox |
|---|---|---|
| Boyut | Yüzlerce MB, her komut ayrı program | Tek dosya, ~1–2 MB |
| Özellik | Bütün seçenekler | En çok kullanılan seçenekler |
| Yapılandırma | — | Kernel gibi `menuconfig` ile hangi applet'lerin derleneceği seçilir |

---

## 12.3 Adım Adım: BusyBox'ı Derlemek

> **Not**
>
> Aşağıdaki ekran görüntüleri **temsilidir**. Çalışma klasörü `~/bbb` , rootfs `~/bbb/rootfs` . Toolchain Bölüm 9'da kurulan Linaro GCC 7.5.0.

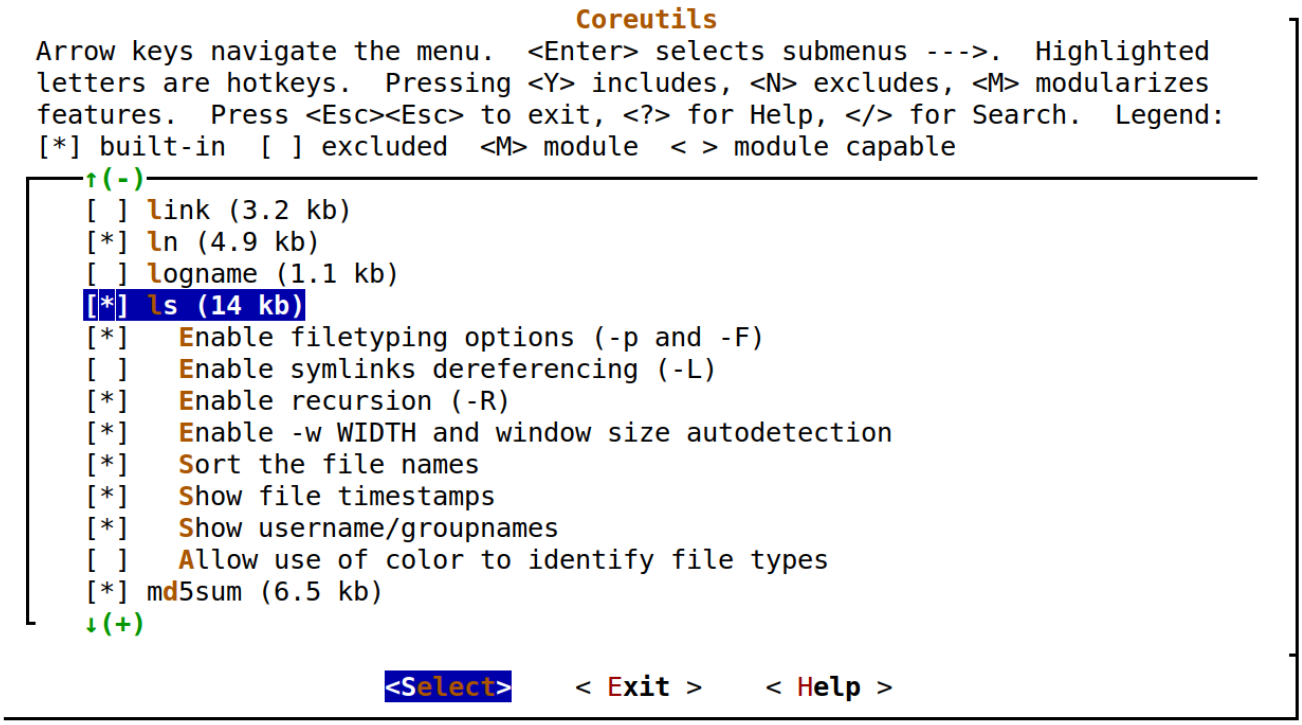
### Adım 1 — İndir ve yapılandır




```
cd ~/bbb
wget https://busybox.net/downloads/busybox-1.36.1.tar.bz2
tar -xjf busybox-1.36.1.tar.bz2
cd busybox-1.36.1
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- defconfig
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig
```

`menuconfig` 'te tek bir ayarı değiştir:

```
Settings  --->
    [*] Build static binary (no shared libs)
```




*Görsel: Bootlin, "Embedded Linux system development" eğitim slaytları (CC BY-SA 3.0)*

| | **Statik ( `CONFIG_STATIC=y` )** | **Dinamik** |
|---|---|---|
| Kütüphaneler | `busybox` dosyasının **içine** gömülür | Çalışırken rootfs'teki `/lib/*.so` dosyalarından yüklenir |
| Rootfs'te `/lib` gerekir mi? | Hayır | Evet (12.11) |
| Ne zaman? | İlk deneme, en basit rootfs. **Kursta kullanılan.** | Rootfs'e başka dinamik programlar da konacaksa |

## Adım 2 — Derle ve rootfs klasörüne kur

```
mehmet@ubuntu: ~/bbb/busybox-1.36.1
mehmet@ubuntu:~/bbb/busybox-1.36.1$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j$(nproc)
  ...
  LINK    busybox_unstripped
Static linking against glibc, can't use --gc-sections
Trying libraries: crypt m resolv rt
Final link with: m resolv rt
mehmet@ubuntu:~/bbb/busybox-1.36.1$ make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=~/bbb/rootfs install
  ~/bbb/rootfs/bin/ash -> busybox
  ~/bbb/rootfs/bin/cat -> busybox
  ...
--------------------------------------------------
You will probably need to make your busybox binary
setuid root to ensure all configured applets will
work properly.
--------------------------------------------------
mehmet@ubuntu:~/bbb/busybox-1.36.1$ file ~/bbb/rootfs/bin/busybox
~/bbb/rootfs/bin/busybox: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
statically linked, for GNU/Linux 3.2.0, stripped      <-- ARM + statik
```

```
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=~/bbb/rootfs install
file ~/bbb/rootfs/bin/busybox
```

`install` hedefi `busybox`'ı `~/bbb/rootfs/bin/` altına koyar ve bütün applet linklerini (`bin/`, `sbin/`, `usr/bin/`, `usr/sbin/`) oluşturur. `file` çıktısında **ARM** ve **statically linked** görmelisin.

> **İpucu**
>
> Statik derlemede `Using 'getpwnam' in statically linked applications requires at runtime the shared libraries...` gibi uyarılar görebilirsin. Bunlar glibc'nin statik bağlamadaki bilinen uyarılarıdır, derleme başarılıdır.

### Adım 3 — Kernel modüllerini kur

Kursun "Kernel Modules installation" dersi: Bölüm 10'da derlediğin modülleri aynı rootfs klasörüne kur. Bunu daha önce yaptıysan atlayabilirsin.

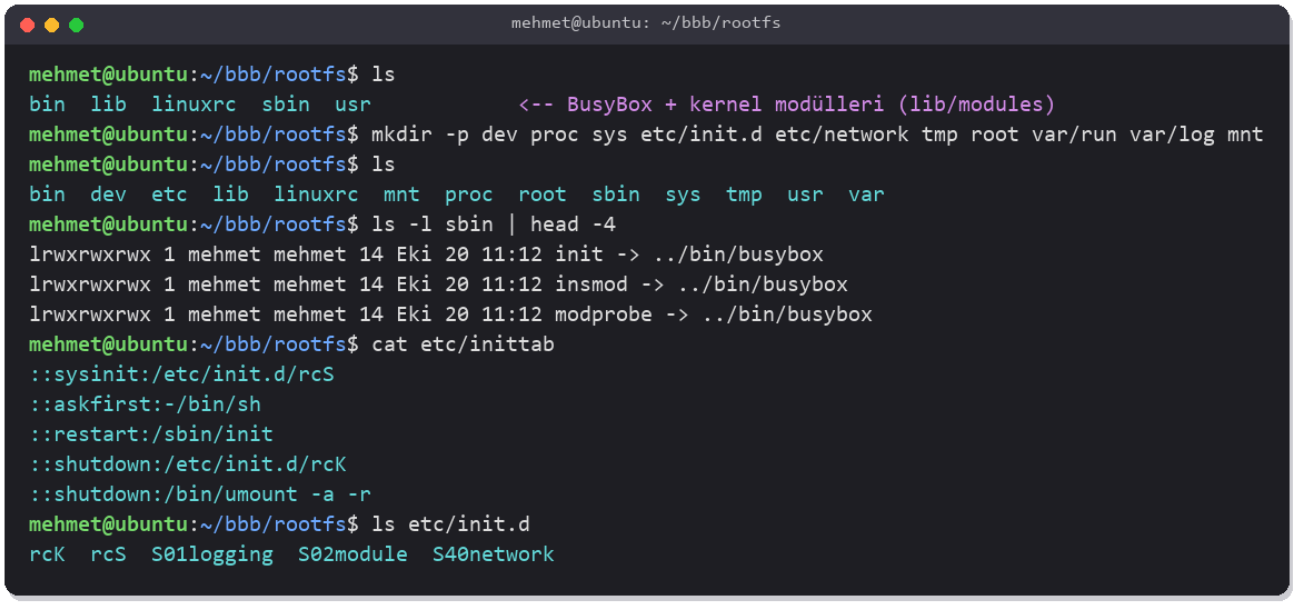
```
cd ~/bbb/linux
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- INSTALL_MOD_PATH=~/bbb/rootfs modules_install
ls ~/bbb/rootfs/lib/modules/          # 5.10.162
```

> **Önemli**
>
> `/lib/modules/` altındaki klasör adı, çalışan kernel'in `uname -r` çıktısıyla **birebir aynı** olmalı. Kernel'i yeniden derleyip sürüm değiştiysen modülleri de yeniden kur. Aksi halde `modprobe` modülleri bulamaz.

---

## 12.4 Rootfs'i Tamamlamak

BusyBox sadece programları kurar. Kalan klasörleri ve ayar dosyalarını elle oluşturuyoruz:




```
cd ~/bbb/rootfs
mkdir -p dev proc sys etc/init.d tmp root var/run var/log mnt
mkdir -p etc/network/if-pre-up.d etc/network/if-up.d etc/network/if-down.d etc/network/if-post-down.d
```

| Klasör | Neden boş da olsa gerekli? |
|---|---|
| `dev` | Kernel buraya `devtmpfs`'i bağlar ve aygıt dosyalarını (`/dev/ttyS0`, `/dev/mmcblk0` ...) kendisi oluşturur. `bb.org_defconfig`'te `CONFIG_DEVTMPFS_MOUNT=y` olduğu için ayrıca bir şey yapmana gerek yok. |
| `proc`, `sys` | `rcS` buraya `proc` ve `sysfs` sanal dosya sistemlerini bağlayacak |
| `tmp`, `var/run`, `var/log` | Geçici dosyalar, PID dosyaları ve loglar |
| `etc/network/if-*.d` | BusyBox'ın `ifup` komutu bu klasörleri arar; yoksa hata verir |

> **Not**
>
> `/dev` klasörünü elle doldurmana (`mknod`) gerek yok. Eskiden aygıt dosyaları elle oluşturulurdu; bugün `devtmpfs` bunu kernel tarafında otomatik yapar.

---

## 12.5 BusyBox `init`, `inittab` ve `rcS`

Kernel rootfs'i bağladıktan sonra şu zincir çalışır:

```
kernel  ──►  /sbin/init  (BusyBox, PID 1)
                 │  /etc/inittab'ı okur
                 ├── ::sysinit   → /etc/init.d/rcS        (bir kez, açılışta)
                 │                    └── /etc/init.d/S??*  sırayla "start" ile
                 ├── ::askfirst  → /bin/sh                (konsolda Enter'a basınca kabuk)
                 └── ::shutdown  → /etc/init.d/rcK        (kapanırken, ters sırayla "stop")
```

### `/etc/inittab`

`inittab` her satırda `<tty>:<runlevel>:<eylem>:<program>` biçimindedir. BusyBox `runlevel` alanını kullanmaz. `tty` boşsa konsol kullanılır.

```
::sysinit:/etc/init.d/rcS
::askfirst:-/bin/sh
::restart:/sbin/init
::shutdown:/etc/init.d/rcK
::shutdown:/bin/umount -a -r
```

| Eylem | Anlamı |
|---|---|
| `sysinit` | Açılışta bir kez çalışır, bitene kadar beklenir |
| `askfirst` | "Please press Enter to activate this console." yazar, Enter'a basınca programı başlatır. Program kapanırsa yeniden başlatır. `-/bin/sh`'teki `-` kabuğu "login shell" yapar. |
| `respawn` | `askfirst` gibi ama sormadan başlatır |
| `restart` | `init` yeniden başlatılırken çalışır |
| `shutdown` | `reboot` / `poweroff` sırasında çalışır |

> **İpucu**
>
> `inittab` hiç yoksa BusyBox varsayılan bir tane kullanır: `rcS`'i çalıştırır, konsolda ve **tty2–tty4**'te kabuk açmaya çalışır. BBB'de bu terminaller olmadığı için `can't open /dev/tty2` hataları görürsün. Kendi `inittab`'ını yazmak bunu önler.

### `/etc/init.d/rcS` — kursun betiği

Kursun reposundaki (`scripts/etc/init.d/rcS`) betik `/proc`'u bağlar ve `/etc/init.d` altındaki `S` ile başlayan iki haneli numaralı betikleri **sırayla** `start` parametresiyle çalıştırır. Sona `.sh` eklenmiş betikler ayrı süreç açmadan, aynı kabukta çalıştırılır:

```
#!/bin/sh

# Start all init scripts in /etc/init.d
# executing them in numerical order.
#
echo "Mounting proc"
mount -t proc /proc /proc

for i in /etc/init.d/S??* ;do

     # Ignore dangling symlinks (if any).
     [ ! -f "$i" ] && continue

     case "$i" in
    *.sh)
        # Source shell script for speed.
        (
        trap - INT QUIT TSTP
        set start
        . $i
        )
        ;;
    *)
        # No sh extension, so fork subprocess.
        $i start
        ;;
    esac
done
```

`echo "Mounting proc"` satırından sonraya şu iki satırı da eklemeni öneririm. `sysfs` birçok aracın ve Bölüm 1'deki LED komutlarının, `devpts` ise SSH'ın (12.10) çalışması için gerekli:

```
mount -t sysfs sysfs /sys
mkdir -p /dev/pts && mount -t devpts devpts /dev/pts
```

`rcK` aynı döngüyü **ters sırada** ve `stop` parametresiyle çalıştırır. Numaralar sırayı belirler: `S01` önce, `S50` sonra başlar; kapanırken tersi.

Kursun reposundaki betikler:

| Betik | Görevi |
|---|---|
| `S01logging` | `syslogd` ve `klogd`'yi başlatır (loglar `/var/log/messages`'a) |
| `S02module` | Açılışta kernel modüllerini yükler (12.7) |
| `S40network` | `/etc/network/interfaces`'teki arayüzleri `ifup -a` ile açar |

| Betik | Görevi |
|---|---|
| `S50sshd` | OpenSSH sunucusunu başlatır. BusyBox rootfs'inde OpenSSH olmadığı için kendi başına çalışmaz (12.10'da yerine Dropbear kullanıyoruz) |

Betiklerin hepsini rootfs'e koy ve **çalıştırılabilir** yap:

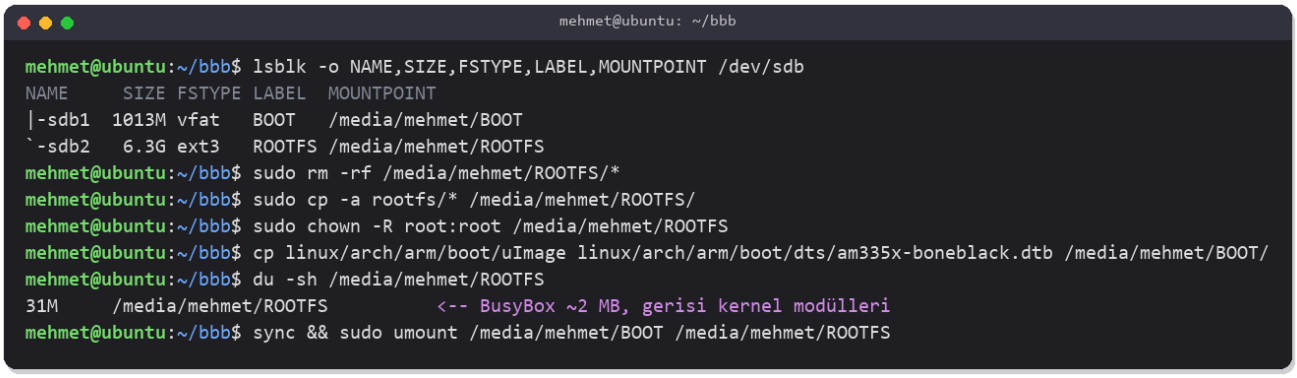
```
cd ~/bbb/rootfs/etc/init.d
# rcS, rcK, S01logging, S02module, S40network dosyalarını buraya kopyala
chmod +x rcS rcK S*
```

---

## 12.6 Kartta Test: Rootfs SD Kartta

Kursun "Testing boot images and busybox on BBB" dersleri: kernel ve DTB SD kartın BOOT bölümünden, rootfs ROOTFS bölümünden gelir.

**1) Rootfs'i SD karta kopyala.**



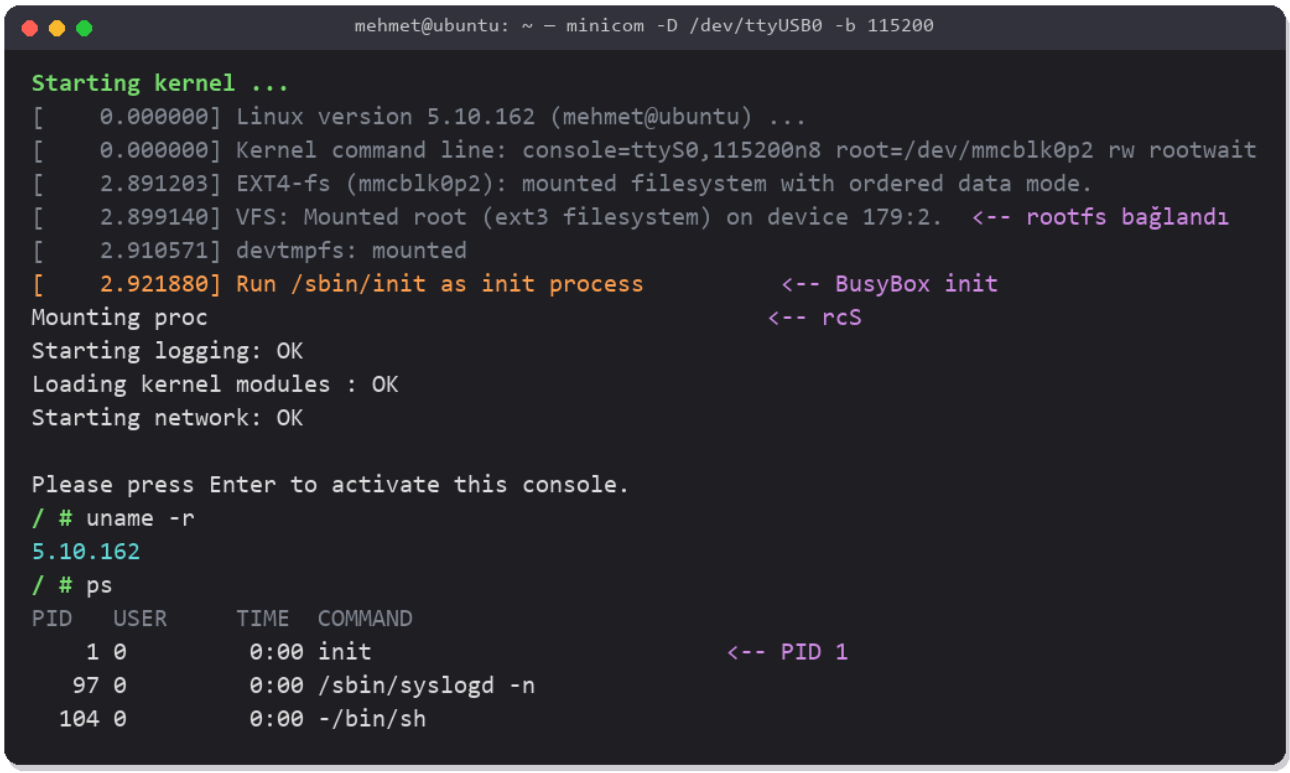

```
cd ~/bbb
sudo rm -rf /media/$USER/ROOTFS/*
sudo cp -a rootfs/* /media/$USER/ROOTFS/
sudo chown -R root:root /media/$USER/ROOTFS
cp linux/arch/arm/boot/uImage linux/arch/arm/boot/dts/am335x-boneblack.dtb /media/$USER/BOOT/
sync && sudo umount /media/$USER/BOOT /media/$USER/ROOTFS
```

> **Dikkat**
>
> `sudo rm -rf /media/$USER/ROOTFS/*` komutundaki yolu iki kez kontrol et. Yol yanlış yazılırsa bilgisayarındaki dosyaları silebilir.

**2) BOOT bölümündeki `uEnv.txt`** (10.9'dakinin rootfs'i SD kart olan hali):

```
console=ttyS0,115200n8
netargs=setenv bootargs console=${console} root=/dev/mmcblk0p2 rw rootwait
netboot=echo Booting from microSD ...; setenv autoload no; load mmc 0:1 ${loadaddr} uImage; load
mmc 0:1 ${fdtaddr} am335x-boneblack.dtb; run netargs; bootm ${loadaddr} - ${fdtaddr}
uenvcmd=run netboot
```

**3) Kartı S2'ye basılı tutarak aç.** `rcS`'in mesajlarından sonra Enter'a bas:




`ps` çıktısında PID 1'in `init` olduğunu görürsün. Artık kartta çalışan her şeyi sen derledin: SPL, U-Boot, kernel, DTB ve rootfs.

---

## 12.7 USB Üzerinden Ethernet ve Sürücüleri Açılışta Yüklemek

Kursun "Enabling ethernet over usb by driver integration" ve "Auto loading drivers during system startup" dersleri.

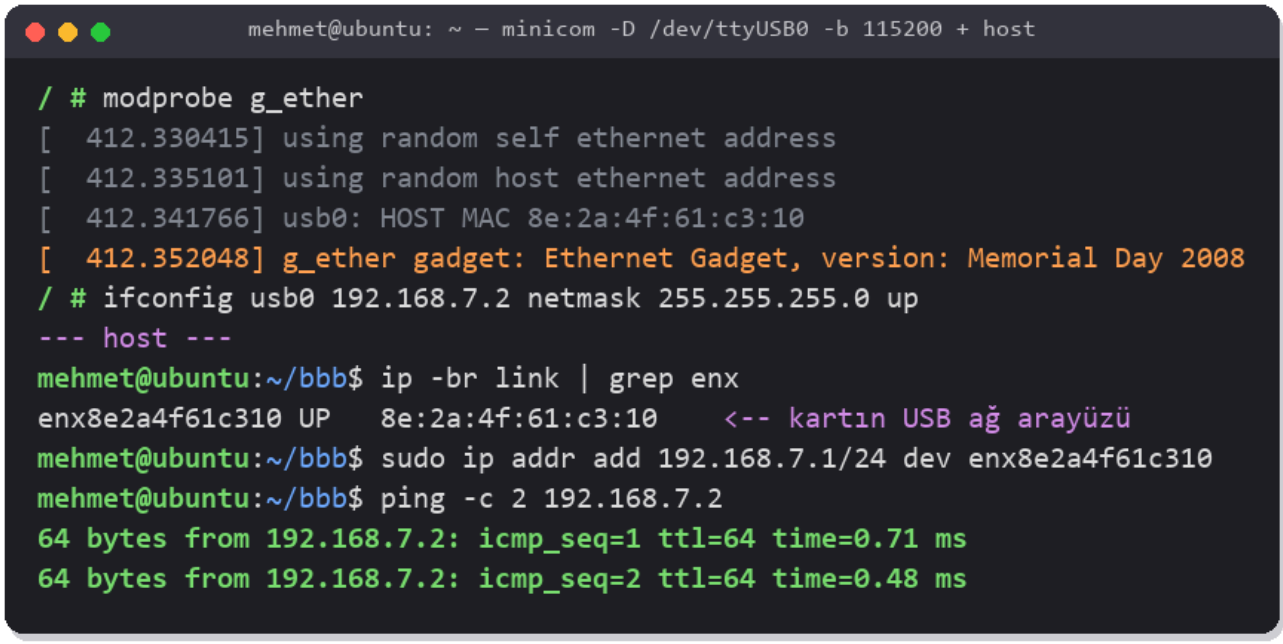
### USB gadget: `g_ether`

BBB'nin mini-USB portu (USB0) **gadget** (cihaz) modunda çalışabilir. `g_ether` sürücüsü yüklenince kart, bilgisayara USB üzerinden bir **ağ kartı** gibi görünür. Ethernet kablosu olmadan SSH, TFTP, NFS yapılabilir. Kursun `192.168.7.x` adresleri bu ağdan gelir.

`bb.org_defconfig`'te bu sürücü **modül** olarak derlenir (`CONFIG_USB_ETH=m`, 10.8). Başka bir defconfig kullanıyorsan `menuconfig`'te açman gerekir:

```
Device Drivers  --->
    [*] USB support  --->
        <*>   USB Gadget Support  --->
            <M>   USB Gadget precomposed configurations
                <M>   Ethernet Gadget (with CDC Ethernet support)
```

Elle denemek için kartta:




```
/ # modprobe g_ether
/ # ifconfig usb0 192.168.7.2 netmask 255.255.255.0 up
```

Bilgisayarda yeni bir ağ arayüzü ( `enx...` veya `usb0` ) belirir. Ona `192.168.7.1` ver ve ping at:

```
ip -br link
sudo ip addr add 192.168.7.1/24 dev enx8e2a4f61c310     # arayüz adını kendi çıktından al
ping 192.168.7.2
```

### Açılışta otomatik yüklemek

Her açılışta bu komutları yazmamak için iki betik kullanılır.

**`S02module`** (kursun reposundan) `g_ether` 'i yükler:

```
#!/bin/sh
#
# load the kernel modules at startup
#

case "$1" in
  start)
    printf "Loading kernel modules : "
    /sbin/modprobe g_ether
    [ $? = 0 ] && echo "OK" || echo "FAIL"
    ;;
  stop)
    printf "Unloading kernel modules : "
    /sbin/rmmod g_ether
    [ $? = 0 ] && echo "OK" || echo "FAIL"
    ;;
  restart|reload)
    "$0" stop
    "$0" start
    ;;
  *)
    echo "Usage: $0 {start|stop|restart}"
    exit 1
esac

exit $?
```

**`S40network`** `ifup -a` ile `/etc/network/interfaces` dosyasındaki arayüzleri açar. Bu dosyayı oluştur:

```
# /etc/network/interfaces
auto lo
iface lo inet loopback

auto usb0
iface usb0 inet static
    address 192.168.7.2
    netmask 255.255.255.0
```

Numaralar sırayı garanti eder: önce `S02` sürücüyü yükler, sonra `S40` arayüzü açar. Kartı yeniden başlatınca ekranda `Loading kernel modules : OK` ve `Starting network: OK` görürsün.

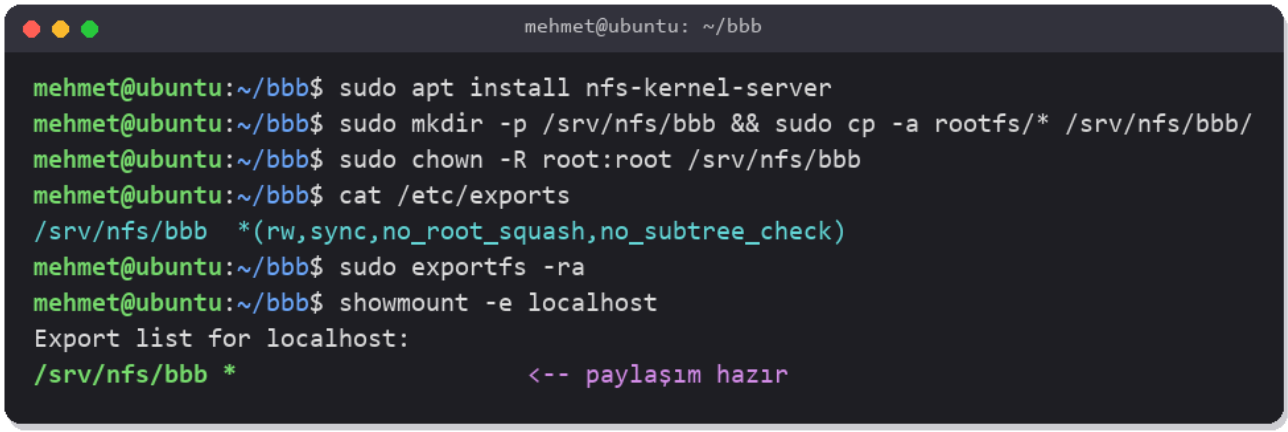
---

## 12.8 Rootfs'i NFS ile Bilgisayardan Bağlamak

Rootfs'i her değişiklikte SD karta kopyalamak yorucu. **NFS** (Network File System) ile kart, rootfs'i doğrudan bilgisayardaki bir klasörden bağlar. Bilgisayarda bir dosyayı değiştirdiğin anda kart da görür. Bölüm 8'deki TFTP ile birlikte geliştirme döngüsü şöyle olur:

```
Host PC                                    BBB
 /var/lib/tftpboot/uImage, .dtb  ── TFTP ──►  U-Boot: RAM'e yükle, bootm
 /srv/nfs/bbb/  (rootfs)     ── NFS  ──►  Kernel: / olarak bağla
```

### Adım 1 — Host'ta NFS sunucusu




```
sudo apt install nfs-kernel-server
sudo mkdir -p /srv/nfs/bbb
sudo cp -a ~/bbb/rootfs/* /srv/nfs/bbb/
sudo chown -R root:root /srv/nfs/bbb
echo "/srv/nfs/bbb  *(rw,sync,no_root_squash,no_subtree_check)" | sudo tee -a /etc/exports
sudo exportfs -ra
showmount -e localhost
```

| `/etc/exports` seçeneği | Anlamı |
|---|---|
| `*` | Her IP'den erişime izin ver (yerine `192.168.27.2` yazarak sadece karta açabilirsin) |
| `rw` | Okuma-yazma |
| `sync` | Yazmalar diske yazıldıktan sonra onay ver |
| `no_root_squash` | Karttaki `root` kullanıcısı paylaşımda da `root` olsun. Rootfs için şart. |
| `no_subtree_check` | Alt klasör kontrollerini kapat (daha hızlı ve sorunsuz) |

### Adım 2 — `uEnv.txt`

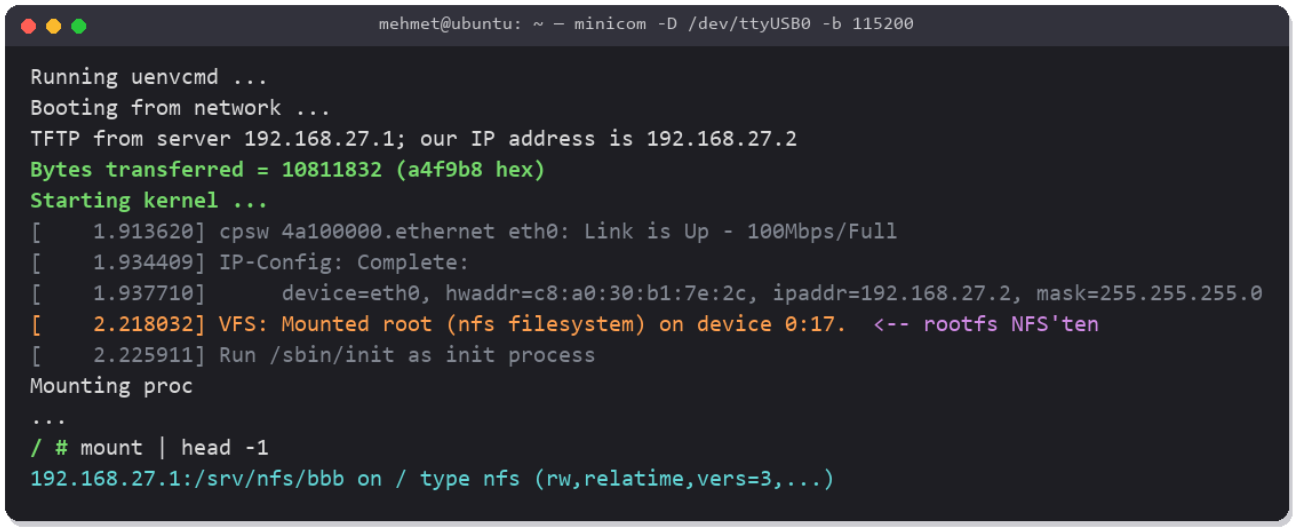
Kursun `uEnv-nfs.txt` dosyası temel alındı. Kursun dosyası USB ağı ( `192.168.7.x` ) kullanıyor; burada Bölüm 8'deki Ethernet kablosu ve adresleri kullanılıyor:

```
console=ttyS0,115200n8
ipaddr=192.168.27.2
serverip=192.168.27.1
rootpath=/srv/nfs/bbb
loadtftp=echo Booting from network ...; tftpboot ${loadaddr} uImage; tftpboot ${fdtaddr} am335x-boneblack.dtb
netargs=setenv bootargs console=${console} root=/dev/nfs rw
nfsroot=${serverip}:${rootpath},nolock,v3,tcp rootwait
ip=${ipaddr}:${serverip}::255.255.255.0::eth0:off init=/sbin/init
uenvcmd=setenv autoload no; run loadtftp; run netargs; bootm ${loadaddr} - ${fdtaddr}
```

| `bootargs` parçası | Anlamı |
|---|---|
| `root=/dev/nfs` | Rootfs bir disk değil, NFS |
| `nfsroot=<sunucu>:<klasör>,...` | NFS sunucusu ve paylaşılan klasör |
| `nolock` | NFS dosya kilitlemesini kapat (kartta kilit servisi yok) |
| `v3,tcp` | NFS sürüm 3, TCP üzerinden. **Önemli:** kernel varsayılan olarak NFSv2 dener; Ubuntu 22.04 ve sonrasında NFSv2 kapalıdır. |
| `ip=<kart>:<sunucu>::<maske>::eth0:off` | Kernel'in IP'sini kendisi ayarlaması. U-Boot'un IP'si kernel'e geçmez (8.7). |
| `init=/sbin/init` | Çalıştırılacak ilk program (varsayılan da budur) |

`uImage` ve `am335x-boneblack.dtb`'yi `/var/lib/tftpboot`'a kopyalamayı unutma.

### Adım 3 — Aç ve doğrula




```
/ # mount | head -1
192.168.27.1:/srv/nfs/bbb on / type nfs (rw,...,vers=3,...)
```

> **Uyarı**
>
> Rootfs NFS'ten geliyorken kartta **`eth0` 'ı yeniden yapılandırma** ( `ifconfig eth0 ...` , `/etc/network/interfaces` 'e `eth0` eklemek). Ağ bir an bile kesilirse kernel rootfs'e ulaşamaz ve sistem donar. `eth0` 'ı kernel `ip=` ile ayarlıyor; `interfaces` dosyasında sadece `lo` ve `usb0` kalsın.

> **Not**
>
> Kursun `uEnv-nfs.txt` 'si NFS'i USB ağı ( `usb0` ) üzerinden yapar. Bunun için `g_ether` 'in modül değil **kernel'e gömülü** ( `CONFIG_USB_ETH=y` ) olması gerekir; çünkü rootfs bağlanmadan modül yüklenemez (10.7). Ethernet kablosunda bu sorun yok, CPSW sürücüsü zaten gömülü.

---

## 12.9 Geliştirme Döngüsü

| Ne değişti? | Ne yaparsın? |
|---|---|
| Kendi uygulaman / bir betik | Dosyayı `/srv/nfs/bbb` içine kopyala. Kart hemen görür, yeniden başlatma yok. |
| Kernel veya DTB | `uImage` / `.dtb` 'yi `/var/lib/tftpboot` 'a kopyala, kartı resetle. |
| Kernel modülleri | `modules_install INSTALL_MOD_PATH=/srv/nfs/bbb` (sudo ile), kartta `modprobe` ya da reset |
| BusyBox ayarları | BusyBox'ı yeniden derle, `CONFIG_PREFIX=/srv/nfs/bbb` ile kur |

## 12.10 Challenge 1: SSH ve TFTP Desteği

Kursun bu bölümdeki ilk challenge'ı:

> *BusyBox ile minimal bir dosya sistemi kurup kartta test ettin. Şimdi onu birkaç araçla zenginleştir: kart ile bilgisayar arasında **TFTP** ve **SSH** ile dosya aktarımı yapılabilsin. BusyBox'ta SSH desteği yoktur; üçüncü parti bir kaynaktan eklenmesi gerekir. İpucu: https://busybox.net/tinyutils.html*

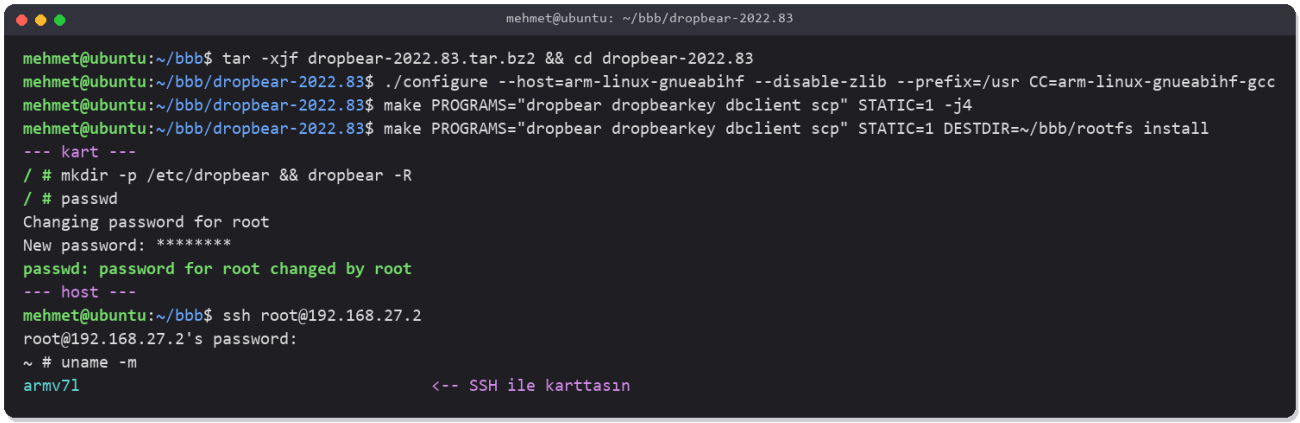
### TFTP

BusyBox'ın `defconfig`'inde `tftp` applet'i zaten açık (`Networking Utilities → tftp`). Kullanımı 8.7'deki gibi:

```
/ # tftp -g -r dosya 192.168.27.1        # host'tan al
/ # tftp -p -l dosya 192.168.27.1        # host'a gönder
```

### SSH: Dropbear

İpucundaki sayfa küçük sistemler için araçları listeler. SSH için önerilen **Dropbear**: SSH sunucusu, istemcisi ve `scp`'yi birkaç yüz KB'ta sunan bir proje. Kursun derleme notlarındaki adımlar:




```
cd ~/bbb
wget https://matt.ucc.asn.au/dropbear/releases/dropbear-2022.83.tar.bz2
tar -xjf dropbear-2022.83.tar.bz2 && cd dropbear-2022.83
./configure --host=arm-linux-gnueabihf --disable-zlib --prefix=/usr CC=arm-linux-gnueabihf-gcc
make PROGRAMS="dropbear dropbearkey dbclient scp" STATIC=1 -j4
make PROGRAMS="dropbear dropbearkey dbclient scp" STATIC=1 DESTDIR=~/bbb/rootfs install
```

| Parça | Anlamı |
|---|---|
| `--host=arm-linux-gnueabihf` | ARM için cross compile |

| Parça | Anlamı |
|---|---|
| `--disable-zlib` | zlib sıkıştırma kütüphanesi olmadan derle (rootfs'te zlib yok) |
| `--prefix=/usr` + `DESTDIR=...` | Kartta `/usr/sbin/dropbear` olacak şekilde rootfs klasörüne kur |
| `STATIC=1` | BusyBox gibi statik derle, kütüphane gerekmesin |

Kartta SSH'a giriş yapabilmek için bir **kullanıcı ve şifre** gerekir. Rootfs'e şu iki dosyayı ekle:

```
cd ~/bbb/rootfs
echo 'root:x:0:0:root:/root:/bin/sh' > etc/passwd
echo 'root::0:0:99999:7:::' > etc/shadow
chmod 600 etc/shadow
```

Kartta şifreyi belirle ve sunucuyu başlat:

```
/ # passwd
/ # mkdir -p /etc/dropbear
/ # dropbear -R
```

`-R`, sunucu anahtarlarını ilk bağlantıda otomatik üretir. Bilgisayardan bağlan ve dosya gönder:

```
ssh root@192.168.27.2
scp dosya root@192.168.27.2:/root/
```

Açılışta otomatik başlaması için `S50sshd` yerine şu küçük betiği koy:

```
#!/bin/sh
# /etc/init.d/S50dropbear
case "$1" in
  start) printf "Starting dropbear: "; mkdir -p /etc/dropbear; /usr/sbin/dropbear -R && echo "OK"
;;
  stop)  printf "Stopping dropbear: "; killall dropbear; echo "OK" ;;
  *)     echo "Usage: $0 {start|stop}"; exit 1 ;;
esac
```

> **Önemli**
>
> Dropbear, kabuk açmak için `/dev/pts` 'e ihtiyaç duyar. 12.5'te `rcS` 'e eklediğimiz `mount -t devpts devpts /dev/pts` satırı yoksa bağlantı `PTY allocation request failed` ile kapanır.

---

## 12.11 Challenge 2: BusyBox'ı Dinamik Derlemek

Kursun ikinci challenge'ı:

> *BusyBox derslerinin hepsinde "statik" derlenmiş programlar kullandık. Dinamik bağlanarak (dynamic linking) cross compile edilmiş bir uygulamayı bu dosya sisteminde çalıştırırsan çalışmaz. BusyBox'ı **dinamik** bağlanmış programlar üretecek şekilde yeniden derle ve dinamik bağlanmış bir uygulamanın çalıştığını test et.*

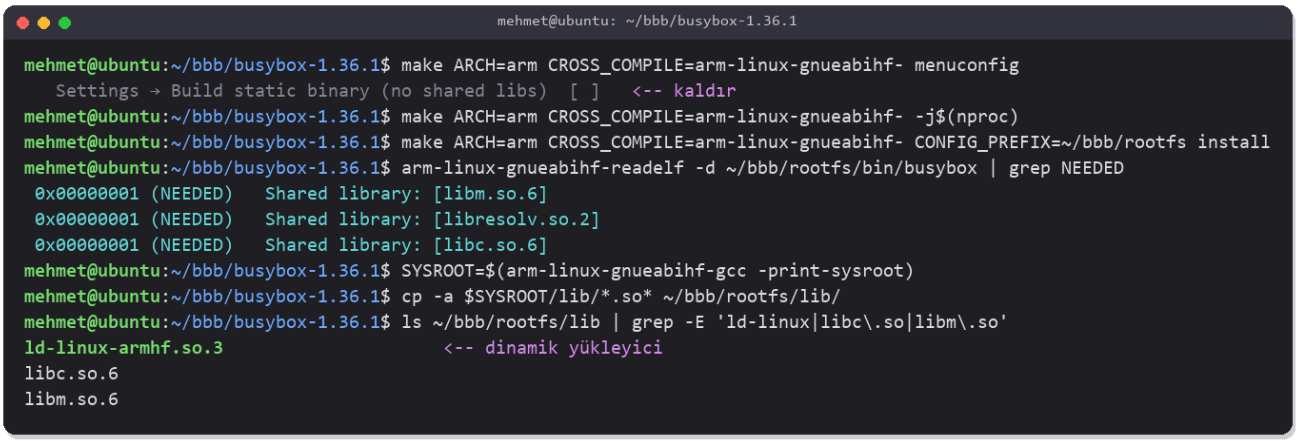
### Neden dinamik program çalışmıyor?

Dinamik bağlanmış bir program, çalışırken iki şeye ihtiyaç duyar:

1. **Dinamik yükleyici**: ARM hard-float'ta `/lib/ld-linux-armhf.so.3`. Kernel programı başlatırken önce bunu çalıştırır.
2. Programın kullandığı **paylaşılan kütüphaneler**: `libc.so.6`, `libm.so.6` ...

Statik BusyBox rootfs'inde `/lib` altında sadece kernel modülleri vardır. Yükleyici yoksa kernel programı hiç başlatamaz ve kabuk yanıltıcı bir şekilde `not found` der.

### Çözüm




**1) Statik ayarı kapat, yeniden derle ve kur:**

```
cd ~/bbb/busybox-1.36.1
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- menuconfig      # Settings → Build static
binary: [ ]
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- -j$(nproc)
make ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf- CONFIG_PREFIX=~/bbb/rootfs install
```

**2) Hangi kütüphanelerin gerektiğine bak:**

```
arm-linux-gnueabihf-readelf -d ~/bbb/rootfs/bin/busybox | grep NEEDED
arm-linux-gnueabihf-readelf -l ~/bbb/rootfs/bin/busybox | grep interpreter
```

**3) Kütüphaneleri toolchain'den rootfs'e kopyala.** Toolchain, ARM için derlenmiş glibc'yi kendi **sysroot** klasöründe taşır:

```
SYSROOT=$(arm-linux-gnueabihf-gcc -print-sysroot)
cp -a $SYSROOT/lib/*.so* ~/bbb/rootfs/lib/
```

> **İpucu**
>
> Bu komut toolchain'deki bütün kütüphaneleri kopyalar (birkaç on MB). Sadece gerekenleri istiyorsan `readelf` çıktısındaki dosyaları ve `ld-linux-armhf.so.3` 'ü kopyala. Kopyaladıktan sonra `arm-linux-gnueabihf-strip` ile küçültebilirsin.

**4) Dinamik bir uygulamayla test et:**

```
arm-linux-gnueabihf-gcc hello.c -o hello          # varsayılan: dinamik
file hello                                            # "dynamically linked, interpreter /lib/ld-
linux-armhf.so.3"
cp hello ~/bbb/rootfs/root/                         # NFS kullanıyorsan /srv/nfs/bbb/root/
```

Kartta `/root/hello` çalışıyorsa challenge tamam.

---

## 12.12 Özet

- BusyBox, yüzlerce komutu tek bir programda toplar; komutlar ona giden sembolik linklerdir.
- Minimal rootfs = BusyBox + `dev proc sys etc tmp var` klasörleri + `/etc/inittab` + `/etc/init.d/rcS` + `/lib/modules`.
- Açılış: kernel → `/sbin/init` (PID 1) → `inittab` → `rcS` → `S??*` betikleri sırayla → konsolda kabuk.
- `g_ether` kartı USB üzerinden bir ağ kartı yapar; `S02module` + `S40network` ile açılışta otomatik yüklenir.
- NFS + TFTP ile rootfs ve kernel bilgisayardan gelir; dosyayı kopyalamak, kartta denemek için yeterli olur.
- SSH için Dropbear, dinamik programlar için `/lib` 'e toolchain sysroot'undaki kütüphaneler gerekir.

## 12.13 Karşılaşılabilecek Problemler

```
mehmet@ubuntu: ~ — minicom -D /dev/ttyUSB0 -b 115200 (temsili hatalar)

Kernel panic - not syncing: No working init found.   <-- init yok / ARM değil / kütüphane eksik

can't open /dev/tty2: No such file or directory      <-- /etc/inittab yok

/etc/init.d/rcS: Permission denied                   <-- chmod +x

modprobe: can't change directory to '/lib/modules': No such file  <-- modules_install

VFS: Unable to mount root fs via NFS.                <-- NFS sürümü / exports / ip=

PTY allocation request failed on channel 0           <-- /dev/pts bağlı değil
```

| Belirti | Sebep | Çözüm |
|---|---|---|
| `Kernel panic - not syncing: No working init found` | `/sbin/init` yok, ARM için derlenmemiş ya da dinamik derlenmiş ama `/lib`'te kütüphane yok | `file bin/busybox` → ARM ve statik olmalı (veya 12.11) |
| `can't open /dev/tty2: No such file or directory` | `/etc/inittab` yok, BusyBox varsayılan olarak tty2–tty4'ü açmaya çalışıyor | 12.5'teki `inittab`'ı ekle |
| `/etc/init.d/rcS: Permission denied` | Betik çalıştırılabilir değil | `chmod +x etc/init.d/rcS etc/init.d/S*` |
| `modprobe: can't change directory to '/lib/modules'` | Modüller kurulmamış ya da sürüm klasörü `uname -r` ile uyuşmuyor | `modules_install INSTALL_MOD_PATH=...` ile yeniden kur |
| `VFS: Unable to mount root fs via NFS` | NFS sürümü (v2), `exports` hatası, yanlış `ip=` / `nfsroot=` | `nfsroot=...,v3,tcp` ; host'ta `showmount -e` ; firewall'da NFS'e izin |
| `ifup: can't open '/etc/network/interfaces'` | Dosya yok | 12.7'deki dosyayı oluştur |
| `run-parts: /etc/network/if-pre-up.d: No such file or directory` | BusyBox `ifup`'ın aradığı klasörler yok | `mkdir -p etc/network/if-{pre-up,up,down,post-down}.d` |
| SSH: `PTY allocation request failed` | `/dev/pts` bağlı değil | `rcS`'e `mount -t devpts devpts /dev/pts` ekle |
| SSH: `Permission denied` | root'un şifresi yok | Kartta `passwd` |
| Derleme: `networking/tc.c: error: 'TCA_CBQ_MAX' undeclared` | Toolchain'in kernel başlıkları çok yeni (6.8+), BusyBox 1.36'nın `tc` applet'i derlenmiyor | `menuconfig` → Networking Utilities → `tc`'yi kapat |
| Kartta dosyalar `root` yerine `nobody` sahipliğinde görünüyor | `no_root_squash` eksik | `/etc/exports`'a ekle, `sudo exportfs -ra` |